

DATA SCIENCE END-TO-END HANDBOOK

Cẩm nang Khoa học Dữ liệu chuẩn Production
từ cơ bản đến nâng cao — kèm code mẫu

Định nghĩa bài toán · Thu thập dữ liệu · EDA · Thống kê suy luận · Tiền xử lý · Chuẩn hóa · Chia dữ liệu
Huấn luyện · Đánh giá & Confusion Matrix · Giải thích mô hình · Trực quan hóa · MLOps

Phiên bản 2.0 — Tháng 10/2026

Bám sát tài liệu chính thức · Mọi khối code đã được chạy kiểm chứng

MỤC LỤC

DATA SCIENCE END-TO-END HANDBOOK

- 0.1. Cách dùng handbook
- 0.2. Kế hoạch nội dung
- 0.3. Phương pháp luận dự án
- 0.4. Vai trò trong một đội dữ liệu
- 0.5. Cấu trúc repository chuẩn
- 0.6. Môi trường phát triển
- 0.7. Tái lập kết quả (Reproducibility)
- 0.8. Bộ dữ liệu xuyên suốt handbook
- 0.9. Bản đồ khóa học chuyên sâu

CHƯƠNG 1. ĐỊNH NGHĨA BÀI TOÁN & THIẾT KẾ GIẢI PHÁP

- 1.1. Có nên dùng Machine Learning không?
- 1.2. Khung chuyển đổi bài toán
- 1.3. Định nghĩa nhãn và cửa sổ thời gian
- 1.4. Hệ thống metric: từ KPI kinh doanh đến hàm loss
- 1.5. Khung giá trị kỳ vọng (Expected Value framework)
- 1.6. Baseline, mức hiệu năng con người và tính khả thi
- 1.7. Đặt bài toán dưới dạng uplift (nâng cao)
- 1.8. Tài liệu thiết kế (ML Design Doc) & quản trị rủi ro

CHƯƠNG 2. THU THẬP DỮ LIỆU (DATA COLLECTION & INGESTION)

- 2.1. Bản đồ nguồn dữ liệu
- 2.2. Định dạng file: lựa chọn có cơ sở
- 2.3. Đọc dữ liệu với pandas: chính xác và tiết kiệm
- 2.4. Khi pandas không đủ: Polars và DuckDB
- 2.5. Đọc từ cơ sở dữ liệu với SQLAlchemy 2.x
- 2.6. Thu thập qua API: timeout, retry, rate limit, phân trang
- 2.7. Web scraping có trách nhiệm
- 2.8. Streaming & Change Data Capture
- 2.9. Data Contract & kiểm tra chất lượng dữ liệu
- 2.10. Versioning dữ liệu với DVC
- 2.11. Dữ liệu nhãn do con người gán
- 2.12. Quyền riêng tư: ẩn danh và giả danh
- 2.13. Pitfalls khi thu thập dữ liệu

CHƯƠNG 3. PHÂN TÍCH KHÁM PHÁ DỮ LIỆU (EDA)

- 3.1. EDA trả lời những câu hỏi nào?
- 3.2. Hồ sơ cấu trúc (Data profiling)
- 3.3. Giá trị thiếu: lượng, mẫu hình và cơ chế
- 3.4. Phân tích đơn biến: biến số
- 3.5. Phân tích đơn biến: biến phân loại
- 3.6. Quan hệ với target
- 3.7. Quan hệ giữa các feature
- 3.8. Ngoại lai (Outliers)
- 3.9. Chiều thời gian
- 3.10. Phát hiện leakage ngay từ EDA
- 3.11. Adversarial validation: train và test có cùng phân phối?
- 3.12. Báo cáo tự động: điểm khởi đầu, không phải điểm kết thúc
- 3.13. Mẫu báo cáo EDA gửi stakeholder

CHƯƠNG 4. PHÂN TÍCH DỮ LIỆU & THỐNG KÊ SUY LUẬN

- 4.1. Bốn cấp độ phân tích
- 4.2. Nền tảng: phân phối mẫu, sai số chuẩn, khoảng tin cậy
- 4.3. Chọn kiểm định và kiểm tra giả định
- 4.4. Kiểm định nhiều giả thuyết
- 4.5. Phương pháp lấy mẫu lại: bootstrap và kiểm định hoán vị

- 4.6. A/B testing chuẩn ngành
- 4.7. Hồi quy để suy luận (statsmodels)
- 4.8. Suy luận nhân quả: nhập môn có thực hành
- 4.9. Phân tích sống còn (Survival analysis): churn theo thời gian
- 4.10. Phân tích chuỗi thời gian cơ bản

CHƯƠNG 5. TIỀN XỬ LÝ DỮ LIỆU & FEATURE ENGINEERING

- 5.1. Nguyên tắc nền tảng: fit trên train, transform trên mọi tập
- 5.2. Làm sạch dữ liệu (stateless)
- 5.3. Xử lý giá trị thiếu (Imputation)
- 5.4. Ngoại lai: clip có học từ train
- 5.5. Mã hóa biến phân loại
- 5.6. Feature Engineering: nơi tạo ra khác biệt lớn nhất
- 5.7. Pipeline tiền xử lý hoàn chỉnh
- 5.8. Lựa chọn đặc trưng (Feature Selection)
- 5.9. Dữ liệu mất cân bằng lớp (Imbalanced data)

CHƯƠNG 6. CHUẨN HÓA, BIẾN ĐỔI PHÂN PHỐI & GIẢM CHIỀU

- 6.1. Vì sao thang đo quan trọng: lý thuyết
- 6.2. Các scaler của scikit-learn
- 6.3. Biến đổi phân phối
- 6.4. Biến đổi target cho hồi quy
- 6.5. Giảm chiều (Dimensionality Reduction)
- 6.6. Ghép nối: scaler khác nhau cho từng nhóm cột
- 6.7. Pitfalls

CHƯƠNG 7. CHIA DỮ LIỆU & CHIẾN LƯỢC VALIDATION

- 7.1. Ba tập dữ liệu và nguyên tắc sử dụng
- 7.2. Bảng chọn chiến lược chia
- 7.3. Hold-out ba tập (stratified)
- 7.4. Out-of-time: chuẩn cho hầu hết bài toán kinh doanh
- 7.5. Cross-validation đúng cách
- 7.6. Group K-Fold: tránh rò rỉ theo thực thể
- 7.7. Chuỗi thời gian: TimeSeriesSplit, gap, purging, embargo
- 7.8. Nested CV: ước lượng không bias khi có tuning
- 7.9. Danh mục data leakage

CHƯƠNG 8. HUẤN LUYỆN MÔ HÌNH (MODEL TRAINING)

- 8.1. Nền tảng lý thuyết
- 8.2. Baseline: luôn bắt đầu từ đây
- 8.3. Các họ mô hình: cơ chế, giả định, tham số chính
- 8.4. So sánh nhiều mô hình trên cùng một CV
- 8.5. Gradient Boosting chuyên sâu
- 8.6. Tối ưu siêu tham số
- 8.7. Ensemble
- 8.8. Chẩn đoán: learning curve và validation curve
- 8.9. Neural network cho dữ liệu bảng
- 8.10. Huấn luyện tái lập được, có tracking (MLflow)
- 8.11. Song song hóa và số luồng CPU: bài học từ chính môi trường viết handbook

CHƯƠNG 9. ĐÁNH GIÁ MÔ HÌNH: METRICS, CONFUSION MATRIX, THRESHOLD & CALIBRATION

- 9.1. Khung lý thuyết: dự đoán và ra quyết định
- 9.2. Confusion matrix: nền tảng của mọi metric phân loại
- 9.3. Metric không phụ thuộc ngưỡng
- 9.4. Chọn ngưỡng quyết định
- 9.5. Metric xếp hạng kinh doanh: Lift, Gain, Precision@K
- 9.6. Calibration: xác suất có đáng tin không?
- 9.7. Metric hồi quy
- 9.8. Metric phân cụm
- 9.9. Metric truy hồi & xếp hạng (search, RAG, recommender)
- 9.10. So sánh mô hình có ý nghĩa thống kê

- 9.11. Phân tích theo phân khúc (slice analysis) và phân tích lỗi
- 9.12. Báo cáo đánh giá cuối cùng trên tập test

CHƯƠNG 10. GIẢI THÍCH MÔ HÌNH (XAI) & FAIRNESS

- 10.1. Vì sao cần giải thích và giải thích cho ai?
- 10.2. Feature importance: MDI và permutation
- 10.3. Partial Dependence (PDP), ICE và hiệu ứng tương tác
- 10.4. SHAP: Shapley values cho từng dự đoán
- 10.5. Mô hình tự giải thích được (glass-box) và mô hình thay thế
- 10.6. Fairness: công bằng giữa các nhóm
- 10.7. Model Card: tài liệu hóa mô hình

CHƯƠNG 11. TRỰC QUAN HÓA DỮ LIỆU & KỂ CHUYỆN BẰNG DỮ LIỆU

- 11.1. Nền tảng nhận thức thị giác
- 11.2. Chọn biểu đồ theo câu hỏi
- 11.3. Kiến trúc matplotlib và phong cách thống nhất
- 11.4. Màu sắc có chủ đích
- 11.5. Biểu đồ EDA chuẩn mực
- 11.6. Dashboard đánh giá mô hình (một hình, sáu panel)
- 11.7. Biểu đồ tương tác với Plotly
- 11.8. Dashboard ứng dụng với Streamlit
- 11.9. Kể chuyện bằng dữ liệu (Data Storytelling)

CHƯƠNG 12. MLOps — ĐƯA MÔ HÌNH VÀO PRODUCTION VÀ GIỮ NÓ ĐÚNG

- 12.1. Vì sao MLOps: nợ kỹ thuật đặc thù của ML
- 12.2. Kiến trúc tham chiếu
- 12.3. Kiểm thử hệ thống ML
- 12.4. Experiment tracking & Model Registry (MLflow 3)
- 12.5. Serving: batch, online, streaming
- 12.6. Đóng gói: Docker và Kubernetes
- 12.7. CI/CD/CT với GitHub Actions
- 12.8. Chiến lược triển khai an toàn
- 12.9. Giám sát (Monitoring)
- 12.10. Retraining và orchestration
- 12.11. Feature store: chống training-serving skew
- 12.12. Tối ưu suy luận
- 12.13. Bảo mật, quyền riêng tư, quản trị và chi phí

CHƯƠNG 13. CHECKLIST PRODUCTION, ANTI-PATTERNS & QUY TRÌNH REVIEW

- 13.1. Checklist trước khi Go-Live
- 13.2. Tự chấm ML Test Score
- 13.3. Danh mục anti-pattern
- 13.4. Bộ câu hỏi review mô hình (dành cho reviewer)
- 13.5. Mẫu postmortem (blameless) cho sự cố mô hình
- 13.6. Lộ trình áp dụng handbook trong một dự án 12 tuần

PHỤ LỤC

- Phụ lục A. Cheatsheet thư viện theo giai đoạn
- Phụ lục B. Thay đổi API cần biết (đã kiểm chứng khi viết handbook)
- Phụ lục C. Bảng chọn metric nhanh
- Phụ lục D. Bảng chọn thuật toán
- Phụ lục E. Thuật ngữ Anh – Việt
- Phụ lục F. Bản đồ khóa học chuyên sâu theo chương
- Phụ lục G. Cách dùng package tham chiếu và kiểm chứng handbook
- Phụ lục H. Tài liệu tham khảo tổng hợp

DATA SCIENCE END-TO-END HANDBOOK

Cẩm nang Khoa học Dữ liệu chuẩn Production: từ cơ bản đến nâng cao

Phiên bản 2.0, tháng 10/2026. Bản này được viết lại để bám sát **tài liệu chính thức** của các thư viện (scikit-learn User Guide, pandas User Guide, LightGBM/XGBoost/CatBoost docs, Optuna, MLflow, imbalanced-learn, statsmodels, SHAP, Evidently, FastAPI) và giáo trình của các khóa học chuyên sâu (Stanford CS229, Google Machine Learning Crash Course & Rules of ML, DeepLearning.AI MLOps Specialization, Made With ML, Full Stack Deep Learning, Harvard CS109, Kohavi về thử nghiệm có kiểm soát).

Code đã được chạy kiểm chứng. Mọi khối code Python không gắn nhãn `norun` đều được script `build/run_snippets.py` thực thi tự động. Package tham chiếu `code/src/churn` có bộ test riêng (`pytest`).

0.1. Cách dùng handbook

Cấu trúc một chương

Mỗi chương đi theo cùng một khung:

- 1. **Mục tiêu & câu hỏi cần trả lời.** Chương này giải quyết vấn đề gì trong dự án thực.
- 2. **Lý thuyết cốt lõi.** Định nghĩa, công thức, giả định. Đây là phần mà khóa học hàn lâm (CS229, CS109) dạy.
- 3. **Kỹ thuật & API chính thức.** Tham số quan trọng, hành vi mặc định, những thay đổi giữa các phiên bản, kèm trích dẫn tài liệu gốc.
- 4. **Code mẫu chạy được.** Viết theo phong cách production: hàm có type hint, pipeline không rò rỉ dữ liệu, seed cố định.
- 5. **Bẫy thường gặp (Pitfalls).** Lấy từ mục *Common pitfalls* của tài liệu chính thức và kinh nghiệm dự án.
- 6. **Checklist** và **Tài liệu tham khảo** của chương.

Quy ước trình bày

Ký hiệu	Ý nghĩa
[Docs]	Nội dung tóm lược/diễn giải từ tài liệu chính thức của thư viện. Đường dẫn ở cuối chương
[Course]	Nội dung tương ứng trong khóa học chuyên sâu
[Kinh nghiệm]	Bài học thực tế, không phải quy định của thư viện
<code>```python</code>	Khối code được chạy kiểm chứng
<code>```python norun</code>	Khối code minh họa cần dịch vụ ngoài (DB, Kafka, API, server) nên không chạy tự động

Phiên bản thư viện đã kiểm chứng

Thư viện	Phiên bản	Thư viện	Phiên bản
Python	3.13	scikit-learn	1.9.1
pandas	3.0.6	numpy	2.x
LightGBM	4.7.0	XGBoost	3.4.1
CatBoost	1.2.10	Optuna	5.0.0
MLflow	3.17.0	SHAP	0.52.0
imbalanced-learn	0.14.2	statsmodels	0.15.0

Thư viện	Phiên bản	Thư viện	Phiên bản
pandera	0.34.1	FastAPI	0.142
fairlearn	0.14.0	Evidently	0.7.x

[Kinh nghiệm] Thư viện ML thay đổi API nhanh. Ví dụ, trong các phiên bản trên: scikit-learn 1.9 deprecate `TargetEncoder(random_state=...)` (thay bằng truyền splitter vào `cv`). MLflow 3 đổi `artifact_path` thành `name` trong `log_model`, đưa file store `./mlruns` vào chế độ bảo trì và mặc định dùng `skops` để serialize scikit-learn. pandas 3 bật *Copy-on-Write* và kiểu `str` mặc định. Luôn **khóa phiên bản** (lock file) và đọc *release notes* trước khi nâng cấp.

0.2. Kế hoạch nội dung

Phần	Chương	Nội dung chính	Nền tảng chính thống
I. Nền tảng	0	Phương pháp luận, vai trò, cấu trúc repo, môi trường, tái lập	CRISP-DM, Microsoft TDSP, scikit-learn <i>Controlling randomness</i>
	1	Định nghĩa bài toán, metric kinh doanh, thiết kế giải pháp	Google <i>ML Problem Framing, Rules of ML</i> (Zinkevich)
II. Dữ liệu	2	Thu thập: file, SQL, API, scraping, streaming; data contract; versioning; nhãn	pandas <i>Scaling to large datasets</i> , Apache Arrow/Parquet, pandera, DVC
	3	EDA: hồ sơ dữ liệu, missing, phân phối, quan hệ, leakage, adversarial validation	Tukey (1977), Harvard CS109, scikit-learn
	4	Thống kê suy luận, A/B test, hồi quy, suy luận nhân quả, survival	statsmodels, SciPy, Kohavi et al. (2020)
	5	Tiền xử lý & feature engineering, imbalanced data	scikit-learn <i>Preprocessing, Imputation, Compose</i> ; imbalanced-learn
	6	Chuẩn hóa, biến đổi phân phối, giảm chiều	scikit-learn <i>Preprocessing, Decomposition</i>
	7	Chia dữ liệu & validation, nested CV, leakage	scikit-learn <i>Cross-validation, Common pitfalls</i>
	8	Huấn luyện: lý thuyết, GBM, tuning, ensemble, deep learning, tracking	CS229; LightGBM/XGBoost/CatBoost docs; Optuna; scikit-learn <i>Ensembles</i>
III. Mô hình	9	Đánh giá: scoring rules, confusion matrix, threshold, calibration, so sánh thống kê	scikit-learn <i>Model evaluation, Calibration, Threshold tuning</i>
	10	Giải thích mô hình & fairness	scikit-learn <i>Inspection</i> ; SHAP; fairlearn; Molnar
	11	Trực quan hóa & kể chuyện bằng dữ liệu	matplotlib, seaborn, plotly; Cleveland & McGill; Knaflitz
	12	MLOps: tracking, registry, serving, Docker, CI/CD, monitoring, retraining	Google <i>MLOps levels</i> ; MLflow; FastAPI; Evidently; Made With ML
IV. Production	13	Checklist production, ML Test Score, anti-patterns	Breck et al. (2017); Sculley et al. (2015)
Phụ lục	A-F	Cheatsheet, chọn metric/thuật toán, thuật ngữ, bản đồ khóa học, tài liệu	

Lộ trình đọc đề xuất

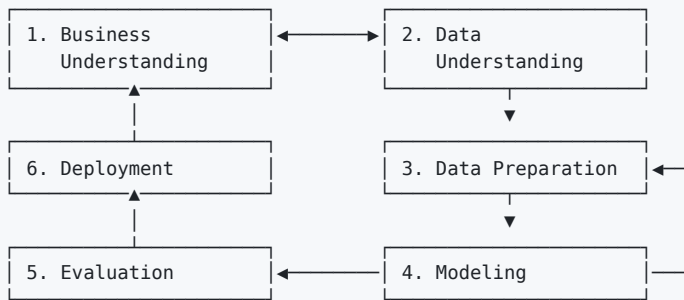
Trình độ	Thứ tự đọc	Trọng tâm
Mới bắt đầu (0-1 năm)	0 → 1 → 2 → 3 → 5 → 6 → 7 → 8.1-8.6 → 9.1-9.5 → 11	Pipeline không leakage, chọn metric đúng

Trình độ	Thứ tự đọc	Trọng tâm
Trung cấp (1–3 năm)	Toàn bộ, đặc biệt 4, 7, 8, 9, 10	Validation đúng kịch bản, tuning, calibration, threshold
Senior / ML Engineer	1, 7, 9, 12, 13 + Phụ lục	Thiết kế hệ thống, kiểm thử ML, giám sát, quản trị

0.3. Phương pháp luận dự án

CRISP-DM (Cross-Industry Standard Process for Data Mining, 1999–2000)

CRISP-DM vẫn là quy trình được dùng nhiều nhất. Nó chia dự án thành 6 pha **có vòng lặp**:

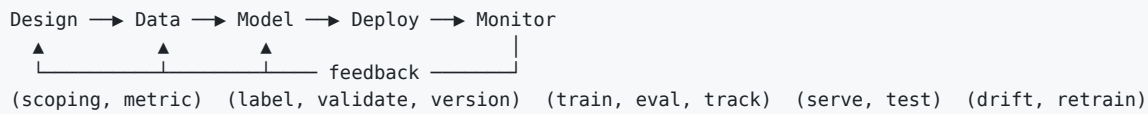


Pha	Nhiệm vụ chính (theo CRISP-DM 1.0 User Guide)	Sản phẩm bàn giao	Chương
1. Business Understanding	Xác định mục tiêu kinh doanh, đánh giá tình hình (tài nguyên, rủi ro, chi phí-lợi ích), mục tiêu data mining, kế hoạch dự án	Business objectives, success criteria, project plan	1
2. Data Understanding	Thu thập dữ liệu ban đầu, mô tả, khám phá, kiểm tra chất lượng	Data description report, data quality report	2, 3
3. Data Preparation	Chọn, làm sạch, xây dựng, tích hợp, định dạng dữ liệu	Dataset + dataset description	4, 5, 6
4. Modeling	Chọn kỹ thuật, thiết kế kiểm thử, xây mô hình, đánh giá kỹ thuật	Test design, model, model assessment	7, 8
5. Evaluation	Đánh giá theo tiêu chí kinh doanh, rà soát quy trình, quyết định bước tiếp	Approved models, list of next actions	9, 10
6. Deployment	Kế hoạch triển khai, giám sát & bảo trì, báo cáo cuối, retrospective	Deployment & monitoring plan, final report	11, 12

Hạn chế của CRISP-DM và cách các khung hiện đại bổ sung:

- CRISP-DM ra đời trước thời kỳ MLOps, nên pha *Deployment* rất mỏng: không nói tới giám sát drift, retraining, CI/CD. Chương 12 bổ sung theo mô hình **MLOps maturity** của Google.
- Không mô tả vai trò và cộng tác nhóm. **Microsoft TDSP** (Team Data Science Process) bổ sung vai trò (Group Manager, Team Lead, Project Lead, Individual Contributor), cấu trúc repo chuẩn và các template tài liệu (Project Charter, Data Report, Model Report, Exit Report).
- Không nhấn mạnh thực nghiệm online. Các công ty công nghệ bổ sung vòng **A/B test** sau triển khai (Chương 4.6).

Vòng đời ML trong production (tổng hợp từ Google, Made With ML, Full Stack Deep Learning)



[Course] *Made With ML* (Goku Mohandas) chia khóa MLOps thành: Design → Data → Model → Develop → Utilities → Test → Reproducibility → Production → Data engineering. Khóa *Machine Learning Engineering for Production* (DeepLearning.AI, Andrew Ng) chia theo vòng đời **Scoping → Data → Modeling → Deployment**. Handbook này đi theo đúng mạch đó.

Tỷ lệ công sức điển hình

Giai đoạn	% thời gian	Ghi chú
Hiểu bài toán, thống nhất metric với stakeholder	10-15%	Sai ở đây thì mọi bước sau đều vô ích
Thu thập, làm sạch, gán nhãn	35-50%	"Data-centric AI" (Andrew Ng): cải thiện dữ liệu thường lợi hơn cải thiện mô hình
EDA & feature engineering	15-20%	
Huấn luyện & tuning	10-15%	Tuning thường chỉ thêm 1-3%
Đánh giá, triển khai, giám sát	15-25%	Bị đánh giá thấp nhất, gây nhiều sự cố nhất

0.4. Vai trò trong một đội dữ liệu

Vai trò	Trách nhiệm chính	Kỹ năng/Công cụ cốt lõi
Data Analyst	Báo cáo, dashboard, phân tích mô tả & chẩn đoán	SQL, Excel/BI, thống kê mô tả, trực quan hóa
Analytics Engineer	Mô hình hóa dữ liệu trong warehouse, chất lượng dữ liệu	SQL, dbt, kiểm thử dữ liệu
Data Engineer	Pipeline thu thập, lưu trữ, xử lý dữ liệu lớn	Spark, Kafka, Airflow, cloud
Data Scientist	Định nghĩa bài toán, EDA, thống kê, mô hình, thực nghiệm	Python, thống kê, ML, giao tiếp kinh doanh
ML Engineer	Đưa mô hình vào production, tối ưu suy luận	Software engineering, serving, Docker, K8s
MLOps / Platform Engineer	Hạ tầng huấn luyện, registry, CI/CD, giám sát	MLflow, Kubeflow, Terraform, observability

[Kinh nghiệm] Data Scientist "chuẩn production" không cần làm mọi việc của ML Engineer. Nhưng họ phải viết được code **có test, đóng gói được, tái lập được**. Nếu không, chi phí bàn giao sẽ ăn hết giá trị của mô hình.

0.5. Cấu trúc repository chuẩn

Dựa trên **Cookiecutter Data Science v2** (DrivenData) và cấu trúc package tham chiếu của handbook (`/`):

```

churn-prediction/
├── .github/workflows/ci.yml      # lint + test + train + quality gate
├── configs/                     # tham số tách khỏi code (YAML)
│   └── train.yaml
├── data/                        # KHÔNG commit dữ liệu vào git -> DVC / object storage
│   ├── raw/                    # bất biến (immutable), chỉ đọc
│   ├── interim/                # trung gian
│   ├── processed/              # sẵn sàng huấn luyện
│   └── external/               # dữ liệu bên thứ ba
├── models/                     # artifact cục bộ (production: MLflow Model Registry)
├── notebooks/                  # đặt tên 1.0-tt-eda-churn.ipynb (thứ tự-tác giả-mô tả)
├── reports/figures/
├── src/churn/                   # package Python: mọi logic dùng lại phải ở đây
│   ├── config.py               # pydantic: cấu hình có kiểu, đọc secret từ env
│   ├── data/                   # synthetic.py, validate.py, split.py
│   ├── features/               # clean.py, transformers.py, build.py
│   ├── models/                 # train.py, evaluate.py, registry.py
│   ├── monitoring/drift.py
│   ├── serving/api.py
│   └── viz/style.py
├── tests/                      # unit / data / model / api
├── Dockerfile
├── Makefile
├── pyproject.toml
├── .pre-commit-config.yaml
└── .env.example                # mẫu biến môi trường (không chứa secret thật)

```

Nguyên tắc của Cookiecutter Data Science:

1. **Dữ liệu thô là bất biến.** Không bao giờ sửa tay file trong `data/raw`. Mọi biến đổi phải là code chạy lại được.
2. **Notebook để khám phá và trao đổi, không để sản xuất.** Logic dùng lại phải chuyển vào `src/` và có test.
3. **Phân tích là một DAG.** Mỗi bước có input/output rõ ràng (Makefile, DVC pipeline, Airflow).
4. **Tách cấu hình và secret khỏi code.** Theo nguyên tắc *Twelve-Factor App* (factor III — Config).

0.6. Môi trường phát triển

Quản lý môi trường & phụ thuộc

Công cụ	Ưu điểm	Khi nào dùng
uv (Astral)	Rất nhanh, quản lý cả Python version, lock file đa nền tảng	Mặc định cho dự án mới
pip + venv + pip-tools	Chuẩn, đơn giản	Môi trường hạn chế công cụ
Poetry / PDM	Quản lý package + build	Thư viện phát hành lên PyPI
conda / mamba	Phụ thuộc C/CUDA/GDAL phức tạp	Deep learning GPU, GIS
Docker	Đồng bộ toàn bộ hệ điều hành	Production, CI

```

# uv: tạo môi trường, cài package ở chế độ editable, khóa phiên bản
uv venv --python 3.12 && source .venv/bin/activate
uv pip install -e ".[dev,extra]"
uv pip compile pyproject.toml -o requirements.lock    # lock file để tái lập
uv pip sync requirements.lock                        # môi trường CI/production

```

pyproject.toml (trích từ package tham chiếu)

```

[project]
name = "churn"
version = "2.0.0"
requires-python = ">=3.11"
dependencies = [

```

```

"numpy>=2.0", "pandas>=2.2", "pyarrow>=15", "scipy>=1.13",
"scikit-learn>=1.6", "lightgbm>=4.3",
"pandera>=0.24", "pydantic>=2.7", "pydantic-settings>=2.3", "pyyaml>=6.0",
"mlflow>=3.0", "fastapi>=0.110", "uvicorn[standard]>=0.30",
]

[project.optional-dependencies]
dev = ["pytest>=8", "pytest-cov", "httpx>=0.27", "ruff>=0.6", "mypy>=1.10", "pre-commit"]
extra = ["xgboost>=2.1", "catboost>=1.2", "optuna>=4.0", "shap>=0.46", "imbalanced-learn>=0.12",
        "statsmodels>=0.14", "fairlearn>=0.11", "evidently>=0.7", "plotly>=5.22"]

[tool.ruff]
line-length = 110
target-version = "py311"

[tool.ruff.lint]
select = ["E", "F", "I", "B", "UP", "SIM", "NPY", "PD"] # PD: pandas-vet, NPY: numpy rules

[tool.pytest.ini_options]
testpaths = ["tests"]
pythonpath = ["src"]

```

Chất lượng code tự động: pre-commit

```

# .pre-commit-config.yaml
repos:
- repo: https://github.com/astral-sh/ruff-pre-commit
  rev: v0.6.9
  hooks:
    - id: ruff
      args: [--fix]
    - id: ruff-format
- repo: https://github.com/kynan/nbstripout
  rev: 0.7.1
  hooks:
    - id: nbstripout
- repo: https://github.com/pre-commit/pre-commit-hooks
  rev: v4.6.0
  hooks:
    - id: check-added-large-files
      args: [--maxkb=5000]
    - id: detect-private-key
    - id: check-yaml

```

[Kinh nghiệm] Notebook khó review vì diff JSON. Dùng **JupyterText** để ghép mỗi notebook với một file `.py` (định dạng *percent*) và review file `.py` đó. Dùng `nbstripout` để không commit output (output có thể chứa dữ liệu khách hàng).

0.7. Tái lập kết quả (Reproducibility)

Một kết quả chỉ đáng tin khi tái lập được từ bốn thành phần: **code** (git commit), **dữ liệu** (phiên bản/snapshot), **môi trường** (lock file/Docker image) và **cấu hình + seed**.

random_state trong scikit-learn: điều tài liệu chính thức nhấn mạnh

[Docs] Mục *Controlling randomness* trong *Common pitfalls and recommended practices* của scikit-learn:

- Truyền **số nguyên** cho `random_state`: mỗi lần gọi `fit` / `split` cho **cùng kết quả**, vì RNG được khởi tạo lại ở đầu mỗi lần gọi.
- Truyền **None hoặc một instance** `RandomState`: mỗi lần gọi cho **kết quả khác nhau**, vì RNG bị "tiêu thụ" và thay đổi trạng thái. Các đối tượng dùng chung một instance sẽ ảnh hưởng lẫn nhau.

- **Khuyến nghị:** với CV splitter, truyền số nguyên để mọi mô hình được so sánh trên **cùng các fold**. Với estimator, khi muốn đánh giá độ bền vững của CV có thể để `None` /instance. Còn khi cần tái lập tuyệt đối giữa các lần chạy thì bỏ mọi `random_state=None`.

```
import numpy as np
from sklearn.model_selection import KFold

X = np.arange(10).reshape(-1, 1)

# Số nguyên: split() gọi nhiều lần cho cùng fold -> so sánh mô hình công bằng
cv_int = KFold(n_splits=2, shuffle=True, random_state=0)
assert [list(v) for _, v in cv_int.split(X)] == [list(v) for _, v in cv_int.split(X)]

# Instance RandomState: mỗi lần split() cho fold KHÁC -> hai mô hình bị đánh giá trên dữ liệu khác nhau
cv_rng = KFold(n_splits=2, shuffle=True, random_state=np.random.RandomState(0))
first = [list(v) for _, v in cv_rng.split(X)]
second = [list(v) for _, v in cv_rng.split(X)]
print("Fold giống nhau giữa 2 lần split()?", first == second)
```

Seed cho toàn bộ stack

```
import os
import random

import numpy as np

def seed_everything(seed: int = 42) -> np.random.Generator:
    """Đặt seed cho các nguồn ngẫu nhiên phổ biến; trả về numpy Generator để dùng tương minh."""
    random.seed(seed)
    np.random.seed(seed) # cho thư viện cũ còn dùng global RNG
    os.environ["PYTHONHASHSEED"] = str(seed) # chỉ có tác dụng nếu đặt TRƯỚC khi khởi động Python
    try:
        import torch

        torch.manual_seed(seed)
        torch.cuda.manual_seed_all(seed)
        torch.use_deterministic_algorithms(True, warn_only=True)
        torch.backends.cudnn.benchmark = False
    except ImportError:
        pass
    return np.random.default_rng(seed) # API khuyến nghị của NumPy (NEP 19)

rng = seed_everything(42)
print(rng.integers(0, 100, 3))
```

Thư viện	Cách đảm bảo tái lập
NumPy	Dùng <code>np.random.default_rng(seed)</code> (Generator, NEP 19) thay cho global <code>np.random.*</code>
scikit-learn	<code>random_state=int</code> cho splitter và estimator
LightGBM	<code>seed / random_state</code> ; <code>deterministic=True</code> + <code>force_row_wise=True</code> nếu cần kết quả giống hệt trên CPU; cố định <code>num_threads</code>
XGBoost	<code>random_state</code> ; kết quả GPU có thể khác CPU
PyTorch	<code>torch.manual_seed</code> , <code>torch.use_deterministic_algorithms(True)</code> , cố định <code>num_workers</code> + <code>worker_init_fn</code> của <code>DataLoader</code>
Optuna	<code>TPESampler(seed=...)</code> ; tối ưu song song thì không tái lập hoàn toàn

0.8. Bộ dữ liệu xuyên suốt handbook

Toàn bộ code dùng bộ dữ liệu **mô phỏng bài toán dự đoán khách hàng rời bỏ (churn)** của một nhà mạng viễn thông. Dữ liệu sinh từ package tham chiếu `churn.data.synthetic`. Vì biết trước **cơ chế sinh dữ liệu thật (ground truth)**, ta kiểm tra được mỗi kỹ thuật có tìm ra sự thật không. Ví dụ: feature nào thực sự quan trọng, leakage làm metric "đẹp ảo" bao nhiêu.

Cột	Kiểu	Ý nghĩa	Vấn đề được cài sẵn
customer_id	str	Mã khách hàng C0000001	100 bản ghi trùng lặp
signup_date	datetime	Ngày đăng ký (2022-01 → 2024-06)	Dùng cho time split
age	float	Tuổi	~5% thiếu
tenure_months	int	Số tháng gắn bó	Tắc động âm lên churn
monthly_charges	float	Cước tháng	30 ngoại lai ×8 (lỗi nhập liệu)
total_charges	float	Tổng cước ≈ monthly × tenure	Đa cộng tuyến
contract	str	month-to-month / one-year / two-year	50 bản ghi sai chuẩn hóa Month-to-Month
payment_method	str	e-wallet / bank-transfer / credit-card / cash	~3% thiếu
region	str	30 vùng R00 ... R29	Cardinality cao, không có tác động thật
support_calls	int	Số cuộc gọi hỗ trợ (Poisson)	Tắc động dương mạnh
data_usage_gb	float	Dung lượng dữ liệu (log-normal)	Lệch phải; ~8% thiếu; không có tác động thật
churn	int	Nhãn: 1 = rời bỏ	~16.5% positive (mất cân bằng)

Cơ chế sinh nhãn (log-odds):

$$\text{logit } P(\text{churn}) = -2.2 + 1.3 \cdot \mathbb{I}[\text{month-to-month}] - 0.03 \cdot \text{tenure} + 0.012 \cdot (\text{charges} - 70) + 0.35 \cdot \text{calls} + 0.4 \cdot \mathbb{I}[\text{cash}] - 0.01 \cdot (\text{age} - 38)$$

```
from churn.data.synthetic import make_churn_data

df = make_churn_data(n=20_000, seed=42)
print(df.shape)                                     # (20100, 12): 20 000 khách + 100 dòng trùng
print(df["churn"].mean().round(3))                  # tỷ lệ churn
print(df.isna().mean().round(3)[lambda s: s > 0]) # các cột có missing
print(df.head(3).T)
```

0.9. Bản đồ khóa học chuyên sâu

Khóa học / Giáo trình	Đơn vị	Phù hợp chương	Ghi chú
CS229 <i>Machine Learning</i>	Stanford (Andrew Ng, Tengyu Ma...)	6, 8, 9	Nền tảng toán: GLM, SVM, bias-variance, learning theory
CS109 <i>Data Science</i>	Harvard	2, 3, 4, 11	Thu thập, EDA, thống kê, trực quan hóa
<i>Machine Learning Specialization</i>	DeepLearning.AI / Stanford Online	5-9	Cập nhật của khóa ML kinh điển
<i>MLOps Specialization</i> (MLEP)	DeepLearning.AI	1, 2, 12	Scoping, data-centric, deployment, monitoring
<i>Machine Learning Crash Course, Problem Framing, Data Prep, Rules of ML</i>	Google for Developers	1, 5, 9, 12	Thực hành, ngắn gọn, nhiều bài học production
<i>Made With ML</i>	Goku Mohandas (Anyscale)	0, 7, 12, 13	MLOps end-to-end, testing, CI/CD

Khóa học / Giáo trình	Đơn vị	Phù hợp chương	Ghi chú
<i>Full Stack Deep Learning</i>	FSDL	1, 12	Dự án ML thực tế, tổ chức đội, hạ tầng
<i>Practical Deep Learning for Coders</i>	fast.ai	8	Deep learning thực hành, tabular & NLP
Kaggle Learn (Pandas, Feature Engineering, Intermediate ML, Data Leakage)	Kaggle	3, 5, 7	Bài tập ngắn có chấm điểm
<i>Trustworthy Online Controlled Experiments</i>	Kohavi, Tang, Xu (2020)	4	Chuẩn mực A/B test trong ngành
<i>Interpretable Machine Learning</i>	Christoph Molnar	10	Sách mở về XAI
<i>Designing Machine Learning Systems</i>	Chip Huyen (Stanford CS329S)	1, 12	Thiết kế hệ thống ML

Checklist Chương 0

- ☐ Repo theo cấu trúc chuẩn; logic dùng lại nằm trong `src/` và có test.
- ☐ Phụ thuộc được khóa phiên bản; có Dockerfile cho production.
- ☐ Secret nằm trong biến môi trường hoặc `.env` (đã có trong `.gitignore`).
- ☐ `pre-commit` chạy ruff, strip notebook output, chặn file lớn và private key.
- ☐ Seed và `random_state=int` được đặt trong config; CV splitter dùng số nguyên.
- ☐ Biết dự án đang ở pha nào của CRISP-DM và sản phẩm bàn giao của pha đó.

Tài liệu tham khảo Chương 0

- Chapman, P. et al. (2000). *CRISP-DM 1.0: Step-by-step data mining guide*. SPSS.
- Microsoft. *Team Data Science Process (TDSP)*. learn.microsoft.com/azure/architecture/data-science-process
- DrivenData. *Cookiecutter Data Science v2*. cookiecutter-data-science.drivendata.org
- scikit-learn User Guide. *Common pitfalls and recommended practices → Controlling randomness*. scikit-learn.org/stable/common_pitfalls.html
- NumPy. *NEP 19 — Random number generator policy*. numpy.org/neps/nep-0019-rng-policy.html
- Wiggins, A. *The Twelve-Factor App*. 12factor.net
- Mohandas, G. *Made With ML — MLOps course*. madewithml.com

CHƯƠNG 1. ĐỊNH NGHĨA BÀI TOÁN & THIẾT KẾ GIẢI PHÁP

"Rule #1: Don't be afraid to launch a product without machine learning." — Martin Zinkevich, *Rules of Machine Learning* (Google).

Mục tiêu chương: biến một mong muốn kinh doanh mơ hồ ("giảm khách rời bỏ") thành một bài toán ML **đo được, khả thi, không rò rỉ dữ liệu**, có metric gắn với tiền và một baseline để so sánh.

1.1. Có nên dùng Machine Learning không?

[Course] Khóa *Introduction to Machine Learning Problem Framing* của Google đề xuất trả lời theo thứ tự:

- 1. **Mục tiêu (goal) là gì?** Phát biểu bằng ngôn ngữ kinh doanh, không nhắc tới mô hình.
- 2. **Kết quả lý tưởng (ideal outcome)?** Sản phẩm/quy trình sẽ thay đổi thế nào nếu có dự đoán tốt.
- 3. **Có giải pháp không dùng ML không?** Heuristic, quy tắc, truy vấn SQL. Theo *Rules of ML* (#1, #3), heuristic là baseline bắt buộc. Chỉ dùng ML khi heuristic trở nên quá phức tạp để bảo trì.
- 4. **Dữ liệu có đủ không?** Có nhãn không, có đủ ví dụ positive không, feature có sẵn **tại thời điểm dự đoán** không.
- 5. **Dự đoán có hành động được không (actionable)?** Ai dùng, lúc nào, làm gì khác đi.

Dấu hiệu nên dùng ML	Dấu hiệu chưa nên dùng ML
Quy luật phức tạp, nhiều tương tác, khó viết thành quy tắc	Quy tắc đơn giản đã đủ tốt
Dữ liệu lớn, có nhãn, lặp lại thường xuyên	Ít dữ liệu, nhãn không đáng tin
Chấp nhận được sai số, có cơ chế sửa sai	Sai một lần là thảm họa, không giải thích được
Môi trường thay đổi, cần cập nhật liên tục	Quyết định hiếm, một lần

1.2. Khung chuyển đổi bài toán

Bước	Câu hỏi	Ví dụ Churn
1. Mục tiêu kinh doanh	KPI nào cần cải thiện?	Giảm tỷ lệ rời bỏ tháng từ 2.1% xuống 1.8% trong 2 quý
2. Hành động	Dự đoán xong ai làm gì?	CSKH gọi + tặng voucher cho top-K khách nguy cơ cao
3. Đơn vị dự đoán	Dự đoán cho ai, khi nào, bao lâu một lần?	Mỗi thuê bao đang hoạt động, ngày 1 hằng tháng (batch)
4. Nhãn	Định nghĩa chính xác positive?	Hủy dịch vụ trong 30 ngày, bắt đầu sau 7 ngày kể từ ngày chấm điểm
5. Dạng bài toán	Phân loại / hồi quy / xếp hạng / uplift?	Phân loại nhị phân để xếp hạng top-K, tiến tới uplift
6. Ràng buộc	Độ trễ, năng lực vận hành, giải thích, pháp lý	Batch; CSKH gọi tối đa 5 000 khách/tháng; phải có lý do cho từng khách
7. Metric offline	Metric nào phản ánh mục tiêu?	PR-AUC, Precision@5000, lợi nhuận kỳ vọng
8. Metric online	Đo hiệu quả thật thế nào?	Tỷ lệ churn nhóm được gọi so với nhóm đối chứng (A/B test)
9. Baseline	Hiện tại làm thế nào?	Quy tắc: hợp đồng tháng và ≥ 3 cuộc gọi khiếu nại

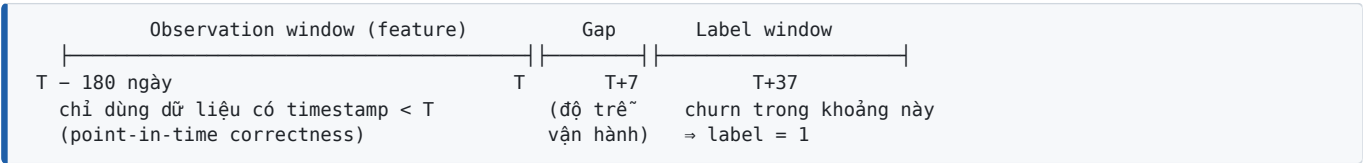
Phân loại bài toán và điểm khởi đầu

Dạng bài toán	Đầu ra	Ví dụ	Khởi đầu (baseline → mạnh)	Loss/metric chuẩn
Phân loại nhị phân	Xác suất $\in [0, 1]$	Churn, gian lận	Logistic Regression → GBM	Log loss; PR-AUC
Phân loại đa lớp	Phân phối trên K lớp	Phân loại ticket	Multinomial LR → GBM/Transformer	Log loss; macro-F1
Đa nhãn	Tập con nhãn	Gắn tag văn bản	One-vs-Rest; BCE	Hamming loss; micro-F1
Hồi quy	Số thực (mean)	Doanh thu khách hàng	Ridge → GBM	MSE (mean), MAE (median)
Hồi quy phân vị	Phân vị	Dự trữ tồn kho P90	GBM objective="quantile"	Pinball loss
Dữ liệu đếm	Số nguyên ≥ 0	Số cuộc gọi	Poisson GLM → GBM Poisson	Poisson deviance
Xếp hạng	Thứ tự	Tìm kiếm, gợi ý	BM25 → LambdaMART	NDCG@K, MAP
Chuỗi thời gian	Giá trị tương lai	Dự báo nhu cầu	Seasonal naive → ETS/GBM	MASE, WAPE
Thời gian đến sự kiện	Hàm sống còn	Khi nào khách rời bỏ	Kaplan-Meier → Cox	C-index
Uplift / nhân quả	Hiệu ứng can thiệp	Ai nên nhận voucher	T-/X-learner, Causal Forest	Qini, AUUC
Phân cụm	Nhóm	Phân khúc khách hàng	K-Means → GMM/HDBSCAN	Silhouette + đánh giá nghiệp vụ
Phát hiện bất thường	Điểm bất thường	Giám sát giao dịch	Isolation Forest	Precision@K trên nhãn kiểm tra

[Kinh nghiệm] Churn thường được đặt thành "phân loại nhị phân", nhưng câu hỏi kinh doanh thật là: *nếu gọi điện, khách nào sẽ đổi ý?* Đó là bài toán **uplift** (mục 1.7). Khách chắc chắn đi dù có gọi ("lost causes") và khách chắc chắn ở lại ("sure things") đều không đáng tốn voucher.

1.3. Định nghĩa nhãn và cửa sổ thời gian

Đây là nơi **leakage** sinh ra nhiều nhất. Phải thiết kế ngay từ đầu.



- Feature** chỉ tính từ dữ liệu có **timestamp < T** và phải **đã có sẵn** trong hệ thống vào lúc T. Dữ liệu đến trễ (late-arriving data) phải được tính theo thời điểm nó *thực sự* có mặt.
- Gap** mô phỏng độ trễ vận hành: ETL chạy xong, CSKH cần thời gian liên hệ.
- Label window** dài bao nhiêu là quyết định nghiệp vụ. Ngắn quá thì ít positive, dài quá thì hành động không kịp.
- Khách **đã rời bỏ trước T** phải bị loại khỏi tập chấm điểm, nếu không mô hình học "dự đoán quá khứ".

```
import numpy as np
import pandas as pd

def make_event_log(n_customers: int = 2_000, seed: int = 0) -> pd.DataFrame:
    """Nhật ký sự kiện mô phỏng: 'activate' khi đăng ký, 'cancel' khi rời bỏ (nếu có)."""
    rng = np.random.default_rng(seed)
    start = pd.Timestamp("2024-01-01") + pd.to_timedelta(rng.integers(0, 300, n_customers), unit="D")
    life = pd.to_timedelta(rng.exponential(240, n_customers).astype(int), unit="D")
    ids = [f"C{i:07d}" for i in range(n_customers)]
```

```

activate = pd.DataFrame({"customer_id": ids, "event_date": start, "event_type": "activate"})
cancel = pd.DataFrame({"customer_id": ids, "event_date": start + life, "event_type": "cancel"})
cancel = cancel[cancel["event_date"] < pd.Timestamp("2025-06-30")]
return pd.concat([activate, cancel], ignore_index=True)

def build_labels(events: pd.DataFrame, snapshot: str, horizon_days: int = 30,
                gap_days: int = 7) -> pd.DataFrame:
    """Nhãn point-in-time: khách đang hoạt động tại T, label = hủy trong [T+gap, T+gap+horizon)."""
    t = pd.Timestamp(snapshot)
    start, end = t + pd.Timedelta(days=gap_days), t + pd.Timedelta(days=gap_days + horizon_days)
    activated = set(events.loc[(events.event_type == "activate") & (events.event_date < t), "customer_id"])
    cancelled_before = set(events.loc[(events.event_type == "cancel") & (events.event_date < t), "customer_id"])
    cancel_dates = events.loc[events.event_type == "cancel"].set_index("customer_id")["event_date"]

    active = sorted(activated - cancelled_before) # loại khách đã rời bỏ trước T
    labels = pd.DataFrame({"customer_id": active, "snapshot_date": t})
    cd = labels["customer_id"].map(cancel_dates)
    labels["churn"] = ((cd >= start) & (cd < end)).astype(int)
    # Khách hủy trong gap: không thể can thiệp kịp -> loại khỏi huấn luyện để nhãn sạch
    in_gap = (cd >= t) & (cd < start)
    return labels[~in_gap].reset_index(drop=True)

events = make_event_log()
for snap in ["2024-06-01", "2024-09-01", "2024-12-01"]:
    lab = build_labels(events, snap)
    print(snap, "n_active =", len(lab), "churn_rate =", round(lab["churn"].mean(), 4))

```

Huấn luyện trên **nhiều snapshot** (ví dụ mỗi tháng một snapshot) làm tăng dữ liệu và giúp mô hình bền với mùa vụ. Khi đó **một khách hàng xuất hiện nhiều lần**, nên bắt buộc phải chia theo nhóm và theo thời gian (Chương 7).

1.4. Hệ thống metric: từ KPI kinh doanh đến hàm loss

```

North-star KPI (doanh thu giữ lại / tháng)
├── Online metric (tỷ lệ churn nhóm được can thiệp vs đối chứng)
│   ├── Offline business metric (lợi nhuận kỳ vọng, Precision@K)
│   │   ├── Offline ML metric (PR-AUC, log loss)
│   │   └── Training loss (binary cross-entropy)
└── A/B test
    ├── chọn ngưỡng/K
    ├── chọn mô hình
    └── tối ưu tham số

```

- **Tính nhất quán theo tầng:** mỗi tầng dưới phải là *proxy* tốt của tầng trên. **Định luật Goodhart:** khi một chỉ số trở thành mục tiêu, nó không còn là chỉ số tốt. Tối ưu PR-AUC quá mức mà quên lợi nhuận là một ví dụ.
- **[Course]** Andrew Ng (*Structuring ML Projects*): chọn **một metric tối ưu (optimizing)** để xếp hạng mô hình, và các **metric thỏa mãn (satisficing)** chỉ cần vượt ngưỡng. Ví dụ: tối đa PR-AUC với điều kiện độ trễ < 100 ms, recall nhóm khách VIP ≥ 0.7, chênh lệch recall giữa nhóm tuổi ≤ 0.05.
- **Guardrail metrics** (Kohavi): các chỉ số *không được xấu đi* khi triển khai, như doanh thu/khách, khiếu nại, tỷ lệ hủy dịch vụ do bị làm phiền.

1.5. Khung giá trị kỳ vọng (Expected Value framework)

[Course/Sách] Provost & Fawcett, *Data Science for Business* (ch. 7) đề xuất đánh giá mô hình bằng **giá trị kỳ vọng**, kết hợp ma trận nhầm lẫn với ma trận lợi ích-chi phí:

$$\mathbb{E}[\text{profit}] = \sum_{(a,y)} P(\hat{y} = a, y) \cdot b(a, y)$$

	Thực tế: churn (y=1)	Thực tế: ở lại (y=0)
Gọi + voucher ($\hat{y}=1$)	$b(1,1) = v \cdot r - c = 500k \cdot 0.3 - 50k = \mathbf{+100k}$	$b(1,0) = -c = \mathbf{-50k}$
Không làm gì ($\hat{y}=0$)	$b(0,1) = 0$ (mất khách là hiện trạng)	$b(0,0) = 0$

với v = giá trị vòng đời khách (500k), r = xác suất giữ chân thành công khi được can thiệp (30%), c = chi phí can thiệp (50k).

Ngưỡng tối ưu. Gọi khách khi lợi ích kỳ vọng dương:

$$p \cdot (vr - c) + (1 - p) \cdot (-c) > 0 \Leftrightarrow p > \frac{c}{vT} = \frac{50}{150} \approx 0.33$$

Ngưỡng này chỉ đúng khi xác suất **đã được hiệu chuẩn (calibrated)**, xem Chương 9. Ngưỡng mặc định 0.5 làm mất lợi nhuận.

```
import numpy as np

def expected_profit_per_customer(p: np.ndarray, value: float = 500, retain_rate: float = 0.3,
                                cost: float = 50) -> np.ndarray:
    """Lợi nhuận kỳ vọng (nghìn VNĐ) nếu can thiệp khách có xác suất churn p (đã calibrate)."""
    return p * (value * retain_rate - cost) + (1 - p) * (-cost)

p = np.array([0.1, 0.2, 0.33, 0.5, 0.8])
print({float(k): float(v) for k, v in zip(p, expected_profit_per_customer(p).round(1))})
print("Ngưỡng hòa vốn p* =", round(50 / (500 * 0.3), 3))
```

1.6. Baseline, mức hiệu năng con người và tính khả thi

Các loại baseline

Baseline	Ví dụ	Mục đích
Ngây thơ (naive)	Dự đoán tỷ lệ churn trung bình; <code>DummyClassifier</code>	Sàn tối thiểu; PR-AUC của nó bằng prevalence
Heuristic nghiệp vụ	Quy tắc CSKH đang dùng	Mô hình phải vượt cái đang có
Mô hình đơn giản	Logistic Regression vài feature	Đo giá trị gia tăng của độ phức tạp
Hiệu năng con người	Chuyên gia CSKH đoán	Ước lượng <i>Bayes error</i> (Andrew Ng)

[Course] Andrew Ng phân tích sai số thành **avoidable bias** (sai số train so với mức con người/Bayes) và **variance** (chênh lệch train-dev). Biết thành phần nào lớn hơn sẽ quyết định nên làm gì tiếp: mô hình lớn hơn hay thêm dữ liệu/regularization (Chương 8.1).

Cỡ mẫu tối thiểu

- Với mô hình hồi quy logistic, tài liệu thống kê y sinh dùng quy tắc **EPV (events per variable) $\geq 10-20$** : mỗi tham số cần ít nhất 10-20 sự kiện positive (Peduzzi et al., 1996; Riley et al., 2019 đề xuất cách tính cỡ mẫu chính xác hơn).
- Với ML linh hoạt (GBM, NN) cần nhiều hơn đáng kể. Cách thực tế là vẽ **learning curve** (Chương 8) để xem thêm dữ liệu có còn giúp không.

1.7. Đặt bài toán dưới dạng uplift (nâng cao)

Gọi $T \in \{0, 1\}$ là can thiệp (gọi điện), Y là churn. Thứ ta cần là **hiệu ứng can thiệp có điều kiện (CATE)**:

$$\tau(x) = \mathbb{E}[Y \mid T = 1, X = x] - \mathbb{E}[Y \mid T = 0, X = x]$$

Chỉ gọi những khách có $\tau(x)$ âm nhiều nhất (giảm churn nhiều nhất). Muốn ước lượng $\tau(x)$ **phải có dữ liệu thực nghiệm ngẫu nhiên** (A/B test) hoặc dùng các phương pháp nhân quả với giả định mạnh (Chương 4.8). Dưới đây là **T-learner**: hai mô hình riêng cho nhóm treatment và control.

```

import numpy as np
import pandas as pd
from sklearn.ensemble import HistGradientBoostingClassifier
from sklearn.model_selection import train_test_split

rng = np.random.default_rng(1)
n = 20_000
X = pd.DataFrame({"tenure": rng.integers(1, 72, n), "calls": rng.poisson(1.5, n),
                  "m2m": rng.integers(0, 2, n)})
T = rng.integers(0, 2, n) # can thiệp NGẪU NHIÊN (A/B test)
base = -2 + 1.2 * X.m2m - 0.03 * X.tenure + 0.3 * X.calls
true_tau_logit = -1.0 * X.m2m * (X.calls >= 2) # chỉ nhóm m2m & nhiều khiêu nại được lợi
Y = (rng.random(n) < 1 / (1 + np.exp(-(base + T * true_tau_logit))))).astype(int)

X_tr, X_te, T_tr, T_te, Y_tr, Y_te = train_test_split(X, T, Y, test_size=0.3, random_state=0)
m1 = HistGradientBoostingClassifier(random_state=0).fit(X_tr[T_tr == 1], Y_tr[T_tr == 1])
m0 = HistGradientBoostingClassifier(random_state=0).fit(X_tr[T_tr == 0], Y_tr[T_tr == 0])
tau_hat = m1.predict_proba(X_te)[:, 1] - m0.predict_proba(X_te)[:, 1]

seg = (X_te.m2m == 1) & (X_te.calls >= 2)
print("τ^trung bình nhóm được lợi :", tau_hat[seg].mean().round(3))
print("τ^trung bình nhóm còn lại   :", tau_hat[~seg].mean().round(3))

```

Thư viện chuyên dụng: **CausalML** (Uber), **EconML** (Microsoft), **scikit-uplift**. Đánh giá bằng **Qini curve** / **AUUC** trên dữ liệu thực nghiệm.

1.8. Tài liệu thiết kế (ML Design Doc) & quản trị rủi ro

```

# [Tên dự án] – ML Design Doc (v0.3)
## 1. Bối cảnh & mục tiêu: KPI, giá trị kỳ vọng (tiền), người hưởng lợi
## 2. Phi mục tiêu (non-goals): những gì dự án KHÔNG làm
## 3. Định nghĩa bài toán: đơn vị, nhãn, cửa sổ thời gian, dạng bài toán
## 4. Dữ liệu: nguồn, chủ sở hữu, độ trễ, khối lượng, PII, rủi ro chất lượng
## 5. Baseline hiện tại & tiêu chí thành công (offline: optimizing + satisficing; online: A/B)
## 6. Phương pháp: feature, mô hình ứng viên, chiến lược validation (out-of-time?)
## 7. Triển khai: batch/online, SLA độ trễ, tần suất retrain, fallback khi lỗi
## 8. Giám sát: drift, hiệu năng khi có nhãn trễ, KPI kinh doanh, người trực (on-call)
## 9. Rủi ro & đạo đức: fairness, giải thích được, tuân thủ (NĐ 13/2023/NĐ-CP, EU AI Act)
## 10. Kế hoạch & mốc: MVP -> shadow -> A/B -> rollout
## 11. Câu hỏi mở / quyết định cần stakeholder

```

Rủi ro	Ví dụ	Giảm thiểu
Dữ liệu	Nhãn churn ghi nhận trễ 2 tuần	Gap trong định nghĩa nhãn; kiểm tra độ trễ
Mô hình	Overfit giai đoạn khuyến mãi	Out-of-time validation nhiều giai đoạn
Vận hành	CSKH không gọi theo danh sách	Theo dõi tỷ lệ thực thi; đào tạo
Đạo đức/pháp lý	Phân biệt đối xử theo tuổi	Đo fairness; loại biến nhạy cảm và biến proxy
Phản hồi (feedback loop)	Khách được gọi thì không churn ⇒ nhãn sau này lệch	Giữ nhóm đối chứng ngẫu nhiên (holdout) cố định

Checklist Chương 1

- ☐ Có mục tiêu kinh doanh đo được, hành động cụ thể sau dự đoán và người chịu trách nhiệm hành động.
- ☐ Đã cân nhắc giải pháp không dùng ML; có baseline heuristic.
- ☐ Nhãn có cửa sổ quan sát, gap, label window; loại khách đã rời bỏ trước T.
- ☐ Hệ thống metric nhất quán từ KPI đến loss; có optimizing, satisficing và guardrail.
- ☐ Ma trận lợi ích-chi phí được stakeholder xác nhận; biết ngưỡng hòa vốn.
- ☐ Có kế hoạch nhóm đối chứng để đo tác động thật và tránh feedback loop.

Tài liệu tham khảo Chương 1

- Google for Developers. *Introduction to Machine Learning Problem Framing*. developers.google.com/machine-learning/problem-framing
- Zinkevich, M. *Rules of Machine Learning: Best Practices for ML Engineering*. developers.google.com/machine-learning/guides/rules-of-ml
- Provost, F. & Fawcett, T. (2013). *Data Science for Business*, ch. 7 "Decision Analytic Thinking". O'Reilly.
- Ng, A. *Structuring Machine Learning Projects* (DeepLearning.AI) và *Machine Learning Yearning*.
- Huyen, C. (2022). *Designing Machine Learning Systems*, ch. 2. O'Reilly.
- Gutierrez, P. & Gérardy, J.-Y. (2017). *Causal Inference and Uplift Modelling: A Review of the Literature*. PMLR 67.
- Riley, R. D. et al. (2019). *Minimum sample size for developing a multivariable prediction model*. *Statistics in Medicine* 38(7).

CHƯƠNG 2. THU THẬP DỮ LIỆU (DATA COLLECTION & INGESTION)

Mục tiêu chương: lấy dữ liệu từ mọi nguồn phổ biến **một cách hiệu quả, đúng thời điểm (point-in-time), có kiểm soát chất lượng, có phiên bản và hợp pháp.**

2.1. Bản đồ nguồn dữ liệu

Nguồn	Ví dụ	Công cụ Python	Lưu ý production
File tĩnh	CSV, Excel, JSON, Parquet	<code>pandas</code> , <code>polars</code> , <code>pyarrow</code>	Ưu tiên Parquet : có schema, nén cột, đọc chọn cột/lọc dòng
CSDL quan hệ (OLTP)	PostgreSQL, MySQL, SQL Server	<code>SQLAlchemy 2.x</code> , <code>psycopg2</code> , <code>connectorx</code> , <code>adbc</code>	Đọc từ replica , không truy vấn nặng trên DB chính
Data Warehouse (OLAP)	BigQuery, Snowflake, Redshift	client chính thức, <code>ibis</code>	Trả tiền theo lượng quét: chọn cột, lọc partition
Data Lake / Lakehouse	S3/GCS/ADLS + Parquet/Delta/Iceberg	<code>pyarrow.dataset</code> , <code>duckdb</code> , <code>polars</code> , <code>deltalake</code>	Partition theo ngày; bảng có transaction log
API (REST/GraphQL)	CRM, cổng thanh toán	<code>httpx</code> , <code>requests</code> , <code>tenacity</code>	Timeout, retry có backoff, rate limit, phân trang, idempotency
Web scraping	Trang công khai	<code>httpx</code> + <code>BeautifulSoup</code> / <code>lxml</code> , <code>Playwright</code> , <code>Scrapy</code>	<code>robots.txt</code> , điều khoản sử dụng, dữ liệu cá nhân
Streaming / CDC	Clickstream, thay đổi DB	<code>confluent-kafka</code> , <code>Debezium</code>	Ngữ nghĩa giao nhận, thứ tự, dữ liệu đến trễ
Phi cấu trúc	Văn bản, ảnh, PDF, âm thanh	<code>pypdf</code> , <code>docling</code> , <code>Pillow</code> , <code>librosa</code>	Lưu file trong object storage, metadata trong bảng
Nhãn do con người	Annotation	Label Studio, Argilla, CVAT	Hướng dẫn gán nhãn, đo độ đồng thuận

2.2. Định dạng file: lựa chọn có cơ sở

Định dạng	Kiểu lưu	Schema	Nén	Đọc một phần	Dùng khi
CSV	Dòng, văn bản	❌ (phải suy luận)	Kém	❌	Trao đổi với con người/hệ thống cũ
JSON / JSONL	Dòng, văn bản	❌	Kém	❌	Dữ liệu lỏng nhau, log API
Excel	Bảng tính	❌	Trung bình	❌	Dữ liệu nghiệp vụ nhập tay
Parquet	Cột (row groups → column chunks → pages)	✅	Tốt (Snappy/ZSTD)	✅ cột + lọc theo min/max statistics	Mặc định cho phân tích
Feather/Arrow IPC	Cột, bộ nhớ	✅	Tùy chọn	✅	Trao đổi nhanh giữa tiến trình
Delta / Iceberg	Parquet + transaction log	✅ + tiến hóa schema	Tốt	✅	Lakehouse: ACID, time-travel

Cấu trúc Parquet và vì sao nó nhanh. Một file Parquet gồm nhiều *row group*; mỗi row group lưu từng cột liên tiếp (*column chunk*) kèm thống kê min/max. Khi đọc với `columns=[...]` thì chỉ các cột cần thiết được đọc (**projection pushdown**). Khi đọc với `filters=[...]`, các row group có min/max nằm ngoài điều kiện bị bỏ qua

(predicate pushdown).

Thiết lập dữ liệu mẫu cho chương

```
from pathlib import Path

import numpy as np
import pandas as pd

from churn.data.synthetic import make_churn_data

Path("data/raw").mkdir(parents=True, exist_ok=True)
df = make_churn_data(n=50_000, seed=42)
df.to_csv("data/raw/churn.csv", index=False)
df.to_parquet("data/raw/churn.parquet", index=False, row_group_size=10_000) # cần pyarrow

# Lake partition theo năm-tháng đăng ký (hive-style: year_month=2023-01/part-0.parquet)
df.assign(year_month=df["signup_date"].dt.strftime("%Y-%m")).to_parquet(
    "data/lake/churn", partition_cols=["year_month"], index=False)
print(sorted(p.name for p in Path("data/lake/churn").iterdir())[3], "...")
```

2.3. Đọc dữ liệu với pandas: chính xác và tiết kiệm

[Docs] pandas User Guide, mục *Scaling to large datasets*, khuyến nghị theo thứ tự: (1) **chỉ đọc dữ liệu cần thiết**: chọn cột, lọc dòng; (2) **dùng kiểu dữ liệu hiệu quả**: category, số nguyên/thực nhỏ hơn; (3) **chia nhỏ (chunking)**; (4) khi vẫn không đủ, chuyển sang thư viện khác như Polars, DuckDB, Dask hoặc Spark.

```
# 1) CSV: khai báo dtype + parse_dates thay vì để pandas tự suy luận (chậm, dễ sai)
dtypes = {"customer_id": "string", "contract": "category", "payment_method": "category",
          "region": "category", "tenure_months": "int16", "support_calls": "int16"}
csv_df = pd.read_csv("data/raw/churn.csv", dtype=dtypes, parse_dates=["signup_date"],
                    usecols=list(dtypes) + ["signup_date", "monthly_charges", "churn"],
                    na_values=["", "NA", "null", "-"], encoding="utf-8")

# 2) Parquet: projection + predicate pushdown
pq_df = pd.read_parquet("data/raw/churn.parquet",
                      columns=["customer_id", "tenure_months", "contract", "churn"],
                      filters=[("tenure_months", ">", 24)])

# 3) Đọc một phần lake theo partition: chỉ các thư mục year_month thỏa điều kiện được mở
lake_df = pd.read_parquet("data/lake/churn", filters=[("year_month", ">=", "2024-01")])

# 4) Chunking: tổng hợp file lớn hơn RAM theo từng phần
agg = None
for chunk in pd.read_csv("data/raw/churn.csv", usecols=["contract", "churn"], chunksize=10_000):
    part = chunk.groupby("contract")["churn"].agg(["sum", "count"])
    agg = part if agg is None else agg.add(part, fill_value=0)
print((agg["sum"] / agg["count"]).round(3))
print(len(csv_df), len(pq_df), len(lake_df))
```

Kiểu dữ liệu và bộ nhớ

```
def memory_report(frame: pd.DataFrame) -> pd.DataFrame:
    return pd.DataFrame({"dtype": frame.dtypes.astype(str),
                        "MB": frame.memory_usage(deep=True, index=False) / 1e6}).round(3)

def optimize_dtypes(frame: pd.DataFrame, max_cat_ratio: float = 0.05) -> pd.DataFrame:
    """Downcast số; chuỗi có ít giá trị phân biệt -> category. Thường giảm 50-90% RAM."""
    out = frame.copy()
    for c in out.select_dtypes("integer").columns:
        out[c] = pd.to_numeric(out[c], downcast="integer")
    for c in out.select_dtypes("float").columns:
        out[c] = pd.to_numeric(out[c], downcast="float")
    for c in out.select_dtypes(["object", "string"]).columns:
        if out[c].nunique(dropna=True) / max(len(out), 1) < max_cat_ratio:
```

```

        out[c] = out[c].astype("category")
    return out

raw = pd.read_csv("data/raw/churn.csv")
opt = optimize_dtypes(raw)
print(f"{raw.memory_usage(deep=True).sum() / 1e6:.1f} MB -> {opt.memory_usage(deep=True).sum() / 1e6:.1f} MB")
print(memory_report(opt).sort_values("MB", ascending=False).head())

```

Lưu ý về float32. Downcast float64 → float32 chỉ còn ~7 chữ số có nghĩa. Không áp dụng cho số tiền cần chính xác tuyệt đối, ID số lớn hay timestamp dạng số.

pandas 3.0: những thay đổi cần biết

Thay đổi	Hệ quả	Cách viết đúng
Copy-on-Write luôn bật	Mọi DataFrame/Series lấy ra từ đối tượng khác hành xử như bản sao. Chained assignment <code>df["a"][mask] = v</code> không còn sửa df	<code>df.loc[mask, "a"] = v</code>
Kiểu chuỗi mặc định <code>str</code> (dùng PyArrow nếu có)	Cột chuỗi không còn là <code>object</code> ; thiếu giá trị là <code>NaN</code>	Kiểm tra <code>pd.api.types.is_string_dtype</code> thay vì <code>== object</code>
Datetime không còn mặc định nano giây	Độ phân giải được suy luận (thường là <code>us</code>) khi đọc/tạo datetime	Không giả định <code>datetime64[ns]</code> trong <code>schema/test</code> . Ép kiểu tường minh khi cần
Nhiều API cũ bị xóa	Code pandas 1.x có thể lỗi	Chạy test với <code>-W error::FutureWarning</code> trên pandas 2.x trước khi nâng cấp

```

s = pd.DataFrame({"a": [1, 2, 3], "b": ["x", "y", "z"]})
print(s.dtypes.to_dict())          # b là kiểu chuỗi chuyên dụng ở pandas 3
s.loc[s["a"] > 1, "a"] = 0         # cách duy nhất đúng để gán có điều kiện
print(s["a"].tolist())

```

`dtype_backend="pyarrow"` : kiểu dữ liệu có hỗ trợ missing thật sự

```

arrow_df = pd.read_parquet("data/raw/churn.parquet", dtype_backend="pyarrow")
print(arrow_df.dtypes.head(4).to_dict())
# int64[pyarrow] giữ được giá trị thiếu mà không bị ép sang float như numpy int

```

2.4. Khi pandas không đủ: Polars và DuckDB

```

import duckdb
import polars as pl

# Polars lazy: xây kế hoạch truy vấn, tối ưu (pushdown, chạy song song) rồi mới thực thi
lazy = (
    pl.scan_parquet("data/raw/churn.parquet")
    .filter(pl.col("tenure_months") > 6)
    .group_by("contract")
    .agg(pl.col("churn").mean().alias("churn_rate"), pl.len().alias("n"))
    .sort("churn_rate", descending=True)
)
print(lazy.explain().splitlines()[0])      # xem kế hoạch truy vấn đã tối ưu
print(lazy.collect())

# DuckDB: SQL trực tiếp trên Parquet/CSV/DataFrame, không cần server
con = duckdb.connect()
print(con.sql("""
    SELECT contract, round(avg(churn), 4) AS churn_rate, count(*) AS n
    FROM read_parquet('data/raw/churn.parquet')
    WHERE tenure_months > 6

```

```
GROUP BY contract ORDER BY churn_rate DESC
""").df())
```

Công cụ	Mô hình thực thi	Mạnh nhất khi
pandas	Eager, phần lớn đơn luồng	Dữ liệu vừa RAM, hệ sinh thái lớn nhất
Polars	Lazy + eager, đa luồng, Arrow	Biến đổi dữ liệu lớn trên một máy
DuckDB	SQL OLAP nhúng, vectorized	Phân tích SQL trên file, join lớn
Spark / Dask	Phân tán	Dữ liệu vượt một máy

2.5. Đọc từ cơ sở dữ liệu với SQLAlchemy 2.x

Nguyên tắc:

1. **Đẩy tính toán xuống database** (JOIN, GROUP BY, WHERE) và chỉ kéo về kết quả.
2. **Tham số hóa truy vấn** bằng bind parameters, không dùng f-string. Cách này vừa chống SQL injection vừa giúp tái sử dụng execution plan.
3. **Point-in-time**: mọi điều kiện thời gian phải `< :snapshot`.
4. Kết nối dùng **pool**, `pool_pre_ping=True` để tự loại kết nối chết. Đọc từ **read replica**.

```
from sqlalchemy import create_engine, text

engine = create_engine("sqlite:///data/warehouse.db") # production: postgresql+psycopg://...
customers = df.drop_duplicates("customer_id")[["customer_id", "signup_date", "contract"]]
rng = np.random.default_rng(0)
tickets = pd.DataFrame({
    "ticket_id": np.arange(30_000),
    "customer_id": rng.choice(customers["customer_id"], 30_000),
    "created_at": pd.Timestamp("2023-01-01") + pd.to_timedelta(rng.integers(0, 600, 30_000), unit="D"),
})
with engine.begin() as conn: # transaction: commit/rollback tự động
    customers.to_sql("customers", conn, if_exists="replace", index=False)
    tickets.to_sql("tickets", conn, if_exists="replace", index=False)

FEATURE_SQL = text("""
    SELECT c.customer_id,
           c.contract,
           COUNT(t.ticket_id) AS tickets_90d
    FROM customers c
    LEFT JOIN tickets t
        ON t.customer_id = c.customer_id
        AND t.created_at >= :window_start
        AND t.created_at < :snapshot -- point-in-time: không nhìn tương lai
    WHERE c.signup_date < :snapshot
    GROUP BY c.customer_id, c.contract
""")

def load_features(snapshot: str, window_days: int = 90) -> pd.DataFrame:
    snap = pd.Timestamp(snapshot)
    params = {"snapshot": str(snap), "window_start": str(snap - pd.Timedelta(days=window_days))}
    with engine.connect() as conn:
        return pd.read_sql(FEATURE_SQL, conn, params=params)

feats = load_features("2024-03-01")
print(feats.shape, feats["tickets_90d"].describe()[["mean", "max"]].round(2).to_dict())
```

Point-in-time join trong pandas: `merge_asof`

Khi feature được cập nhật theo thời gian (ví dụ hạng thành viên thay đổi), mỗi dòng nhãn tại thời điểm T phải lấy **giá trị gần nhất trước T**, không được lấy giá trị mới nhất.

```

labels = pd.DataFrame({"customer_id": ["C1", "C1", "C2"],
                       "snapshot_date": pd.to_datetime(["2024-02-01", "2024-05-01", "2024-05-01"])}))
tier_history = pd.DataFrame({"customer_id": ["C1", "C1", "C2"],
                             "valid_from": pd.to_datetime(["2024-01-10", "2024-04-15", "2024-06-01"]),
                             "tier": ["silver", "gold", "platinum"]})

pit = pd.merge_asof(
    labels.sort_values("snapshot_date"), tier_history.sort_values("valid_from"),
    left_on="snapshot_date", right_on="valid_from", by="customer_id",
    direction="backward", allow_exact_matches=False, # chỉ lấy bản ghi có hiệu lực TRƯỚC T
)
print(pit) # C2 tại 2024-05-01 chưa có tier (NaN): đúng, vì tier platinum có từ 2024-06-01

```

[Kinh nghiệm] Bảng nghiệp vụ thường chỉ lưu **trạng thái hiện tại** (bị ghi đè). Nếu tính feature từ bảng như vậy cho các snapshot quá khứ, bạn đang nhìn trộm tương lai. Cần bảng lịch sử (**SCD Type 2**: `valid_from`, `valid_to`), event log, hoặc snapshot định kỳ. Feature store (Chương 12) giải quyết vấn đề này một cách hệ thống.

2.6. Thu thập qua API: timeout, retry, rate limit, phân trang

[Docs] HTTPX: luôn đặt **timeout** (mặc định của httpx là 5 giây). Tái sử dụng `Client` để có connection pooling. `tenacity` cung cấp retry có **exponential backoff + jitter**. Chỉ retry các lỗi **tạm thời**: lỗi mạng, 429, 5xx. Không retry lỗi 4xx khác vì đó là lỗi phía client.

```

import time

import httpx
from tenacity import retry, retry_if_exception, stop_after_attempt, wait_random_exponential

def is_transient(exc: BaseException) -> bool:
    if isinstance(exc, httpx.TransportError):
        return True
    return isinstance(exc, httpx.HTTPStatusError) and exc.response.status_code in {429, 500, 502, 503, 504}

class APIClient:
    def __init__(self, base_url: str, token: str, rate_per_sec: float = 10.0,
                 transport: httpx.BaseTransport | None = None):
        self.client = httpx.Client(base_url=base_url, headers={"Authorization": f"Bearer {token}"},
                                   timeout=httpx.Timeout(10.0, connect=3.0), transport=transport)
        self.min_interval, self._last = 1.0 / rate_per_sec, 0.0

    def _throttle(self) -> None:
        wait = self.min_interval - (time.monotonic() - self._last)
        if wait > 0:
            time.sleep(wait)
        self._last = time.monotonic()

    @retry(retry=retry_if_exception(is_transient), wait=wait_random_exponential(multiplier=0.01, max=1),
          stop=stop_after_attempt(5), reraise=True)
    def get(self, path: str, params: dict | None = None) -> dict:
        self._throttle()
        resp = self.client.get(path, params=params)
        resp.raise_for_status()
        return resp.json()

    def paginate(self, path: str, page_size: int = 100):
        cursor = None
        while True:
            payload = self.get(path, {"limit": page_size, **({"cursor": cursor} if cursor else {})})
            yield from payload["data"]
            if not (cursor := payload.get("next_cursor")):
                break

```

```
# Kiểm thử không cần server thật: MockTransport giả lập API có phân trang và lỗi 503 thoáng qua
calls = {"n": 0}

def fake_api(request: httpx.Request) -> httpx.Response:
    calls["n"] += 1
    if calls["n"] == 2:
        return httpx.Response(503) # lỗi tạm thời -> được retry
    cursor = int(request.url.params.get("cursor", 0))
    data = [{"id": i} for i in range(cursor, min(cursor + 100, 250))]
    nxt = cursor + 100 if cursor + 100 < 250 else None
    return httpx.Response(200, json={"data": data, "next_cursor": nxt})

api = APIClient("https://crm.example.com", token="****", rate_per_sec=1000,
               transport=httpx.MockTransport(fake_api))
records = list(api.paginate("/customers"))
print(len(records), "bản ghi;", calls["n"], "request (có 1 lần retry)")
```

Thu thập song song có giới hạn (async)

```
import asyncio

async def fetch_all(ids: list[int], max_concurrency: int = 20) -> list[dict]:
    sem = asyncio.Semaphore(max_concurrency) # giới hạn số kết nối đồng thời
    transport = httpx.MockTransport(lambda r: httpx.Response(200, json={"id": r.url.path.split("/")[-1]}))
    async with httpx.AsyncClient(base_url="https://crm.example.com", transport=transport,
                                timeout=10) as client:
        async def one(i: int) -> dict:
            async with sem:
                r = await client.get(f"/customers/{i}")
                r.raise_for_status()
                return r.json()
        return await asyncio.gather(*(one(i) for i in ids))

print(len(asyncio.run(fetch_all(list(range(500))))))
```

2.7. Web scraping có trách nhiệm

```
from bs4 import BeautifulSoup

HTML = """<table class="plans"><tr><th>Gói</th><th>Giá</th><th>Data</th></tr>
<tr><td>Basic</td><td>70.000</td><td>5 GB</td></tr>
<tr><td>Pro</td><td>150.000</td><td>30 GB</td></tr></table>"""

def parse_plans(html: str) -> pd.DataFrame:
    soup = BeautifulSoup(html, "lxml")
    rows = soup.select("table.plans tr")
    header = [th.get_text(strip=True) for th in rows[0].select("th")]
    data = [[td.get_text(strip=True) for td in r.select("td")] for r in rows[1:]]
    out = pd.DataFrame(data, columns=header)
    out["Giá"] = out["Giá"].str.replace(".", "", regex=False).astype(int) # chuẩn hóa định dạng VN
    return out

print(parse_plans(HTML))
```

Quy tắc bắt buộc:

- Kiểm tra `robots.txt` (`urllib.robotparser`) và **điều khoản sử dụng** của trang. Đặt `User-Agent` có thông tin liên hệ.
- Giới hạn tốc độ, cache kết quả, chạy ngoài giờ cao điểm.

- Không thu thập **dữ liệu cá nhân** khi không có cơ sở pháp lý. Tại Việt Nam phải tuân thủ **Nghị định 13/2023/NĐ-CP** về bảo vệ dữ liệu cá nhân; với khách hàng EU là **GDPR**.
- Trang render bằng JavaScript thì dùng **Playwright**. Hệ thống crawl lớn thì dùng **Scrapy** (có sẵn throttling, pipeline, retry).

2.8. Streaming & Change Data Capture

Khái niệm	Ý nghĩa	Hệ quả cho Data Scientist
At-most-once	Có thể mất sự kiện	Feature đếm bị thiếu
At-least-once (phổ biến)	Có thể trùng sự kiện	Xử lý phải idempotent (dedup theo <code>event_id</code>)
Exactly-once	Không mất, không trùng	Cần transaction (Kafka EOS, Flink checkpoint)
Event time vs processing time	Thời điểm xảy ra vs thời điểm hệ thống nhận	Feature phải dùng event time
Watermark	Ngưỡng chờ dữ liệu đến trễ	Quyết định khi nào "đóng" cửa sổ tổng hợp
CDC (Debezium)	Đọc log thay đổi của DB thành luồng sự kiện	Xây lại lịch sử để có feature point-in-time

```
import json

from confluent_kafka import Consumer

consumer = Consumer({
    "bootstrap.servers": "kafka:9092",
    "group.id": "churn-feature-builder",
    "auto.offset.reset": "earliest",
    "enable.auto.commit": False,      # commit thủ công SAU khi xử lý xong -> at-least-once
    "isolation.level": "read_committed", # bỏ qua message của transaction bị hủy
})
consumer.subscribe(["customer-events"])
try:
    while True:
        msg = consumer.poll(1.0)
        if msg is None:
            continue
        if msg.error():
            raise RuntimeError(msg.error())
        event = json.loads(msg.value())
        upsert_feature(event)          # idempotent: khóa theo event["event_id"]
        consumer.commit(message=msg, asynchronous=False)
finally:
    consumer.close()
```

2.9. Data Contract & kiểm tra chất lượng dữ liệu

Data contract là thỏa thuận giữa bên sản xuất và bên tiêu thụ dữ liệu về schema, ngữ nghĩa, chất lượng và SLA. Kiểm tra phải đặt **tại biên** (ingestion) và **trước khi chấm điểm**.

Sáu chiều chất lượng dữ liệu

Chiều	Câu hỏi	Ví dụ kiểm tra
Completeness	Thiếu bao nhiêu?	% null mỗi cột < ngưỡng
Uniqueness	Có trùng không?	<code>customer_id</code> duy nhất
Validity	Đúng kiểu, định dạng, miền giá trị?	<code>age</code> ∈ [18, 100], regex mã khách
Consistency	Logic giữa cột/bảng?	<code>total</code> ≈ <code>monthly</code> × <code>tenure</code> ; khóa ngoại tồn tại
Timeliness	Dữ liệu có mới?	<code>max(event_time)</code> > <code>now</code> - 1 ngày
Accuracy	Đúng với thực tế?	Đối soát với nguồn chuẩn (sổ cái kế toán)

pandera: schema dạng class (DataFrameModel)

Package tham chiếu định nghĩa contract trong churn/data/validate.py :

```
import pandera.pandas as pa
from pandera.typing import Series

class ChurnSchema(pa.DataFrameModel):
    customer_id: Series[str] = pa.Field(unique=True, str_matches=r"^\d{7}$")
    signup_date: Series[pd.Timestamp] = pa.Field(le=pd.Timestamp("2100-01-01"))
    age: Series[float] = pa.Field(ge=18, le=100, nullable=True)
    tenure_months: Series[int] = pa.Field(ge=0, le=600)
    monthly_charges: Series[float] = pa.Field(gt=0, le=2_000, nullable=True)
    contract: Series[str] = pa.Field(isin=["month-to-month", "one-year", "two-year"])
    churn: Series[int] = pa.Field(isin=[0, 1])
    # ... (xem file đầy đủ)

@pa.dataframe_check(error="total_charges inconsistent with monthly_charges * tenure")
def charges_consistent(cls, df: pd.DataFrame) -> bool:
    expected = df["monthly_charges"] * df["tenure_months"]
    ok = (df["total_charges"] - expected).abs() <= 0.05 * expected + 1
    return bool(ok[expected.notna()].mean() > 0.99)

class Config:
    coerce = True      # ép kiểu trước khi kiểm tra
    strict = False     # cho phép cột thừa
```

Chạy contract trên **dữ liệu thô** (có lỗi cài sẵn) và trên dữ liệu đã làm sạch:

```
import pandera.errors

from churn.data.validate import validate
from churn.features.clean import clean_churn

try:
    validate(df)                                # lazy=True: gom TẤT CẢ lỗi một lần
except pandera.errors.SchemaErrors as err:
    fc = err.failure_cases
    print(fc.groupby(["column", "check"], dropna=False).size().sort_values(ascending=False).head(6))

validate(clean_churn(df))                       # sau khi làm sạch: hợp lệ
print("Dữ liệu sạch thỏa mãn contract")
```

Contract phát hiện `customer_id` trùng (100 dòng lặp, tính cả hai bản) và `contract` sai chuẩn hóa (Month-to-Month). Lỗi được bắt **tại biên**, rồi xử lý có chủ đích ở bước làm sạch (Chương 5).

Bài học quan trọng. 30 ngoại lai `monthly_charges` $\times 8$ (tối đa $\sim 1\,600$) **không** bị bắt, vì vẫn nằm trong miền hợp lệ (0; 2 000]. Kiểm tra miền giá trị chỉ bắt giá trị **không thể có**, không bắt được giá trị **bất thường**. Cần thêm kiểm tra thống kê: phân vị, z-score robust, so sánh phân phối với lô tham chiếu (Chương 3.7, 12.9).

Kiểm tra cấp lô dữ liệu (volume, freshness, phân phối)

```
def batch_checks(batch: pd.DataFrame, reference_rows: int, max_age_days: int = 2,
                 now: pd.Timestamp | None = None) -> dict[str, bool]:
    now = now or pd.Timestamp.now()
    return {
        "volume_ok": 0.7 * reference_rows <= len(batch) <= 1.3 * reference_rows,
        "fresh_ok": (now - batch["signup_date"].max()).days <= max_age_days,
        "null_rate_ok": bool((batch.isna().mean() < 0.2).all()),
        "target_rate_ok": 0.05 <= batch["churn"].mean() <= 0.4,
    }

print(batch_checks(clean_churn(df), reference_rows=50_000, now=pd.Timestamp("2024-06-20")))
```

Công cụ cho hệ thống lớn: Great Expectations (Expectation Suites, Data Docs), Soda Core (SodaCL), dbt tests (unique, not_null, relationships, accepted_values), Deequ trên Spark.

2.10. Versioning dữ liệu với DVC

[Docs] DVC lưu dữ liệu lớn ở remote (S3/GCS/Azure/SSH). Git chỉ giữ file `.dvc` nhỏ chứa hash nội dung (MD5). Nhờ vậy *mỗi commit git xác định chính xác phiên bản dữ liệu*.

```
pip install "dvc[s3]"
dvc init
dvc remote add -d storage s3://ml-bucket/dvc-store
dvc add data/raw/churn.parquet # sinh data/raw/churn.parquet.dvc
git add data/raw/churn.parquet.dvc data/raw/.gitignore && git commit -m "data: churn snapshot 2026-10"
dvc push # đẩy dữ liệu thật lên remote
git checkout <commit-cũ> && dvc checkout # quay lại đúng dữ liệu của commit đó
```

```
# dvc.yaml – pipeline có cache: chỉ chạy lại stage có deps/params thay đổi
stages:
  featurize:
    cmd: python -m churn.features.make --in data/raw/churn.parquet --out data/processed/features.parquet
    deps: [src/churn/features, data/raw/churn.parquet]
    outs: [data/processed/features.parquet]
  train:
    cmd: python -m churn.models.train --config configs/train.yaml
    deps: [src/churn/models, data/processed/features.parquet]
    params: [configs/train.yaml:model_params]
    outs: [models/model.joblib]
    metrics: [reports/metrics.json: {cache: false}]
```

dvc repro chạy lại pipeline. dvc metrics diff và dvc params diff so sánh giữa các commit.

2.11. Dữ liệu nhãn do con người gán

[Course] *MLOps Specialization* (Andrew Ng) nhấn mạnh **tính nhất quán của nhãn** ("label consistency"): nhãn không nhất quán gây hại như nhiều. Quy trình:

1. Viết **hướng dẫn gán nhãn** có ví dụ biên (edge cases).
2. Cho 2-3 người gán cùng một mẫu, đo **độ đồng thuận**.
3. Thảo luận các ca bất đồng và cập nhật hướng dẫn. Lặp lại tới khi đạt ngưỡng.

```
from sklearn.metrics import cohen_kappa_score

rng = np.random.default_rng(3)
truth = rng.integers(0, 3, 500) # 3 loại khiêu nại
annot_a = np.where(rng.random(500) < 0.85, truth, rng.integers(0, 3, 500))
annot_b = np.where(rng.random(500) < 0.75, truth, rng.integers(0, 3, 500))
print("Tỷ lệ trùng khớp thô:", (annot_a == annot_b).mean().round(3))
print("Cohen's k      :", round(cohen_kappa_score(annot_a, annot_b), 3))
# k hiệu chỉnh cho đồng thuận ngẫu nhiên. Thang Landis & Koch (1977):
# <0.2 kém | 0.21-0.4 khá | 0.41-0.6 trung bình | 0.61-0.8 tốt | >0.8 gần hoàn hảo
```

Với nhiều người gán hoặc nhãn thứ bậc, dùng **Fleiss' κ** hoặc **Krippendorff's α** . Khi nhãn nhiều, các kỹ thuật *confident learning* (thư viện `cleanlab`) giúp tìm mẫu bị gán sai.

2.12. Quyền riêng tư: ẩn danh và giả danh

Kỹ thuật	Mô tả	Lưu ý
Giả danh (pseudonymization)	Thay định danh bằng mã, ví dụ HMAC có khóa bí mật	Vẫn là dữ liệu cá nhân theo GDPR/NĐ 13 nếu còn khóa
Tổng quát hóa	Tuổi → nhóm tuổi; địa chỉ → tỉnh	Giảm rủi ro tái định danh
k-anonymity	Mỗi tổ hợp quasi-identifier xuất hiện $\geq k$ lần	Không chống được tấn công đồng nhất thuộc tính
Differential privacy	Thêm nhiễu có kiểm soát vào thống kê	Chuẩn mực mạnh nhất cho dữ liệu công bố

```
import hashlib
import hmac

def pseudonymize(values: pd.Series, secret: bytes) -> pd.Series:
    """HMAC-SHA256: ôñ ñịnh (cùng input -> cùng mã) ñể join; không ñảo ngược ñược khi không có khóa."""
    return values.map(lambda v: hmac.new(secret, str(v).encode(), hashlib.sha256).hexdigest()[:16])

def k_anonymity(frame: pd.DataFrame, quasi_identifiers: list[str]) -> int:
    return int(frame.groupby(quasi_identifiers, observed=True).size().min())

SECRET = b"load-from-secret-manager" # production: lấy từ Vault/Secrets Manager
safe = df[["customer_id", "age", "region"]].dropna().copy()
safe["customer_id"] = pseudonymize(safe["customer_id"], SECRET)
safe["age_band"] = pd.cut(safe["age"], [17, 25, 35, 45, 55, 65, 100]).astype(str)
print("k (age, region) =", k_anonymity(safe, ["age", "region"]))
print("k (age_band, region) =", k_anonymity(safe, ["age_band", "region"]))
```

2.13. Pitfalls khi thu thập dữ liệu

- Survivorship bias:** chỉ lấy khách *đang hoạt động* nên thiếu hẳn nhóm ñã churn.
- Nhìn trộm tương lai qua bảng bị ghi ñề:** dùng trạng thái hiện tại ñể tính feature cho quá khứ.
- Múi giờ:** lưu UTC, chỉ ñổi sang Asia/Ho_Chi_Minh khi hiển thị. Cẩn thận ranh giới ngày khi tổng hợp.
- Unicode tiếng Việt:** dữ liệu lẫn NFC/NFD làm "Hà Nội" != "Hà Nội". Chuẩn hóa bằng `unicodedata.normalize("NFC", s)`.
- Sampling bias:** dữ liệu từ một kênh/vùng không ñại diện cho quần thể chấm ñiểm.
- Thay ñổi ñịnh nghĩa ở nguồn:** bên sản xuất ñổi đơn vị (VNĐ → nghìn VNĐ) mà không báo. ðây là lý do cần data contract và giám sát phân phối.

Checklist Chương 2

- ☐ Biết nguồn, chủ sở hữu, tần suất cập nhật, ñộ trễ của mọi bảng.
- ☐ Dùng ñịnh dạng cột (Parquet), chỉ ñọc cột/dòng cần; kiểu dữ liệu tối ưu.
- ☐ Truy vấn tham số hóa; feature ñảm bảo point-in-time (`< snapshot` , `merge_asof`).
- ☐ API client có timeout, retry chỉ cho lỗi tạm thời, rate limit, test bằng mock.
- ☐ Data contract chạy tự ñộng tại ingestion và trước khi chấm ñiểm.
- ☐ Dữ liệu thô bất biến, có phiên bản (DVC/snapshot).
- ☐ Nhãn thủ công có hướng dẫn và ño ñộ ñồng thuận.
- ☐ PII ñược giả danh/ẩn danh; tuân thủ NĐ 13/2023/NĐ-CP.

Tài liệu tham khảo Chương 2

- pandas User Guide: *IO tools, Scaling to large datasets, Copy-on-Write, PyArrow functionality*. pandas.pydata.org/docs/user_guide
- Apache Parquet. *File format documentation*. parquet.apache.org/docs
- Polars User Guide: *Lazy API*. docs.pola.rs · DuckDB: *Parquet import*. duckdb.org/docs
- SQLAlchemy 2.0. *Unified Tutorial*. docs.sqlalchemy.org/en/20/tutorial
- HTTPX documentation: *Timeouts, Transports (MockTransport)*. python-httpx.org · Tenacity docs.
- pandera documentation: *DataFrame Models*. pandera.readthedocs.io
- DVC documentation: *Data versioning, Pipelines*. dvc.org/doc
- Kleppmann, M. (2017). *Designing Data-Intensive Applications*, ch. 11 "Stream Processing". O'Reilly.
- Landis, J. R. & Koch, G. G. (1977). *The Measurement of Observer Agreement for Categorical Data*. Biometrics 33(1).
- Chính phủ Việt Nam. *Nghị định 13/2023/NĐ-CP về bảo vệ dữ liệu cá nhân*.

CHƯƠNG 3. PHÂN TÍCH KHÁM PHÁ DỮ LIỆU (EDA)

"Exploratory data analysis is detective work." — John W. Tukey, *Exploratory Data Analysis* (1977).

Mục tiêu chương: hiểu cấu trúc, chất lượng, phân phối và quan hệ trong dữ liệu; phát hiện rủi ro (leakage, drift, bias) **trước khi** mô hình hóa; và kết thúc bằng các **quyết định cụ thể** cho bước tiền xử lý.

3.1. EDA trả lời những câu hỏi nào?

Nhóm	Câu hỏi	Kỹ thuật	Quyết định dẫn tới
Cấu trúc	Đơn vị quan sát? khóa? trùng lặp? kiểu dữ liệu?	Profile bảng, kiểm tra khóa	Dedup, ép kiểu, định nghĩa grain
Chất lượng	Thiếu ở đâu, vì sao? giá trị vô lý? sai chuẩn hóa?	Missing matrix, miền giá trị	Chiến lược impute, quy tắc làm sạch
Phân phối	Lệch, đa đỉnh, đuôi dày, tập trung ở 0?	Histogram, ECDF, Q-Q, skew/kurtosis	Biến đổi log/Yeo-Johnson, binning
Quan hệ với target	Biến nào có tín hiệu, tuyến tính hay không?	Rate-by-bin, IV, MI, AUC đơn biến	Feature engineering, chọn mô hình
Quan hệ giữa feature	Đa cộng tuyến, dư thừa?	Spearman, Cramér's V, VIF, phân cụm feature	Loại/gộp feature, diễn giải cẩn thận
Thời gian	Phân phối có trôi theo thời gian? mùa vụ?	Chuỗi theo tháng, cohort	Out-of-time split, giám sát drift
Rủi ro	Leakage? train/test khác phân phối?	AUC đơn biến, adversarial validation	Loại feature, sửa cách chia

EDA và CDA. Tukey phân biệt *Exploratory* (tìm giả thuyết) với *Confirmatory Data Analysis* (kiểm định giả thuyết, Chương 4). Một mẫu hình tìm được khi "đào bới" dữ liệu cần được **xác nhận trên dữ liệu độc lập**. Nếu không, đó có thể chỉ là kết quả của việc thử nhiều lần (*garden of forking paths*).

[Docs] scikit-learn *Common pitfalls* nhắc: mọi quyết định dựa trên dữ liệu (chọn biến, ngưỡng cắt ngoại lai, cách impute) đều là một dạng "fit". Vì vậy **EDA chi tiết làm trên tập train**, sau khi đã tách test.

Thiết lập

```
import matplotlib.pyplot as plt
import numpy as np
import pandas as pd
import seaborn as sns
from scipy import stats

from churn.data.synthetic import make_churn_data
from churn.viz.style import set_style

set_style()
raw = make_churn_data(n=20_000, seed=42)

# Chỉ dedup chính xác (stateless) trước khi tách; mọi phân tích sâu làm trên train
raw = raw.drop_duplicates().reset_index(drop=True)
train = raw[raw["signup_date"] < "2023-10-01"].copy()
test = raw[raw["signup_date"] >= "2023-10-01"].copy()
print(len(train), len(test))
```

3.2. Hồ sơ cấu trúc (Data profiling)

```
def profile(frame: pd.DataFrame) -> pd.DataFrame:
    """Bảng hồ sơ từng cột: thứ đầu tiên nên chạy với mọi dataset."""
    n = len(frame)
    top = frame.apply(lambda s: s.value_counts(dropna=True).iloc[0] / n if s.notna().any() else np.nan)
    return pd.DataFrame({
        "dtype": frame.dtypes.astype(str),
        "n_missing": frame.isna().sum(),
        "pct_missing": (frame.isna().mean() * 100).round(2),
        "n_unique": frame.nunique(dropna=True),
        "pct_unique": (frame.nunique(dropna=True) / n * 100).round(2),
        "top_freq_pct": (top * 100).round(2), # ~100% => cột hằng/gần hằng
        "examples": [frame[c].dropna().unique()[:3].tolist() for c in frame.columns],
    })

print(profile(train).to_string())
print("Trùng toàn dòng:", train.duplicated().sum(),
      "| Trùng khóa customer_id:", train["customer_id"].duplicated().sum())
print("Tỷ lệ churn train/test:", round(train["churn"].mean(), 4), round(test["churn"].mean(), 4))
```

Cần đọc ra từ bảng này:

- `pct_unique ≈ 100%` ở cột không phải ID: có thể là timestamp hoặc số liên tục. Nếu là ID trá hình thì phải loại.
- `top_freq_pct > 95%`: cột gần như hằng số (*quasi-constant*), gần như không mang thông tin.
- Cột số mang kiểu `str` / `object`: thường do giá trị rác như `"N/A"` hay dấu phân cách hàng nghìn.

3.3. Giá trị thiếu: lượng, mẫu hình và cơ chế

Ba cơ chế thiếu (Rubin, 1976) quyết định cách xử lý:

Cơ chế	Định nghĩa	Ví dụ	Hệ quả
MCAR	P(thiếu) không phụ thuộc dữ liệu nào	Lỗi đồng bộ ngẫu nhiên	Xóa dòng không gây bias, chỉ mất hiệu quả
MAR	P(thiếu) phụ thuộc biến quan sát được	Khách trẻ hay bỏ trống tuổi	Impute có điều kiện (KNN, MICE)
MNAR	P(thiếu) phụ thuộc chính giá trị bị thiếu	Người thu nhập cao không khai thu nhập	Không thể sửa chỉ bằng dữ liệu; dùng cờ missing, mô hình hóa cơ chế, thu thập thêm

MCAR không thể chứng minh, chỉ có thể tìm bằng chứng chống lại. Cách thực tế: kiểm tra xem **cờ thiếu có dự đoán được từ các biến khác không**. Nếu có, dữ liệu không phải MCAR.

```
import missingno as msno
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import roc_auc_score
from sklearn.model_selection import cross_val_predict

msno.matrix(train.sample(1500, random_state=0), figsize=(10, 4), fontsize=9, sparkline=False)

num_cols = ["age", "tenure_months", "monthly_charges", "total_charges", "support_calls", "data_usage_gb"]
cat_cols = ["contract", "payment_method", "region"]

# (1) Thiêu có liên quan tới target không? -> cờ missing có thể là feature
for col in ["age", "data_usage_gb", "payment_method"]:
    rates = train.groupby(train[col].isna())["churn"].agg(["mean", "size"])
    print(f"{col:15s} churn|có={rates.loc[False, 'mean']:.3f} churn|thiếu={rates.loc[True, 'mean']:.3f}"
          f" (n_thiếu={rates.loc[True, 'size']})")

# (2) Bằng chứng chống MCAR: dự đoán cờ thiếu từ các biến khác (AUC ≈ 0.5 -> không bác bỏ MCAR)
others = train[["tenure_months", "monthly_charges", "support_calls"]]
for col in ["age", "data_usage_gb"]:
```

```
miss = train[col].isna().astype(int)
p = cross_val_predict(LogisticRegression(max_iter=1000), others, miss, cv=5, method="predict_proba")[:, 1]
print(f"AUC dự đoán cờ thiếu {col}: {roc_auc_score(miss, p):.3f}")
```

Diễn giải với dữ liệu mô phỏng. Thiếu được bơm ngẫu nhiên (MCAR), nên $AUC \approx 0.5$ và tỷ lệ churn hai nhóm gần nhau. Trong dữ liệu thật, AUC này thường > 0.6 , tức là dữ liệu thiếu có hệ thống.

3.4. Phân tích đơn biến: biến số

Thống kê mô tả cổ điển và bền vững

Khía cạnh	Thống kê cổ điển	Thống kê bền vững (robust)
Vị trí	Mean	Median , trimmed mean
Độ phân tán	Std	IQR , MAD = median(x – median)
Hình dạng	Skewness, kurtosis	Quantile skewness (Bowley)

Breakdown point của mean là 0%: một giá trị cực đoan đủ làm nó lệch tùy ý. Của median là 50%. Vì vậy dữ liệu có ngoại lai (như `monthly_charges`) nên được mô tả bằng thống kê bền vững.

```
def describe_numeric(s: pd.Series) -> pd.Series:
    s = s.dropna()
    q1, q2, q3 = s.quantile([0.25, 0.5, 0.75])
    mad = stats.median_abs_deviation(s, scale="normal") # scale="normal": ước lượng σ khi phân phối chuẩn
    return pd.Series({
        "mean": s.mean(), "median": q2, "std": s.std(), "mad_sigma": mad, "iqr": q3 - q1,
        "skew": stats.skew(s), "kurtosis_excess": stats.kurtosis(s),
        "bowley_skew": (q3 + q1 - 2 * q2) / (q3 - q1) if q3 > q1 else 0.0,
        "p01": s.quantile(0.01), "p99": s.quantile(0.99), "max": s.max(),
        "pct_zero": (s == 0).mean() * 100,
    })

print(train[num_cols].apply(describe_numeric).T.round(2).to_string())
```

Cách đọc:

- `|skew| > 1`: lệch mạnh, nên cân nhắc log hoặc Yeo-Johnson (Chương 6). `data_usage_gb` (log-normal) là ví dụ điển hình.
- `std >> mad_sigma` và kurtosis rất lớn: có ngoại lai kéo std lên. Đây là trường hợp của `monthly_charges` (kurtosis ≈ 179) do các giá trị bị nhân 8.
- `pct_zero` cao: phân phối *zero-inflated*, hoặc số 0 đang đóng vai trò "không có dữ liệu".

Hình dạng phân phối: histogram, ECDF, Q-Q

Số bin của histogram ảnh hưởng mạnh tới diễn giải. Quy tắc **Freedman-Diaconis**, độ rộng bin $h = 2 \text{ IQR} \cdot n^{-1/3}$, bền với ngoại lai (`bins="fd"` trong NumPy/matplotlib). **ECDF** không cần chọn bin và cho phép đọc trực tiếp phân vị.

```
fig, axes = plt.subplots(2, 3, figsize=(15, 8))
for ax, col in zip(axes[0], ["monthly_charges", "data_usage_gb", "tenure_months"]):
    sns.histplot(train[col].dropna(), bins="fd", ax=ax)
    ax.set_title(f"{col} | skew={stats.skew(train[col].dropna()):.2f}")
sns.ecdfplot(data=train, x="data_usage_gb", hue="churn", ax=axes[1, 0])
axes[1, 0].set_xscale("log")
stats.probplot(train["data_usage_gb"].dropna(), dist="norm", plot=axes[1, 1])
axes[1, 1].set_title("Q-Q (góc): cong -> không chuẩn")
stats.probplot(np.log(train["data_usage_gb"].dropna()), dist="norm", plot=axes[1, 2])
```

```
axes[1, 2].set_title("Q-Q log: gâh thắg -> log-normal")
plt.tight_layout()
```

Kiểm định phân phối chuẩn và giới hạn của nó

```
x = np.log(train["data_usage_gb"].dropna())
print("Shapiro-Wilk (n=500) p =", round(stats.shapiro(x.sample(500, random_state=0)).pvalue, 4))
print("D'Agostino K² (toàn bộ) p =", round(stats.normaltest(x).pvalue, 4))
ad = stats.anderson(x, dist="norm", method="interpolate") # SciPy >= 1.17: phải chọn cách tính p-value
print("Anderson-Darling A² =", round(ad.statistic, 3), "| p ≈", ad.pvalue)
```

[Kinh nghiệm] Với n lớn, kiểm định chuẩn **gần như luôn bác bỏ** vì độ lệch nhỏ không đáng kể vẫn có ý nghĩa thống kê. Với n nhỏ thì ngược lại, thiếu power. Hãy ra quyết định bằng **Q-Q plot + độ lớn skew/kurtosis**. p-value chỉ là thông tin phụ.

3.5. Phân tích đơn biến: biến phân loại

```
for col in cat_cols:
    vc = train[col].value_counts(dropna=False)
    share = vc / len(train)
    entropy = stats.entropy(share) # 0 = một mức; ln(K) = đều tuyệt đối
    rare = (share < 0.01).sum()
    print(f"\n{col}: {train[col].nunique()} mức | entropy={entropy:.2f}/{np.log(len(vc)):.2f}"
          f" | mức hiếm (<1%)={rare}")
    print(pd.DataFrame({"n": vc, "%": (share * 100).round(2)}).head(6).to_string())
```

Cần phát hiện: **sai chuẩn hóa** (Month-to-Month và month-to-month), **mức hiếm** (gom thành "other" hoặc dùng min_frequency của OneHotEncoder), **cardinality cao** (region có 30 mức, cần target/frequency encoding).

3.6. Quan hệ với target

Biến số và target nhị phân

```
def numeric_vs_binary(frame: pd.DataFrame, cols: list[str], target: str) -> pd.DataFrame:
    rows = []
    for c in cols:
        d = frame[[c, target]].dropna()
        pos, neg = d.loc[d[target] == 1, c], d.loc[d[target] == 0, c]
        auc = roc_auc_score(d[target], d[c])
        rows.append({
            "feature": c,
            "median_pos": pos.median(), "median_neg": neg.median(),
            "ks_stat": stats.ks_2samp(pos, neg).statistic, # khác biệt phân phối
            "auc_univariate": max(auc, 1 - auc), # sức xếp hạng đơn lẻ
            "direction": "+" if auc >= 0.5 else "-",
        })
    return pd.DataFrame(rows).sort_values("auc_univariate", ascending=False)

print(numeric_vs_binary(train, num_cols, "churn").round(3).to_string(index=False))
```

Tỷ lệ target theo nhóm, có khoảng tin cậy Wilson

Khoảng tin cậy Wald $\hat{p} \pm 1.96\sqrt{\hat{p}(1-\hat{p})/n}$ cho kết quả sai khi n nhỏ hoặc p gần 0/1. **Khoảng Wilson** (mặc định nên dùng, Agresti & Coull 1998) đáng tin hơn.

```
from statsmodels.stats.proportion import proportion_confint
```

```
def rate_by_group(frame: pd.DataFrame, col: str, target: str = "churn") -> pd.DataFrame:
    g = frame.groupby(col, observed=True, dropna=False)[target].agg(["sum", "count"])
    lo, hi = proportion_confint(g["sum"], g["count"], alpha=0.05, method="wilson")
    out = pd.DataFrame({"n": g["count"], "rate": g["sum"] / g["count"], "ci_low": lo, "ci_high": hi})
    out["lift"] = out["rate"] / frame[target].mean()
    return out.sort_values("rate", ascending=False)

print(rate_by_group(train, "contract").round(3))
print(rate_by_group(train, "payment_method").round(3))

# Biến số -> chia decile -> tỷ lệ churn (phát hiện quan hệ phi tuyến, ngưỡng)
binned = train.assign(bin=pd.qcut(train["tenure_months"], 10, duplicates="drop"))
fig, ax = plt.subplots()
r = rate_by_group(binned, "bin").sort_index()
ax.errorbar(range(len(r)), r["rate"], yerr=[r["ci_low"] - r["rate"], r["ci_high"] - r["rate"]],
            fmt="o-", capsize=3)
ax.set(title="Churn giảm đều theo tenure", xlabel="Decile tenure", ylabel="Tỷ lệ churn")
```

Weight of Evidence & Information Value

Kỹ thuật chuẩn trong credit scoring (Siddiqi, *Intelligent Credit Scoring*):

$$WoE_i = \ln \frac{\%Non-event_i}{\%Event_i}, \quad IV = \sum_i (\%Non-event_i - \%Event_i) \cdot WoE_i$$

```
def woe_iv(frame: pd.DataFrame, feature: str, target: str, bins: int = 10) -> tuple[pd.DataFrame, float]:
    x = frame[feature]
    if pd.api.types.is_numeric_dtype(x) and x.nunique() > bins:
        x = pd.qcut(x, q=bins, duplicates="drop")
    x = x.astype("object").where(x.notna(), "MISSING").astype(str) # missing là một nhóm riêng
    tab = pd.crosstab(x, frame[target])
    good = (tab[0] + 0.5) / (tab[0].sum() + 0.5) # +0.5: làm tròn tránh log(0)
    bad = (tab[1] + 0.5) / (tab[1].sum() + 0.5)
    table = pd.DataFrame({"n": tab.sum(axis=1), "event_rate": tab[1] / tab.sum(axis=1), "woe": np.log(good / bad)})
    return table, float(((good - bad) * table["woe"]).sum())

iv = pd.Series({c: woe_iv(train, c, "churn")[1] for c in num_cols + cat_cols}).sort_values(ascending=False)
print(iv.round(3))
```

IV	Sức mạnh dự đoán (Siddiqi)
< 0.02	Không có
0.02-0.1	Yếu
0.1-0.3	Trung bình
0.3-0.5	Mạnh
> 0.5	Đáng ngờ, kiểm tra leakage

Kết quả trên dữ liệu mô phỏng khớp với cơ chế sinh: `contract`, `tenure_months`, `support_calls` mạnh; `region` và `data_usage_gb` có IV ≈ 0 , đúng với việc chúng **không có tác động thật**. Đây là cách kiểm tra EDA trên dữ liệu biết trước đáp án.

Mutual Information: bắt quan hệ phi tuyến

[Docs] `mutual_info_classif` ước lượng MI bằng phương pháp k-láng giềng (Kraskov et al., 2004) cho biến liên tục. Cần khai báo `discrete_features` đúng, và kết quả có tính ngẫu nhiên nên phải đặt `random_state`.

```
from sklearn.feature_selection import mutual_info_classif

X_mi = train[num_cols].fillna(train[num_cols].median())
mi = mutual_info_classif(X_mi, train["churn"], discrete_features=[False, True, False, False, True, False],
```

```
n_neighbors=3, random_state=0)
print(pd.Series(mi, index=num_cols).sort_values(ascending=False).round(4))
```

3.7. Quan hệ giữa các feature

Cặp biến	Thước đo	Ghi chú
Số – Số, tuyến tính	Pearson r	Nhạy ngoại lai; chỉ đo quan hệ tuyến tính
Số – Số, đơn điệu	Spearman ρ , Kendall τ	Dùng thứ hạng, bền vững; nên dùng mặc định
Phân loại – Phân loại	Cramér's V (hiệu chỉnh bias)	Dựa trên χ^2
Số – Phân loại	Correlation ratio η	η^2 = phương sai giữa nhóm / tổng phương sai
Mọi loại, phi tuyến	Mutual information, ϕK (phik)	

```
def cramers_v(x: pd.Series, y: pd.Series) -> float:
    """Cramér's V với hiệu chỉnh bias (Bergsma, 2013)."""
    tab = pd.crosstab(x, y)
    chi2 = stats.chi2_contingency(tab, correction=False)[0]
    n = tab.to_numpy().sum()
    r, k = tab.shape
    phi2 = max(0.0, chi2 / n - (k - 1) * (r - 1) / (n - 1))
    rc, kc = r - (r - 1) ** 2 / (n - 1), k - (k - 1) ** 2 / (n - 1)
    return float(np.sqrt(phi2 / max(min(kc - 1, rc - 1), 1e-12)))

def correlation_ratio(categories: pd.Series, values: pd.Series) -> float:
    d = pd.DataFrame({"c": categories, "v": values}).dropna()
    grand = d["v"].mean()
    between = d.groupby("c", observed=True)["v"].agg(lambda g: len(g) * (g.mean() - grand) ** 2).sum()
    total = ((d["v"] - grand) ** 2).sum()
    return float(np.sqrt(between / total)) if total else 0.0

corr = train[num_cols].corr(method="spearman")
fig, ax = plt.subplots(figsize=(7, 6))
sns.heatmap(corr, mask=np.triu(np.ones_like(corr, dtype=bool)), annot=True, fmt=".2f",
            cmap="RdBu_r", center=0, vmin=-1, vmax=1, square=True, ax=ax)
ax.set_title("Spearman: total_charges gắn chặt với tenure")
print("Cramér's V(contract, payment_method) =", round(cramers_v(train["contract"], train["payment_method"]), 3))
print("η(contract -> monthly_charges) =", round(correlation_ratio(train["contract"],
train["monthly_charges"]), 3))
```

Đa cộng tuyến: VIF và phân cụm feature

$$VIF_j = \frac{1}{1 - R_j^2}$$

trong đó R_j^2 là R^2 khi hồi quy feature j theo các feature còn lại. $VIF > 5-10$ là đa cộng tuyến đáng kể.

```
from scipy.cluster import hierarchy
from scipy.spatial.distance import squareform
from statsmodels.stats.outliers_influence import variance_inflation_factor
from statsmodels.tools.tools import add_constant

X_vif = add_constant(train[num_cols].dropna())
vif = pd.Series([variance_inflation_factor(X_vif.values, i) for i in range(X_vif.shape[1])],
                index=X_vif.columns).drop("const")
print(vif.sort_values(ascending=False).round(2))

# [Docs] scikit-learn example "Permutation Importance with Multicollinear or Correlated Features":
# phân cụm phân cấp trên khoảng cách 1 - |Spearman|, giữ 1 feature mỗi cụm
dist = 1 - np.abs(corr.to_numpy())
np.fill_diagonal(dist, 0)
linkage = hierarchy.ward(squareform(dist, checks=False))
```

```
clusters = hierarchy.fcluster(linkage, t=0.5, criterion="distance")
print(dict(zip(num_cols, clusters)))
```

$\text{total_charges} \approx \text{monthly_charges} \times \text{tenure_months}$ nên VIF cao. Với **mô hình tuyến tính**, đa cộng tuyến làm hệ số không ổn định và khó diễn giải. Với **mô hình cây**, dự đoán ít bị ảnh hưởng, nhưng **feature importance bị chia nhỏ** giữa các biến tương quan (Chương 10).

3.8. Ngoại lai (Outliers)

Phương pháp	Loại	Quy tắc	Ghi chú
IQR (Tukey fences)	Đơn biến	$x < Q_1 - 1.5 IQR$ hoặc $x > Q_3 + 1.5 IQR$	Không giả định phân phối
Z-score	Đơn biến	$ z > 3$	Mean/std bị chính ngoại lai làm sai lệch (<i>masking</i>)
Robust z (MAD)	Đơn biến	$0.6745 x - \bar{x} / \text{MAD} > 3.5$	Iglewicz & Hoaglin (1993)
Mahalanobis bền vững	Đa biến	Khoảng cách với hiệp phương sai MCD	Giả định elip
Isolation Forest	Đa biến	Điểm dễ bị cô lập bằng cây ngẫu nhiên	Không giả định phân phối, nhanh
Local Outlier Factor	Đa biến	Mật độ cục bộ thấp hơn láng giềng	Tốt khi có nhiều cụm

```
from sklearn.covariance import MinCovDet
from sklearn.ensemble import IsolationForest
from sklearn.neighbors import LocalOutlierFactor

def univariate_outliers(s: pd.Series) -> dict:
    s = s.dropna()
    q1, q3 = s.quantile([0.25, 0.75])
    iqr = q3 - q1
    med, mad = s.median(), stats.median_abs_deviation(s)
    return {
        "iqr_1.5": int(((s < q1 - 1.5 * iqr) | (s > q3 + 1.5 * iqr)).sum()),
        "z_3": int((np.abs(stats.zscore(s)) > 3).sum()),
        "robust_z_3.5": int((0.6745 * np.abs(s - med) / (mad or 1e-9) > 3.5).sum()),
    }

print(pd.DataFrame({c: univariate_outliers(train[c]) for c in num_cols}).T)

Xo = train[["monthly_charges", "tenure_months", "support_calls"]].dropna()
mcd = MinCovDet(random_state=0).fit(Xo)
d2 = mcd.mahalanobis(Xo) # bình phương khoảng cách Mahalanobis bền vững
flag_mcd = d2 > stats.chi2.ppf(0.999, df=Xo.shape[1]) # ngưỡng  $\chi^2$  với 3 bậc tự do
# contamination="auto" dùng ngưỡng của bài báo gốc -> thường gán cờ quá nhiều; đặt theo tỷ lệ kỳ vọng
flag_if = IsolationForest(contamination=0.005, random_state=0).fit_predict(Xo) == -1
flag_lof = LocalOutlierFactor(n_neighbors=35).fit_predict(Xo) == -1
print({"mcd": int(flag_mcd.sum()), "isolation_forest": int(flag_if.sum()), "lof": int(flag_lof.sum())})
print("Ngoại lai cài sẵn (>500) trong train:", int((Xo["monthly_charges"] > 500).sum()),
      "| bị MCD bắt:", int((flag_mcd & (Xo["monthly_charges"] > 500).to_numpy()).sum()))
```

Quy trình ra quyết định với ngoại lai:

1. **Có thể xảy ra về mặt vật lý/nghiệp vụ không?** Không (tuổi 250) thì là lỗi, chuyển thành NaN.
2. **Có thể xảy ra nhưng hiếm?** Hỏi domain expert. Đó có thể là khách VIP hoặc gian lận, tức là **tín hiệu quý**.
3. **Mô hình có nhạy với ngoại lai không?** Mô hình tuyến tính/KNN thì clip (winsorize) hoặc biến đổi. Mô hình cây ít nhạy với ngoại lai ở X.
4. **Không bao giờ** xóa ngoại lai trên tập test để "làm đẹp" kết quả.

3.9. Chiều thời gian

```
by_month = (raw.set_index("signup_date").resample("MS")
            .agg(n=("churn", "size"), churn_rate=("churn", "mean"),
```

```

        median_charges=("monthly_charges", "median"),
        pct_m2m=("contract", lambda s: (s.str.lower() == "month-to-month").mean()))
fig, axes = plt.subplots(3, 1, figsize=(11, 8), sharex=True)
for ax, col in zip(axes, ["churn_rate", "median_charges", "pct_m2m"]):
    ax.plot(by_month.index, by_month[col], marker="o", ms=3)
    ax.set_ylabel(col)
axes[0].set_title("Theo dõi target và feature theo thời gian: nền tảng cho out-of-time validation")
plt.tight_layout()
print(by_month[["churn_rate"]].describe().T.round(3))

```

Nếu tỷ lệ target hoặc phân phối feature thay đổi rõ theo thời gian, **bắt buộc** chia dữ liệu theo thời gian (Chương 7) và thiết lập giám sát drift (Chương 12).

3.10. Phát hiện leakage ngay từ EDA

Dấu hiệu nghi ngờ:

- Một feature đơn lẻ có AUC > 0.9 hoặc IV > 0.5.
- Feature được ghi nhận **sau** sự kiện target: `cancellation_reason`, `refund_amount`, `last_status = "closed"`.
- ID, số thứ tự hoặc timestamp tương quan với target (dữ liệu được sắp xếp theo nhãn).

Minh họa: thêm một feature rò rỉ `days_to_contract_end_at_export`, được tính tại thời điểm **xuất dữ liệu** (sau khi khách đã hủy).

```

leaky = train.copy()
rng = np.random.default_rng(0)
# Khách đã hủy có "số ngày còn lại của hợp đồng" = 0 vì hệ thống đóng hợp đồng khi hủy
leaky["days_to_contract_end_at_export"] = np.where(leaky["churn"] == 1, 0, rng.integers(1, 365, len(leaky)))
leaky["row_number"] = np.arange(len(leaky))

single_auc = numeric_vs_binary(leaky, num_cols + ["days_to_contract_end_at_export", "row_number"], "churn")
print(single_auc[["feature", "auc_univariate"]].head(4).round(3).to_string(index=False))
# AUC = 1.0 -> không có thật ở thời điểm dự đoán: loại bỏ và truy nguyên cách feature được tạo

```

3.11. Adversarial validation: train và test có cùng phân phối?

Huấn luyện một bộ phân loại phân biệt **dòng thuộc train** với **dòng thuộc test**. AUC $\approx$ 0.5 nghĩa là hai tập giống nhau. AUC càng cao, phân phối càng khác. Feature importance của bộ phân loại chỉ ra **feature nào trôi**.

```

from lightgbm import LGBMClassifier
from sklearn.model_selection import StratifiedKFold

def adversarial_validation(a: pd.DataFrame, b: pd.DataFrame, features: list[str]) -> tuple[float, pd.Series]:
    X = pd.concat([a[features], b[features]], ignore_index=True)
    for c in X.columns:
        if not pd.api.types.is_numeric_dtype(X[c]):
            X[c] = X[c].astype("category")
    y = np.r_[np.zeros(len(a)), np.ones(len(b))]
    clf = LGBMClassifier(n_estimators=200, learning_rate=0.05, num_leaves=15, importance_type="gain",
                        verbose=-1, random_state=0)
    p = cross_val_predict(clf, X, y, cv=StratifiedKFold(5, shuffle=True, random_state=0),
                        method="predict_proba")[:, 1]
    imp = pd.Series(clf.fit(X, y).feature_importances_, index=features).sort_values(ascending=False)
    return roc_auc_score(y, p), imp

feats = ["age", "tenure_months", "monthly_charges", "support_calls", "data_usage_gb", "contract", "payment_method"]
perm = np.random.default_rng(1).permutation(len(train)) # hai nửa RỒI NHAU của train
half_a, half_b = train.iloc[perm[: len(train) // 2]], train.iloc[perm[len(train) // 2:]]
auc_same, _ = adversarial_validation(half_a, half_b, feats)
with_time = [*feats, "signup_ordinal"]
tr_t = train.assign(signup_ordinal=train["signup_date"].map(pd.Timestamp.toordinal))
te_t = test.assign(signup_ordinal=test["signup_date"].map(pd.Timestamp.toordinal))

```

```
auc_time, imp = adversarial_validation(tr_t, te_t, with_time)
print(f"AUC (hai nửa ngẫu nhiên của train) = {auc_same:.3f}")
print(f"AUC (train vs test theo thời gian, có ngày đăng ký) = {auc_time:.3f} | feature trôi nhất: {imp.index[0]}")
```

Feature trôi theo thời gian (như ngày đăng ký tuyệt đối) cho AUC ≈ 1 . Đó là lý do **không đưa timestamp thô vào mô hình**. Thay vào đó dùng feature tương đối, đo tới **thời điểm chấm điểm của từng dòng**, như `tenure_months` (xem bẫy "ngày tham chiếu cố định" ở Chương 5.6).

3.12. Báo cáo tự động: điểm khởi đầu, không phải điểm kết thúc

```
from ydata_profiling import ProfileReport

ProfileReport(train, title="Churn EDA", minimal=len(train) > 100_000,
              explorative=True).to_file("reports/eda_profile.html")
# sweetviz.compare([train, "train"], [test, "test"], target_feat="churn"): so sánh 2 tập
```

3.13. Mẫu báo cáo EDA gửi stakeholder

```
## Tóm tắt EDA: Churn (dữ liệu train đến 2023-09)
**Dữ liệu:** ~13 000 khách, 12 cột; 100 dòng trùng chính xác (đã loại). Tỷ lệ churn train 16–17%.
**Chất lượng:**
- data_usage_gb thiếu 8%, age 5%, payment_method 3%; không có bằng chứng thiếu có hệ thống (AUC  $\approx 0.5$ ).
- Một số bản ghi monthly_charges gấp ~8 lần bình thường (lỗi nhập liệu, ~30 trên toàn bộ dữ liệu): winsorize.
- Một số bản ghi contract sai chuẩn hóa ("Month-to-Month"): chuẩn hóa chữ thường.
**Tín hiệu:** contract (IV $\approx 0.3$ ), tenure_months (IV $\approx 0.3$ ), support_calls (IV $\approx 0.15$ ).
region và data_usage_gb gần như không có tín hiệu.
**Rủi ro:** total_charges đa cộng tuyến với tenure  $\times$  monthly_charges; ngày đăng ký trôi theo thời gian.
**Đề xuất:** out-of-time split; winsorize + median impute; one-hot cho contract/payment;
target encoding có cross-fitting cho region; không dùng timestamp thô làm feature.
```

Checklist Chương 3

- ☐ EDA chi tiết làm trên tập train; test chỉ dùng cho adversarial validation.
- ☐ Đã kiểm tra khóa, trùng lặp, kiểu dữ liệu, cột hằng/gần hằng.
- ☐ Missing: lượng, mẫu hình, quan hệ với target, bằng chứng về cơ chế.
- ☐ Phân phối mô tả bằng thống kê bền vững; quyết định biến đổi dựa trên Q-Q và skew.
- ☐ Quan hệ với target (rate + CI, IV, MI, AUC đơn biến) và giữa các feature (Spearman, Cramér's V, VIF).
- ☐ Ngoại lai được phân loại: lỗi hay hiện tượng thật (đã hỏi domain expert).
- ☐ Đã kiểm tra drift theo thời gian, leakage, adversarial validation.
- ☐ Có báo cáo tóm tắt kèm quyết định cho bước tiếp xử lý.

Tài liệu tham khảo Chương 3

- Tukey, J. W. (1977). *Exploratory Data Analysis*. Addison-Wesley.
- Rubin, D. B. (1976). *Inference and missing data*. Biometrika 63(3).
- van Buuren, S. (2018). *Flexible Imputation of Missing Data*, 2nd ed. (stefvanbuuren.name/fimd)
- scikit-learn User Guide: *Mutual information, Novelty and Outlier Detection, Covariance estimation*; Example: *Permutation Importance with Multicollinear or Correlated Features*.
- SciPy reference: `scipy.stats` (shapiro, normaltest, anderson, ks_2samp, median_abs_deviation).
- statsmodels: `proportion_confint`, `variance_inflation_factor`.
- Agresti, A. & Coull, B. (1998). *Approximate is better than "exact" for interval estimation of binomial proportions*. The American Statistician 52(2).
- Iglewicz, B. & Hoaglin, D. (1993). *How to Detect and Handle Outliers*. ASQC.
- Siddiqi, N. (2017). *Intelligent Credit Scoring*, 2nd ed. Wiley.
- Harvard CS109 *Data Science*, các bài EDA & Visualization. • Kaggle Learn: *Data Leakage*.

CHƯƠNG 4. PHÂN TÍCH DỮ LIỆU & THỐNG KÊ SUY LUẬN

Mục tiêu chương: EDA giúp *nhìn thấy* mẫu hình. Thống kê suy luận giúp *khẳng định* mẫu hình đó không do ngẫu nhiên, *định lượng* độ chắc chắn và, khi có thiết kế phù hợp, *trả lời câu hỏi nhân quả*. Đây là kỹ năng phân biệt Data Scientist với người chỉ "chạy mô hình".

4.1. Bốn cấp độ phân tích

Cấp độ	Câu hỏi	Kỹ thuật	Chương
Mô tả (Descriptive)	Chuyện gì đã xảy ra?	Thống kê mô tả, KPI, cohort	3, 4.2
Chẩn đoán (Diagnostic)	Tại sao xảy ra?	Kiểm định, hồi quy, phân tích phân khúc	4.3-4.7
Dự đoán (Predictive)	Chuyện gì sẽ xảy ra?	Machine Learning, chuỗi thời gian	8
Đề xuất (Prescriptive)	Nên làm gì?	A/B test, suy luận nhân quả, uplift, tối ưu hóa	1.7, 4.6, 4.8

Thiết lập

```
import matplotlib.pyplot as plt
import numpy as np
import pandas as pd
import statsmodels.api as sm
import statsmodels.formula.api as smf
from scipy import stats

from churn.data.synthetic import make_churn_data
from churn.features.clean import clean_churn

pd.set_option("display.width", 160)
pd.set_option("display.max_columns", 12)
df = clean_churn(make_churn_data(n=20_000, seed=42))
rng = np.random.default_rng(2026)
print(df.shape, round(df["churn"].mean(), 4))
```

4.2. Nền tảng: phân phối mẫu, sai số chuẩn, khoảng tin cậy

- **Tham số (parameter)** là đại lượng của quần thể, ví dụ tỷ lệ churn thật p . **Thống kê (statistic)** tính từ mẫu, ví dụ $\hat{p}$.
- **Phân phối mẫu (sampling distribution)** là phân phối của thống kê qua các lần lấy mẫu lặp lại. Độ lệch chuẩn của nó là **sai số chuẩn (SE)**. Với trung bình: $SE = \sigma/\sqrt{n}$.
- **Định lý giới hạn trung tâm (CLT)**: với n đủ lớn, trung bình mẫu xấp xỉ phân phối chuẩn **bất kể phân phối gốc**, miễn là phương sai hữu hạn. Phân phối càng lệch thì càng cần n lớn.

```
population = rng.lognormal(mean=2.5, sigma=0.8, size=1_000_000) # rất lệch phải (data usage)
for n in [5, 30, 200]:
    means = rng.choice(population, size=(5_000, n)).mean(axis=1)
    print(f"n={n:3d} skew(mean)={stats.skew(means):5.2f} SE thực nghiệm={means.std():.3f}"
          f" SE lý thuyết={population.std() / np.sqrt(n):.3f}")
```

Diễn giải đúng khoảng tin cậy 95% (tần suất): nếu lặp lại quy trình lấy mẫu và tính khoảng nhiều lần, **95% các khoảng** sẽ chứa tham số thật. Câu "tham số có 95% xác suất nằm trong khoảng này" là sai theo trường phái tần suất. Đó là cách diễn giải của *khoảng khả tín (credible interval)* Bayes.

```

true_p, n, covered = 0.165, 400, 0
for _ in range(2_000):
    sample = rng.random(n) < true_p
    p_hat = sample.mean()
    se = np.sqrt(p_hat * (1 - p_hat) / n)
    covered += (p_hat - 1.96 * se <= true_p <= p_hat + 1.96 * se)
print("Tỷ lệ khoảng Wald 95% chứa p thật:", covered / 2_000)

```

p-value: định nghĩa và sáu nguyên tắc của ASA (2016)

p-value là xác suất, **giả sử H_0 đúng**, quan sát được thống kê kiểm định cực đoan bằng hoặc hơn giá trị thực tế. Tuyên bố của Hiệp hội Thống kê Hoa Kỳ (Wasserstein & Lazar, 2016):

1. p-value cho biết dữ liệu **không tương thích** với một mô hình thống kê cụ thể đến mức nào.
2. p-value **không** đo xác suất giả thuyết đúng, cũng không đo xác suất dữ liệu do ngẫu nhiên tạo ra.
3. Kết luận khoa học và quyết định kinh doanh **không nên** chỉ dựa trên việc p có vượt ngưỡng hay không.
4. Suy luận đúng đòi hỏi **báo cáo đầy đủ và minh bạch**. Không được chọn lọc kết quả (p-hacking).
5. p-value **không** đo độ lớn hiệu ứng hay tầm quan trọng của kết quả.
6. Tự thân p-value không phải là thước đo bằng chứng tốt cho một mô hình hay giả thuyết.

	H_0 đúng	H_0 sai
Bác bỏ H_0	Sai lầm loại I (α), "false positive"	Đúng. Power = $1 - \beta$
Không bác bỏ	Đúng	Sai lầm loại II (β), "false negative"

4.3. Chọn kiểm định và kiểm tra giả định

Mục đích	Dữ liệu	Tham số	Phi tham số / thay thế
2 nhóm độc lập	Số	Welch's t-test (mặc định, không giả định phương sai bằng nhau)	Mann-Whitney U
2 nhóm ghép cặp	Số	Paired t-test	Wilcoxon signed-rank
≥ 3 nhóm	Số	One-way ANOVA / Welch ANOVA	Kruskal-Wallis
2 biến phân loại	Bảng tần số	χ^2 test of independence	Fisher exact (ô kỳ vọng < 5)
2 tỷ lệ	Nhị phân	z-test cho tỷ lệ	Fisher exact, Barnard
Ghép cặp nhị phân	Trước/sau	—	McNemar
Phân phối chuẩn?	Số	Shapiro-Wilk, D'Agostino	Q-Q plot
2 phân phối giống nhau?	Số	—	Kolmogorov-Smirnov, Anderson-Darling k-sample
Phương sai bằng nhau?	Số	Bartlett (nhạy với phi chuẩn)	Levene / Brown-Forsythe
Tương quan	Số	Pearson	Spearman, Kendall

Lưu ý về Mann-Whitney U. Kiểm định này **không phải** "t-test cho median". Nó kiểm định $P(X > Y) = 0.5$ (stochastic equality). Chỉ khi hai phân phối có cùng hình dạng thì nó mới trở thành kiểm định về vị trí (median).

```

churn_yes = df.loc[df["churn"] == 1, "monthly_charges"].dropna().to_numpy()
churn_no = df.loc[df["churn"] == 0, "monthly_charges"].dropna().to_numpy()

print("Levene (phương sai bằng nhau?) p =", f"{stats.levene(churn_yes, churn_no, center='median').pvalue:.3g}")
welch = stats.ttest_ind(churn_yes, churn_no, equal_var=False)
ci = welch.confidence_interval(confidence_level=0.95)
# CI cho hiệu hai trung bình

```

```
mw = stats.mannwhitneyu(churn_yes, churn_no, alternative="two-sided")
print(f"Welch t={welch.statistic:.2f} p={welch.pvalue:.2e} | Δmean 95% CI=({ci.low:.2f}, {ci.high:.2f})")
print(f"Mann-Whitney U p={mw.pvalue:.2e}")
```

Effect size: độ lớn hiệu ứng (bắt buộc báo cáo cùng p-value)

Bối cảnh	Effect size	Diễn giải quy ước (Cohen, 1988)
Hai trung bình	Cohen's d, Hedges' g (hiệu chỉnh mẫu nhỏ)	0.2 nhỏ · 0.5 vừa · 0.8 lớn
Hai phân phối (phi tham số)	Rank-biserial $r = 2 \cdot \text{AUC} - 1$	0.1 nhỏ · 0.3 vừa · 0.5 lớn
Hai tỷ lệ	Chênh lệch tuyệt đối, relative risk , odds ratio	Theo ngữ cảnh kinh doanh
Bảng phân loại	Cramér's V	Phụ thuộc bậc tự do
ANOVA	η^2 , ω^2 (ít bias hơn)	0.01 · 0.06 · 0.14

```
def hedges_g(a: np.ndarray, b: np.ndarray) -> float:
    na, nb = len(a), len(b)
    pooled = np.sqrt(((na - 1) * a.var(ddof=1) + (nb - 1) * b.var(ddof=1)) / (na + nb - 2))
    d = (a.mean() - b.mean()) / pooled
    return d * (1 - 3 / (4 * (na + nb) - 9)) # hiệu chỉnh mẫu nhỏ

rank_biserial = 1 - 2 * mw.statistic / (len(churn_yes) * len(churn_no))
print(f"Hedges' g = {hedges_g(churn_yes, churn_no):.3f} | rank-biserial = {abs(rank_biserial):.3f}")
# p rất nhỏ nhưng g ~ 0.2: khác biệt CÓ THẬT nhưng NHỎ -> p-value không nói về độ lớn
```

Bảng phân loại và tỷ lệ

```
from statsmodels.stats.proportion import confint_proportions_2indep, proportions_ztest

table = pd.crosstab(df["contract"], df["churn"])
chi2, p_chi, dof, expected = stats.chi2_contingency(table)
print(f"χ²={chi2:.1f}, dof={dof}, p={p_chi:.2e}; ô kỳ vọng nhỏ nhất = {expected.min():.1f}")

cash = (df["payment_method"] == "cash").to_numpy()
count = np.array([df.loc[cash, "churn"].sum(), df.loc[~cash, "churn"].sum()])
nobs = np.array([cash.sum(), (~cash).sum()])
z, p_prop = proportions_ztest(count, nobs)
lo, hi = confint_proportions_2indep(count[0], nobs[0], count[1], nobs[1], method="newcomb")
rr = (count[0] / nobs[0]) / (count[1] / nobs[1])
print(f"Cash vs khác: Δp 95% CI=({lo:.3f}, {hi:.3f}), RR={rr:.2f}, p={p_prop:.2e}")

kw = stats.kruskal(*[g["support_calls"].to_numpy() for _, g in df.groupby("contract")])
print("Kruskal-Wallis support_calls ~ contract: p =", round(kw.pvalue, 3))
```

4.4. Kiểm định nhiều giả thuyết

Kiểm định 20 giả thuyết đúng- H_0 với $\alpha = 0.05$ thì xác suất có **ít nhất một** "phát hiện" giả là $1 - 0.95^{20} \approx 64\%$.

Mục tiêu kiểm soát	Phương pháp	Khi nào dùng
FWER: $P(\geq 1 \text{ sai lầm loại I})$	Bonferroni (α/m), Holm (luôn mạnh hơn Bonferroni)	Ít giả thuyết, sai lầm rất đắt (y tế, pháp lý)
FDR: tỷ lệ kỳ vọng phát hiện sai	Benjamini-Hochberg , Benjamini-Yekutieli (phụ thuộc tùy ý)	Sàng lọc nhiều feature/phân khúc

```
from statsmodels.stats.multitest import multipletests

# Mô phỏng: 200 phân khúc, chỉ 10 phân khúc có hiệu ứng thật (p nhỏ), 190 phân khúc H0 đúng
p_values = np.r_[rng.uniform(0, 0.0004, 10), rng.uniform(0, 1, 190)]
is_true = np.r_[np.ones(10, bool), np.zeros(190, bool)]
print(f"{'none':10s}: phát hiện {(p_values < 0.05).sum():3d} | sai {(p_values < 0.05) & ~is_true).sum():3d}")
```

```
for method in ["bonferroni", "holm", "fdr_bh"]:
    reject = multipletests(p_values, alpha=0.05, method=method)[0]
    print(f"{method:10s}: phát hiện {reject.sum():3d} | sai {(reject & ~is_true).sum()} | đúng {(reject & is_true).sum()}/10")
```

4.5. Phương pháp lấy mẫu lại: bootstrap và kiểm định hoán vị

Bootstrap (Efron, 1979) ước lượng phân phối mẫu của *bất kỳ* thống kê nào bằng cách lấy mẫu có hoàn lại từ dữ liệu. **[Docs]** `scipy.stats.bootstrap` mặc định dùng phương pháp **BCa** (bias-corrected and accelerated), chính xác hơn phương pháp percentile khi phân phối lệch.

```
res = stats.bootstrap((churn_yes,), np.median, n_resamples=5_000, method="BCa", rng=rng)
print("95% CI BCa cho median cước (khách churn):", np.round(res.confidence_interval, 2))

def median_diff(a, b, axis=-1):
    return np.median(a, axis=axis) - np.median(b, axis=axis)

# Hai mẫu lớn: BCa cần jackknife O(n) lần tính thống kê -> rất chậm; percentile đủ tốt khi n lớn
res2 = stats.bootstrap((churn_yes, churn_no), median_diff, n_resamples=2_000, method="percentile", rng=rng)
print("95% CI chênh lệch median:", np.round(res2.confidence_interval, 2))
```

Kiểm định hoán vị (permutation test) kiểm định H_0 "hai nhóm có cùng phân phối" một cách chính xác, không cần giả định phân phối: xáo trộn nhãn nhóm nhiều lần để tạo phân phối null.

```
def mean_diff(a, b, axis=-1):
    return np.mean(a, axis=axis) - np.mean(b, axis=axis)

small_a, small_b = churn_yes[:60], churn_no[:80] # mẫu nhỏ: minh họa kiểm định chính xác
perm = stats.permutation_test((small_a, small_b), mean_diff, n_resamples=9_999,
                              alternative="two-sided", rng=rng)
print(f"Permutation test: Δmean={perm.statistic:.2f}, p={perm.pvalue:.4f}")
```

4.6. A/B testing chuẩn ngành

[Sách] Kohavi, Tang & Xu, *Trustworthy Online Controlled Experiments* (2020). Đây là chuẩn mực được Microsoft, Google, Booking.com, LinkedIn áp dụng.

Thiết kế

Thành phần	Quyết định	Ví dụ Churn
OEC (Overall Evaluation Criterion)	Một metric chính phản ánh giá trị dài hạn	Tỷ lệ giữ chân 60 ngày
Đơn vị ngẫu nhiên hóa	Người dùng, phiên, cụm (địa bàn)	Khách hàng
Guardrail metrics	Không được xấu đi	Doanh thu/khách, khiếu nại, hủy do bị làm phiền
MDE (Minimum Detectable Effect)	Hiệu ứng nhỏ nhất có ý nghĩa kinh doanh	Giảm churn 2 điểm %
α , power	Thường 0.05 và 0.8	
Thời lượng	Đủ cỡ mẫu và đủ chu kỳ tuần	≥ 2 tuần đầy đủ

Bước 1: tính cỡ mẫu trước khi chạy

$$n \approx \frac{2(z_{1-\alpha/2} + z_{1-\beta})^2 \bar{p}(1-\bar{p})}{\delta^2}$$

```
from statsmodels.stats.power import NormalIndPower
from statsmodels.stats.proportion import proportion_effectsize

baseline, mde = 0.18, 0.02
effect = proportion_effectsize(baseline, baseline - mde) # Cohen's h
n_per_group = NormalIndPower().solve_power(effect_size=effect, alpha=0.05, power=0.8, ratio=1.0)
print(f"Cần ~{int(np.ceil(n_per_group))}, khách mỗi nhóm để phát hiện giảm {mde:.0%} từ {baseline:.0%}")
for m in [0.01, 0.02, 0.04]:
    n_m = NormalIndPower().solve_power(proportion_effectsize(baseline, baseline - m), alpha=0.05, power=0.8)
    print(f" MDE={m:.0%} -> n/nhóm={int(np.ceil(n_m))}") # MDE giảm 1/2 -> n tăng ~4 lần
```

Bước 2: kiểm tra tính hợp lệ (trustworthiness)

- **A/A test:** chạy hai nhóm giống hệt nhau. Tỷ lệ "có ý nghĩa" phải xấp xỉ α . Nếu không, hạ tầng thí nghiệm có lỗi.
- **Sample Ratio Mismatch (SRM):** tỷ lệ phân bổ thực tế lệch khỏi thiết kế là dấu hiệu lỗi nghiêm trọng (bot, lỗi redirect, lỗi log). **Không** phân tích tiếp khi có SRM.

```
def srm_pvalue(n_control: int, n_treatment: int, expected_ratio: float = 0.5) -> float:
    total = n_control + n_treatment
    return stats.chisquare([n_control, n_treatment],
                           f_exp=[total * (1 - expected_ratio), total * expected_ratio]).pvalue

print("SRM p (50 000 vs 50 300):", round(srm_pvalue(50_000, 50_300), 4)) # ôi
print("SRM p (50 000 vs 51 200):", f"{srm_pvalue(50_000, 51_200):.2e}") # p < 0.001 -> dừng, điều tra
```

Bước 3: vấn đề "nhìn trộm" (peeking)

Xem kết quả mỗi ngày và dừng ngay khi $p < 0.05$ sẽ **lạm phát sai lầm loại I** rất nhiều. Mô phỏng A/A test (không có hiệu ứng thật):

```
def peeking_false_positive_rate(n_sims: int = 1_000, n_days: int = 20, users_per_day: int = 500) -> float:
    false_pos = 0
    for _ in range(n_sims):
        a = rng.random((n_days, users_per_day)) < 0.18
        b = rng.random((n_days, users_per_day)) < 0.18 # cùng tỷ lệ: H0 đúng
        ca, cb = a.sum(axis=1).cumsum(), b.sum(axis=1).cumsum()
        n = users_per_day * np.arange(1, n_days + 1)
        pooled = (ca + cb) / (2 * n)
        z = (ca / n - cb / n) / np.sqrt(pooled * (1 - pooled) * 2 / n)
        false_pos += np.any(np.abs(z) > 1.96) # dừng ngay khi "có ý nghĩa"
    return false_pos / n_sims

print("Tỷ lệ false positive khi nhìn trộm 20 lần:", peeking_false_positive_rate()) # >> 0.05
```

Giải pháp: cố định cỡ mẫu và chỉ phân tích một lần; hoặc dùng **sequential testing** có kiểm soát α (alpha spending O'Brien-Fleming, mSPRT / always-valid p-values). Các nền tảng Optimizely, Eppo, Statsig đều có sẵn những phương pháp này.

Bước 4: phân tích, với CUPED để giảm phương sai

CUPED (Deng et al., 2013, Microsoft) dùng covariate **trước thí nghiệm** X (không bị can thiệp ảnh hưởng) để giảm phương sai:

$$Y^{cuped} = Y - \theta(X - \bar{X}), \quad \theta = \frac{\text{cov}(X, Y)}{\text{var}(X)}, \quad \text{Var}(Y^{cuped}) = \text{Var}(Y)(1 - \rho^2)$$

```

n_exp = 8_000
pre_usage = rng.gamma(4, 5, 2 * n_exp) # sử dụng trước thí nghiệm
group = np.r_[np.zeros(n_exp), np.ones(n_exp)]
post_usage = 0.8 * pre_usage + rng.normal(0, 4, 2 * n_exp) + 0.6 * group # hiệu ứng thật = +0.6

theta = np.cov(pre_usage, post_usage)[0, 1] / np.var(pre_usage, ddof=1)
y_cuped = post_usage - theta * (pre_usage - pre_usage.mean())
for name, y in [("raw", post_usage), ("CUPED", y_cuped)]:
    t = stats.ttest_ind(y[group == 1], y[group == 0], equal_var=False)
    ci = t.confidence_interval()
    print(f"{name:6s} Δ={y[group == 1].mean() - y[group == 0].mean():.3f} "
          f"CI={ci.low:.3f}, {ci.high:.3f} p={t.pvalue:.2e}")

```

Phân tích Bayes (Beta-Binomial)

```

def bayes_ab(conv_a: int, n_a: int, conv_b: int, n_b: int, draws: int = 200_000) -> dict:
    a = rng.beta(1 + conv_a, 1 + n_a - conv_a, draws) # prior Beta(1,1)
    b = rng.beta(1 + conv_b, 1 + n_b - conv_b, draws)
    lift = b / a - 1
    return {"P(B<A)": float((b < a).mean()), # churn: B tốt hơn nếu thấp hơn
            "expected_rel_change": float(lift.mean()),
            "95% credible": np.round(np.quantile(lift, [0.025, 0.975]), 3).tolist()}

print(bayes_ab(conv_a=1_800, n_a=10_000, conv_b=1_650, n_b=10_000))

```

Metric dạng tỷ số (doanh thu/piền, CTR theo trang) có đơn vị phân tích khác đơn vị ngẫu nhiên hóa. Khi đó phương sai phải tính bằng **delta method** hoặc bootstrap theo người dùng. Dùng công thức nhị thức thông thường sẽ cho CI quá hẹp.

4.7. Hồi quy để suy luận (statsmodels)

Mô hình ML tối ưu **dự đoán**. Hồi quy thống kê tối ưu **diễn giải**: hệ số, sai số chuẩn, CI, kiểm định.

Hồi quy logistic: odds ratio và marginal effects

```

d = df.dropna(subset=["payment_method", "age"]).copy()
logit = smf.logit(
    "churn ~ C(contract, Treatment('two-year')) + tenure_months + monthly_charges"
    + " + support_calls + C(payment_method, Treatment('credit-card')) + age",
    data=d,
).fit(dispatch=False)

or_table = pd.DataFrame({"OR": np.exp(logit.params), "CI_low": np.exp(logit.conf_int()[0]),
                        "CI_high": np.exp(logit.conf_int()[1]), "p": logit.pvalues}).round(3)
print(or_table)
print("Pseudo R² (McFadden):", round(logit.prsquared, 4))

# Average marginal effects: thay đổi XÁC SUẤT (không phải odds) khi biến tăng 1 đơn vị
print(logit.get_margeff(at="overall").summary_frame().round(4).head(6))

```

Đối chiếu với cơ chế sinh dữ liệu (Chương 0.8): hệ số log-odds thật của `support_calls` là 0.35, tương ứng $OR = e^{0.35} \approx 1.42$. Ước lượng hồi quy phải bao khoảng giá trị này. Đây là cách kiểm tra mô hình thống kê "đúng đặc tả".

Hồi quy tuyến tính và chẩn đoán phần dư

```

from statsmodels.stats.diagnostic import het_breuschpagan

ols = smf.ols("np.log(data_usage_gb) ~ age + tenure_months + C(contract)", data=d.dropna()).fit()

```

```
bp = het_breuschpagan(ols.resid, ols.model.exog)
print(f"Breusch-Pagan p={bp[1]:.3f} (p nhỏ -> phương sai sai số không đều)")
robust = ols.get_robustcov_results(cov_type="HC3") # sai số chuẩn bền với heteroscedasticity
print(pd.DataFrame({"coef": ols.params, "se_classic": ols.bse, "se_HC3": robust.bse}).round(4))
```

Giả định OLS	Kiểm tra	Hậu quả khi vi phạm	Khắc phục
Tuyến tính	Residual vs fitted, partial residual plot	Hệ số sai lệch	Biến đổi, spline, tương tác
Sai số độc lập	Durbin-Watson, cấu trúc nhóm/thời gian	SE quá nhỏ	Cluster-robust SE, mô hình hỗn hợp
Phương sai đều	Breusch-Pagan, White	SE sai	HC3 robust SE , WLS
Sai số chuẩn	Q-Q residual	Ảnh hưởng ít khi n lớn	Bootstrap
Không đa cộng tuyến hoàn hảo	VIF	Hệ số không ổn định	Bỏ/gộp biến, ridge

GLM cho dữ liệu đếm (Poisson)

```
poisson = smf.glm("support_calls ~ C(contract) + tenure_months", data=d,
                  family=sm.families.Poisson()).fit()
dispersion = poisson.pearson_chi2 / poisson.df_resid
print("Incidence rate ratios:", np.exp(poisson.params).round(3).to_dict())
print("Hệ số phân tán:", round(dispersion, 2), "(>1 -> overdispersion, dùng Negative Binomial)")
```

4.8. Suy luận nhân quả: nhập môn có thực hành

Tương quan $\neq$ nhân quả. Hệ số hồi quy chỉ có nghĩa nhân quả khi không còn **biến gây nhiễu (confounder)** bị bỏ sót, tức là giả định *no unmeasured confounding*.

Nghịch lý Simpson

```
# Dữ liệu quan sát: chương trình khuyến mãi được áp dụng nhiều cho nhóm RỦI RO CAO
n_obs = 20_000
risk_high = rng.random(n_obs) < 0.5
promo = rng.random(n_obs) < np.where(risk_high, 0.8, 0.2) # gây nhiễu: risk -> promo
p_churn = np.where(risk_high, 0.40, 0.10) - 0.05 * promo # promo GIẢM churn 5 điểm %
y_obs = rng.random(n_obs) < p_churn
sim = pd.DataFrame({"risk_high": risk_high, "promo": promo, "churn": y_obs})

print("Gộp chung:", sim.groupby("promo")["churn"].mean().round(3).to_dict()) # promo "có hại"!
print("Theo nhóm rủi ro:\n", sim.groupby(["risk_high", "promo"])["churn"].mean().unstack().round(3))
```

Inverse Propensity Weighting (IPW)

Ước lượng **propensity score** $e(x) = P(T = 1 \mid X = x)$, rồi gán trọng số $1/e(x)$ cho nhóm treatment và $1/(1 - e(x))$ cho nhóm control để tạo "quần thể giả" cân bằng về X.

```
from sklearn.linear_model import LogisticRegression

X_conf = sim[["risk_high"]].astype(int).to_numpy()
e = LogisticRegression().fit(X_conf, sim[["promo"]]).predict_proba(X_conf)[:, 1]
t, y = sim[["promo"]].to_numpy(), sim[["churn"]].to_numpy()
ate_naive = y[t].mean() - y[~t].mean()
ate_ipw = np.mean(t * y / e) - np.mean((1 - t) * y / (1 - e))
print(f"ATE ngày thơ = {ate_naive:+.3f} | ATE IPW = {ate_ipw:+.3f} | ATE thật = -0.050")
```

Difference-in-Differences (DiD)

Khi một chính sách áp dụng cho một nhóm (vùng A) từ một thời điểm, DiD so sánh **thay đổi** của nhóm được áp dụng với **thay đổi** của nhóm đối chứng. Giả định quan trọng: **xu hướng song song (parallel trends)**.

```
n_did = 4_000
did = pd.DataFrame({"treated": rng.integers(0, 2, n_did), "post": rng.integers(0, 2, n_did)})
did["churn_rate"] = (0.20 + 0.03 * did["treated"] + 0.02 * did["post"]
                    - 0.04 * did["treated"] * did["post"] + rng.normal(0, 0.05, n_did))
did_fit = smf.ols("churn_rate ~ treated * post", data=did).fit(cov_type="HC3")
print("Hiệu ứng DiD (treated:post):", round(did_fit.params["treated:post"], 4),
      "CI:", np.round(did_fit.conf_int().loc["treated:post"].to_numpy(), 4))
```

Phương pháp	Giả định chính	Công cụ
Thí nghiệm ngẫu nhiên (RCT/A/B)	Ngẫu nhiên hóa đúng	statsmodels, scipy
Hiệu chỉnh hồi quy / IPW / AIPW	Không có confounder không đo được; overlap ($0 < e(x) < 1$)	DoWhy, EconML
Matching (PSM)	Như trên	causalml, DoWhy
DiD	Xu hướng song song	statsmodels, linearmodels
Regression Discontinuity	Không thao túng được ngưỡng	rdrobust
Instrumental Variables	Biến công cụ hợp lệ (relevance + exclusion)	linearmodels.IV2SLS
Synthetic Control	Nhóm đối chứng tổng hợp khớp tiền can thiệp	pysyncon

4.9. Phân tích sống còn (Survival analysis): churn theo thời gian

Churn về bản chất là bài toán **thời gian đến sự kiện**. Khách chưa rời bỏ là quan sát **bị kiểm duyệt phải (right-censored)**: ta chỉ biết họ "sống" ít nhất đến hiện tại. Bỏ khách bị kiểm duyệt, hoặc coi họ là "không churn", đều gây bias.

```
from statsmodels.duration.hazard_regression import PHReg
from statsmodels.duration.survfunc import SurvfuncRight, survdiff

n_s = 3_000
m2m = rng.integers(0, 2, n_s)
calls = rng.poisson(1.5, n_s)
true_time = rng.exponential(scale=np.exp(3.5 - 0.8 * m2m - 0.2 * calls)) # tháng đến khi rời bỏ
censor_time = rng.uniform(0, 48, n_s) # thời gian quan sát được
surv = pd.DataFrame({"time": np.minimum(true_time, censor_time),
                    "event": (true_time <= censor_time).astype(int), "m2m": m2m, "calls": calls})
print("Tỷ lệ quan sát bị kiểm duyệt:", round(1 - surv["event"].mean(), 3))

# Kaplan-Meier theo loại hợp đồng + log-rank test
fig, ax = plt.subplots()
for g, sub in surv.groupby("m2m"):
    km = SurvfuncRight(sub["time"], sub["event"])
    km.plot(ax=ax)
    print(f"m2m={g}: median survival = {km.quantile(0.5):.1f} tháng")
chisq, p_lr = survdiff(surv["time"], surv["event"], surv["m2m"])
print(f"Log-rank  $\chi^2$ ={chisq:.1f}, p={p_lr:.2e}")

# Cox Proportional Hazards: hazard ratio cho từng biến
cox = PHReg.from_formula("time ~ m2m + calls", data=surv, status=surv["event"].to_numpy()).fit()
print(pd.DataFrame({"HR": np.exp(cox.params), "p": cox.pvalues},
                    index=cox.model.exog_names).round(3)) # HR thật:  $e^{0.8} \approx 2.23$ ,  $e^{0.2} \approx 1.22$ 
```

Thư viện chuyên dụng: **lifelines** (Kaplan-Meier, Cox, AFT, kiểm tra giả định proportional hazards), **scikit-survival** (Random Survival Forest, metric C-index tương thích scikit-learn).

4.10. Phân tích chuỗi thời gian cơ bản

```
from statsmodels.tsa.seasonal import STL
from statsmodels.tsa.stattools import acf, adfuller, kpss

days = pd.date_range("2024-01-01", periods=365, freq="D")
signups = pd.Series(200 + 0.2 * np.arange(365) + 25 * np.sin(2 * np.pi * np.arange(365) / 7)
                    + rng.normal(0, 8, 365), index=days)

stl = STL(signups, period=7, robust=True).fit() # Trend + Seasonal + Residual
strength_seasonal = max(0, 1 - stl.resid.var() / (stl.seasonal + stl.resid).var())
print("Độ mạnh mùa vụ tuần (Hyndman):", round(strength_seasonal, 3))
# statsmodels 0.15: result_object=True trả về đối tượng có tên trường (sẽ là mặc định từ 0.16)
adf = adfuller(signups, result_object=True)
kp = kpss(signups, regression="c", nlags="auto", result_object=True)
print("ADF p =", round(adf.pvalue, 4), "(H0: có nghiệm đơn vị / không dừng)")
print("KPSS p =", round(kp.pvalue, 4), "(H0: dừng; p bị chặn ở 0.01 bởi bảng tra)")
print("ACF lag 7 =", round(acf(signups.diff().dropna(), nlags=7)[7], 3))
```

Kết hợp ADF và KPSS: ADF không bác bỏ **và** KPSS bác bỏ thì chuỗi không dừng, cần sai phân hoặc khử xu hướng trước khi mô hình ARIMA.

Checklist Chương 4

- ☐ Phát biểu giả thuyết và α **trước khi** nhìn dữ liệu; chọn kiểm định phù hợp kiểu dữ liệu và giả định.
- ☐ Báo cáo **effect size + CI**, không chỉ p-value; diễn giải p-value đúng tinh thần ASA.
- ☐ Hiệu chỉnh đa kiểm định (Holm/BH) khi kiểm định nhiều giả thuyết.
- ☐ A/B test: tính cỡ mẫu trước, kiểm tra SRM, không nhìn trộm, dùng CUPED khi có covariate.
- ☐ Hồi quy: kiểm tra giả định, dùng robust SE khi cần, diễn giải bằng OR / marginal effects.
- ☐ Không diễn giải nhân quả từ dữ liệu quan sát khi chưa xử lý confounder.
- ☐ Dữ liệu thời gian-đến-sự-kiện được xử lý bằng phương pháp survival (tôn trọng kiểm duyệt).

Tài liệu tham khảo Chương 4

- Wasserstein, R. L. & Lazar, N. A. (2016). *The ASA Statement on p-Values: Context, Process, and Purpose*. The American Statistician 70(2).
- SciPy reference: `scipy.stats` — *Hypothesis tests, bootstrap, permutation_test*. docs.scipy.org
- statsmodels User Guide: *Regression, GLM, Stats (power, proportion, multitest), Duration (survival), Time Series Analysis*. statsmodels.org
- Kohavi, R., Tang, D. & Xu, Y. (2020). *Trustworthy Online Controlled Experiments*. Cambridge University Press.
- Deng, A. et al. (2013). *Improving the Sensitivity of Online Controlled Experiments by Utilizing Pre-Experiment Data (CUPED)*. WSDM.
- Efron, B. & Tibshirani, R. (1993). *An Introduction to the Bootstrap*. Chapman & Hall.
- Benjamini, Y. & Hochberg, Y. (1995). *Controlling the False Discovery Rate*. JRSS-B 57(1).
- Hernán, M. A. & Robins, J. M. (2020). *Causal Inference: What If*. Chapman & Hall/CRC (bản miễn phí).
- Cunningham, S. (2021). *Causal Inference: The Mixtape*. Yale University Press.
- Hyndman, R. J. & Athanasopoulos, G. (2021). *Forecasting: Principles and Practice*, 3rd ed. (otexts.com/fpp3)
- Harvard CS109, Stanford STATS 200/STATS 361 (tài liệu bài giảng về suy luận và nhân quả).

CHƯƠNG 5. TIỀN XỬ LÝ DỮ LIỆU & FEATURE ENGINEERING

"Coming up with features is difficult, time-consuming, requires expert knowledge. 'Applied machine learning' is basically feature engineering." — Andrew Ng.

Mục tiêu chương: biến dữ liệu thô thành ma trận feature **sạch, giàu thông tin, không rò rỉ**, được đóng gói thành một Pipeline duy nhất dùng chung cho huấn luyện, đánh giá và production.

5.1. Nguyên tắc nền tảng: fit trên train, transform trên mọi tập

[Docs] scikit-learn Common pitfalls:

- *Inconsistent preprocessing.* Phép biến đổi dùng lúc train phải được áp dụng **y hệt** lúc test và trong production. Nếu không, không gian feature thay đổi và mô hình hoạt động sai.
- *Data leakage during pre-processing.* Ví dụ chính thức của scikit-learn: chọn feature bằng SelectKBest trên **toàn bộ** dữ liệu ngẫu nhiên (X và y độc lập) cho accuracy **0.76** thay vì ~0.5. Khi đặt cùng bước đó trong Pipeline và CV, accuracy về đúng ~0.5.
- Khuyến nghị: dùng **Pipeline** để mọi bước có trạng thái (stateful) chỉ được **fit** trên phần train của từng fold.

Loại bước	Ví dụ	Làm trước khi chia dữ liệu?
Stateless (không học gì từ dữ liệu)	Chuẩn hóa chuỗi, parse ngày, sửa giá trị vô lý → NaN, dedup chính xác, feature tỷ lệ a/b	✅ Được
Stateful (học tham số)	Impute mean/median, scaling, encoding, binning theo phân vị, chọn feature, PCA, SMOTE	❌ Phải nằm trong Pipeline

```
Dữ liệu thô → clean_churn (stateless) → split train/valid/test → Pipeline[ ChurnFeatures → ColumnTransformer(
    num: Winsorizer → SimpleImputer(+indicator) →
Scaler
    low-card: Imputer → OneHotEncoder
    high-card: Imputer → TargetEncoder (cross-
fitting)
) → Model ]
```

Thiết lập

```
import numpy as np
import pandas as pd
from sklearn.compose import ColumnTransformer, make_column_selector
from sklearn.impute import SimpleImputer
from sklearn.linear_model import LogisticRegression
from sklearn.model_selection import StratifiedKFold, cross_val_score, cross_validate
from sklearn.pipeline import Pipeline, make_pipeline
from sklearn.preprocessing import OneHotEncoder, StandardScaler

from churn.data.split import time_split
from churn.data.synthetic import make_churn_data
from churn.features.build import RAW_FEATURES, build_preprocessor
from churn.features.clean import clean_churn

pd.set_option("display.width", 160)
raw = make_churn_data(n=20_000, seed=42)
df = clean_churn(raw)
train_df, valid_df, test_df = time_split(df, "signup_date", "2023-10-01", "2024-01-01")
X_train, y_train = train_df[RAW_FEATURES], train_df["churn"]
X_valid, y_valid = valid_df[RAW_FEATURES], valid_df["churn"]
```

```
cv = StratifiedKFold(n_splits=5, shuffle=True, random_state=0)
print(len(X_train), len(X_valid), len(test_df))
```

5.2. Làm sạch dữ liệu (stateless)

Mã nguồn đầy đủ ở `code/src/churn/features/clean.py`. Các quy tắc chính:

Vấn đề	Quy tắc	Lý do
Trùng chính xác	<code>drop_duplicates()</code>	Bản ghi lặp làm lệch thống kê và gây leakage giữa train/test
Trùng theo khóa nghiệp vụ	Giữ bản ghi mới nhất theo thời gian	Một khách hàng = một dòng (theo grain đã định)
Chuỗi không nhất quán	Unicode NFC → strip → gộp khoảng trắng → lowercase → bằng ánh xạ alias	"Month-to-Month" = "month-to-month"
Giá trị không thể có	Chuyển thành NaN (tuổi 150, cước ≤ 0)	Để imputer xử lý thống nhất; không xóa dòng âm thầm
Kiểu dữ liệu	<code>pd.to_datetime(errors="coerce")</code> , ép số	Lỗi parse trở thành NaN có thể đếm được

```
before = raw["contract"].value_counts().to_dict()
after = df["contract"].value_counts().to_dict()
print("Trước:", before)
print("Sau   :", after)
print("Số dòng:", len(raw), "->", len(df))
```

5.3. Xử lý giá trị thiếu (Imputation)

Phương pháp	Khi nào dùng	scikit-learn
Xóa dòng	Thiếu rất ít, MCAR, dữ liệu đối dào	<code>dropna</code> (chỉ trên train; production vẫn phải xử lý NaN)
Xóa cột	Thiếu > 60–70% và không mang tín hiệu	—
Mean / Median	Baseline; median bền với ngoại lai	<code>SimpleImputer(strategy="median")</code>
Hằng số / mode	Biến phân loại	<code>SimpleImputer(strategy="constant", fill_value="missing")</code>
Cờ thiếu	Thiếu mang thông tin (MAR/MNAR)	<code>SimpleImputer(add_indicator=True)</code> , <code>MissingIndicator</code>
KNN	Quan hệ cục bộ, dữ liệu vừa	<code>KNNImputer</code> (cần scale trước, $O(n^2)$)
Iterative (MICE-like)	MAR, quan hệ đa biến	<code>IterativeImputer</code> (experimental)
Không impute	Mô hình xử lý NaN gốc	<code>HistGradientBoosting*</code> , <code>DecisionTree*</code> / <code>RandomForest*</code> (≥ 1.4), <code>LightGBM</code> , <code>XGBoost</code> , <code>CatBoost</code>

[Docs] Về *Multiple vs. Single Imputation*: `IterativeImputer` lấy cảm hứng từ gói R **MICE** nhưng trả về **một** lần impute. Muốn multiple imputation thì chạy lặp với `sample_posterior=True` và các seed khác nhau. Tài liệu cũng ghi rõ: trong bối cảnh **dự đoán** (không cần định lượng bất định do thiếu), lợi ích của multiple imputation vẫn là câu hỏi mở. **[Docs]** Từ phiên bản 1.2, tham số `keep_empty_features` quyết định cột toàn NaN bị bỏ hay giữ (điền 0). Điều này ảnh hưởng tới số cột đầu ra.

So sánh chiến lược impute bằng chính metric cuối cùng

```
from sklearn.ensemble import ExtraTreesRegressor, HistGradientBoostingClassifier
from sklearn.experimental import enable_iterative_imputer # noqa: F401 (bật API experimental)
from sklearn.impute import IterativeImputer, KNNImputer

num_cols = ["age", "tenure_months", "monthly_charges", "support_calls", "data_usage_gb"]
Xn, yn = X_train[num_cols].iloc[:6000], y_train.iloc[:6000] # mẫu con để chạy nhanh

strategies = {
    "median": SimpleImputer(strategy="median"),
    "median+indicator": SimpleImputer(strategy="median", add_indicator=True),
    "knn(k=10)": make_pipeline(StandardScaler(), KNNImputer(n_neighbors=10)),
    "iterative(ET)": IterativeImputer(estimator=ExtraTreesRegressor(n_estimators=30, random_state=0),
                                     max_iter=5, random_state=0),
}

for name, imp in strategies.items():
    pipe = make_pipeline(imp, StandardScaler(), LogisticRegression(max_iter=2000))
    s = cross_val_score(pipe, Xn, yn, cv=cv, scoring="average_precision")
    print(f"{name:18s} PR-AUC = {s.mean():.4f} ± {s.std():.4f}")
s = cross_val_score(HistGradientBoostingClassifier(random_state=0), Xn, yn, cv=cv, scoring="average_precision")
print(f"'HGB (NaN gốc)':18s PR-AUC = {s.mean():.4f} ± {s.std():.4f}")
```

[Kinh nghiệm] Chọn cách impute bằng **CV trên metric cuối cùng**, không phải bằng việc giá trị điển "giống thật" nhất. Trên dữ liệu MCAR này, bốn chiến lược impute cho kết quả gần như bằng nhau, nên chọn phương án **đơn giản nhất** (median + indicator). Với dữ liệu có cơ chế MAR mạnh, KNN/Iterative mới thể hiện ưu thế. HistGradientBoosting với tham số mặc định thấp hơn ở đây vì mô hình chưa được tinh chỉnh trên mẫu 6 000 dòng, quan hệ thật lại gần tuyến tính. Đừng kết luận "xử lý NaN gốc kém": so sánh này đối cả *mô hình* chứ không chỉ cách impute.

5.4. Ngoại lai: clip có học từ train

Winsorizer trong package (code/src/churn/features/transformers.py) tuân thủ **hợp đồng estimator** của scikit-learn (*Developing scikit-learn estimators*):

```
class Winsorizer(OneToOneFeatureMixin, TransformerMixin, BaseEstimator):
    def __init__(self, q_low: float = 0.01, q_high: float = 0.99):
        self.q_low = q_low # __init__ CHỈ lưu tham số: không validate, không tính toán
        self.q_high = q_high

    def fit(self, X, y=None):
        arr = np.asarray(X, dtype=float)
        self.n_features_in_ = arr.shape[1]
        self.lower_ = np.nanquantile(arr, self.q_low, axis=0) # thuộc tính học được: hậu tố "_"
        self.upper_ = np.nanquantile(arr, self.q_high, axis=0)
        return self # fit trả về self

    def transform(self, X):
        check_is_fitted(self, ["lower_", "upper_"])
        return np.clip(np.asarray(X, dtype=float), self.lower_, self.upper_) # NaN được giữ nguyên
```

OneToOneFeatureMixin tự cung cấp get_feature_names_out, nhờ đó transformer hoạt động với set_output(transform="pandas"). Package có test check_transformer_general của scikit-learn để bảo đảm tuân thủ API.

```
from churn.features.transformers import Winsorizer

w = Winsorizer(0.01, 0.99).set_output(transform="pandas").fit(X_train[["monthly_charges"]])
clipped = w.transform(X_valid[["monthly_charges"]])
print("Ngưỡng học từ train:", w.lower_.round(2), w.upper_.round(2))
print("Max trước/sau:", X_valid["monthly_charges"].max(), "->", clipped["monthly_charges"].max())
```

5.5. Mã hóa biến phân loại

Encoder	Cardinality	Mô hình phù hợp	Ghi chú (theo tài liệu chính thức)
OneHotEncoder	Thấp	Tuyến tính, NN, SVM	<code>handle_unknown="infrequent_if_exist"</code> , <code>min_frequency</code> , <code>max_categories</code> , <code>drop="if_binary"</code> ; <code>sparse_output</code>
OrdinalEncoder	Có thứ tự / mô hình cây	Cây, GBM	<code>categories=[...]</code> để cố định thứ tự; <code>handle_unknown="use_encoded_value"</code> , <code>encoded_missing_value</code>
TargetEncoder	Cao	Mọi mô hình	Có cross-fitting trong <code>fit_transform</code> ; <code>smooth="auto"</code> (empirical Bayes)
Frequency/count	Cao	Cây	Không dùng target nên không có leakage từ target
Hashing	Rất cao (URL, ID)	Tuyến tính	FeatureHasher; chấp nhận va chạm hash
Native categorical	Bất kỳ	HistGB (<code>categorical_features="from_dtype"</code>), LightGBM, CatBoost	CatBoost dùng <i>ordered target statistics</i>
Embedding	Rất cao	Neural network	Học biểu diễn

OneHotEncoder với nhóm hiếm và giá trị chưa thấy

```

ohe = OneHotEncoder(handle_unknown="infrequent_if_exist", min_frequency=0.01,
                    sparse_output=False).set_output(transform="pandas")
ohe.fit(X_train[["contract"]].fillna("missing"))
print(ohe.categories_, "| infrequent:", ohe.infrequent_categories_)
print(ohe.transform(pd.DataFrame({"contract": ["one-year", "lifetime-plan"]}))
# Không có mức nào < 1% nên không tồn tại nhóm infrequent: mức lạ "lifetime-plan" được mã hóa toàn 0
# (hành vi giống handle_unknown="ignore"). Nếu có nhóm infrequent, mức lạ sẽ được gán vào nhóm đó.

```

TargetEncoder: công thức và vì sao phải cross-fitting

[Docs] Với target nhị phân, mã hóa của mức i :

$$S_i = \lambda_i \frac{n_{iY}}{n_i} + (1 - \lambda_i) \frac{n_Y}{n}, \quad \lambda_i = \frac{n_i}{m + n_i}$$

Trong đó m là hệ số làm tròn. `smooth="auto"` ước lượng $m = \sigma_i^2 / \tau^2$ theo empirical Bayes. `fit(X, y).transform(X)` khác `fit_transform(X, y)`: `fit_transform` chia train thành k fold, rồi mã hóa mỗi fold bằng thống kê học từ $k-1$ fold còn lại. Cách này ngăn mô hình "học thuộc" nhãn qua các mức hiếm. Thí nghiệm dưới đây tái hiện ví dụ chính thức *Target Encoder's Internal Cross fitting* của scikit-learn: thêm một biến **hoàn toàn ngẫu nhiên** có 3 000 mức.

```

from sklearn.preprocessing import TargetEncoder

from sklearn.metrics import roc_auc_score

rng = np.random.default_rng(0)
useful = ["tenure_months", "support_calls", "monthly_charges"]
sc = StandardScaler().fit(X_train[useful])
U_tr, U_va = sc.transform(X_train[useful]), sc.transform(X_valid[useful])
noise_tr = pd.DataFrame({"noise_id": rng.integers(0, 3000, len(X_train)).astype(str)})
noise_va = pd.DataFrame({"noise_id": rng.integers(0, 3000, len(X_valid)).astype(str)})

# SAI: fit trên train rồi transform chính train -> mức hiếm "nhỏ" nhãn của chính dòng đó
te_wrong = TargetEncoder(target_type="binary").fit(noise_tr, y_train)
Z_wrong_tr = np.c_[U_tr, te_wrong.transform(noise_tr)]
# ĐÚNG: fit_transform -> mỗi fold được mã hóa bằng thống kê của các fold khác
te_right = TargetEncoder(target_type="binary", cv=StratifiedKFold(5, shuffle=True, random_state=0))
Z_right_tr = np.c_[U_tr, te_right.fit_transform(noise_tr, y_train)]

```

```
for name, enc, Z_tr in [("fit + transform (leak)", te_wrong, Z_wrong_tr),
                      ("fit_transform (đúng)", te_right, Z_right_tr)]:
    m = LogisticRegression(max_iter=1000).fit(Z_tr, y_train)
    auc_tr = roc_auc_score(y_train, m.predict_proba(Z_tr)[:, 1])
    auc_va = roc_auc_score(y_valid, m.predict_proba(np.c_[U_va, enc.transform(noise_va)][:, 1])
    print(f"{name:22s} AUC train={auc_tr:.3f} valid={auc_va:.3f} | hệ số của biến nhiễu={m.coef_[0, -1]:+.2f}")
# Bản leak: mô hình dồn trọng số vào biến nhiễu (AUC train ảo cao) -> AUC valid giảm rõ.
# Bản đúng: hệ số biến nhiễu ~0, AUC train ≈ valid.
```

Lưu ý phiên bản. Từ scikit-learn **1.9**, `TargetEncoder(shuffle=..., random_state=...)` bị deprecate (sẽ xóa ở 1.11). Hãy truyền splitter vào `cv`, ví dụ `cv=StratifiedKFold(5, shuffle=True, random_state=0)`, như package tham chiếu.

5.6. Feature Engineering: nơi tạo ra khác biệt lớn nhất

a) Feature nghiệp vụ (domain features)

`ChurnFeatures` trong package tạo các feature có ý nghĩa nghiệp vụ, tính **theo từng dòng** (stateless):

Feature	Công thức	Giả thuyết nghiệp vụ
<code>avg_charge_per_month</code>	$\text{total} / \max(\text{tenure}, 1)$	Giá trị trung bình thực trả
<code>charge_vs_expected</code>	$\text{total} / (\text{monthly} \times \text{tenure})$	Lệch so với kỳ vọng: thay đổi gói, khuyến mãi
<code>calls_per_year</code>	$\text{support_calls} / (\text{tenure}/12)$	Cường độ khiếu nại chuẩn hóa theo thời gian
<code>is_new_customer</code>	$\text{tenure} \leq 3$	Giai đoạn onboarding rủi ro cao
<code>signup_month, signup_dow</code>	Từ ngày đăng ký	Mùa vụ chiến dịch

```
from churn.features.transformers import ChurnFeatures

feat = ChurnFeatures().fit(X_train).transform(X_valid)
print(feat[["avg_charge_per_month", "charge_vs_expected", "calls_per_year", "is_new_customer"]]
      .describe().T.round(2))
```

Bẫy thật đã gặp khi viết handbook: "số ngày kể từ một ngày cố định". Phiên bản đầu của package có feature `days_since_signup = reference_date - signup_date`, với `reference_date` học từ tập train (ngày lớn nhất = 2023-09-30). Trên tập validation (khách đăng ký *sau* ngày đó), feature này **âm** và nằm ngoài miền giá trị mô hình từng thấy:

```
ref = X_train["signup_date"].max()
print((ref - X_valid["signup_date"]).dt.days.describe()[["min", "max"]]) # toàn giá trị âm
```

Nguyên tắc: feature "thời gian trôi qua" phải đo **tới thời điểm chấm điểm của chính dòng đó** (snapshot date). Ở đây `tenure_months` đã làm đúng điều này, nên feature kia vừa sai vừa thừa và đã bị loại khỏi package. Adversarial validation (Chương 3.11) và kiểm tra miền giá trị feature giữa train/valid giúp phát hiện sớm lỗi này.

b) Mã hóa chu kỳ (cyclical encoding)

Tháng 12 và tháng 1 gần nhau, nhưng mã hóa số nguyên đặt chúng xa nhau nhất. Ảnh xạ lên vòng tròn đơn vị giải quyết điều này. **[Docs]** Ví dụ chính thức *Time-related feature engineering* của scikit-learn so sánh one-hot, sin/cos và `SplineTransformer(extrapolation="periodic")` cho dữ liệu theo giờ.

```
from sklearn.preprocessing import FunctionTransformer, SplineTransformer
```

```
def sin_transformer(period: int) -> FunctionTransformer:
    return FunctionTransformer(lambda x: np.sin(x / period * 2 * np.pi), feature_names_out="one-to-one")

def cos_transformer(period: int) -> FunctionTransformer:
    return FunctionTransformer(lambda x: np.cos(x / period * 2 * np.pi), feature_names_out="one-to-one")

months = pd.DataFrame({"month": np.arange(1, 13)})
cyc = pd.DataFrame({"sin": sin_transformer(12).fit_transform(months)["month"],
                    "cos": cos_transformer(12).fit_transform(months)["month"]})
print("Khoảng cách 12↔1:", np.linalg.norm(cyc.iloc[11] - cyc.iloc[0]).round(3),
      "| 6↔7:", np.linalg.norm(cyc.iloc[5] - cyc.iloc[6]).round(3)) # bằng nhau: đúng tính chu kỳ
periodic_spline = SplineTransformer(degree=3, n_knots=13, knots=np.linspace(1, 13, 13).reshape(-1, 1),
                                   extrapolation="periodic")
print("Periodic spline features:", periodic_spline.fit_transform(months).shape)
```

c) Tổng hợp theo cửa sổ thời gian từ dữ liệu giao dịch (point-in-time)

```
def window_aggregates(tx: pd.DataFrame, snapshot: str, windows=(7, 30, 90)) -> pd.DataFrame:
    """tx: customer_id, ts, amount. Chỉ dùng giao dịch TRƯỚC snapshot."""
    t = pd.Timestamp(snapshot)
    tx = tx[tx["ts"] < t]
    parts = []
    for w in windows:
        sub = tx[tx["ts"] >= t - pd.Timedelta(days=w)]
        agg = sub.groupby("customer_id")["amount"].agg(["count", "sum", "mean", "max"])
        agg.columns = [f"amt_{c}_{w}d" for c in agg.columns]
        parts.append(agg)
    out = pd.concat(parts, axis=1).fillna(0)
    out["trend_7_vs_90"] = out["amt_sum_7d"] / (out["amt_sum_90d"] * 7 / 90 + 1e-9) # tăng tốc / giảm tốc
    out["recency_days"] = (t - tx.groupby("customer_id")["ts"].max()).dt.days
    return out

tx = pd.DataFrame({"customer_id": rng.choice([f"C{i:07d}" for i in range(500)], 20_000),
                  "ts": pd.Timestamp("2024-01-01") + pd.to_timedelta(rng.integers(0, 200, 20_000), unit="D"),
                  "amount": rng.gamma(2, 50, 20_000).round(0)})
agg = window_aggregates(tx, "2024-06-01")
print(agg.shape)
print(agg.iloc[:3, [0, 1, 4, 5, -2, -1]].round(1))
```

Mẫu feature RFM (*Recency, Frequency, Monetary*) và *trend* (hoạt động gần đây so với dài hạn) là nhóm feature mạnh nhất cho churn trong thực tế.

d) Feature văn bản

```
from sklearn.feature_extraction.text import HashingVectorizer, TfidfVectorizer

tickets = ["mạng chậm vào buổi tối", "cước tháng này quá cao", "muốn hủy hợp đồng",
           "mạng rất liên tục", "hỏi về gói cước mới", "cước cao hơn quảng cáo"]
tfidf = TfidfVectorizer(ngram_range=(1, 2), min_df=1, sublinear_tf=True)
M = tfidf.fit_transform(tickets)
print(M.shape, "| top features:", tfidf.get_feature_names_out()[:6])
hv = HashingVectorizer(n_features=2**10, alternate_sign=False) # không lưu vocabulary: phù hợp streaming
print(hv.transform(tickets).shape)
```

Với tiếng Việt, nên **tách từ** trước (`underthesea` , `pyvi`) để có token như "hợp_đồng". Ở mức nâng cao, dùng embedding câu (sentence-transformers với mô hình tiếng Việt, PhoBERT) rồi giảm chiều.

e) Quan hệ phi tuyến cho mô hình tuyến tính: spline và tương tác

[Docs] `SplineTransformer` tạo cơ sở B-spline, giúp mô hình tuyến tính học được đường cong mượt. `PolynomialFeatures(interaction_only=True)` tạo các tích chéo.

```

from sklearn.preprocessing import PolynomialFeatures

base_cols = ["tenure_months", "monthly_charges", "support_calls"]
Xb = X_train[base_cols]
linear = make_pipeline(StandardScaler(), LogisticRegression(max_iter=2000))
spline = make_pipeline(SplineTransformer(n_knots=6, degree=3), LogisticRegression(max_iter=2000))
inter = make_pipeline(StandardScaler(), PolynomialFeatures(degree=2, interaction_only=True, include_bias=False),
                      LogisticRegression(max_iter=2000))
for name, m in [("tuyến tính", linear), ("spline", spline), ("tương tác bậc 2", inter)]:
    s = cross_val_score(m, Xb, y_train, cv=cv, scoring="roc_auc")
    print(f"{name:16s} ROC-AUC = {s.mean():.4f}")

```

f) Binning (rời rạc hóa)

```

from sklearn.preprocessing import KBinsDiscretizer

kb = KBinsDiscretizer(n_bins=8, encode="ordinal", strategy="quantile", quantile_method="averaged_inverted_cdf")
binned = kb.fit_transform(X_train[["tenure_months"]])
print("Cạnh bin (học từ train):", np.round(kb.bin_edges_[0], 1))

```

Binning làm mất thông tin, nhưng hữu ích khi cần **diễn giải** (scorecard), khi quan hệ dạng bậc thang, hoặc khi cần bền vững với ngoại lai. Binning tối ưu theo target có ở thư viện `optbinning`.

5.7. Pipeline tiền xử lý hoàn chỉnh

Package tham chiếu (`code/src/churn/features/build.py`) đóng gói toàn bộ thành một đối tượng:

```

def build_preprocessor(scale: bool = True, random_state: int = 0) -> Pipeline:
    numeric = Pipeline([
        ("winsor", Winsorizer(0.01, 0.99)), # clip trước khi impute
        ("impute", SimpleImputer(strategy="median", add_indicator=True)),
        ("scale", StandardScaler() if scale else "passthrough"), # cây không cần scale
    ])
    low_card = Pipeline([
        ("impute", SimpleImputer(strategy="constant", fill_value="missing")),
        ("ohe", OneHotEncoder(handle_unknown="infrequent_if_exist", min_frequency=0.01, sparse_output=False)),
    ])
    high_card = Pipeline([
        ("impute", SimpleImputer(strategy="constant", fill_value="missing")),
        ("te", TargetEncoder(target_type="binary",
                           cv=StratifiedKFold(5, shuffle=True, random_state=random_state))),
    ])
    columns = ColumnTransformer(
        [
            ("num", numeric, NUM_COLS),
            ("low", low_card, LOW_CARD_COLS),
            ("high", high_card, HIGH_CARD_COLS),
            ("pass", "passthrough", PASSTHROUGH)],
        remainder="drop", # cột lạ trong production bị bỏ qua thay vì làm hỏng mô hình
        verbose_feature_names_out=False,
    )
    return Pipeline([("domain", ChurnFeatures()), ("columns", columns)])

```

```

prep = build_preprocessor(scale=True)
Xt = prep.fit_transform(X_train, y_train) # TargetEncoder cần y; cross-fitting tự động
Xv = prep.transform(X_valid)
names = prep.get_feature_names_out()
print(Xt.shape, Xv.shape)
print(list(names))

clf = make_pipeline(build_preprocessor(scale=True), LogisticRegression(max_iter=3000))
res = cross_validate(clf, X_train, y_train, cv=cv, scoring=["roc_auc", "average_precision"])
print({k: round(v.mean(), 4) for k, v in res.items() if k.startswith("test_")})

```

[Docs] Các tính năng `ColumnTransformer` hay dùng:

- `make_column_selector(dtype_include=...)` chọn cột theo kiểu dữ liệu, không cần liệt kê tên.

- `remainder="passthrough" | "drop" | <transformer>` xử lý các cột không được liệt kê.
- `verbose_feature_names_out=False` giữ tên cột gọn (không thêm tiền tố `num_`).
- `set_output(transform="pandas")` (≥ 1.2) trả về DataFrame có tên cột, dễ debug và giải thích.
- `force_int_remainder_cols` và `n_jobs` cho bằng rất rộng.

```
auto = ColumnTransformer([
    ("num", SimpleImputer(strategy="median"), make_column_selector(dtype_include="number")),
    ("cat", OneHotEncoder(handle_unknown="ignore", sparse_output=False),
     make_column_selector(dtype_include=["object", "string", "category"])),
], verbose_feature_names_out=False).set_output(transform="pandas")
sample = X_train.drop(columns=["customer_id", "signup_date"]).head(1000)
print(auto.fit_transform(sample).columns[:10].tolist())
```

5.8. Lựa chọn đặc trưng (Feature Selection)

Nhóm	Phương pháp (scikit-learn)	Ưu	Nhược
Filter	VarianceThreshold, SelectKBest(f_classif/chi2/mutual_info_classif), SelectPercentile	Nhanh	Bỏ qua tương tác và dư thừa
Wrapper	RFE, RFECV, SequentialFeatureSelector	Tính đến mô hình	Chậm; dễ overfit với CV nhỏ
Embedded	SelectFromModel (L1, feature_importances_)	Cân bằng	Phụ thuộc mô hình; importance của cây bị bias
Model-agnostic	Permutation importance, SHAP, null importance, Boruta	Đáng tin cậy hơn	Tốn tính toán

Nguyên tắc vàng: feature selection là **một bước của pipeline**. Nó phải được thực hiện **bên trong** CV, không làm trước trên toàn bộ dữ liệu (ví dụ chính thức trong *Common pitfalls*).

```
from sklearn.feature_selection import RFECV, SelectFromModel, SelectKBest, mutual_info_classif

# Thêm 10 biến nhiễu để xem các phương pháp có loại được không
noise = pd.DataFrame(rng.normal(size=(len(Xt), 10)), columns=[f"noise_{i}" for i in range(10)])
Xsel = pd.concat([pd.DataFrame(Xt, columns=names), noise], axis=1)

# scikit-learn >= 1.8: `penalty` bị deprecate -> dùng l1_ratio=1 cho L1 (liblinear/saga hỗ trợ)
l1 = SelectFromModel(LogisticRegression(l1_ratio=1, solver="liblinear", C=0.02)).fit(Xsel, y_train)
print("L1 giữ:", [c for c, k in zip(Xsel.columns, l1.get_support()) if k]) # L1 vẫn có thể giữ vài biến nhiễu

rfecv = RFECV(LogisticRegression(max_iter=2000), step=2, cv=cv, scoring="average_precision",
              min_features_to_select=3, n_jobs=-1).fit(Xsel, y_train)
print("RFECV số feature tối ưu:", rfecv.n_features_,
      "| còn biến nhiễu:", [c for c, k in zip(Xsel.columns, rfecv.support_) if k and c.startswith("noise")])

kbest = make_pipeline(SelectKBest(mutual_info_classif, k=10), LogisticRegression(max_iter=2000))
print("SelectKBest(MI) trong pipeline, CV PR-AUC:",
      cross_val_score(kbest, Xsel, y_train, cv=cv, scoring="average_precision").mean().round(4))
```

Null importance: phát hiện feature "ảo" của mô hình cây

```
from lightgbm import LGBMClassifier

def null_importance(X: pd.DataFrame, y: pd.Series, n_runs: int = 15, seed: int = 0) -> pd.DataFrame:
    """So sánh importance thật với phân phối importance khi target bị xáo trộn (Altmann et al., 2010)."""
    r = np.random.default_rng(seed)
    params = dict(n_estimators=150, learning_rate=0.05, num_leaves=15, importance_type="gain", verbose=-1)
    actual = LGBMClassifier(**params, random_state=seed).fit(X, y).feature_importances_
    null = np.array([LGBMClassifier(**params, random_state=i).fit(X, r.permutation(y.to_numpy()))
                     .feature_importances_ for i in range(n_runs)])
    return pd.DataFrame({"actual": actual, "null_p90": np.percentile(null, 90, axis=0),
```

```
"p_value": (null >= actual).mean(axis=0)}, index=X.columns
).sort_values("actual", ascending=False)
```

```
print(null_importance(Xsel, y_train).round(3).head(12))
# Feature có actual <= null_p90 (p_value lớn) không tốt hơn ngẫu nhiên -> ứng viên loại bỏ
```

5.9. Dữ liệu mất cân bằng lớp (Imbalanced data)

[Docs] imbalanced-learn, mục *Common pitfalls and recommended practices*: lỗi phổ biến nhất là **resample toàn bộ dữ liệu trước khi chia**. Lỗi này gây hai vấn đề: (1) mô hình không được đánh giá trên phân phối lớp thật; (2) mẫu tổng hợp sinh ra từ dữ liệu sẽ rơi vào tập test, gây leakage. Giải pháp là dùng `imblearn.pipeline.Pipeline`, trong đó bước resampling **chỉ chạy khi fit** và bị bỏ qua khi `predict / transform`.

Kỹ thuật	Cơ chế	Khi nào dùng
Không resample + metric đúng + chỉnh ngưỡng	PR-AUC, threshold tuning (Chương 9)	Nên thử đầu tiên
Class/sample weights	Phạt lỗi lớp thiểu số nặng hơn	<code>class_weight="balanced"</code> , <code>scale_pos_weight</code>
Random under-sampling	Bớt lớp đa số	Dữ liệu rất lớn
Random over-sampling	Nhân bản lớp thiểu số	Dữ liệu nhỏ
SMOTE / Borderline-SMOTE / ADASYN	Nội suy mẫu thiểu số	Mô hình tuyến tính/KNN; ít giúp GBM
SMOTENC / SMOTEN	SMOTE cho dữ liệu có biến phân loại	
Ensemble resampling	<code>BalancedRandomForestClassifier</code> , <code>EasyEnsembleClassifier</code>	
Focal loss	Tập trung vào mẫu khó	Deep learning

```
from imblearn.ensemble import BalancedRandomForestClassifier
from imblearn.over_sampling import SMOTE
from imblearn.pipeline import Pipeline as ImbPipeline
from imblearn.under_sampling import RandomUnderSampler
from sklearn.metrics import average_precision_score, brier_score_loss

candidates = {
    "baseline": make_pipeline(build_preprocessor(), LogisticRegression(max_iter=3000)),
    "class_weight": make_pipeline(build_preprocessor(), LogisticRegression(max_iter=3000, class_weight="balanced")),
    # imblearn >= 0.14 không cho phép Pipeline lồng làm bước trung gian -> "trải phẳng" các bước
    "SMOTE": ImbPipeline([*build_preprocessor().steps, ("smote", SMOTE(random_state=0)),
                          ("clf", LogisticRegression(max_iter=3000))]),
    "undersample": ImbPipeline([*build_preprocessor().steps, ("rus", RandomUnderSampler(random_state=0)),
                               ("clf", LogisticRegression(max_iter=3000))]),
    "BalancedRF": make_pipeline(build_preprocessor(scale=False),
                               BalancedRandomForestClassifier(n_estimators=200, sampling_strategy="all",
                                                                replacement=True, bootstrap=False,
                                                                random_state=0, n_jobs=-1)),
}

rows = []
for name, m in candidates.items():
    p = m.fit(X_train, y_train).predict_proba(X_valid)[ :, 1]
    rows.append({"method": name, "pr_auc": average_precision_score(y_valid, p),
                 "brier": brier_score_loss(y_valid, p), "mean_pred": p.mean(), "true_rate": y_valid.mean()})
print(pd.DataFrame(rows).set_index("method").round(4))
```

Độc kết quả: PR-AUC (khả năng xếp hạng) gần như không đổi giữa các phương pháp. Nhưng resampling và class weight đẩy xác suất trung bình lên xa tỷ lệ thật, làm **Brier score tệ đi**, tức là **mất calibration**. Nếu xác suất được dùng trực tiếp (tính lợi nhuận kỳ vọng như Chương 1.5) thì phải calibrate lại (Chương 9.6). Trong thực tế với GBM, kết hợp `scale_pos_weight` (hoặc không gì cả) với tối ưu ngưỡng thường tốt ngang hoặc hơn SMOTE.

Checklist Chương 5

- ☐ Bước stateless trước split; mọi bước stateful nằm trong `Pipeline`, fit chỉ trên train/fold.
- ☐ Chiến lược missing được chọn bằng CV trên metric cuối; có cờ missing khi thiếu mang tín hiệu.
- ☐ Encoding phù hợp cardinality; `TargetEncoder` dùng `fit_transform` (cross-fitting); xử lý mức lạ/hiếm.
- ☐ Custom transformer tuân thủ API scikit-learn (tham số trong `__init__`, thuộc tính `_, get_feature_names_out`) và có test.
- ☐ Feature nghiệp vụ point-in-time, có tài liệu ý nghĩa; không dùng timestamp tuyệt đối.
- ☐ Feature selection nằm trong CV; đã kiểm tra bằng biến nhiễu / null importance.
- ☐ Xử lý mất cân bằng bằng `imblearn.pipeline`; đã đánh giá ảnh hưởng tới calibration.

Tài liệu tham khảo Chương 5

- scikit-learn User Guide: *Common pitfalls and recommended practices; Pipelines and composite estimators* (`Pipeline`, `ColumnTransformer`, `TransformedTargetRegressor`); *Preprocessing data* (encoders, `TargetEncoder`, `KBinsDiscretizer`, `SplineTransformer`); *Imputation of missing values; Feature selection; Developing scikit-learn estimators*.
- scikit-learn Examples: *Target Encoder's Internal Cross fitting; Time-related feature engineering; Imputing missing values before building an estimator*.
- imbalanced-learn User Guide: *Over-sampling, Under-sampling, Ensemble methods, Common pitfalls*. imbalanced-learn.org
- Micci-Barreca, D. (2001). *A preprocessing scheme for high-cardinality categorical attributes*. ACM SIGKDD Explorations 3(1).
- Chawla, N. et al. (2002). *SMOTE: Synthetic Minority Over-sampling Technique*. JAIR 16.
- Altmann, A. et al. (2010). *Permutation importance: a corrected feature importance measure*. Bioinformatics 26(10).
- Kuhn, M. & Johnson, K. (2019). *Feature Engineering and Selection*. CRC Press (bookdown.org/max/FES).
- Zheng, A. & Casari, A. (2018). *Feature Engineering for Machine Learning*. O'Reilly.
- Google *Machine Learning Crash Course: Working with numerical/categorical data*. · Kaggle Learn: *Feature Engineering*.

CHƯƠNG 6. CHUẨN HÓA, BIẾN ĐỔI PHÂN PHỐI & GIẢM CHIỀU

Mục tiêu chương: hiểu **khi nào** và **vì sao** cần đưa feature về cùng thang đo hoặc thay đổi hình dạng phân phối; chọn đúng scaler theo dữ liệu và mô hình; dùng giảm chiều đúng mục đích (nén feature hay trực quan hóa).

6.1. Vì sao thang đo quan trọng: lý thuyết

Cơ chế	Mô hình bị ảnh hưởng	Giải thích
Khoảng cách	KNN, K-Means, SVM-RBF, DBSCAN, LOF	$\ x - x'\ ^2 = \sum_j (x_j - x'_j)^2$: feature có thang lớn (VNĐ) lấn át feature thang nhỏ (số cuộc gọi)
Regularization	Ridge, Lasso, ElasticNet, Logistic (mặc định L2)	Penalty $\lambda \sum \beta_j^2$ phạt các hệ số như nhau, nhưng độ lớn của hệ số phụ thuộc đơn vị đo. Chưa scale thì regularization không công bằng
Tối ưu bằng gradient	Logistic/Linear (solver lặp), Neural Network	Mất loss "dẹt" (điều kiện số kém) làm gradient descent hội tụ chậm hoặc dao động
Phương sai	PCA, LDA	PCA tìm hướng phương sai lớn nhất nên feature có thang lớn chiếm các thành phần đầu
Thứ tự giá trị	Cây, Random Forest, GBM	Chỉ phụ thuộc thứ tự để chọn điểm cắt, bất biến với mọi phép biến đổi đơn điệu

Thiết lập & thực nghiệm: scaling ảnh hưởng mô hình nào?

```
import matplotlib.pyplot as plt
import numpy as np
import pandas as pd
from scipy import stats
from sklearn.ensemble import HistGradientBoostingClassifier
from sklearn.linear_model import LogisticRegression
from sklearn.model_selection import StratifiedKFold, cross_val_score
from sklearn.neighbors import KNeighborsClassifier
from sklearn.pipeline import make_pipeline
from sklearn.preprocessing import (MaxAbsScaler, MinMaxScaler, Normalizer, PowerTransformer,
                                   QuantileTransformer, RobustScaler, StandardScaler)
from sklearn.svm import SVC

from churn.data.synthetic import make_churn_data
from churn.features.clean import clean_churn

pd.set_option("display.width", 160)
df = clean_churn(make_churn_data(n=20_000, seed=42))
num_cols = ["age", "tenure_months", "monthly_charges", "total_charges", "support_calls", "data_usage_gb"]
X = df[num_cols].fillna(df[num_cols].median()).iloc[:6000]
y = df["churn"].iloc[:6000]
cv = StratifiedKFold(5, shuffle=True, random_state=0)

models = {"KNN(k=25)": KNeighborsClassifier(25),
          "SVC-RBF": SVC(C=1.0, gamma="scale"),
          "Logistic(L2)": LogisticRegression(C=0.05, max_iter=5000),
          "HistGB (cây)": HistGradientBoostingClassifier(max_iter=100, random_state=0)}

rows = []
for name, m in models.items():
    raw = cross_val_score(m, X, y, cv=cv, scoring="roc_auc").mean()
    scaled = cross_val_score(make_pipeline(StandardScaler(), m), X, y, cv=cv, scoring="roc_auc").mean()
    rows.append({"model": name, "no_scaling": raw, "standard_scaled": scaled, "gain": scaled - raw})
print(pd.DataFrame(rows).set_index("model").round(4))
```

Kết quả cần quan sát: KNN và SVM cải thiện rõ khi scale. Mô hình cây cho kết quả **giống hệt**. Logistic ở đây gần như không đổi về AUC vì n lớn so với số feature nên regularization yếu. Ảnh hưởng của scaling lên regularization thể hiện ở **hệ số** (xem mục 6.2).

6.2. Các scaler của scikit-learn

Scaler	Công thức	Miền	Bền ngoại lai	Giữ sparse	Dùng khi
StandardScaler	$z = (x - \mu) / \sigma$	$\sim (-3, 3)$	✗	Chỉ khi <code>with_mean=False</code>	Mặc định cho tuyến tính, SVM, NN, PCA
MinMaxScaler	$(x - x_{\min}) / (x_{\max} - x_{\min})$	$[0, 1]$ (<code>feature_range</code>)	✗	✗	NN cần miền cố định, ảnh; dùng <code>clip=True</code> cho production
MaxAbsScaler	$x / \max x $	$[-1, 1]$	✗	✓	Dữ liệu sparse (khuyến nghị chính thức)
RobustScaler	$(x - \text{median}) / \text{IQR}$	Không cố định	✓	Chỉ <code>transform</code>	Nhiều ngoại lai
Normalizer	$x / \ x\ _p$ (theo dòng)	Vector đơn vị	—	✓	Văn bản TF-IDF, cosine similarity
QuantileTransformer	$F_{\text{target}}^{-1}(\hat{F}(x))$	Uniform/Normal	✓✓	✓ (một phần)	Ép phân phối, rất bền ngoại lai; phi tuyến
PowerTransformer	Box-Cox / Yeo-Johnson + chuẩn hóa	~Chuẩn	Một phần	✗	Giảm skew, ổn định phương sai

[Docs] *Scaling sparse data*: centering phá vỡ cấu trúc sparse và có thể làm tràn bộ nhớ. `MaxAbsScaler` được thiết kế cho sparse. `StandardScaler` chỉ nhận sparse khi `with_mean=False` (nếu không sẽ báo `ValueError`). `RobustScaler` không `fit` được trên sparse nhưng `transform` được.

So sánh các scaler trên dữ liệu có ngoại lai

Mô phỏng theo ví dụ chính thức *Compare the effect of different scalers on data with outliers*:

```
x = df[["monthly_charges"]].dropna() # có ~30 giá trị x8
scalers = {
    "Standard": StandardScaler(),
    "MinMax": MinMaxScaler(),
    "MaxAbs": MaxAbsScaler(),
    "Robust": RobustScaler(quantile_range=(25, 75)),
    "Quantile-Normal": QuantileTransformer(output_distribution="normal", n_quantiles=1000, random_state=0),
    "Yeo-Johnson": PowerTransformer(method="yeo-johnson"),
}
summary = {}
for name, s in scalers.items():
    z = s.fit_transform(x).ravel()
    body = np.quantile(z, [0.05, 0.95])
    summary[name] = {"p05": body[0], "p95": body[1], "body_width": body[1] - body[0],
                    "max": z.max(), "skew": stats.skew(z)}
print(pd.DataFrame(summary).T.round(3))
```

Cách đọc `body_width` (độ rộng phần thân 90% dữ liệu): với `MinMax/MaxAbs`, phần thân bị ép vào một khoảng rất hẹp vì một ngoại lai quyết định thang đo, làm **mất độ phân giải**. `RobustScaler` giữ phần thân đúng tỷ lệ nhưng ngoại lai vẫn rất lớn. `Quantile-Normal` kéo ngoại lai về phần đuôi của phân phối chuẩn.

Scaling và regularization: penalty "không công bằng" khi chưa scale

Đo **mức co rút (shrinkage)** của từng hệ số: tỷ số giữa hệ số khi có regularization mạnh ($C = 0.001$) và hệ số gần như không regularization ($C = 10^6$). Không scale thì feature có thang nhỏ (support_calls , 0-8) bị co mạnh, còn feature thang lớn (monthly_charges , data_usage_gb) gần như không bị phạt. Scale xong, mọi feature chịu cùng một "luật chơi".

```
def shrinkage_table(data: pd.DataFrame, scale: bool) -> pd.Series:
    pre = [StandardScaler()] if scale else []
    strong = make_pipeline(*pre, LogisticRegression(C=1e-3, max_iter=10_000)).fit(data, y)[-1].coef_[0]
    weak = make_pipeline(*pre, LogisticRegression(C=1e6, max_iter=10_000)).fit(data, y)[-1].coef_[0]
    return pd.Series(strong / weak, index=data.columns)
```

```
X_nc = X.drop(columns="total_charges") # bỏ biến đa cộng tuyến để so sánh co rút cho rõ
print(pd.DataFrame({"chưa scale": shrinkage_table(X_nc, scale=False),
                    "đã scale": shrinkage_table(X_nc, scale=True)}).round(3))
# Chưa scale: support_calls (thang nhỏ) bị co mạnh nhất, các biến thang lớn gần như không bị phạt
# -> mức phạt phụ thuộc đơn vị đo. Đã scale: mọi hệ số bị co theo cùng một thang (độ lệch chuẩn).
```

6.3. Biến đổi phân phối

Họ biến đổi lũy thừa

[Docs] Mapping to a Gaussian distribution: power transform là họ biến đổi **đơn điệu, có tham số**, nhằm đưa dữ liệu về gần phân phối chuẩn để **ổn định phương sai và giảm skew**. λ được ước lượng bằng **maximum likelihood**.

$$\text{Box-Cox } (x > 0): x^{(\lambda)} = \begin{cases} \frac{x^\lambda - 1}{\lambda} & \lambda \neq 0 \\ \ln x & \lambda = 0 \end{cases}$$

$$\text{Yeo-Johnson (mọi } x): x^{(\lambda)} = \begin{cases} [(x+1)^\lambda - 1]/\lambda & \lambda \neq 0, x \geq 0 \\ \ln(x+1) & \lambda = 0, x \geq 0 \\ -[(-x+1)^{2-\lambda} - 1]/(2-\lambda) & \lambda \neq 2, x < 0 \\ -\ln(-x+1) & \lambda = 2, x < 0 \end{cases}$$

Biến đổi	Điều kiện	Ghi chú
$\log x, \log(1+x)$	$x > 0 / x \geq 0$	Dễ diễn giải (hiệu ứng theo %); np.log1p
$\sqrt{x}, x^{1/3}$	$x \geq 0 / \text{mọi } x$	Dữ liệu đếm (Poisson: căn bậc hai ổn định phương sai)
Box-Cox	$x > 0$	λ tối ưu bằng MLE
Yeo-Johnson	Mọi x	Mặc định của PowerTransformer
QuantileTransformer	Mọi x	Phi tham số; làm méo khoảng cách tuyến tính; cần nhiều mẫu (n_quantiles)

```
usage = df[["data_usage_gb"]].dropna()
transforms = {
    "gốc": None,
    "log1p": lambda a: np.log1p(a),
    "box-cox": PowerTransformer(method="box-cox"),
    "yeo-johnson": PowerTransformer(method="yeo-johnson"),
    "quantile-normal": QuantileTransformer(output_distribution="normal", n_quantiles=1000, random_state=0),
}
for name, t in transforms.items():
    z = usage.to_numpy().ravel() if t is None else (t(usage).to_numpy().ravel() if callable(t) and not hasattr(t, "fit")
                                                    else t.fit_transform(usage).ravel())
    print(f"{name:16s} skew={stats.skew(z):7.3f} kurtosis={stats.kurtosis(z):7.3f}")
```

```
pt = PowerTransformer(method="box-cox", standardize=False).fit(usage)
print("λ Box-Cox (MLE) =", np.round(pt.lambdas_, 3), "-> gần 0 nghĩa là log là lựa chọn gần tối ưu")
```

Khi nào biến đổi phân phối thực sự cần?

- **Mô hình tuyến tính / GLM:** giúp quan hệ gần tuyến tính hơn và giảm ảnh hưởng ngoại lai. Cần thiết khi biến lệch mạnh.
- **Mô hình cây: không cần**, vì biến đổi đơn điệu không đổi thứ tự.
- **Neural network:** có ích, vì input có phân phối "đẹp" giúp tối ưu ổn định.
- **Kiểm định thống kê có giả định chuẩn:** có ích, nhưng thường nên dùng phương pháp bền vững hoặc phi tham số.

6.4. Biến đổi target cho hồi quy

Khi target lệch phải (doanh thu, giá nhà, thời gian xử lý), huấn luyện trên $\log y$ thường cải thiện mô hình. **[Docs]** `TransformedTargetRegressor` tự áp dụng `func` khi `fit` và `inverse_func` khi `predict`.

Cảnh báo thống kê (bất đẳng thức Jensen): $\exp(\mathbb{E}[\log y]) \leq \mathbb{E}[y]$. Dự đoán ngược từ thang log ước lượng **trung vị**, không phải **trung bình**, nên hệ thống bị **ước lượng thấp**. Nếu metric cần trung bình (MSE, tổng doanh thu), hãy hiệu chỉnh bằng **smearing estimator** của Duan (1983): nhân với $\exp(\hat{\epsilon})$.

```
from sklearn.compose import TransformedTargetRegressor
from sklearn.ensemble import HistGradientBoostingRegressor
from sklearn.metrics import mean_absolute_error, mean_squared_error
from sklearn.model_selection import train_test_split

rng = np.random.default_rng(0)
n = 8_000
Xr = pd.DataFrame({"tenure": rng.integers(1, 72, n), "calls": rng.poisson(1.5, n), "plan": rng.integers(0, 3, n)})
revenue = np.exp(3 + 0.02 * Xr["tenure"] + 0.3 * Xr["plan"] + rng.normal(0, 0.6, n)) # lệch phải, nhiễu nhân
Xa, Xb, ya, yb = train_test_split(Xr, revenue, test_size=0.3, random_state=0)

raw_model = HistGradientBoostingRegressor(random_state=0).fit(Xa, ya)
log_model = TransformedTargetRegressor(regressor=HistGradientBoostingRegressor(random_state=0),
                                       func=np.log, inverse_func=np.exp).fit(Xa, ya)
resid_log = np.log(ya) - np.log(log_model.predict(Xa))
smear = np.mean(np.exp(resid_log)) # hệ số smearing của Duan

for name, pred in [("y gốc", raw_model.predict(Xb)), ("log y", log_model.predict(Xb)),
                  ("log y + smearing", log_model.predict(Xb) * smear)]:
    print(f"{name:17s} MAE={mean_absolute_error(yb, pred):7.2f} RMSE={mean_squared_error(yb, pred) ** 0.5:7.2f}"
          f" tổng dự đoán/thực tế={pred.sum() / yb.sum():.3f}")
```

Mô hình trên thang log có **MAE tốt** (tối ưu trung vị) nhưng **tổng dự đoán thấp hơn thực tế**. Smearing khôi phục tổng. Điều này quan trọng khi dự báo dùng cho kế hoạch tài chính. Một lựa chọn khác là huấn luyện trực tiếp với **loss phù hợp phân phối**: `loss="poisson" / "gamma"` trong `HistGradientBoostingRegressor`, `objective="tweedie"` trong `LightGBM` (Chương 8).

6.5. Giảm chiều (Dimensionality Reduction)

Phương pháp	Loại	Mục đích chính	Ghi chú (theo tài liệu)
PCA	Tuyến tính, không giám sát	Nén, khử tương quan, khử nhiễu	Cần scale trước; <code>n_components</code> là số nguyên hoặc tỷ lệ phương sai; <code>whiten=True</code> cho phương sai đơn vị
IncrementalPCA	Tuyến tính	PCA trên dữ liệu không vừa RAM	<code>partial_fit</code> theo batch
TruncatedSVD	Tuyến tính	Ma trận sparse (TF-IDF → LSA)	Không center, giữ sparse

Phương pháp	Loại	Mục đích chính	Ghi chú (theo tài liệu)
KernelPCA	Phi tuyến	Cấu trúc phi tuyến	$O(n^2)$ bộ nhớ
NMF	Tuyến tính, không âm	Chủ đề văn bản, thành phần diễn giải được	Dữ liệu ≥ 0
LDA (Linear Discriminant)	Có giám sát	Tối đa phân tách lớp	Tối đa $K-1$ chiều
t-SNE	Phi tuyến	Chỉ trực quan hóa	Không có <code>transform</code> cho dữ liệu mới; khoảng cách giữa cụm và kích thước cụm không có nghĩa
UMAP (<code>umap-learn</code>)	Phi tuyến	Trực quan hóa, có thể làm feature	Có <code>transform</code> ; nhanh hơn t-SNE

PCA: toán học và thực hành

PCA tìm các hướng trục giao w_k tối đa phương sai, tức là các vector riêng của ma trận hiệp phương sai, tính qua SVD của ma trận đã center: $X_c = U\Sigma V^T$. Tỷ lệ phương sai giải thích của thành phần k bằng $\sigma_k^2 / \sum_j \sigma_j^2$.

```
from sklearn.decomposition import PCA

Xp = df[num_cols].dropna()
for name, pipe in [("không scale", PCA()), ("có scale", make_pipeline(StandardScaler(), PCA()))]:
    pipe.fit(Xp)
    pca = pipe if isinstance(pipe, PCA) else pipe[-1]
    print(f"{name:12s} explained variance ratio:", pca.explained_variance_ratio_.round(3))
# Không scale: PC1 ~ 100% vì total_charges (thang hàng nghìn) chiếm toàn bộ phương sai

pca_pipe = make_pipeline(StandardScaler(), PCA(n_components=0.90, svd_solver="full")).fit(Xp)
pca = pca_pipe[-1]
print("Số thành phần giữ 90% phương sai:", pca.n_components_)
loadings = pd.DataFrame(pca.components_.T * np.sqrt(pca.explained_variance_), index=num_cols,
                        columns=[f"PC{i + 1}" for i in range(pca.n_components_)])
print(loadings.round(2)) # loading = tương quan giữa feature (đã scale) và thành phần
```

Ví dụ chính thức liên quan: *Importance of Feature Scaling* của scikit-learn cho thấy PCA trên dữ liệu chưa scale bị chi phối hoàn toàn bởi feature có thang lớn nhất. Pipeline `StandardScaler` → `PCA` → `classifier` cho độ chính xác cao hơn hẳn.

TruncatedSVD cho văn bản (LSA) và t-SNE để nhìn dữ liệu

```
from sklearn.decomposition import TruncatedSVD
from sklearn.feature_extraction.text import TfidfVectorizer
from sklearn.manifold import TSNE

docs = ["mạng chậm buổi tối", "mạng rớt liên tục", "cước tháng cao", "cước cao hơn quảng cáo",
        "muốn hủy hợp đồng", "chuyển sang nhà mạng khác", "gói cước mới", "khuyến mãi gói cước"] * 25
tfidf = TfidfVectorizer().fit_transform(docs)
lsa = TruncatedSVD(n_components=3, random_state=0).fit(tfidf)
print("LSA explained variance:", lsa.explained_variance_ratio_.round(3), "| input sparse:", tfidf.format)

sample = make_pipeline(StandardScaler()).fit_transform(Xp.sample(1500, random_state=0))
emb = TSNE(n_components=2, perplexity=30, init="pca", learning_rate="auto", random_state=0).fit_transform(sample)
fig, ax = plt.subplots(figsize=(6, 5))
ax.scatter(emb[:, 0], emb[:, 1], s=4, c=df.loc[Xp.sample(1500, random_state=0).index, "churn"], cmap="coolwarm")
ax.set_title("t-SNE: chỉ đề'nhìn, không dùng khoảng cách/kích thước cụm để'kết luận")
print("t-SNE embedding:", emb.shape)
```

6.6. Ghép nối: scaler khác nhau cho từng nhóm cột

```
from sklearn.compose import ColumnTransformer
from sklearn.impute import SimpleImputer
```

```

from sklearn.pipeline import Pipeline

scaling = ColumnTransformer([
    ("skewed", Pipeline([("imp", SimpleImputer(strategy="median")),
                        ("pow", PowerTransformer(method="yeo-johnson"))]), ["data_usage_gb", "total_charges"]),
    ("outliers", Pipeline([("imp", SimpleImputer(strategy="median")),
                        ("rob", RobustScaler())]), ["monthly_charges"]),
    ("regular", Pipeline([("imp", SimpleImputer(strategy="median")),
                        ("std", StandardScaler())]), ["age", "tenure_months", "support_calls"]),
], verbose_feature_names_out=False).set_output(transform="pandas")
out = scaling.fit_transform(df[num_cols])
print(out.describe().T[["mean", "std", "min", "max"]].round(2))

```

6.7. Pitfalls

1. **Leakage do scale trên toàn bộ dữ liệu** trước khi chia: `StandardScaler().fit(X_all)` khiến mean/std chứa thông tin tập test. Luôn đặt scaler trong `Pipeline`.
2. **Quên lưu scaler**: production phải dùng đúng scaler đã fit lúc train. Hãy lưu **cả pipeline** (`joblib` / `MLflow`), không lưu riêng mô hình.
3. **MinMaxScaler trong production**: giá trị mới ngoài `[min, max]` của train cho output ngoài `[0, 1]`. Dùng `clip=True` hoặc giám sát drift.
4. **Box-Cox với giá trị ≤ 0** : lỗi. Dùng Yeo-Johnson hoặc `log1p`.
5. **Scale biến one-hot**: thường không cần và làm khó diễn giải hệ số. Với L1/L2 mạnh, cân nhắc kỹ.
6. **QuantileTransformer với ít dữ liệu**: `n_quantiles` lớn hơn số mẫu sẽ bị điều chỉnh, tạo các bậc thang. Phép biến đổi phi tuyến làm méo khoảng cách nên không phù hợp mọi mô hình.
7. **Diễn giải t-SNE như phân cụm**: sai. t-SNE không bảo toàn khoảng cách toàn cục, và kết quả thay đổi theo perplexity.
8. **Biến đổi log target rồi báo cáo tổng/mean** mà không hiệu chỉnh Jensen.

Checklist Chương 6

- ☐ Chỉ scale khi mô hình cần (khoảng cách, regularization, gradient, PCA); mô hình cây có thể bỏ qua.
- ☐ Chọn scaler theo phân phối và ngoại lai; dữ liệu sparse dùng `MaxAbsScaler` / `with_mean=False`.
- ☐ Biến lệch mạnh dùng log/Yeo-Johnson cho mô hình tuyến tính/NN; kiểm tra bằng skew và Q-Q.
- ☐ Target lệch: cân nhắc `TransformedTargetRegressor` + smearing, hoặc loss Poisson/Gamma/Tweedie.
- ☐ PCA fit trong pipeline sau khi scale; báo cáo explained variance; t-SNE/UMAP chỉ để trực quan.
- ☐ Scaler và mọi transformer được serialize cùng mô hình.

Tài liệu tham khảo Chương 6

- scikit-learn User Guide: *Preprocessing data* (Standardization, Scaling sparse data, Scaling data with outliers, Non-linear transformation, Mapping to a Gaussian distribution, Normalization); *Transforming target in regression*; *Decomposing signals in components* (PCA, IncrementalPCA, KernelPCA, TruncatedSVD, NMF); *Manifold learning* (t-SNE).
- scikit-learn Examples: *Compare the effect of different scalers on data with outliers*; *Importance of Feature Scaling*; *Effect of transforming the targets in regression model*.
- Box, G. E. P. & Cox, D. R. (1964). *An analysis of transformations*. JRSS-B 26(2). · Yeo, I.-K. & Johnson, R. (2000). *A new family of power transformations*. Biometrika 87(4).
- Duan, N. (1983). *Smearing Estimate: A Nonparametric Retransformation Method*. JASA 78(383).
- Wattenberg, M., Viégas, F. & Johnson, I. (2016). *How to Use t-SNE Effectively*. Distill.
- Stanford CS229 Lecture Notes: *Principal Components Analysis*. · Hastie, Tibshirani & Friedman. *The Elements of Statistical Learning*, §3.4 (shrinkage và scaling), §14.5 (PCA).

CHƯƠNG 7. CHIA DỮ LIỆU & CHIẾN LƯỢC VALIDATION

"Your validation strategy matters more than your model."

Mục tiêu chương: ước lượng **trung thực** hiệu năng của mô hình trên dữ liệu tương lai mà nó chưa từng thấy. Muốn vậy, cách chia dữ liệu phải **mô phỏng đúng cách mô hình được dùng trong production**: dự đoán cho thời điểm sau, cho khách hàng mới, với thông tin có sẵn lúc đó.

7.1. Ba tập dữ liệu và nguyên tắc sử dụng

Tập	Vai trò	Dùng để	Tỷ lệ điển hình
Train	Học tham số	fit	60-80%
Validation (hoặc CV trên train)	Ra quyết định: chọn mô hình, siêu tham số, ngưỡng, early stopping, feature	So sánh	10-20%
Test (hold-out)	Ước lượng cuối cùng, một lần	Báo cáo	10-20%

Quy tắc vàng. Tập test bị "đốt" ngay khi bạn dùng nó để ra **bất kỳ** quyết định nào. Lặp lại vòng "train → xem test → chỉnh → xem test" biến test thành validation và kết quả báo cáo bị lạc quan. Andrew Ng gọi đây là *overfitting the dev set*. Khi điều đó xảy ra, cần một tập dev mới.

Thiết lập

```
import numpy as np
import pandas as pd
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import roc_auc_score
from sklearn.model_selection import (GroupKFold, KFold, RepeatedStratifiedKFold, StratifiedGroupKFold,
                                     StratifiedKFold, TimeSeriesSplit, cross_val_score, cross_validate,
                                     train_test_split)
from sklearn.pipeline import make_pipeline

from churn.data.split import assert_no_leakage, purged_time_splits, time_split
from churn.data.synthetic import make_churn_data
from churn.features.build import RAW_FEATURES, build_preprocessor
from churn.features.clean import clean_churn

pd.set_option("display.width", 160)
df = clean_churn(make_churn_data(n=20_000, seed=42))
X, y = df[RAW_FEATURES], df["churn"]
rng = np.random.default_rng(7)
```

7.2. Bảng chọn chiến lược chia

Tình huống	Chiến lược	scikit-learn
i.i.d., cân bằng	Random split / K-Fold	<code>train_test_split</code> , <code>KFold(shuffle=True)</code>
Phân loại mất cân bằng	Stratified	<code>StratifiedKFold</code> , <code>train_test_split(stratify=y)</code>
Nhiều bản ghi / một thực thể	Group : một nhóm chỉ thuộc một tập	<code>GroupKFold</code> , <code>StratifiedGroupKFold</code> , <code>GroupShuffleSplit</code> , <code>LeaveOneGroupOut</code>
Dự đoán tương lai	Out-of-time	Chia theo ngày; <code>TimeSeriesSplit</code>

Tình huống	Chiến lược	scikit-learn
Nhấn có cửa sổ tương lai (chống lẩn)	Time split + purging + embargo	Tự viết (<code>churn.data.split.purged_time_splits</code>)
Dữ liệu ít	Repeated stratified K-Fold	<code>RepeatedStratifiedKFold</code>
Vừa tune vừa ước lượng không bias	Nested CV	<code>GridSearchCV</code> bên trong <code>cross_val_score</code>
Có sẵn tập validation cố định	Predefined split	<code>PredefinedSplit</code>
Dữ liệu không gian	Spatial/block CV	<code>GroupKFold</code> theo ô lưới

[Docs] *A note on shuffling*: nếu thứ tự dữ liệu không ngẫu nhiên (ví dụ dữ liệu được sắp theo nhãn), cần shuffle để CV có ý nghĩa. Ngược lại, nếu các mẫu **không i.i.d.**, ví dụ bài báo xếp theo thời gian xuất bản, thì shuffle lại làm **điểm validation bị thổi phồng**, vì mô hình được kiểm tra trên mẫu "giống" mẫu huấn luyện một cách giả tạo.

7.3. Hold-out ba tập (stratified)

```
X_tr, X_tmp, y_tr, y_tmp = train_test_split(X, y, test_size=0.30, stratify=y, random_state=42)
X_va, X_te, y_va, y_te = train_test_split(X_tmp, y_tmp, test_size=0.50, stratify=y_tmp, random_state=42)
for name, t in [("train", y_tr), ("valid", y_va), ("test", y_te)]:
    print(f"{name:5s} n={len(t):6d} churn_rate={t.mean():.4f}")
```

Tập validation/test cần lớn bao nhiêu?

Sai số chuẩn của AUC (công thức Hanley & McNeil, 1982) cho biết **độ chính xác của chính phép đo**:

$$SE(AUC) = \frac{\sqrt{A(1-A) + (n_1 - 1)(Q_1 - A^2) + (n_0 - 1)(Q_2 - A^2)}}{n_1 n_0}, \quad Q_1 = \frac{A}{2-A}, \quad Q_2 = \frac{2A^2}{1+A}$$

```
def auc_standard_error(auc: float, n_pos: int, n_neg: int) -> float:
    q1, q2 = auc / (2 - auc), 2 * auc**2 / (1 + auc)
    return np.sqrt((auc * (1 - auc) + (n_pos - 1) * (q1 - auc**2) + (n_neg - 1) * (q2 - auc**2)) / (n_pos * n_neg))

for n in [500, 2_000, 10_000]:
    se = auc_standard_error(0.75, int(n * 0.165), int(n * 0.835))
    print(f"n_test={n:6d}: AUC 0.75 ± {1.96 * se:.3f} (95% CI)")
# Với 500 mẫu, hai mô hình chênh 0.02 AUC là KHÔNG phân biệt được
```

7.4. Out-of-time: chuẩn cho hầu hết bài toán kinh doanh

```
train_df, valid_df, test_df = time_split(df, "signup_date", "2023-10-01", "2024-01-01", gap_days=0)
assert_no_leakage(train_df, test_df, id_col="customer_id", date_col="signup_date")
print({k: (len(v), v["signup_date"].min().date(), v["signup_date"].max().date())
      for k, v in [("train", train_df), ("valid", valid_df), ("test", test_df)]})
```

Vì sao random split có thể đánh lừa: mô phỏng concept drift

Khi quan hệ $X \rightarrow y$ **thay đổi theo thời gian** (concept drift), random split trộn tương lai vào train và cho điểm lạc quan. Điểm out-of-time mới phản ánh thực tế.

```
n = 12_000
t = np.sort(rng.uniform(0, 1, n)) # thời gian chuẩn hóa [0, 1]
x1, x2 = rng.normal(size=n), rng.normal(size=n)
beta1 = 1.5 - 2.5 * t # tác động của x1 đảo chiều theo thời gian
p = 1 / (1 + np.exp(-(-1 + beta1 * x1 + 0.8 * x2)))
drift = pd.DataFrame({"x1": x1, "x2": x2, "t": t, "y": (rng.random(n) < p).astype(int)})
```

```
model = LogisticRegression()
random_cv = cross_val_score(model, drift[["x1", "x2"]], drift["y"],
                             cv=KFold(5, shuffle=True, random_state=0), scoring="roc_auc").mean()
past, future = drift[drift.t < 0.8], drift[drift.t >= 0.8]
oot = roc_auc_score(future["y"], model.fit(past[["x1", "x2"]], past["y"]).predict_proba(future[["x1", "x2"]])[:, 1])
print(f"Random 5-fold CV AUC = {random_cv:.3f} | Out-of-time AUC = {oot:.3f}")
```

[Kinh nghiệm] Đánh giá production nên vừa **out-of-time** (giai đoạn sau) vừa **out-of-sample** (khách hàng mới). Báo cáo hiệu năng **theo từng tháng** của giai đoạn test để thấy xu hướng suy giảm.

7.5. Cross-validation đúng cách

Hồ sơ các bộ chia (minh họa chỉ số)

```
toy = np.arange(12)
toy_y = np.array([0] * 8 + [1] * 4)
toy_groups = np.repeat(["A", "B", "C", "D", "E", "F"], 2)
splitters = {
    "KFold(3)": KFold(3),
    "StratifiedKFold(3)": StratifiedKFold(3),
    "GroupKFold(3)": GroupKFold(3),
    "TimeSeriesSplit(3)": TimeSeriesSplit(3),
    "TimeSeriesSplit(3,gap=1)": TimeSeriesSplit(3, gap=1),
}
for name, sp in splitters.items():
    folds = [f"train={tr.tolist()} val={va.tolist()}" for tr, va in sp.split(toy, toy_y, groups=toy_groups)]
    print(f"{name}:\n    " + "\n    ".join(folds))
```

cross_validate : nhiều metric, điểm train, chỉ số fold

```
pipe = make_pipeline(build_preprocessor(), LogisticRegression(max_iter=3000))
cv = StratifiedKFold(n_splits=5, shuffle=True, random_state=42) # số nguyên: cùng fold cho mọi mô hình
res = cross_validate(pipe, X_tr, y_tr, cv=cv, n_jobs=-1, return_train_score=True, return_indices=True,
                     scoring={"roc_auc": "roc_auc", "pr_auc": "average_precision", "neg_log_loss": "neg_log_loss"})
summary = pd.DataFrame({k: v for k, v in res.items() if k.startswith(("train_", "test_"))})
print(summary.agg(["mean", "std"]).T.round(4))
# train >> test: overfitting | std lớn: mô hình không ổn định hoặc dữ liệu ít
```

[Docs] *Note on inappropriate usage of cross_val_predict* : kết quả của `cross_val_predict` có thể khác `cross_val_score`, vì nó trả về dự đoán của **nhiều mô hình khác nhau** gộp chung lại. Do đó nó **không phải** thước đo sai số tổng quát hóa phù hợp. Hãy dùng nó cho: (1) out-of-fold predictions để **chọn ngưỡng / calibrate / stacking**, (2) trực quan hóa.

```
from sklearn.model_selection import cross_val_predict

oof = cross_val_predict(pipe, X_tr, y_tr, cv=cv, method="predict_proba", n_jobs=-1)[:, 1]
print("OOF predictions:", oof.shape, "| dùng cho threshold tuning & calibration (Chương 9)")
```

Bao nhiêu fold? Lặp lại CV

- K nhỏ (5): mỗi mô hình huấn luyện trên ít dữ liệu hơn, nên ước lượng hơi **bi quan** (bias), nhưng nhanh.
- K lớn (10, LOOCV): ít bias, nhưng **phương sai cao** (các tập train gần trùng nhau) và chậm.
- Thực hành tốt: K = 5 hoặc 10. Với dữ liệu nhỏ, dùng **lặp lại** (RepeatedStratifiedKFold) để giảm phương sai của ước lượng.

```
small = X_tr.iloc[:1500], y_tr.iloc[:1500]
for name, splitter in [("5-fold x1", StratifiedKFold(5, shuffle=True, random_state=0)),
                      ("5-fold x5", RepeatedStratifiedKFold(n_splits=5, n_repeats=5, random_state=0))]:
    s = cross_val_score(pipe, *small, cv=splitter, scoring="roc_auc", n_jobs=-1)
    print(f"{name:10s}: {s.mean():.4f} ± {s.std():.4f} (n_fits={len(s)})")
```

Lưu ý thống kê. Các fold CV **không độc lập** (tập train chồng lấn), nên `std` qua các fold **đánh giá thấp** độ bất định thật. So sánh hai mô hình bằng t-test thông thường trên điểm fold là sai. Cần dùng *corrected resampled t-test* (Nadeau & Bengio, 2003), xem Chương 9.10.

Permutation test score: mô hình có học được gì không?

[Docs] `permutation_test_score` xáo trộn nhãn n lần để tạo phân phối null. H_0 là "mô hình không khai thác được phụ thuộc nào giữa feature và target". Hữu ích khi dữ liệu nhỏ và muốn chắc chắn điểm số không do may mắn.

```
from sklearn.model_selection import permutation_test_score

score, perm_scores, pvalue = permutation_test_score(
    make_pipeline(build_preprocessor(), LogisticRegression(max_iter=2000)), X_tr.iloc[:800], y_tr.iloc[:800],
    scoring="roc_auc", cv=StratifiedKFold(5, shuffle=True, random_state=0), n_permutations=50,
    random_state=0, n_jobs=-1)
print(f"AUC thật={score:.3f} | AUC khi xáo nhãn={perm_scores.mean():.3f}±{perm_scores.std():.3f} | p={pvalue:.3f}")
```

7.6. Group K-Fold: tránh rò rỉ theo thực thể

Khi một khách hàng có nhiều snapshot (mỗi tháng một dòng), random split để cùng khách hàng xuất hiện ở cả train và validation. Mô hình "nhớ" khách hàng thay vì học quy luật tổng quát.

```
# Dữ liệu panel: 1 500 khách × 8 tháng; mỗi khách có "độ trung thành" ổn định theo thời gian
n_cust, n_months = 1_500, 8
cust = np.repeat(np.arange(n_cust), n_months)
latent = np.repeat(rng.normal(0, 2.5, n_cust), n_months) # đặc tính riêng, ổn định của từng khách
fingerprint = np.repeat(rng.uniform(0, 1, n_cust), n_months) # feature định danh vô nghĩa (vd: mã vùng chỉ
tiết)
usage = rng.normal(0, 1, n_cust * n_months)
yp = (rng.random(n_cust * n_months) < 1 / (1 + np.exp(-(-1.5 + latent + 1.0 * usage)))).astype(int)
panel = pd.DataFrame({"fingerprint": fingerprint, "usage": usage, "y": yp, "customer": cust})

from sklearn.ensemble import RandomForestClassifier

rf = RandomForestClassifier(n_estimators=200, n_jobs=-1, random_state=0)
Xp, yp_ = panel[["fingerprint", "usage"]], panel["y"]
random_auc = cross_val_score(rf, Xp, yp_, cv=StratifiedKFold(5, shuffle=True, random_state=0),
scoring="roc_auc").mean()
group_auc = cross_val_score(rf, Xp, yp_, cv=StratifiedGroupKFold(5, shuffle=True, random_state=0),
groups=panel["customer"], scoring="roc_auc").mean()
print(f"Random K-Fold AUC = {random_auc:.3f} <- ảo: mô hình nhận ra khách qua 'fingerprint'")
print(f"Group K-Fold AUC = {group_auc:.3f} <- trung thực với khách hàng mới")
```

7.7. Chuỗi thời gian: TimeSeriesSplit, gap, purging, embargo

[Docs] `TimeSeriesSplit` tạo các fold **expanding window**: train luôn ở trước validation. Các tham số quan trọng:

- `gap`: số **mẫu** bị loại giữa train và validation (không phải số ngày; dữ liệu phải được sắp theo thời gian).
- `max_train_size`: giới hạn cửa sổ train (**sliding window**) khi dữ liệu cũ không còn đại diện.
- `test_size`: kích thước mỗi fold validation.

Expanding window (mặc định)	Sliding window (max_train_size)
Fold 1: [TRAIN]—gap—[VAL]	Fold 1: [TRAIN]—gap—[VAL]
Fold 2: [TRAIN TRAIN]—gap—[VAL]	Fold 2: [TRAIN]—gap—[VAL]
Fold 3: [TRAIN TRAIN TRAIN]—gap—[VAL]	Fold 3: [TRAIN]—gap—[VAL]

```
ordered = df.sort_values("signup_date").reset_index(drop=True)
tscv = TimeSeriesSplit(n_splits=4, gap=300, test_size=2_000)
rows = []
```

```
for k, (tr, va) in enumerate(tscv.split(ordered)):
    m = make_pipeline(build_preprocessor(), LogisticRegression(max_iter=3000)).fit(
        ordered.loc[tr, RAW_FEATURES], ordered.loc[tr, "churn"])
    auc = roc_auc_score(ordered.loc[va, "churn"], m.predict_proba(ordered.loc[va, RAW_FEATURES])[:, 1])
    rows.append({"fold": k, "train_end": ordered.loc[tr[-1], "signup_date"].date(),
                "valid": f"{ordered.loc[va[0], 'signup_date'].date()} → {ordered.loc[va[-1],
                'signup_date'].date()}",
                "n_train": len(tr), "auc": round(auc, 4)})
print(pd.DataFrame(rows).to_string(index=False)) # backtest: hiệu năng qua từng giai đoạn
```

Purged K-Fold với Embargo (nhãn chống lẩn)

Khi nhãn tại thời điểm t phụ thuộc dữ liệu trong khoảng $[t, t+h]$ (churn trong 30 ngày tới, lợi nhuận 5 ngày tới), mẫu train **gần ranh giới** có nhãn "nhìn thấy" giai đoạn validation. López de Prado (2018, ch. 7) đề xuất **purge** (loại mẫu train có cửa sổ nhãn chồng lên validation) và **embargo** (loại thêm một khoảng sau validation):

```
dates = df["signup_date"].reset_index(drop=True)
for k, (tr, va) in enumerate(purged_time_splits(dates, n_splits=5, horizon=pd.Timedelta(days=30),
                                              embargo=pd.Timedelta(days=7))):
    removed = len(dates) - len(tr) - len(va)
    print(f"fold {k}: train={len(tr):5d} valid={len(va):5d} bị purge/embargo={removed:4d}")
```

7.8. Nested CV: ước lượng không bias khi có tuning

Dùng cùng một CV để vừa **chọn siêu tham số** vừa **báo cáo điểm** cho kết quả lạc quan, vì ta đã chọn cấu hình "may mắn" nhất trên chính các fold đó. **[Docs]** Ví dụ chính thức *Nested versus non-nested cross-validation* (bộ Iris, SVC) cho thấy điểm non-nested cao hơn có hệ thống.

```
from sklearn.datasets import load_breast_cancer
from sklearn.model_selection import GridSearchCV
from sklearn.preprocessing import StandardScaler
from sklearn.svm import SVC

Xb, yb = load_breast_cancer(return_X_y=True)
idx = np.random.default_rng(0).permutation(len(yb))[:120] # dữ liệu nhỏ: bias lạc quan lộ rõ
Xb, yb = Xb[idx], yb[idx]
grid = {"svc__C": [0.01, 0.1, 1, 10, 100], "svc__gamma": [1e-4, 1e-3, 1e-2, 0.1, 1]}
non_nested, nested = [], []
for trial in range(5):
    inner = StratifiedKFold(4, shuffle=True, random_state=trial)
    outer = StratifiedKFold(4, shuffle=True, random_state=100 + trial)
    search = GridSearchCV(make_pipeline(StandardScaler(), SVC()), grid, cv=inner, scoring="accuracy")
    search.fit(Xb, yb)
    non_nested.append(search.best_score_) # chọn và báo cáo trên cùng fold
    nested.append(cross_val_score(search, Xb, yb, cv=outer, scoring="accuracy").mean())
diff = np.array(non_nested) - np.array(nested)
print(f"Non-nested={np.mean(non_nested):.4f} | Nested={np.mean(nested):.4f} | chênh lệch lạc quan={diff.mean():+.4f}")
```

Chênh lệch tăng khi **dữ liệu ít** và **lưới tìm kiếm lớn** (càng nhiều cấu hình, càng dễ gặp cấu hình "may mắn"). Trong thực tế: dùng nested CV để **báo cáo** hiệu năng của *quy trình* (bao gồm tuning). Dùng `search.best_estimator_` refit trên toàn bộ train làm mô hình cuối, rồi kiểm tra một lần trên test out-of-time.

7.9. Danh mục data leakage

Kapoor & Narayanan (2023), *Leakage and the reproducibility crisis in ML-based science*, tổng hợp 294 bài báo bị ảnh hưởng bởi leakage. Có thể phân loại như sau:

Loại	Ví dụ	Phòng tránh
Không tách train/test	Feature selection, impute, scale trên toàn bộ dữ liệu	Pipeline + CV (Chương 5)
Tiền xử lý dùng thông tin test	Target encoding không cross-fitting	TargetEncoder.fit_transform
Feature không hợp lệ (target leakage)	Feature chỉ có sau sự kiện (refund, lý do hủy)	Kiểm tra thời điểm sinh feature; AUC đơn biến
Phân phối test không khớp phạm vi áp dụng	Test ngẫu nhiên khi production là tương lai	Out-of-time, out-of-sample
Phụ thuộc thời gian	Random split dữ liệu chuỗi thời gian; dùng dữ liệu tương lai	Time split, point-in-time join
Phụ thuộc giữa train và test	Cùng khách hàng/bệnh nhân ở cả hai tập	Group split
Trùng lặp	Bản ghi trùng hoặc gần trùng giữa train/test	Dedup trước khi chia (cả near-duplicate)
Tuning trên test	Chọn mô hình theo điểm test	Validation riêng / nested CV

```
def leakage_audit(train: pd.DataFrame, test: pd.DataFrame, id_col: str, date_col: str,
                 feature_cols: list[str]) -> dict:
    """Các kiểm tra tự động nên chạy trong CI cho mọi lần chia dữ liệu."""
    dup = train[feature_cols].merge(test[feature_cols], how="inner").shape[0]
    return {
        "id_overlap": len(set(train[id_col]) & set(test[id_col])),
        "time_overlap": bool(train[date_col].max() >= test[date_col].min()),
        "exact_feature_duplicates": int(dup),
        "test_rate_vs_train": round(test["churn"].mean() / train["churn"].mean(), 3),
    }

print(leakage_audit(train_df, test_df, "customer_id", "signup_date",
                    ["age", "tenure_months", "monthly_charges", "contract", "support_calls"]))
```

Checklist Chương 7

- ☐ Cách chia mô phỏng đúng kịch bản production (thời gian, thực thể mới, thông tin có sẵn).
- ☐ Stratify cho phân loại mất cân bằng; group split khi một thực thể có nhiều dòng.
- ☐ Dữ liệu thời gian: out-of-time / TimeSeriesSplit (+ gap, purge, embargo khi cần cửa sổ tương lai).
- ☐ random_state là số nguyên cho splitter; mọi mô hình được so sánh trên cùng fold.
- ☐ Báo cáo mean $\pm$ std (hoặc CI) qua fold; đủ cỡ mẫu validation để phân biệt các mô hình.
- ☐ Tuning dùng validation/inner CV; báo cáo bằng nested CV hoặc test out-of-time một lần.
- ☐ cross_val_predict chỉ dùng cho OOF (threshold, calibration, stacking), không để báo cáo điểm.
- ☐ Có kiểm tra leakage tự động (ID, thời gian, trùng lặp) trong CI.

Tài liệu tham khảo Chương 7

- scikit-learn User Guide: *Cross-validation: evaluating estimator performance* (iterators, cross_validate, cross_val_predict warning, A note on shuffling, Permutation test score); *Tuning the hyper-parameters of an estimator; Common pitfalls*.
- scikit-learn Examples: *Nested versus non-nested cross-validation; Visualizing cross-validation behavior in scikit-learn*.
- Hanley, J. A. & McNeil, B. J. (1982). *The meaning and use of the area under a ROC curve*. Radiology 143(1).
- Nadeau, C. & Bengio, Y. (2003). *Inference for the Generalization Error*. Machine Learning 52.
- Cawley, G. C. & Talbot, N. L. C. (2010). *On Over-fitting in Model Selection and Subsequent Selection Bias in Performance Evaluation*. JMLR 11.
- Kaufman, S. et al. (2012). *Leakage in Data Mining: Formulation, Detection, and Avoidance*. ACM TKDD 6(4).
- Kapoor, S. & Narayanan, A. (2023). *Leakage and the reproducibility crisis in machine-learning-based science*. Patterns 4(9).

- López de Prado, M. (2018). *Advances in Financial Machine Learning*, ch. 7. Wiley.
- Ng, A. *Machine Learning Yearning*, ch. 5–12 (dev/test sets). · Google ML Crash Course: *Datasets, generalization, and overfitting*.

CHƯƠNG 8. HUẤN LUYỆN MÔ HÌNH (MODEL TRAINING)

Mục tiêu chương: chọn họ mô hình phù hợp, huấn luyện đúng cách (baseline, early stopping, regularization), tối ưu siêu tham số hiệu quả, chẩn đoán bias/variance, và đóng gói quá trình huấn luyện thành **một lệnh tái lập được, có tracking**.

8.1. Nền tảng lý thuyết

Tối thiểu hóa rủi ro thực nghiệm (ERM)

Học có giám sát tìm hàm f trong một lớp giả thuyết $\mathcal{F}$ sao cho tối thiểu **rủi ro thực nghiệm có regularization**:

$$\hat{f} = \arg \min_{f \in \mathcal{F}} \frac{1}{n} \sum_{i=1}^n L(y_i, f(x_i)) + \lambda \Omega(f)$$

Thứ ta thực sự quan tâm là **rủi ro kỳ vọng** trên phân phối thật $\mathbb{E}_{(x,y) \sim P}[L(y, f(x))]$. Khoảng cách giữa hai đại lượng này là **generalization gap**. Toàn bộ Chương 7 nhằm ước lượng đúng khoảng cách đó.

Chọn hàm loss theo đại lượng cần dự đoán

[Docs] scikit-learn *Which scoring function should I use?* (dựa trên Gneiting, 2011): chọn **hàm loss nhất quán chặt (strictly consistent)** với đại lượng thống kê (functional) cần dự đoán. Nên dùng cùng hàm đó cho cả huấn luyện và đánh giá.

Đại lượng cần dự đoán	Loss nhất quán	scikit-learn / LightGBM
Xác suất lớp (mean của Y nhị phân)	Log loss , Brier	<code>LogisticRegression</code> , <code>objective="binary"</code>
Mean của Y liên tục	Squared error	<code>loss="squared_error"</code> , <code>objective="regression"</code>
Median	Absolute error	<code>loss="absolute_error"</code> , <code>objective="l1"</code>
Quantile α	Pinball loss	<code>loss="quantile"</code> , <code>quantile=α</code> , <code>objective="quantile"</code>
Mean của dữ liệu đếm ≥ 0	Poisson deviance	<code>loss="poisson"</code> , <code>objective="poisson"</code>
Mean của Y dương lệch phải	Gamma / Tweedie deviance	<code>loss="gamma"</code> , <code>objective="tweedie"</code>
Xếp hạng	Pairwise/listwise (LambdaRank)	<code>LGBMRanker(objective="lambdarank")</code>

Phân rã bias-variance

Với loss bình phương: $\mathbb{E}[(y - \hat{f}(x))^2] = \underbrace{\sigma^2}_{\text{nhiều}} + \underbrace{(\mathbb{E}\hat{f}(x) - f(x))^2}_{\text{bias}^2} + \underbrace{\text{Var}(\hat{f}(x))}_{\text{variance}}$.

[Course] Andrew Ng (*Machine Learning Yearning; Deep Learning Specialization*, khóa 3) đưa ra quy trình chẩn đoán:

Train error	Dev error	So với mức tối ưu (Bayes/con người)	Chẩn đoán	Hành động
Cao	Cao ($\approx$ train)	Xa	Avoidable bias	Mô hình lớn hơn, thêm feature, giảm regularization, train lâu hơn
Thấp	Cao	Gần	Variance	Thêm dữ liệu, regularization, early stopping, giảm độ phức tạp, bớt feature
Thấp	Thấp, nhưng test/production cao	—	Data mismatch / leakage	Sửa cách chia (Chương 7), adversarial validation

Thiết lập

```
import time

import lightgbm as lgb
import numpy as np
import pandas as pd
from sklearn.dummy import DummyClassifier
from sklearn.ensemble import HistGradientBoostingClassifier, RandomForestClassifier
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import average_precision_score, log_loss, roc_auc_score
from sklearn.model_selection import StratifiedKFold, cross_validate
from sklearn.pipeline import Pipeline, make_pipeline

from churn.data.split import time_split
from churn.data.synthetic import make_churn_data
from churn.features.build import RAW_FEATURES, build_preprocessor
from churn.features.clean import clean_churn

pd.set_option("display.width", 160)
df = clean_churn(make_churn_data(n=20_000, seed=42))
train_df, valid_df, test_df = time_split(df, "signup_date", "2023-10-01", "2024-01-01")
X_train, y_train = train_df[RAW_FEATURES], train_df["churn"]
X_valid, y_valid = valid_df[RAW_FEATURES], valid_df["churn"]
cv = StratifiedKFold(n_splits=5, shuffle=True, random_state=42)
pos_weight = float((y_train == 0).sum() / (y_train == 1).sum())
print(X_train.shape, X_valid.shape, round(pos_weight, 2))
```

8.2. Baseline: luôn bắt đầu từ đây

```
dummy = DummyClassifier(strategy="prior").fit(X_train, y_train)
p_dummy = dummy.predict_proba(X_valid)[:, 1]

def rule_based_score(X: pd.DataFrame) -> np.ndarray:
    """Heuristic CSKH đang dùng: hợp đồng tháng + nhiều khiếu nại + khách mới."""
    return ((X["contract"] == "month-to-month").astype(int) * 2 + (X["support_calls"] >= 3).astype(int)
            + (X["tenure_months"] < 6).astype(int)).to_numpy()

baselines = pd.DataFrame({
    "dummy(prior)": [roc_auc_score(y_valid, p_dummy), average_precision_score(y_valid, p_dummy)],
    "rule-based": [roc_auc_score(y_valid, rule_based_score(X_valid)),
                  average_precision_score(y_valid, rule_based_score(X_valid))],
}, index=["roc_auc", "pr_auc"]).T
print(baselines.round(4)) # PR-AUC của dummy = prevalence: sần tuyệt đối
```

8.3. Các họ mô hình: cơ chế, giả định, tham số chính

Họ	Cơ chế	Mạnh	Yếu	Tham số quan trọng
Logistic / Linear	Tổ hợp tuyến tính + link function	Nhanh, diễn giải tốt, xác suất calibrate tốt khi đúng đặc tả	Cần feature engineering cho phi tuyến	C, l1_ratio, solver, class_weight
Decision Tree	Chia không gian theo ngưỡng	Diễn giải trực quan	Variance rất cao	max_depth, min_samples_leaf, ccp_alpha
Random Forest	Bagging cây sâu + chọn ngẫu nhiên feature	Bền vững, ít cần tuning	Kém calibrate (dồn về giữa), mô hình lớn	n_estimators, max_features, min_samples_leaf

Họ	Cơ chế	Mạnh	Yếu	Tham số quan trọng
Gradient Boosting (HGB, LightGBM, XGBoost, CatBoost)	Cộng dồn cây nâng sửa sai số theo gradient	Mạnh nhất cho dữ liệu bảng	Nhiều tham số, dễ overfit nếu không early stopping	learning_rate , num_leaves / max_depth , min_child_samples , subsampling, L1/L2
SVM	Siêu phẳng lẻ cực đại (+ kernel)	Tốt với dữ liệu vừa, chiều cao	$O(n^2-n^3)$, không ra xác suất trực tiếp	C , gamma , kernel
KNN	Bỏ phiếu láng giếng	Không giả định	Chậm khi dự đoán, nhạy thang đo	n_neighbors , weights , metric
Naive Bayes	Bayes + giả định độc lập có điều kiện	Rất nhanh, văn bản	Giả định mạnh, xác suất cực đoan	alpha (smoothing)
Neural Network (MLP)	Hàm hợp phi tuyến	Đa phương thức, dữ liệu lớn	Cần nhiều dữ liệu và tuning	Kiến trúc, learning_rate , alpha , early stopping

Logistic Regression trong scikit-learn ≥ 1.8 : đặc tả penalty mới

[Docs] Từ phiên bản 1.8, tham số `penalty` của `LogisticRegression` **bị deprecate** (sẽ xóa ở 1.10). Thay bằng `l1_ratio + C`: `l1_ratio=0` là L2 (mặc định), `l1_ratio=1` là L1, `0 < l1_ratio < 1` là Elastic-Net, `C=np.inf` là không regularization. Bảng tương thích solver:

solver	l1_ratio hỗ trợ	Đa lớp multinomial	Ghi chú tài liệu
lbfgs (mặc định)	0	✓	Mặc định tốt cho đa số bài toán
liblinear	0 hoặc 1	✗	Tốt cho dữ liệu nhỏ
newton-cg	0	✓	
newton-cholesky	0	✓	Tốt khi $n_{\text{samples}} \gg n_{\text{features}}$, nhiều one-hot hiếm
sag	0	✓	Nhanh với dữ liệu lớn; cần feature cùng thang
saga	0 $\rightarrow$ 1	✓	Duy nhất hỗ trợ Elastic-Net

```
logit_variants = {
    "L2 (lbfgs)": LogisticRegression(C=1.0, max_iter=3000),
    "L1 (liblinear)": LogisticRegression(C=0.1, l1_ratio=1, solver="liblinear", max_iter=3000),
    "ElasticNet (saga)": LogisticRegression(C=0.1, l1_ratio=0.5, solver="saga", max_iter=5000),
    "Không penalty": LogisticRegression(C=np.inf, max_iter=3000),
}
for name, est in logit_variants.items():
    pipe = make_pipeline(build_preprocessor(scale=True), est).fit(X_train, y_train)
    coef = pipe[-1].coef_.ravel()
    print(f"{name:18s} PR-AUC={average_precision_score(y_valid, pipe.predict_proba(X_valid)[: , 1]):.4f}"
          f" hệ số = 0: {(np.abs(coef) < 1e-8).sum()}/{coef.size}")
```

8.4. So sánh nhiều mô hình trên cùng một CV

```
from catboost import CatBoostClassifier
from xgboost import XGBClassifier

candidates = {
    "logreg": make_pipeline(build_preprocessor(scale=True), LogisticRegression(max_iter=3000)),
    "random_forest": make_pipeline(build_preprocessor(scale=False), RandomForestClassifier(
        n_estimators=300, min_samples_leaf=10, max_features="sqrt", n_jobs=1, random_state=0)),
    "hist_gb": make_pipeline(build_preprocessor(scale=False), HistGradientBoostingClassifier(
        learning_rate=0.05, max_leaf_nodes=15, min_samples_leaf=50, max_iter=300, random_state=0)),
    "lightgbm": make_pipeline(build_preprocessor(scale=False), lgb.LGBMClassifier(
        n_estimators=300, learning_rate=0.03, num_leaves=15, min_child_samples=50, subsample=0.8,
        subsample_freq=1, colsample_bytree=0.8, reg_lambda=1.0, n_jobs=1, verbose=-1, random_state=0)),
```

```

"xbgboost": make_pipeline(build_preprocessor(scale=False), XGBClassifier(
    n_estimators=300, learning_rate=0.03, max_depth=4, min_child_weight=5, subsample=0.8,
    colsample_bytree=0.8, tree_method="hist", random_state=0, n_jobs=1)),
"catboost": make_pipeline(build_preprocessor(scale=False), CatBoostClassifier(
    iterations=300, learning_rate=0.05, depth=5, random_seed=0, verbose=0, thread_count=1)),
}
rows = []
for name, model in candidates.items():
    # Song song ở cấp fold (joblib), mỗi mô hình 1 luồng: tránh oversubscription (mục 8.11)
    r = cross_validate(model, X_train, y_train, cv=cv, n_jobs=-1,
        scoring={"roc_auc": "roc_auc", "pr_auc": "average_precision", "log_loss": "neg_log_loss"})
    rows.append({"model": name, "roc_auc": r["test_roc_auc"].mean(), "pr_auc": r["test_pr_auc"].mean(),
        "pr_auc_std": r["test_pr_auc"].std(), "log_loss": -r["test_log_loss"].mean(),
        "fit_s": r["fit_time"].mean()})
leaderboard = pd.DataFrame(rows).set_index("model").sort_values("pr_auc", ascending=False)
print(leaderboard.round(4))

```

Đọc kết quả với tư duy thống kê. Cơ chế sinh dữ liệu mô phỏng **gần tuyến tính trên thang log-odds** (Chương 0.8), nên Logistic Regression đứng ngang hoặc trên các mô hình boosting. Đây là bài học quan trọng: GBM không tự động thắng. Chênh lệch giữa các mô hình nhỏ hơn độ lệch chuẩn giữa các fold thì **chưa đủ bằng chứng** để chọn mô hình phức tạp hơn (Chương 9.10). Trong dữ liệu thật có tương tác và phi tuyến mạnh, GBM thường dẫn đầu.

8.5. Gradient Boosting chuyên sâu

Thuật toán (Friedman, 2001) và cải tiến bậc hai (XGBoost)

Khởi tạo $F_0(x) = \arg \min_c \sum L(y_i, c)$. Tại bước m :

1. Tính **pseudo-residual** $g_i = \partial L(y_i, F) / \partial F|_{F_{m-1}}$ (và Hessian h_i với XGBoost/LightGBM).
2. Khớp cây h_m để dự đoán $-g_i$. Giá trị lá tối ưu theo xấp xỉ Taylor bậc hai là $w_j^* = -\frac{\sum_{i \in j} g_i}{\sum_{i \in j} h_i + \lambda}$.
3. Cập nhật $F_m = F_{m-1} + \eta h_m$, với η là **learning rate (shrinkage)**.

Gain khi tách một nút (XGBoost): $\frac{1}{2}[\frac{G_L^2}{H_L + \lambda} + \frac{G_R^2}{H_R + \lambda} - \frac{(G_L + G_R)^2}{H_L + H_R + \lambda}] - \gamma$. Đây là nguồn gốc ý nghĩa của các tham số `reg_lambda` (λ), `min_split_gain` / `gamma` (γ), `min_child_weight` (tổng Hessian tối thiểu ở lá).

So sánh các triển khai

	HistGradientBoosting (sklearn)	LightGBM	XGBoost	CatBoost
Tăng trưởng cây	Leaf-wise (max_leaf_nodes)	Leaf-wise	Depth-wise (mặc định), grow_policy="lossguide"	Symmetric (oblivious)
Histogram binning	✅ (255 bins)	✅	✅ (tree_method="hist")	✅
Biến phân loại gốc	categorical_features="from_dtype"	✅	enable_categorical=True	✅ Ordered target statistics
Missing gốc	✅	✅	✅	✅
Monotonic constraints	monotonic_cst	monotone_constraints	monotone_constraints	monotone_constraints
Interaction constraints	interaction_cst	interaction_constraints	interaction_constraints	—
Early stopping mặc định	Bật nếu n > 10 000	Cần validation set	Cần validation set	use_best_model

	HistGradientBoosting (sklearn)	LightGBM	XGBoost	CatBoost
Điểm đặc biệt	Không phụ thuộc ngoài sklearn	Nhanh nhất, GOSS, EFB	Hệ sinh thái lớn, GPU	Ít overfit với biến phân loại, ít cần tuning

Hướng dẫn tuning chính thức của LightGBM (*Parameters Tuning*)

[Docs] LightGBM dùng **leaf-wise growth**: hội tụ nhanh hơn depth-wise nhưng dễ overfit nếu tham số không phù hợp. Ba tham số quan trọng nhất:

1. **num_leaves** : tham số chính điều khiển độ phức tạp. Về lý thuyết $\text{num_leaves} = 2^{\text{max_depth}}$ tương đương cây depth-wise, nhưng thực tế **nên nhỏ hơn**. Ví dụ tài liệu đưa ra: $\text{max_depth}=7$ thì 127 lá có thể overfit, còn 70-80 lá có thể tốt hơn.
2. **min_data_in_leaf** (**min_child_samples**): rất quan trọng để chống overfit. Với dữ liệu lớn, đặt hàng trăm đến hàng nghìn.
3. **max_depth** : giới hạn độ sâu tường minh. Nếu đặt thì nên đặt $\text{num_leaves} \leq 2^{\text{max_depth}}$.

Mục tiêu	Khuyến nghị của tài liệu LightGBM
Chính xác hơn	max_bin lớn; learning_rate nhỏ + num_iterations lớn; num_leaves lớn (rủi ro overfit); thêm dữ liệu; thử dart
Chống overfit	max_bin nhỏ; num_leaves nhỏ; min_data_in_leaf và min_sum_hessian_in_leaf; bagging (bagging_fraction + bagging_freq); feature_fraction; lambda_l1, lambda_l2, min_gain_to_split; max_depth; extra_trees; tăng path_smooth
Nhanh hơn	ít lá/ít tầng; tăng min_gain_to_split, min_data_in_leaf; early stopping; giảm max_bin; giảm feature_fraction; bagging; số luồng = số lõi vật lý

Hướng dẫn tuning chính thức của XGBoost (*Notes on Parameter Tuning*)

[Docs] Có hai hướng kiểm soát overfit: (1) **trực tiếp giới hạn độ phức tạp**: max_depth, min_child_weight, gamma; (2) **thêm ngẫu nhiên**: subsample, colsample_bytree, giảm eta (và tăng số vòng). Với **dữ liệu mất cân bằng**:

- Nếu chỉ quan tâm **thứ hạng (AUC)**: cân bằng trọng số bằng scale_pos_weight và đánh giá bằng AUC.
- Nếu cần **xác suất đúng**: không cân bằng lại dữ liệu. Đặt max_delta_step hữu hạn (ví dụ 1) để giúp hội tụ.

Early stopping đúng cách

```

prep = build_preprocessor(scale=False)
Xtr = prep.fit_transform(X_train, y_train)          # fit tiền xử lý CHỈ trên train
Xva = prep.transform(X_valid)
feature_names = list(prep.get_feature_names_out())

booster = lgb.LGBMClassifier(
    n_estimators=5_000, learning_rate=0.02, num_leaves=15, min_child_samples=80,
    subsample=0.8, subsample_freq=1, colsample_bytree=0.8, reg_lambda=1.0,
    verbose=-1, random_state=0,
)
booster.fit(
    pd.DataFrame(Xtr, columns=feature_names), y_train,
    eval_set=[(pd.DataFrame(Xva, columns=feature_names), y_valid)],
    eval_metric=["average_precision", "binary_logloss"],
    callbacks=[lgb.early_stopping(stopping_rounds=200, first_metric_only=True, verbose=False),
               lgb.log_evaluation(period=0)],
)
hist = booster.eval_result["valid_0"]
print("best_iteration_ =", booster.best_iteration,
      "| PR-AUC tốt nhất =", round(max(hist["average_precision"]), 4))

```

Lưu ý. Tập dùng cho early stopping đã "bị nhìn", nên **không** dùng nó để báo cáo điểm cuối. Khi huấn luyện lại trên train + valid, đặt $n_estimators \approx best_iteration \times (1 + \text{tỷ lệ dữ liệu tăng thêm})$.

XGBoost và CatBoost với early stopping

```
xgb_model = XGBClassifier(
    n_estimators=5_000, learning_rate=0.02, max_depth=4, min_child_weight=5, subsample=0.8,
    colsample_bytree=0.8, reg_lambda=1.0, tree_method="hist", eval_metric="aucpr",
    early_stopping_rounds=200, random_state=0, n_jobs=2,
)
xgb_model.fit(Xtr, y_train, eval_set=[(Xva, y_valid)], verbose=False)
print("XGBoost best_iteration =", xgb_model.best_iteration,
      "| PR-AUC valid =", round(average_precision_score(y_valid, xgb_model.predict_proba(Xva[:, 1]), 4))

# CatBoost: đưa thẳng cột phân loại dạng chuỗi; ordered boosting giảm target leakage nội bộ
cat_cols = ["contract", "payment_method", "region"]
num_raw = ["age", "tenure_months", "monthly_charges", "total_charges", "support_calls", "data_usage_gb"]

def to_catboost(X: pd.DataFrame) -> pd.DataFrame:
    out = X[num_raw + cat_cols].copy()
    out[cat_cols] = out[cat_cols].fillna("missing").astype(str)
    return out

cb = CatBoostClassifier(iterations=3_000, learning_rate=0.03, depth=5, l2_leaf_reg=3, eval_metric="PRAUC",
    early_stopping_rounds=200, random_seed=0, verbose=0, thread_count=2)
cb.fit(to_catboost(X_train), y_train, cat_features=cat_cols, eval_set=(to_catboost(X_valid), y_valid),
    use_best_model=True)
print("CatBoost best_iteration =", cb.get_best_iteration(),
      "| PR-AUC valid =", round(average_precision_score(y_valid, cb.predict_proba(to_catboost(X_valid))[:, 1]), 4))
```

Monotonic constraints: mô hình phải "hợp lý" về nghiệp vụ

[Docs] `monotonic_cst` của `HistGradientBoosting*` nhận giá trị 1 (đồng biến), -1 (nghịch biến), 0 (không ràng buộc) cho từng feature. Ràng buộc giúp mô hình **hợp lý về nghiệp vụ, bền vững với nhiễu và dễ được duyệt** (ngân hàng, bảo hiểm).

```
mono_features = ["support_calls", "tenure_months", "monthly_charges", "age"]
Xm_tr, Xm_va = X_train[mono_features], X_valid[mono_features]
free = HistGradientBoostingClassifier(max_iter=300, learning_rate=0.05, random_state=0).fit(Xm_tr, y_train)
mono = HistGradientBoostingClassifier(max_iter=300, learning_rate=0.05, random_state=0,
    monotonic_cst={"support_calls": 1, "tenure_months": -1}).fit(Xm_tr, y_train)

grid_calls = Xm_va.median().to_frame().T.loc[np.repeat(0, 9)].assign(support_calls=np.arange(9))
for name, m in [("không ràng buộc", free), ("monotonic", mono)]:
    p = m.predict_proba(grid_calls)[:, 1]
    print(f"{name:16s} P(churn) theo support_calls 0..8: {np.round(p, 3)} | đơn điệu: {bool(np.all(np.diff(p) >=
-1e-12))}")
    f" | PR-AUC={average_precision_score(y_valid, m.predict_proba(Xm_va[:, 1]):.4f}")
# Dữ liệu mô phỏng ít nhiễu nên mô hình tự do cũng đã gần đơn điệu. Trên dữ liệu thật, ràng buộc loại bỏ
# các đoạn "răng cưa" phi lý (thêm 1 cuộc gọi khiến nại mà rủi ro lại giảm) và thường còn tăng tính tổng quát.
```

8.6. Tối ưu siêu tham số

Phương pháp	Ý tưởng	Ưu	Nhược	Công cụ
Grid search	Thử mọi tổ hợp	Toàn diện, song song tốt	Bùng nổ tổ hợp	GridSearchCV
Random search	Lấy mẫu ngẫu nhiên	Hiệu quả hơn grid khi ít tham số thực sự quan trọng (Bergstra & Bengio, 2012)	Không học từ lịch sử	RandomizedSearchCV

Phương pháp	Ý tưởng	Ưu	Nhược	Công cụ
Successive halving / Hyperband	Cấp ít tài nguyên cho nhiều cấu hình, loại dần	Tiết kiệm lớn	Cấu hình "khởi động chậm" bị loại oan	HalvingRandomSearchCV, HyperbandPruner
Bayesian (TPE, GP)	Mô hình hóa quan hệ tham số → điểm	Ít lần thử	Tuần tự hơn	Optuna , Hyperopt
CMA-ES	Chiến lược tiến hóa	Tốt với tham số liên tục, nhiều trial		CmaEsSampler

Nguyên tắc thực hành:

1. Không tune `n_estimators` : dùng **early stopping**.
2. Không gian tìm kiếm theo **thang log** cho learning rate và regularization.
3. Cố định **ngân sách** (số trial, thời gian). Tuning thường chỉ thêm 1-3%.
4. Tuning dùng **CV hoặc validation**; báo cáo trên dữ liệu chưa thấy (Chương 7.8).

Random search và Successive Halving (scikit-learn)

```

from scipy.stats import loguniform, randint, uniform
from sklearn.experimental import enable_halving_search_cv # noqa: F401
from sklearn.model_selection import HalvingRandomSearchCV, RandomizedSearchCV

lgbm_pipe = Pipeline([("prep", build_preprocessor(scale=False)),
                      ("clf", lgb.LGBMClassifier(n_estimators=300, subsample_freq=1, n_jobs=1, verbose=-1,
                                                  random_state=0))])

space = {
    "clf__num_leaves": randint(7, 64),
    "clf__min_child_samples": randint(20, 300),
    "clf__learning_rate": loguniform(0.01, 0.2),
    "clf__colsample_bytree": uniform(0.5, 0.5),
    "clf__subsample": uniform(0.6, 0.4),
    "clf__reg_lambda": loguniform(1e-3, 10),
}

t0 = time.perf_counter()
rs = RandomizedSearchCV(lgbm_pipe, space, n_iter=12, cv=StratifiedKFold(3, shuffle=True, random_state=0),
                        scoring="average_precision", random_state=0, n_jobs=-1).fit(X_train, y_train)
print(f"Random search: best={rs.best_score_:.4f} ({time.perf_counter() - t0:.0f}s)")

t0 = time.perf_counter()
hs = HalvingRandomSearchCV(lgbm_pipe, space, n_candidates=24, factor=3, resource="clf__n_estimators",
                           min_resources=30, max_resources=300, cv=StratifiedKFold(3, shuffle=True, random_state=0),
                           scoring="average_precision", random_state=0, n_jobs=-1).fit(X_train, y_train)
print(f"Halving search: best={hs.best_score_:.4f} | {hs.n_candidates_} ứng viên qua các vòng "
      f"({time.perf_counter() - t0:.0f}s)")

```

Optuna: Bayesian optimization với pruning

[Docs] Các khái niệm của Optuna: **Study** (một phiên tối ưu), **Trial** (một lần thử), **Sampler** (để xuất tham số: TPESampler mặc định, GPSampler, CmaEsSampler, NSGAIIISampler cho đa mục tiêu), **Pruner** (dừng sớm trial kém: MedianPruner, SuccessiveHalvingPruner, HyperbandPruner, WilcoxonPruner), **Storage** (RDB để lưu, tiếp tục và chạy phân tán).

```

import optuna
from optuna.importance import get_param_importances

optuna.logging.set_verbosity(optuna.logging.WARNING)
inner_cv = StratifiedKFold(n_splits=3, shuffle=True, random_state=0)
fold_data = []
for tr, va in inner_cv.split(X_train, y_train): # tiên xử lý mỗi fold một lần, dùng lại
    p = build_preprocessor(scale=False)
    fold_data.append((p.fit_transform(X_train.iloc[tr], y_train.iloc[tr]), y_train.iloc[tr],
                      p.transform(X_train.iloc[va]), y_train.iloc[va]))

```

```
def objective(trial: optuna.Trial) -> float:
    params = {
        "n_estimators": 2_000,
        "learning_rate": trial.suggest_float("learning_rate", 0.01, 0.2, log=True),
        "num_leaves": trial.suggest_int("num_leaves", 7, 127, log=True),
        "min_child_samples": trial.suggest_int("min_child_samples", 10, 400, log=True),
        "subsample": trial.suggest_float("subsample", 0.5, 1.0),
        "subsample_freq": 1,
        "colsample_bytree": trial.suggest_float("colsample_bytree", 0.4, 1.0),
        "reg_alpha": trial.suggest_float("reg_alpha", 1e-8, 10.0, log=True),
        "reg_lambda": trial.suggest_float("reg_lambda", 1e-8, 10.0, log=True),
        "verbose": -1, "random_state": 0,
    }
    scores = []
    for step, (xtr, ytr, xva, yva) in enumerate(fold_data):
        m = lgb.LGBMClassifier(**params).fit(
            xtr, ytr, eval_set=[(xva, yva)], eval_metric="average_precision",
            callbacks=[lgb.early_stopping(100, verbose=False)])
        scores.append(average_precision_score(yva, m.predict_proba(xva)[:, 1]))
        trial.set_user_attr(f"best_iter_{step}", int(m.best_iteration))
        trial.report(float(np.mean(scores)), step=step)  # báo cáo kết quả trung gian
        if trial.should_prune():  # pruner quyết định dừng sớm
            raise optuna.TrialPruned()
    return float(np.mean(scores))

study = optuna.create_study(
    direction="maximize", study_name="churn-lgbm",
    sampler=optuna.samplers.TPESampler(seed=0, multivariate=True, n_startup_trials=8),
    pruner=optuna.pruners.MedianPruner(n_startup_trials=5, n_warmup_steps=1),
    storage="sqlite:///optuna.db", load_if_exists=True,  # lưu lịch sử, tiếp tục được, chạy song song được
)
study.optimize(objective, n_trials=25, timeout=600)
pruned = sum(t.state == optuna.trial.TrialState.PRUNED for t in study.trials)
print(f"Best PR-AUC={study.best_value:.4f} | trials={len(study.trials)} (pruned {pruned})")
print("Best params:", {k: round(v, 4) if isinstance(v, float) else v for k, v in study.best_params.items()})
print("Tầm quan trọng tham số (fANOVA):", {k: round(v, 3) for k, v in get_param_importances(study).items()})
```

```
# Trực quan hóa (cần plotly) và dashboard
optuna.visualization.plot_optimization_history(study).show()
optuna.visualization.plot_param_importances(study).show()
optuna.visualization.plot_parallel_coordinate(study).show()
# optuna-dashboard sqlite:///optuna.db -> giao diện web theo dõi study
```

Đa mục tiêu (ví dụ tối đa PR-AUC và tối thiểu độ trễ suy luận): `optuna.create_study(directions=["maximize", "minimize"])` với `NSGAIISampler`, rồi chọn từ **mặt Pareto** `study.best_trials`.

8.7. Ensemble

Kỹ thuật	Cơ chế	Hiệu quả khi
Bagging	Trung bình mô hình trên mẫu bootstrap	Giảm variance (cây sâu)
Boosting	Mô hình sau sửa lỗi mô hình trước	Giảm bias
Soft voting / blending	Trung bình (có trọng số) xác suất	Các mô hình đa dạng , lỗi ít tương quan
Rank averaging	Trung bình thứ hạng	Metric dựa trên thứ hạng (AUC)
Stacking	Meta-model học trên OOF predictions	Thi đấu; production cần cân nhắc chi phí
Seed averaging	Cùng mô hình, nhiều seed	Ổn định dự đoán, rẻ

[Docs] `StackingClassifier` dùng `cross_val_predict` nội bộ (tham số `cv`) để tạo đặc trưng cho `final_estimator`, nhờ vậy meta-model không học trên dự đoán bị overfit. Sau đó các base estimator được fit lại trên toàn bộ dữ liệu.

```
from sklearn.ensemble import StackingClassifier, VotingClassifier
```

```

base = [("logreg", candidates["logreg"]), ("lightgbm", candidates["lightgbm"]), ("catboost",
candidates["catboost"])]
voting = VotingClassifier(base, voting="soft", weights=[2, 1, 1])
stacking = StackingClassifier(base, final_estimator=LogisticRegression(C=1.0), cv=StratifiedKFold(5, shuffle=True,
random_state=1),
                             stack_method="predict_proba")
for name, m in [("voting", voting), ("stacking", stacking)]:
    p = m.fit(X_train, y_train).predict_proba(X_valid)[0, 1]
    print(f"{name:9s} PR-AUC valid={average_precision_score(y_valid, p):.4f} | ROC-AUC={roc_auc_score(y_valid,
p):.4f}")

# Tương quan dự đoán giữa các base model: thấp -> ensemble có lợi
preds = pd.DataFrame({n: m.fit(X_train, y_train).predict_proba(X_valid)[0, 1] for n, m in base})
print(preds.corr(method="spearman").round(3))

```

Production trade-off. Stacking 3 mô hình nghĩa là 3 lần độ trễ, 3 lần bảo trì và khó giải thích hơn. Chỉ dùng khi lợi ích kinh doanh đo được vượt chi phí.

8.8. Chẩn đoán: learning curve và validation curve

```

import matplotlib.pyplot as plt
from sklearn.model_selection import LearningCurveDisplay, ValidationCurveDisplay

fig, axes = plt.subplots(1, 2, figsize=(13, 4.5))
LearningCurveDisplay.from_estimator(
    candidates["lightgbm"], X_train, y_train, train_sizes=np.linspace(0.1, 1.0, 6),
    cv=StratifiedKFold(3, shuffle=True, random_state=0), scoring="average_precision",
    score_type="both", n_jobs=-1, ax=axes[0])
axes[0].set_title("Learning curve: còn tăng khi thêm dữ liệu?")
ValidationCurveDisplay.from_estimator(
    candidates["lightgbm"], X_train, y_train, param_name="lgbmclassifier_num_leaves",
    param_range=[3, 7, 15, 31, 63, 127], cv=StratifiedKFold(3, shuffle=True, random_state=0),
    scoring="average_precision", n_jobs=-1, ax=axes[1])
axes[1].set_xscale("log", base=2)
axes[1].set_title("Validation curve: độ phức tạp tối ưu")
plt.tight_layout()

```

Hình dạng learning curve	Chẩn đoán	Hành động
Train cao, validation thấp, khoảng cách lớn, validation còn tăng	High variance	Thêm dữ liệu , regularization
Hai đường hội tụ ở mức thấp	High bias	Feature tốt hơn, mô hình mạnh hơn
Hai đường hội tụ ở mức chấp nhận được	Ổn	Thêm dữ liệu ít lợi ích; tập trung vào feature/nhãn

Phân tích lỗi (error analysis)

[Course] Andrew Ng: lấy khoảng 100 mẫu bị dự đoán sai trên dev set, phân loại thủ công theo nguyên nhân, đếm tỷ lệ, rồi ưu tiên sửa nhóm lỗi lớn nhất. Với dữ liệu bảng, dùng phiên bản tự động: **slice analysis** (Chương 9.11) để tìm phân khúc có lỗi cao bất thường.

8.9. Neural network cho dữ liệu bảng

Grinsztajn, Oyallon & Varoquaux (NeurIPS 2022) chỉ ra mô hình cây vẫn vượt deep learning trên dữ liệu bảng cỡ vừa, vì: (1) NN thiên về hàm trơn còn dữ liệu bảng thường có hàm "bậc thang"; (2) NN nhạy với feature vô dụng; (3) NN không bất biến với phép quay. Deep learning đáng thử khi dữ liệu rất lớn, đa phương thức (bảng + văn bản + ảnh), hoặc cần embedding cho biến cardinality cực cao. Các kiến trúc chuyên dụng: **FT-Transformer**, **TabNet**, **TabPFN** (rất mạnh với dữ liệu nhỏ).

```

from sklearn.neural_network import MLPClassifier

mlp = make_pipeline(build_preprocessor(scale=True), MLPClassifier(
    hidden_layer_sizes=(64, 32), alpha=1e-3, learning_rate_init=1e-3, batch_size=256,
    early_stopping=True, validation_fraction=0.15, n_iter_no_change=10, max_iter=300, random_state=0))
mlp.fit(X_train, y_train)
print(f"MLP: {mlp[-1].n_iter_} epochs | PR-AUC valid={average_precision_score(y_valid, mlp.predict_proba(X_valid)[: ,
1]):.4f}")

```

Vòng huấn luyện PyTorch chuẩn mực (khởi minh họa, không chạy trong môi trường kiểm chứng vì không cài PyTorch):

```

import torch
from torch import nn
from torch.utils.data import DataLoader, TensorDataset

class TabularMLP(nn.Module):
    def __init__(self, n_in: int, hidden=(128, 64), dropout: float = 0.2):
        super().__init__()
        layers, d = [], n_in
        for h in hidden:
            layers += [nn.Linear(d, h), nn.BatchNorm1d(h), nn.SiLU(), nn.Dropout(dropout)]
            d = h
        self.net = nn.Sequential(*layers, nn.Linear(d, 1))

    def forward(self, x):
        return self.net(x).squeeze(-1) # logits

def train(model, train_dl, X_val, y_val, epochs=100, patience=10, lr=1e-3, device="cpu"):
    opt = torch.optim.AdamW(model.parameters(), lr=lr, weight_decay=1e-4)
    sched = torch.optim.lr_scheduler.ReduceLROnPlateau(opt, mode="max", factor=0.5, patience=3)
    loss_fn = nn.BCEWithLogitsLoss() # ổn định số hơn sigmoid + BCELoss
    best, best_state, wait = -1.0, None, 0
    for epoch in range(epochs):
        model.train()
        for xb, yb in train_dl:
            opt.zero_grad(set_to_none=True)
            loss = loss_fn(model(xb.to(device)), yb.to(device))
            loss.backward()
            nn.utils.clip_grad_norm_(model.parameters(), 1.0)
            opt.step()
        model.eval()
        with torch.inference_mode():
            p = torch.sigmoid(model(X_val.to(device))).cpu().numpy()
            score = average_precision_score(y_val, p)
            sched.step(score)
            if score > best + 1e-4:
                best, wait = score, 0
                best_state = {k: v.detach().clone() for k, v in model.state_dict().items()}
            elif (wait := wait + 1) >= patience:
                break
        model.load_state_dict(best_state)
    return model, best

```

8.10. Huấn luyện tái lập được, có tracking (MLflow)

Package tham chiếu có lệnh huấn luyện production `python -m churn.models.train --config configs/train.yaml`. Lệnh này thực hiện tuần tự: đọc config (pydantic), nạp dữ liệu, làm sạch, kiểm tra **data contract**, chia theo thời gian + kiểm tra leakage, huấn luyện pipeline, tính metric, rồi lưu `model.joblib + metrics.json`. Toàn bộ params, metrics, git commit và mô hình được **log vào MLflow**. Chạy trực tiếp từ Python:

```

import json
import os
from pathlib import Path

```

```

os.environ["MLFLOW_TRACKING_URI"] = "sqlite:///mlflow.db" # MLflow 3: file store ./mlruns ở chế độ bảo trì
Path("configs").mkdir(exist_ok=True)
Path("configs/train.yaml").write_text("""
target: churn
random_state: 42
split: {date_col: signup_date, train_end: "2023-10-01", valid_end: "2024-01-01", gap_days: 0}
model_params: {n_estimators: 400, learning_rate: 0.03, num_leaves: 15, min_child_samples: 50,
               subsample: 0.8, subsample_freq: 1, colsample_bytree: 0.8, reg_lambda: 1.0}
""")

from churn.models.train import main as train_main

metrics = train_main("configs/train.yaml", out_dir=".")
print(json.dumps({k: round(v, 4) for k, v in metrics.items()}, indent=1))

import mlflow

runs = mlflow.search_runs(experiment_names=["churn-prediction"])
print(runs[["run_id", "metrics.valid_pr_auc", "params.num_leaves", "tags.git_commit"]].head(3))

```

[Docs] Những điểm của MLflow 3 cần biết:

- `mlflow.sklearn.log_model(sk_model=..., name="model")`: tham số `artifact_path` được thay bằng `name`. Mô hình trở thành một **LoggedModel** có `model_id`, nạp bằng `models:/<model_id>`.
- Backend file `./mlruns` ở chế độ bảo trì: dùng `sqlite:///mlflow.db` hoặc PostgreSQL.
- Ngoài Databricks, scikit-learn được serialize mặc định bằng **skops** (an toàn khi nạp). Pipeline có custom class hoặc LightGBM booster phải khai báo `skops_trusted_types` hoặc dùng `serialization_format="cloudpickle"` kèm `code_paths`. Khi đó chỉ nạp model từ registry đáng tin.

Chiến lược mô hình cuối cùng

1. Chọn cấu hình bằng CV/validation (đã tune, có early stopping).
2. **Đánh giá một lần** trên test out-of-time, báo cáo có CI (Chương 9).
3. **Huấn luyện lại** trên toàn bộ dữ liệu có nhãn mới nhất (train + valid + test) với cấu hình đã cố định. Số cây hiệu chỉnh theo lượng dữ liệu.
4. Đăng ký vào registry và triển khai qua shadow/canary (Chương 12).

8.11. Song song hóa và số luồng CPU: bài học từ chính môi trường viết handbook

Khi chạy kiểm chứng chương này trong container (báo `nproc = 4`), `HistGradientBoostingClassifier` với 300 vòng trên 14 000 dòng mất **17-146 giây**. Giới hạn OpenMP còn 1-2 luồng thì chỉ mất **0.2 giây**. Nguyên nhân: container được cấp **hạn mức CPU (cgroup quota) thấp hơn số vCPU nhìn thấy**. Các luồng OpenMP *spin-wait* chờ nhau trong khi bị hệ điều hành tạm dừng, nên hiệu năng sụp đổ. Hiện tượng này rất phổ biến trên Kubernetes, CI runner và notebook dùng chung.

Nguồn song song	Điều khiển bởi	Mặc định
joblib (CV, grid search, RF)	<code>n_jobs</code>	1 (None); <code>-1</code> = mọi CPU
OpenMP (HistGB, LightGBM, XGBoost)	<code>OMP_NUM_THREADS</code> , <code>n_jobs / num_threads / nthread</code>	Mọi CPU
BLAS (numpy, scipy, Logistic lbfgs)	<code>OPENBLAS_NUM_THREADS</code> , <code>MKL_NUM_THREADS</code>	Mọi CPU
CatBoost	<code>thread_count</code>	Mọi CPU

[Docs] scikit-learn *Parallelism, resource management, and configuration*: tổng số luồng = (số tiến trình joblib) × (số luồng OpenMP/BLAS mỗi tiến trình). Khi vượt số lỗi thật sẽ xảy ra **oversubscription**. scikit-learn tự giới hạn luồng của tiến trình con loky, nhưng không kiểm soát được thư viện ngoài. **[Docs]** LightGBM: đặt `num_threads` bằng số **lỗi vật lý**, không phải số luồng hyper-threading.

Phép đo thứ hai trong cùng môi trường: `cross_val_score(..., n_jobs=2)` với pipeline LightGBM để số luồng mặc định mất **451 giây**, trong khi một lần `fit` đơn lẻ chỉ mất **0.47 giây**. Mỗi worker của joblib lại mở nhiều luồng OpenMP, nên tổng số luồng vượt xa số lõi thật. Đặt `LGBMClassifier(n_jobs=1)` khi đã song song ở cấp fold sẽ khắc phục hoàn toàn.

Quy tắc thực hành:

1. **Song song ở một cấp:** hoặc nhiều fold (`n_jobs=-1` ở `cross_validate / GridSearchCV`) với mô hình 1 luồng, hoặc 1 fold với mô hình đa luồng. Không làm cả hai.
2. Trong container, đặt `OMP_NUM_THREADS` (và `n_jobs` của mô hình) bằng **số CPU được cấp**. Trên Kubernetes, giá trị này là `resources.limits.cpu`.
3. Kiểm tra bằng `threadpoolctl.threadpool_info()` và giới hạn cục bộ bằng `threadpool_limits(limits=1)`.

```
from threadpoolctl import threadpool_info, threadpool_limits

print({i["internal_api"], i["num_threads"]} for i in threadpool_info())
with threadpool_limits(limits=1): # giới hạn OpenMP/BLAS trong một khối code
    t0 = time.perf_counter()
    HistGradientBoostingClassifier(max_iter=100, random_state=0).fit(Xtr, y_train)
    print(f"HGB 1 luồng: {time.perf_counter() - t0:.2f}s")
```

Checklist Chương 8

- ☐ Có baseline ngẫu nhiên và baseline nghiệp vụ; mô hình phải vượt rõ rệt.
- ☐ Loss/metric nhất quán với đại lượng cần dự đoán (xác suất, mean, median, quantile).
- ☐ Mọi mô hình được so sánh trên cùng CV, cùng metric, báo cáo mean $\pm$ std.
- ☐ Early stopping dùng validation riêng; không tune `n_estimators` trực tiếp.
- ☐ Tuning có ngân sách, không gian log-scale, lưu lịch sử (Optuna storage), seed cố định.
- ☐ Đã xem learning/validation curve để quyết định thêm dữ liệu hay đổi mô hình.
- ☐ Ràng buộc đơn điệu cho feature có quan hệ nghiệp vụ rõ ràng (khi cần).
- ☐ Huấn luyện là một lệnh tái lập được; params, metrics, commit, mô hình được log vào MLflow.
- ☐ Dùng API hiện hành (`l1_ratio` thay `penalty` ; MLflow `name=` ; TargetEncoder `cv=`).
- ☐ Song song hóa ở một cấp; số luồng khớp CPU được cấp (tránh oversubscription trong container).

Tài liệu tham khảo Chương 8

- scikit-learn User Guide: *Parallelism, resource management, and configuration*; *Linear Models* (Logistic regression, solvers); *Ensembles: Gradient boosting, random forests, bagging, voting, stacking* (HistGradientBoosting: early stopping, categorical, monotonic & interaction constraints); *Tuning the hyper-parameters* (Randomized, Successive Halving); *Validation curves*; *Which scoring function should I use?*
- LightGBM documentation: *Parameters, Parameters Tuning, Features* (leaf-wise, GOSS, EFB). lightgbm.readthedocs.io
- XGBoost documentation: *Notes on Parameter Tuning, Introduction to Boosted Trees*. xgboost.readthedocs.io
- CatBoost documentation: *Transforming categorical features, Ordered boosting*. catboost.ai/docs
- Optuna documentation: *Key Features* (samplers, pruners, storage, multi-objective). optuna.readthedocs.io
- MLflow 3 documentation: *Tracking, Models (LoggedModel), scikit-learn flavor*. mlflow.org/docs
- Friedman, J. (2001). *Greedy Function Approximation: A Gradient Boosting Machine*. Annals of Statistics 29(5).
- Chen, T. & Guestrin, C. (2016). *XGBoost: A Scalable Tree Boosting System*. KDD.
- Ke, G. et al. (2017). *LightGBM*. NeurIPS.
- Prokhorenkova, L. et al. (2018). *CatBoost: unbiased boosting with categorical features*. NeurIPS.
- Bergstra, J. & Bengio, Y. (2012). *Random Search for Hyper-Parameter Optimization*. JMLR 13.
- Akiba, T. et al. (2019). *Optuna*. KDD.
- Grinsztajn, L., Oyallon, E. & Varoquaux, G. (2022). *Why do tree-based models still outperform deep learning on typical tabular data?* NeurIPS.

- Stanford CS229 Lecture Notes (supervised learning, regularization, bias-variance). · Ng, A. *Machine Learning Yearning*.

CHƯƠNG 9. ĐÁNH GIÁ MÔ HÌNH: METRICS, CONFUSION MATRIX, THRESHOLD & CALIBRATION

Mục tiêu chương: đo hiệu năng **đúng thứ cần đo, đúng cách** và **trung thực về độ bất định**. Tách bạch hai bài toán mà tài liệu scikit-learn nhấn mạnh: **dự đoán** (ước lượng xác suất tốt) và **ra quyết định** (chọn ngưỡng để hành động).

9.1. Khung lý thuyết: dự đoán và ra quyết định

[Docs] scikit-learn, *Tuning the decision threshold for class prediction*: bài toán phân loại nên chia làm hai phần:

1. **Bài toán thống kê:** học mô hình ước lượng tốt $P(y | X)$. Kết quả đến từ `predict_proba / decision_function`. Ví dụ: "khả năng mưa ngày mai là bao nhiêu?"
2. **Bài toán quyết định:** dựa vào xác suất đó để hành động. Kết quả đến từ `predict`, mặc định với ngưỡng cứng 0.5. Ví dụ: "có nên mang ô không?"

Ngưỡng 0.5 "hầu như chắc chắn không lý tưởng cho đa số trường hợp sử dụng". Ví dụ của tài liệu: phát hiện khối u thì bác sĩ ưu tiên recall, nên ngưỡng phải thấp hơn nhiều.

[Docs] *Which scoring function should I use?*: đánh giá **chất lượng xác suất** bằng **strictly proper scoring rule** (log loss, Brier score). Hai thước đo này đo cả **calibration** lẫn **discrimination (resolution)**. Còn **chất lượng quyết định** được đo bằng các metric từ confusion matrix tại ngưỡng đã chọn, hoặc bằng lợi nhuận/chi phí.

Bảng tra metric theo bài toán

Bài toán	Metric chính	Bổ sung
Phân loại, cần xác suất đúng	Log loss, Brier	ECE, reliability diagram
Phân loại mất cân bằng, xếp hạng	PR-AUC (AP) , ROC-AUC	Recall@Precision, Precision@K
Quyết định nhị phân	Lợi nhuận kỳ vọng, F-β, MCC	Confusion matrix tại ngưỡng
Tín dụng	Gini = 2·AUC – 1, KS	PSI (ổn định)
Hồi quy (mean)	RMSE, MAE	R ² , D ² , MAPE/sMAPE/WAPE
Hồi quy phân vị / khoảng	Pinball loss	Coverage, độ rộng khoảng
Truy hồi / gợi ý	NDCG@K, MAP@K, MRR, Recall@K	F2 (ưu tiên recall)
Phân cụm	Silhouette, Davies–Bouldin, Calinski–Harabasz	ARI/NMI khi có nhãn

Thiết lập: hai mô hình để so sánh suốt chương

```
import lightgbm as lgb
import matplotlib.pyplot as plt
import numpy as np
import pandas as pd
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import (ConfusionMatrixDisplay, average_precision_score, brier_score_loss,
                             classification_report, confusion_matrix, f1_score, fbeta_score, log_loss,
                             matthews_corrcoef, precision_recall_curve, precision_score, recall_score,
                             roc_auc_score, roc_curve)
from sklearn.model_selection import StratifiedKFold, cross_val_predict
from sklearn.pipeline import make_pipeline

from churn.data.split import time_split
from churn.data.synthetic import make_churn_data
from churn.features.build import RAW_FEATURES, build_preprocessor
from churn.features.clean import clean_churn
```

```

from churn.models.evaluate import binary_report, expected_calibration_error, lift_table, probabilistic_report

pd.set_option("display.width", 170)
pd.set_option("display.max_columns", 20)
df = clean_churn(make_churn_data(n=30_000, seed=42))
train_df, valid_df, test_df = time_split(df, "signup_date", "2023-10-01", "2024-01-01")
X_train, y_train = train_df[RAW_FEATURES], train_df["churn"]
X_valid, y_valid = valid_df[RAW_FEATURES], valid_df["churn"]
X_test, y_test = test_df[RAW_FEATURES], test_df["churn"]

logit = make_pipeline(build_preprocessor(scale=True), LogisticRegression(max_iter=3000)).fit(X_train, y_train)
# n_jobs=1: mô hình sẽ được fit lại song song trong CV ở các mục sau -> tránh oversubscription (Chương 8.11)
gbm = make_pipeline(build_preprocessor(scale=False), lgb.LGBMClassifier(
    n_estimators=400, learning_rate=0.03, num_leaves=15, min_child_samples=50, subsample=0.8,
    subsample_freq=1, colsample_bytree=0.8, n_jobs=1, verbose=-1, random_state=0)).fit(X_train, y_train)
p_logit, p_gbm = logit.predict_proba(X_valid)[0, 1], gbm.predict_proba(X_valid)[0, 1]
print(pd.DataFrame({"logit": probabilistic_report(y_valid, p_logit),
                    "lightgbm": probabilistic_report(y_valid, p_gbm)}).round(4))

```

9.2. Confusion matrix: nền tảng của mọi metric phân loại

		DỰ ĐOÁN		
		Positive (1)	Negative (0)	
THỰC TẾ	P(1)	TP – bắt đúng churn	FN – bỏ sót churn	← sai lầm loại II
	N(0)	FP – báo nhầm	TN – đúng: ở lại	
		↑ sai lầm loại I		

Quy ước của scikit-learn: `confusion_matrix(y_true, y_pred)` trả về **hàng = thực tế, cột = dự đoán**, nhấn theo thứ tự tăng dần `[0, 1]`, tức là `[[TN, FP], [FN, TP]]`. Thứ tự này **ngược** với hình trên. Dùng `.ravel()` để lấy `tn, fp, fn, tp`.

Các chỉ số dẫn xuất

Chỉ số	Công thức	Câu hỏi trả lời	Tên khác
Accuracy	$\frac{TP + TN}{N}$	Tỷ lệ đúng tổng thể	Vô dụng khi mất cân bằng
Precision	$\frac{TP}{TP + FP}$	Trong các dự đoán positive, bao nhiêu đúng?	PPV
Recall	$\frac{TP}{TP + FN}$	Trong các positive thật, bắt được bao nhiêu?	Sensitivity, TPR
Specificity	$\frac{TN}{TN + FP}$	Trong các negative thật, nhận đúng bao nhiêu?	TNR
FPR	$\frac{FP}{FP + TN}$	Tỷ lệ báo động nhầm	Fall-out, 1 – Specificity
FNR	$\frac{FN}{FN + TP}$	Tỷ lệ bỏ sót	Miss rate
NPV	$\frac{TN}{TN + FN}$	Dự đoán negative đáng tin đến đâu?	
F1	$\frac{2PR}{P + R}$	Trung bình điều hòa P và R	
F-β	$\frac{(1 + \beta^2)PR}{\beta^2 P + R}$	$\beta = 2$: recall nặng gấp 4 lần precision	
Balanced accuracy	$\frac{TPR + TNR}{2}$	Accuracy công bằng giữa các lớp	
MCC	$\frac{TP \cdot TN - FP \cdot FN}{\sqrt{(TP + FP)(TP + FN)(TN + FP)(TN + FN)}}$	Tương quan dự đoán-thực tế $\in [-1, 1]$	Phi coefficient

Chỉ số	Công thức	Câu hỏi trả lời	Tên khác
Cohen's κ	$\frac{p_o - p_e}{1 - p_e}$	Đồng thuận vượt mức ngẫu nhiên	
Youden's J	TPR – FPR	Chọn ngưỡng trên ROC	Informedness
Markedness	PPV + NPV – 1		

Phụ thuộc prevalence: Precision, NPV, accuracy và F1 **thay đổi khi tỷ lệ positive thay đổi**, kể cả khi mô hình không đổi. TPR, FPR và ROC-AUC thì **không** phụ thuộc prevalence. Hệ quả: precision đo trên tập cân bằng giả tạo (sau resample) không phản ánh precision trong production.

$$\text{Precision} = \frac{\text{TPR} \cdot \pi}{\text{TPR} \cdot \pi + \text{FPR} \cdot (1 - \pi)}, \quad \pi = \text{prevalence}$$

Ví dụ tính tay

Tập kiểm tra 1 000 khách, 200 churn. Mô hình dự đoán 250 churn, trong đó 150 đúng.

	Dự đoán 1	Dự đoán 0	Tổng
Thực tế 1	TP = 150	FN = 50	200
Thực tế 0	FP = 100	TN = 700	800

- Accuracy = 850/1000 = **0.85**; Precision = 150/250 = **0.60**; Recall = 150/200 = **0.75**; Specificity = 700/800 = **0.875**
- F1 = $2 \cdot 0.6 \cdot 0.75 / 1.35$ = **0.667**; F2 = $5 \cdot 0.6 \cdot 0.75 / (4 \cdot 0.6 + 0.75)$ = **0.714**
- MCC = $(150 \cdot 700 - 100 \cdot 50) / \sqrt{(250 \cdot 200 \cdot 800 \cdot 750)}$ = 100 000/173 205 = **0.577**
- Mô hình "luôn đoán 0": Accuracy = **0.80**, Recall = 0, MCC = 0. Accuracy gây hiểu lầm.

```
y_true_toy = np.r_[np.ones(200), np.zeros(800)].astype(int)
y_pred_toy = np.r_[np.ones(150), np.zeros(50), np.ones(100), np.zeros(700)].astype(int)
print(binary_report(y_true_toy, y_pred_toy).round(3).to_dict())
```

Confusion matrix của mô hình thật tại một ngưỡng

```
threshold = 0.5
pred = (p_gbm >= threshold).astype(int)
print(classification_report(y_valid, pred, target_names=["stay", "churn"], digits=4))

fig, axes = plt.subplots(1, 3, figsize=(16, 4.5))
for ax, (norm, fmt, title) in zip(axes, [(None, "d", "Số lượng"), ("true", ".1%", "Chuẩn hóa theo hàng = recall"),
                                       ("pred", ".1%", "Chuẩn hóa theo cột = precision")]):
    ConfusionMatrixDisplay.from_predictions(y_valid, pred, display_labels=["stay", "churn"], normalize=norm,
                                           values_format=fmt, cmap="Blues", colorbar=False, ax=ax)
    ax.set_title(f"{title} @ {threshold}")
plt.tight_layout()
print("Ở ngưỡng 0.5 với prevalence ~16%: recall rất thấp -> ngưỡng mặc định không phù hợp")
```

Đa lớp: confusion matrix và cách lấy trung bình

```
from sklearn.datasets import load_wine
from sklearn.metrics import multilabel_confusion_matrix
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler

Xw, yw = load_wine(return_X_y=True)
Xa, Xb, ya, yb = train_test_split(Xw, yw, test_size=0.4, stratify=yw, random_state=0)
yb_pred = make_pipeline(StandardScaler(), LogisticRegression(C=0.05, max_iter=1000)).fit(Xa, ya).predict(Xb)
print(confusion_matrix(yb, yb_pred))
print(multilabel_confusion_matrix(yb, yb_pred)[0]) # one-vs-rest 2x2 cho lớp 0
```

```
for avg in ["macro", "weighted", "micro"]:
    print(f"F1 {avg:8s} = {f1_score(yb, yb_pred, average=avg):.4f}")
```

Kiểu trung bình	Cách tính	Khi nào dùng
macro	Trung bình đơn giản metric từng lớp	Mọi lớp quan trọng như nhau, kể cả lớp hiếm
weighted	Trung bình có trọng số theo support	Phản ánh phân phối dữ liệu; che giấu lớp hiếm kém
micro	Gộp TP/FP/FN toàn cục rồi tính	Đa nhãn; với đa lớp đơn nhãn, micro-F1 = accuracy

Đọc confusion matrix để phân tích lỗi: hàng có nhiều giá trị ngoài đường chéo cho biết lớp đó khó nhận diện (recall thấp). Cột có nhiều giá trị ngoài đường chéo cho biết mô hình hay "đổ" về lớp đó (precision thấp). Cặp ô đối xứng lớn $A \leftrightarrow B$ cho biết hai lớp nhầm lẫn lẫn nhau: cần feature phân biệt, hoặc gộp lớp nếu nghiệp vụ cho phép.

9.3. Metric không phụ thuộc ngưỡng

ROC-AUC

Đường ROC vẽ TPR theo FPR khi quét mọi ngưỡng. **AUC bằng xác suất một mẫu positive ngẫu nhiên có điểm cao hơn một mẫu negative ngẫu nhiên.** Đây chính là thống kê Mann-Whitney U đã chuẩn hóa:

$$AUC = U / (n_1 n_0).$$

```
from scipy.stats import mannwhitneyu

u = mannwhitneyu(p_gbm[y_valid.to_numpy() == 1], p_gbm[y_valid.to_numpy() == 0]).statistic
print("AUC = ", round(roc_auc_score(y_valid, p_gbm), 6), " | U/(n1·n0) =",
      round(u / ((y_valid == 1).sum() * (y_valid == 0).sum()), 6))
```

Precision-Recall curve và Average Precision

[Docs] `average_precision_score` tính $AP = \sum_n (R_n - R_{n-1})P_n$. Đây là tổng có trọng số của precision tại mỗi ngưỡng, **không nội suy tuyến tính** (nội suy hình thang trên đường PR cho kết quả lạc quan). **Baseline của AP bằng prevalence**, không phải 0.5.

```
from sklearn.metrics import DetCurveDisplay, PrecisionRecallDisplay, RocCurveDisplay

fig, axes = plt.subplots(1, 3, figsize=(17, 4.8))
for name, p in [("Logistic", p_logit), ("LightGBM", p_gbm)]:
    RocCurveDisplay.from_predictions(y_valid, p, name=name, ax=axes[0])
    PrecisionRecallDisplay.from_predictions(y_valid, p, name=name, ax=axes[1])
    DetCurveDisplay.from_predictions(y_valid, p, name=name, ax=axes[2])
axes[0].plot([0, 1], [0, 1], "k--", lw=0.8)
axes[1].axhline(y_valid.mean(), color="k", ls="--", lw=0.8, label="prevalence")
axes[2].set_title("DET: FNR vs FPR (thang probit)")
plt.tight_layout()
```

Metric	Đo cái gì	Khi nào ưu tiên
ROC-AUC	Khả năng xếp hạng toàn cục	Lớp tương đối cân bằng; so sánh mô hình giữa các tập có prevalence khác nhau
PR-AUC (AP)	Xếp hạng, tập trung vào lớp positive	Positive hiếm , quan tâm vùng top
DET curve	FNR theo FPR trên thang probit	So sánh hệ thống phát hiện (sinh trắc học, gian lận)
Log loss	Chất lượng xác suất; phạt rất nặng dự đoán sai mà tự tin	Cần xác suất đúng; huấn luyện
Brier	MSE của xác suất $\in [0, 1]$	Dễ diễn giải hơn log loss
D^2 log loss / D^2 Brier	Tỷ lệ "giải thích" so với mô hình dự đoán tỷ lệ cơ sở	Giống R^2 cho phân loại (sklearn $\geq 1.5/1.7$)

Phân rã Brier (Murphy, 1973): Brier = **Reliability** (lỗi calibration, càng nhỏ càng tốt) – **Resolution** (khả năng phân biệt, càng lớn càng tốt) + **Uncertainty** (độ khó cố hữu của dữ liệu). **[Docs]** Chính vì vậy, Brier thấp hơn **chưa chắc** nghĩa là calibration tốt hơn. Có thể đó là mô hình phân biệt tốt hơn nhưng calibrate kém hơn.

```
from sklearn.metrics import d2_brier_score, d2_log_loss_score

def brier_decomposition(y, p, n_bins: int = 10) -> dict:
    y, p = np.asarray(y), np.asarray(p)
    bins = np.clip(pd.qcut(p, n_bins, labels=False, duplicates="drop"), 0, None)
    base = y.mean()
    rel = res = 0.0
    for b in np.unique(bins):
        m = bins == b
        w = m.mean()
        rel += w * (p[m].mean() - y[m].mean()) ** 2
        res += w * (y[m].mean() - base) ** 2
    return {"brier": brier_score_loss(y, p), "reliability": rel, "resolution": res,
            "uncertainty": base * (1 - base)}

for name, p in [("logit", p_logit), ("lightgbm", p_gbm)]:
    print(name, {k: round(v, 5) for k, v in brier_decomposition(y_valid, p).items()},
          "| D²(log loss)=", round(d2_log_loss_score(y_valid, np.c_[1 - p, p]), 4),
          "| D²(Brier)=", round(d2_brier_score(y_valid, p), 4))
```

KS statistic và Gini (chuẩn ngành tín dụng)

```
from scipy.stats import ks_2samp

ks = ks_2samp(p_gbm[y_valid == 1], p_gbm[y_valid == 0]).statistic
print(f"KS = {ks:.4f} | Gini = {2 * roc_auc_score(y_valid, p_gbm) - 1:.4f}")
```

9.4. Chọn ngưỡng quyết định

Quan trọng: chọn ngưỡng trên **validation hoặc OOF predictions**, rồi cố định và đánh giá **một lần** trên test.

Quét ngưỡng với `metric_at_thresholds` (scikit-learn ≥ 1.9)

```
from sklearn.metrics import metric_at_thresholds

oof = cross_val_predict(make_pipeline(build_preprocessor(scale=False), lgb.LGBMClassifier(
    n_estimators=400, learning_rate=0.03, num_leaves=15, min_child_samples=50, n_jobs=1, verbose=-1,
    random_state=0)),
    X_train, y_train, cv=StratifiedKFold(5, shuffle=True, random_state=0), method="predict_proba")[:, 1]

# metric_at_thresholds gọi metric tại MỌI giá trị điểm duy nhất (~20 000 ngưỡng -> rất chậm).
# Làm tròn điểm về 3 chữ số 'đề' chỉ còn ≤ 1 000 ngưỡng: đủ mịn cho quyết định kinh doanh.
oof_r = np.round(oof, 3)
f2_vals, thr = metric_at_thresholds(y_train, oof_r, fbeta_score, metric_params={"beta": 2})
mcc_vals, thr_mcc = metric_at_thresholds(y_train, oof_r, matthews_corrcoef)
print("Số ngưỡng được quét:", len(thr))
print(f"Ngưỡng tối đa F2 = {thr[np.argmax(f2_vals)]:.3f} (F2 = {f2_vals.max():.4f})")
print(f"Ngưỡng tối đa MCC = {thr_mcc[np.argmax(mcc_vals)]:.3f} (MCC = {mcc_vals.max():.4f})")
```

Các chiến lược chọn ngưỡng

```
prec, rec, thr_pr = precision_recall_curve(y_train, oof) # len(thr_pr) = len(prec) - 1
f1_curve = 2 * prec[:-1] * rec[:-1] / np.clip(prec[:-1] + rec[:-1], 1e-12, None)
fpr, tpr, thr_roc = roc_curve(y_train, oof)
ok = prec[:-1] >= 0.40 # ràng buộc: precision tối thiểu 40%
```

```

def expected_profit(y, p, t, value=500.0, retain=0.3, cost=50.0):
    """Ma trận lợi ích Chương 1.5: TP = v·r - c, FP = -c, FN = TN = 0 (nghìn VNĐ)."""
    act = p >= t
    y = np.asarray(y)
    return float(np.sum(act & (y == 1)) * (value * retain - cost) - np.sum(act & (y == 0)) * cost)

grid = np.linspace(0.02, 0.9, 89)
profits = np.array([expected_profit(y_train, oof, t) for t in grid])
strategies = {
    "mặc định 0.5": 0.5,
    "max F1": thr_pr[np.argmax(f1_curve)],
    "Youden J (ROC)": thr_roc[np.argmax(tpr - fpr)],
    "recall max | precision ≥ 0.40": thr_pr[ok][np.argmax(rec[:-1][ok])] if ok.any() else np.nan,
    "năng lực CSKH: top 10%": np.quantile(oof, 0.90),
    "max lợi nhuận kỳ vọng": grid[np.argmax(profits)],
    "lý thuyết c/(v·r) (cân calibrate)": 50 / (500 * 0.3),
}
rows = []
for name, t in strategies.items():
    pv = (p_gbm >= t).astype(int)
    rows.append({"strategy": name, "threshold": t, "precision": precision_score(y_valid, pv, zero_division=0),
                "recall": recall_score(y_valid, pv), "f1": f1_score(y_valid, pv),
                "pct_flagged": pv.mean(), "profit_valid": expected_profit(y_valid, p_gbm, t)})
print(pd.DataFrame(rows).set_index("strategy").round(3))

```

Ngưỡng tối ưu theo chi phí (Elkan, 2001). Với ma trận chi phí tổng quát và xác suất **đã calibrate**:

$$t^* = \frac{C_{FP} - C_{TN}}{(C_{FP} - C_{TN}) + (C_{FN} - C_{TP})}$$

Trường hợp churn ở Chương 1.5 cho $t^* = c/(vr) \approx 0.333$.

TunedThresholdClassifierCV và FixedThresholdClassifier

[Docs] TunedThresholdClassifierCV tìm ngưỡng tối ưu cho một scoring bằng **CV nội bộ** (mặc định 5-fold stratified). Lưu ý của tài liệu: **không bao giờ** dùng cùng dữ liệu để huấn luyện bộ phân loại và chọn ngưỡng. Muốn tối ưu lợi nhuận, dùng `make_scorer` với hàm lợi ích tự định nghĩa, như ví dụ chính thức *Post-hoc tuning the cut-off point of decision function* và *cost-sensitive learning*. FixedThresholdClassifier đóng gói một ngưỡng cố định vào mô hình để deploy nhất quán. Bọc FrozenEstimator nếu không muốn fit lại mô hình gốc.

```

from sklearn.frozen import FrozenEstimator
from sklearn.metrics import make_scorer
from sklearn.model_selection import FixedThresholdClassifier, TunedThresholdClassifierCV

def profit_metric(y_true, y_pred, value=500.0, retain=0.3, cost=50.0):
    y_true, y_pred = np.asarray(y_true), np.asarray(y_pred)
    return float(np.sum((y_pred == 1) & (y_true == 1)) * (value * retain - cost)
                - np.sum((y_pred == 1) & (y_true == 0)) * cost)

profit_scorer = make_scorer(profit_metric)
tuned = TunedThresholdClassifierCV(gbm, scoring=profit_scorer, thresholds=100,
                                  cv=StratifiedKFold(5, shuffle=True, random_state=0),
                                  store_cv_results=True, random_state=0).fit(X_train, y_train)
print(f"Ngưỡng tối ưu lợi nhuận (CV nội bộ) = {tuned.best_threshold_:.3f}")
print(f"Lợi nhuận valid: mặc định 0.5 = {profit_metric(y_valid, gbm.predict(X_valid)):.0f} | "
      f"tuned = {profit_metric(y_valid, tuned.predict(X_valid)):.0f} (nghìn VNĐ)")

deployable = FixedThresholdClassifier(FrozenEstimator(gbm), threshold=float(tuned.best_threshold_),
                                     response_method="predict_proba").fit(X_train, y_train)
assert (deployable.predict(X_valid) == tuned.predict(X_valid)).mean() > 0.99

```

9.5. Metric xếp hạng kinh doanh: Lift, Gain, Precision@K

```
lt = lift_table(y_valid, p_gbm, n_bins=10)
print(lt.round(3))

def precision_recall_at_k(y, p, k_frac: float) -> tuple[float, float]:
    y = np.asarray(y)
    top = np.argsort(-p)[: int(len(p) * k_frac)]
    return float(y[top].mean()), float(y[top].sum() / y.sum())

for k in [0.05, 0.10, 0.20]:
    pk, rk = precision_recall_at_k(y_valid, p_gbm, k)
    print(f"Top {k:.0%}: precision={pk:.3f} recall={rk:.3f} lift={pk / y_valid.mean():.2f}")
```

Diễn giải cho stakeholder: "Gọi 10% khách có điểm cao nhất sẽ tiếp cận được X% số khách sắp rời bỏ, hiệu quả gấp Y lần gọi ngẫu nhiên."

9.6. Calibration: xác suất có đáng tin không?

[Docs] Một bộ phân loại **calibrated tốt**: trong các mẫu có `predict_proba` ≈ 0.8 , khoảng 80% thuộc lớp positive. Các dạng lệch điển hình theo tài liệu:

- **Logistic Regression** thường calibrate tốt khi đúng đặc tả và regularization phù hợp, nhờ *balance property* của canonical link.
- **Random Forest, bagging** có đường calibration hình chữ **S**: dự đoán bị kéo khỏi 0 và 1, vì phương sai của các cây thành phần khó tạo ra xác suất cực trị (Niculescu-Mizil & Caruana, 2005).
- **SVM / mô hình max-margin** cũng có dạng chữ S, do tập trung vào mẫu gần biên.
- **Naive Bayes** cho xác suất cực đoan do giả định độc lập.
- **Resampling / class weight** làm lệch toàn bộ xác suất (Chương 5.9).

Các phương pháp calibration (scikit-learn)

method	Mô hình calibrator	Khi nào dùng
"sigmoid" (Platt)	Logistic 1 chiều trên điểm số	Ít dữ liệu; lệch dạng chữ S
"isotonic"	Hàm bậc thang không giảm	Nhiều dữ liệu (tài liệu: isotonic tốt bằng hoặc hơn sigmoid khi đủ dữ liệu; dễ overfit khi ít); tạo ties nên có thể làm giảm nhẹ ROC-AUC
"temperature" (sklearn ≥ 1.8)	Chia logit cho T	Đa lớp , giữ nguyên argmax

[Docs] Calibrator phải được fit trên dữ liệu **độc lập** với dữ liệu huấn luyện bộ phân loại. `CalibratedClassifierCV` lo việc này bằng CV:

- `ensemble=True` (mặc định khi không frozen): mỗi fold sinh một cặp (classifier, calibrator), dự đoán là trung bình. Thường **calibrate tốt hơn và chính xác hơn một chút**.
- `ensemble=False`: dùng `cross_val_predict` để fit một calibrator, một classifier trên toàn bộ dữ liệu. **Rẻ hơn** khi suy luận.
- Mô hình **đã huấn luyện**: bọc `FrozenEstimator(model)` và fit calibrator trên tập validation riêng. Cách này thay cho `cv="prefit"` (đã deprecate từ 1.6).

```
from sklearn.calibration import CalibratedClassifierCV, CalibrationDisplay

# Mô hình bị lệch calibration có chủ đích: class_weight làm xác suất bị đẩy lên
skewed = make_pipeline(build_preprocessor(scale=False), lgb.LGBMClassifier(
    n_estimators=400, learning_rate=0.03, num_leaves=15, min_child_samples=50, class_weight="balanced",
```

```

verbose=-1, random_state=0)).fit(X_train, y_train)

cal_sigmoid = CalibratedClassifierCV(FrozenEstimator(skewed), method="sigmoid").fit(X_valid, y_valid)
cal_isotonic = CalibratedClassifierCV(FrozenEstimator(skewed), method="isotonic").fit(X_valid, y_valid)

fig, ax = plt.subplots(figsize=(6.5, 6))
rows = []
for name, m in [("LightGBM chuẩn", gbm), ("class_weight=balanced", skewed),
                ("+ sigmoid", cal_sigmoid), ("+ isotonic", cal_isotonic)]:
    p = m.predict_proba(X_test)[: , 1] # đánh giá trên TEST, độc lập với calibrator
    CalibrationDisplay.from_predictions(y_test, p, n_bins=10, strategy="quantile", name=name, ax=ax)
    rows.append({"model": name, "mean_pred": p.mean(), "ECE": expected_calibration_error(y_test, p),
                "brier": brier_score_loss(y_test, p), "log_loss": log_loss(y_test, p),
                "roc_auc": roc_auc_score(y_test, p)})
ax.set_title("Reliability diagram (test)")
print("Tỷ lệ churn thật (test):", round(y_test.mean(), 4))
print(pd.DataFrame(rows).set_index("model").round(4))

```

Kết luận từ thí nghiệm: `class_weight="balanced"` giữ nguyên khả năng xếp hạng (ROC-AUC gần như không đổi) nhưng làm xác suất trung bình lệch xa tỷ lệ thật, nên ECE, Brier và log loss tệ đi. Calibration sau huấn luyện khôi phục xác suất mà **không đổi thứ hạng**. Riêng isotonic có thể đổi nhẹ do tạo ties.

9.7. Metric hồi quy

[Docs] Bảng *strictly consistent scoring functions* của scikit-learn:

Đại lượng cần dự đoán	Hàm đánh giá nhất quán	scikit-learn
Mean	Squared error (RMSE, R^2)	<code>root_mean_squared_error</code> , <code>r2_score</code>
Mean (dữ liệu đếm)	Poisson deviance	<code>mean_poisson_deviance</code>
Mean (dương, lệch)	Gamma / Tweedie deviance	<code>mean_gamma_deviance</code> , <code>mean_tweedie_deviance</code>
Median	Absolute error	<code>mean_absolute_error</code>
Quantile α	Pinball loss	<code>mean_pinball_loss(alpha=α)</code>

Metric	Công thức	Đặc điểm
MAE	$\frac{1}{n} \sum y - \hat{y} $	Cùng đơn vị; bền ngoại lai
RMSE	$\sqrt{\frac{1}{n} \sum (y - \hat{y})^2}$	Phạt nặng lỗi lớn
R^2	$1 - SS_{res} / SS_{tot}$	Có thể âm; phụ thuộc phương sai của tập đánh giá
MAPE	$\frac{100}{n} \sum (y - \hat{y}) / y $	Vỡ khi $y \approx 0$; bất đối xứng (phạt over-forecast nhẹ hơn)
sMAPE	$\frac{100}{n} \sum \frac{2 y - \hat{y} }{ y + \hat{y} }$	Đối xứng hơn
WAPE	$\sum y - \hat{y} / \sum y $	Chuẩn ngành bán lẻ
MASE	$MAE / MAE(\text{naive})$	Chuỗi thời gian; < 1 là tốt hơn naive (Hyndman & Koehler, 2006)
Pinball	$\max(\alpha(y - \hat{y}), (\alpha - 1)(y - \hat{y}))$	Dự báo phân vị

```

from sklearn.ensemble import HistGradientBoostingRegressor
from sklearn.metrics import (mean_absolute_error, mean_absolute_percentage_error, mean_pinball_loss,
                             mean_poisson_deviance, r2_score, root_mean_squared_error)

rng = np.random.default_rng(0)
n = 6_000
Xr = pd.DataFrame({"tenure": rng.integers(1, 72, n), "plan": rng.integers(0, 3, n), "calls": rng.poisson(1.5, n)})
y_rev = rng.gamma(shape=2.0, scale=np.exp(3 + 0.02 * Xr["tenure"] + 0.3 * Xr["plan"]) / 2.0)
Xa, Xb, ya, yb = train_test_split(Xr, y_rev, test_size=0.3, random_state=0)

models = {
    "squared_error (mean)": HistGradientBoostingRegressor(loss="squared_error", random_state=0),
    "absolute_error (median)": HistGradientBoostingRegressor(loss="absolute_error", random_state=0),

```

```

"gamma (mean, dương)": HistGradientBoostingRegressor(loss="gamma", random_state=0),
}
rows = []
for name, m in models.items():
    pred = m.fit(Xa, ya).predict(Xb)
    rows.append({"loss": name, "RMSE": root_mean_squared_error(yb, pred), "MAE": mean_absolute_error(yb, pred),
                "R2": r2_score(yb, pred), "MAPE_%": 100 * mean_absolute_percentage_error(yb, pred),
                "PoissonDev": mean_poisson_deviance(yb, np.clip(pred, 1e-6, None)),
                "sum_pred/sum_true": pred.sum() / yb.sum()})
print(pd.DataFrame(rows).set_index("loss").round(3)) # mỗi loss thắng ở metric nhất quán với nó

# Dự báo khoảng 80% bằng hai mô hình phân vị + kiểm tra coverage
lo = HistGradientBoostingRegressor(loss="quantile", quantile=0.1, random_state=0).fit(Xa, ya).predict(Xb)
hi = HistGradientBoostingRegressor(loss="quantile", quantile=0.9, random_state=0).fit(Xa, ya).predict(Xb)
print(f"Coverage khoảng [P10, P90] = {np.mean((yb >= lo) & (yb <= hi)):.3f} (mục tiêu 0.80) | "
      f"pinball@0.9 = {mean_pinball_loss(yb, hi, alpha=0.9):.3f}")
# Hồi quy phân vị KHÔNG bảo đảm coverage trên dữ liệu mới (thường thiếu hụt) -> cần kiểm tra, hoặc dùng conformal

```

Conformal prediction: khoảng dự đoán có bảo đảm coverage

Split conformal cho bảo đảm $P(y \in \hat{C}(x)) \geq 1 - \alpha$ dưới giả định **exchangeability**, không cần giả định phân phối. Thư viện chuyên dụng: **MAPIE**.

```

Xfit, Xcal, yfit, ycal = train_test_split(Xa, ya, test_size=0.3, random_state=1)
point = HistGradientBoostingRegressor(random_state=0).fit(Xfit, yfit)
scores = np.abs(ycal - point.predict(Xcal)) # nonconformity score
alpha = 0.2
q = np.quantile(scores, np.ceil((len(scores) + 1) * (1 - alpha)) / len(scores), method="higher")
pred_b = point.predict(Xb)
print(f"Split conformal 80%: coverage={np.mean(np.abs(yb - pred_b) <= q):.3f} | độ rộng khoảng={2 * q:.1f}")

```

9.8. Metric phân cụm

```

from sklearn.cluster import KMeans
from sklearn.metrics import (adjusted_rand_score, calinski_harabasz_score, davies_bouldin_score,
                             normalized_mutual_info_score, silhouette_score)

Xc = StandardScaler().fit_transform(X_train[["tenure_months", "monthly_charges", "support_calls"]])
rows = []
for k in range(2, 8):
    labels = KMeans(n_clusters=k, n_init=10, random_state=0).fit_predict(Xc)
    rows.append({"k": k, "silhouette": silhouette_score(Xc, labels, sample_size=4000, random_state=0),
                "davies_bouldin": davies_bouldin_score(Xc, labels),
                "calinski_harabasz": calinski_harabasz_score(Xc, labels)})
print(pd.DataFrame(rows).set_index("k").round(3))
true_groups = X_train["contract"].astype("category").cat.codes
km3 = KMeans(n_clusters=3, n_init=10, random_state=0).fit_predict(Xc)
print("ARI so với contract:", round(adjusted_rand_score(true_groups, km3), 4),
      "| NMI:", round(normalized_mutual_info_score(true_groups, km3), 4))
# ARI ≈ 0: các cụm (dựa trên tenure/cước/cuộc gọi) không trùng với loại hợp đồng -> phân cụm tìm cấu trúc khác

```

Metric nội tại chỉ là gợi ý. Phân cụm **có giá trị khi các cụm khác biệt về hành vi và hành động được**. Hãy đánh giá bằng profile cụm và sự đồng thuận của nghiệp vụ.

9.9. Metric truy hồi & xếp hạng (search, RAG, recommender)

```

from sklearn.metrics import ndcg_score

def recall_at_k(relevant: set, ranked: list, k: int) -> float:
    return len(relevant & set(ranked[:k])) / len(relevant) if relevant else 0.0

def average_precision_at_k(relevant: set, ranked: list, k: int) -> float:

```

```

hits, score = 0, 0.0
for i, doc in enumerate(ranked[:k], start=1):
    if doc in relevant:
        hits += 1
        score += hits / i
return score / min(len(relevant), k) if relevant else 0.0

def reciprocal_rank(relevant: set, ranked: list) -> float:
    return next((1.0 / i for i, d in enumerate(ranked, 1) if d in relevant), 0.0)

def f_beta_sets(relevant: set, retrieved: set, beta: float = 2.0) -> float:
    """F2 ưu tiên recall (trọng số  $\beta^2 = 4$ ): phù hợp truy hồi văn bản pháp luật, nơi bỏ sót đắt hơn thừa."""
    tp = len(relevant & retrieved)
    if tp == 0:
        return 0.0
    p, r = tp / len(retrieved), tp / len(relevant)
    return (1 + beta**2) * p * r / (beta**2 * p + r)

queries = [{("d1", "d4"), ("d4", "d2", "d1", "d7")}, ({("d3"), ("d5", "d3", "d9", "d1")}]
print("Recall@3 :", np.mean([recall_at_k(r, ranked, 3) for r, ranked in queries]))
print("MAP@3      :", np.mean([average_precision_at_k(r, ranked, 3) for r, ranked in queries]).round(4))
print("MRR       :", np.mean([reciprocal_rank(r, ranked) for r, ranked in queries]))
print("F2 (top3):", np.mean([f_beta_sets(r, set(ranked[:3])) for r, ranked in queries]).round(4))
print("NDCG@3    :", round(ndcg_score([[3, 2, 0, 0, 1]], [[0.9, 0.8, 0.7, 0.1, 0.6]], k=3), 4))

```

9.10. So sánh mô hình có ý nghĩa thống kê

Trên validation, Logistic có PR-AUC ≈ 0.375 và LightGBM ≈ 0.369 . Chênh lệch nhỏ như vậy có thật không, hay chỉ do nhiễu?

```

from scipy import stats
from sklearn.model_selection import RepeatedStratifiedKFold, cross_val_score
from statsmodels.stats.contingency_tables import mcnemar

def paired_bootstrap(y, p_a, p_b, metric=average_precision_score, n_boot=2000, seed=0) -> dict:
    """CI cho metric(B) - metric(A) trên CÙNG tập test (ghép cặp)."""
    r = np.random.default_rng(seed)
    y = np.asarray(y)
    diffs = []
    for _ in range(n_boot):
        i = r.integers(0, len(y), len(y))
        if y[i].min() != y[i].max():
            diffs.append(metric(y[i], p_b[i]) - metric(y[i], p_a[i]))
    diffs = np.asarray(diffs)
    return {"diff": float(metric(y, p_b) - metric(y, p_a)), "ci95": np.round(np.quantile(diffs, [0.025, 0.975]), 4),
            "P(B>A)": float((diffs > 0).mean())}

def corrected_resampled_ttest(scores_a, scores_b, n_train: int, n_test: int, k: int) -> tuple[float, float]:
    """Nadeau & Bengio (2003): hiệu chỉnh phương sai vì các fold CV không độc lập."""
    d = np.asarray(scores_b) - np.asarray(scores_a)
    var = d.var(ddof=1) * (1 / k + n_test / n_train)
    t = d.mean() / np.sqrt(var)
    return float(t), float(2 * stats.t.sf(abs(t), df=k - 1))

p_logit_te, p_gbm_te = logit.predict_proba(X_test)[: , 1], gbm.predict_proba(X_test)[: , 1]
print("Paired bootstrap PR-AUC (LightGBM - Logistic):", paired_bootstrap(y_test, p_logit_te, p_gbm_te))

rcv = RepeatedStratifiedKFold(n_splits=5, n_repeats=3, random_state=0)
s_logit = cross_val_score(logit, X_train, y_train, cv=rcv, scoring="average_precision", n_jobs=-1)
s_gbm = cross_val_score(gbm, X_train, y_train, cv=rcv, scoring="average_precision", n_jobs=-1)
naive_p = stats.ttest_rel(s_gbm, s_logit).pvalue
t, p_corr = corrected_resampled_ttest(s_logit, s_gbm, n_train=int(len(X_train) * 0.8),
                                      n_test=int(len(X_train) * 0.2), k=len(s_gbm))
print(f"t-test ghép cặp NGÂY THỜ p={naive_p:.4f} | corrected resampled t-test p={p_corr:.4f}")

```

```
a_ok = (p_logit_te >= 0.3) == y_test.to_numpy()
b_ok = (p_gbm_te >= 0.3) == y_test.to_numpy()
table = [[np.sum(a_ok & b_ok), np.sum(a_ok & ~b_ok)], [np.sum(~a_ok & b_ok), np.sum(~a_ok & ~b_ok)]]
print("McNemar (nhân cứng @0.3) p =", round(mcnemar(table, exact=False, correction=True).pvalue, 4))
```

Đọc kết quả: bootstrap ghép cặp trên test cho CI của chênh lệch PR-AUC (LightGBM – Logistic) **nằm hoàn toàn dưới 0**, nên trên dữ liệu gần tuyến tính này Logistic tốt hơn thật. t-test ngây thơ trên các fold CV cho $p \approx 0$. Corrected t-test vẫn có ý nghĩa nhưng p lớn hơn hàng trăm lần, vì t-test ngây thơ đánh giá thấp phương sai. McNemar trên nhân cứng tại ngưỡng 0.3 **không** có ý nghĩa: hai mô hình đưa ra quyết định gần như giống nhau ở ngưỡng đó. Mỗi kiểm định trả lời một câu hỏi khác nhau.

[Docs] Ví dụ chính thức *Statistical comparison of models using grid search* của scikit-learn trình bày cả cách tiếp cận tần suất (corrected t-test) và Bayes (xác suất một mô hình tốt hơn, *ROPE*: region of practical equivalence). Nguyên tắc: nếu CI của chênh lệch chứa 0 hoặc nằm trong vùng tương đương thực tế, **giữ mô hình đơn giản hơn**.

9.11. Phân tích theo phân khúc (slice analysis) và phân tích lỗi

```
def slice_report(X: pd.DataFrame, y, p, col: str, threshold: float) -> pd.DataFrame:
    d = X[[col]].copy()
    d["y"], d["p"] = np.asarray(y), p
    d["pred"] = (d["p"] >= threshold).astype(int)
    rows = []
    for key, g in d.groupby(col, observed=True, dropna=False):
        if g["y"].nunique() < 2 or len(g) < 50:
            continue
        rows.append({col: key, "n": len(g), "prevalence": g["y"].mean(), "roc_auc": roc_auc_score(g["y"], g["p"]),
                    "pr_auc": average_precision_score(g["y"], g["p"]), "recall": recall_score(g["y"], g["pred"]),
                    "precision": precision_score(g["y"], g["pred"], zero_division=0),
                    "calib_gap": g["p"].mean() - g["y"].mean()})
    return pd.DataFrame(rows).sort_values("pr_auc")

t_star = float(tuned.best_threshold_)
print(slice_report(X_valid, y_valid, p_gbm, "contract", t_star).round(3).to_string(index=False))
X_slices = X_valid.assign(tenure_band=pd.cut(X_valid["tenure_months"], [0, 6, 24, 48, 72]))
print(slice_report(X_slices, y_valid, p_gbm, "tenure_band", t_star).round(3).to_string(index=False))

errors = X_valid.assign(y=y_valid.to_numpy(), p=p_gbm)
print("FN tự tin nhất (churn thật, điềm thấp):")
print(errors[errors.y == 1].nsmallest(5, "p")[["contract", "tenure_months", "support_calls", "monthly_charges",
"p"]])
```

9.12. Báo cáo đánh giá cuối cùng trên tập test

```
from churn.models.evaluate import bootstrap_metric_ci

final_p = gbm.predict_proba(X_test)[:, 1]
final_pred = (final_p >= t_star).astype(int)
metrics = {"ROC-AUC": roc_auc_score, "PR-AUC": average_precision_score, "Brier": brier_score_loss}
rows = []
for name, fn in metrics.items():
    point, lo, hi = bootstrap_metric_ci(y_test, final_p, fn, n_boot=500)
    rows.append({"metric": name, "value": point, "ci95_low": lo, "ci95_high": hi})
for name, fn in {"Recall": recall_score, "Precision": precision_score}.items():
    point, lo, hi = bootstrap_metric_ci(y_test, final_pred, fn, n_boot=500)
    rows.append({"metric": f"{name} @ {t_star:.2f}", "value": point, "ci95_low": lo, "ci95_high": hi})
print(pd.DataFrame(rows).set_index("metric").round(4))
print("Lợi nhuận kỳ vọng trên test (nghìn VNĐ):", f"{expected_profit(y_test, final_p, t_star):.0f}")
```

Checklist Chương 9

- ☐ Tách đánh giá **xác suất** (log loss, Brier, calibration) và **quyết định** (ngưỡng, confusion matrix, lợi nhuận).
- ☐ Không dùng Accuracy làm metric chính khi mất cân bằng; nhớ precision phụ thuộc prevalence.
- ☐ Confusion matrix phân tích tại **ngưỡng thực sự triển khai**, theo cả hàng (recall) và cột (precision).
- ☐ Ngưỡng chọn trên validation/OOF theo chi phí hoặc năng lực vận hành; đóng gói bằng `FixedThresholdClassifier`.
- ☐ Kiểm tra calibration (reliability diagram, ECE); calibrate bằng `CalibratedClassifierCV` (+ `FrozenEstimator`) trên dữ liệu độc lập.
- ☐ Hồi quy: metric nhất quán với đại lượng dự đoán; dự báo khoảng có kiểm tra coverage.
- ☐ So sánh mô hình bằng kiểm định ghép cặp / corrected t-test; báo cáo CI.
- ☐ Slice analysis cho các phân khúc quan trọng; xem các lỗi tự tin nhất.

Tài liệu tham khảo Chương 9

- scikit-learn User Guide: *Metrics and scoring: quantifying the quality of predictions* (Which scoring function should I use?; classification, regression, ranking metrics); *Tuning the decision threshold for class prediction*; *Probability calibration*.
- scikit-learn Examples: *Post-tuning the decision threshold for cost-sensitive learning*; *Probability Calibration curves*; *Statistical comparison of models using grid search*; *Prediction Intervals for Gradient Boosting Regression*.
- Gneiting, T. & Raftery, A. (2007). *Strictly Proper Scoring Rules, Prediction, and Estimation*. JASA 102.
- Gneiting, T. (2011). *Making and Evaluating Point Forecasts*. JASA 106.
- Elkan, C. (2001). *The Foundations of Cost-Sensitive Learning*. IJCAI.
- Saito, T. & Rehmsmeier, M. (2015). *The Precision-Recall Plot Is More Informative than the ROC Plot When Evaluating Binary Classifiers on Imbalanced Datasets*. PLOS ONE.
- Chicco, D. & Jurman, G. (2020). *The advantages of the Matthews correlation coefficient (MCC) over F1 score and accuracy*. BMC Genomics.
- Niculescu-Mizil, A. & Caruana, R. (2005). *Predicting Good Probabilities with Supervised Learning*. ICML.
- Murphy, A. H. (1973). *A New Vector Partition of the Probability Score*. J. Applied Meteorology.
- Nadeau, C. & Bengio, Y. (2003). *Inference for the Generalization Error*. Machine Learning 52.
- Dietterich, T. (1998). *Approximate Statistical Tests for Comparing Supervised Classification Learning Algorithms*. Neural Computation.
- Angelopoulos, A. & Bates, S. (2023). *Conformal Prediction: A Gentle Introduction*. Foundations and Trends in ML.
- MAPIE documentation.
- Hyndman, R. & Koehler, A. (2006). *Another look at measures of forecast accuracy*. IJF 22(4).
- Google ML Crash Course: *Classification: Accuracy, precision, recall; ROC and AUC; Prediction bias*.

CHƯƠNG 10. GIẢI THÍCH MÔ HÌNH (XAI) & FAIRNESS

Mục tiêu chương: trả lời **vì sao** mô hình dự đoán như vậy, ở cấp toàn cục lẫn từng khách hàng; phát hiện mô hình học "sai lý do" (leakage, proxy); và kiểm tra mô hình có **công bằng** giữa các nhóm không.

10.1. Vì sao cần giải thích và giải thích cho ai?

Người dùng	Câu hỏi	Kỹ thuật phù hợp
Data Scientist	Mô hình có học đúng quy luật không? Có leakage không?	Permutation importance, PDP/ICE, SHAP, error analysis
Nhân viên vận hành (CSKH)	Vì sao khách này rủi ro cao? Nên nói gì với khách?	Reason codes từ SHAP; counterfactual
Lãnh đạo / nghiệp vụ	Yếu tố nào thúc đẩy churn?	Global importance + PDP, có diễn giải nghiệp vụ
Kiểm toán / pháp chế	Quyết định có giải thích được, có phân biệt đối xử không?	Model card, fairness metrics, mô hình glass-box

Phân loại phương pháp (Molnar, *Interpretable Machine Learning*):

Trục	Lựa chọn
Bản chất	Intrinsic (mô hình tự giải thích được: tuyến tính, cây nông, GAM/EBM) và post-hoc (giải thích mô hình hộp đen)
Phạm vi	Global (toàn mô hình) và local (một dự đoán)
Phụ thuộc mô hình	Model-specific (TreeSHAP, hệ số) và model-agnostic (permutation, PDP, KernelSHAP, LIME)

Cảnh báo quan trọng. Mọi phương pháp ở chương này giải thích **mô hình**, không giải thích **thế giới thật**. Feature quan trọng với mô hình **không có nghĩa** là thay đổi feature đó sẽ thay đổi kết quả thật. Muốn kết luận nhân quả, xem Chương 4.8.

Thiết lập: thêm hai feature ngẫu nhiên để kiểm tra phương pháp

Theo ví dụ chính thức *Permutation Importance vs Random Forest Feature Importance (MDI)* của scikit-learn, ta thêm `random_num` (liên tục) và `random_cat` (phân loại, 3 mức). Cả hai **không liên quan** tới target. Phương pháp tốt phải xếp chúng gần 0.

```
import lightgbm as lgb
import matplotlib.pyplot as plt
import numpy as np
import pandas as pd
from sklearn.compose import ColumnTransformer
from sklearn.ensemble import RandomForestClassifier
from sklearn.impute import SimpleImputer
from sklearn.inspection import permutation_importance
from sklearn.metrics import average_precision_score, roc_auc_score
from sklearn.pipeline import Pipeline, make_pipeline
from sklearn.preprocessing import OrdinalEncoder

from churn.data.split import time_split
from churn.data.synthetic import make_churn_data
from churn.features.clean import clean_churn

pd.set_option("display.width", 160)
rng = np.random.default_rng(0)
df = clean_churn(make_churn_data(n=20_000, seed=42))
```

```

df["random_num"] = rng.normal(size=len(df))
df["random_cat"] = rng.choice(["a", "b", "c"], size=len(df))
num = ["age", "tenure_months", "monthly_charges", "support_calls", "data_usage_gb", "random_num"]
cat = ["contract", "payment_method", "region", "random_cat"]
features = num + cat
train_df, valid_df, test_df = time_split(df, "signup_date", "2023-10-01", "2024-01-01")
X_train, y_train = train_df[features], train_df["churn"]
X_test, y_test = pd.concat([valid_df, test_df])[features], pd.concat([valid_df, test_df])["churn"]

prep = ColumnTransformer([
    ("num", SimpleImputer(strategy="median"), num),
    ("cat", make_pipeline(SimpleImputer(strategy="constant", fill_value="missing"),
                          OrdinalEncoder(handle_unknown="use_encoded_value", unknown_value=-1)), cat),
], verbose_feature_names_out=False).set_output(transform="pandas")

rf = Pipeline([("prep", prep), ("clf", RandomForestClassifier(
    n_estimators=200, min_samples_leaf=1, n_jobs=-1, random_state=0))]).fit(X_train, y_train)
gbm = Pipeline([("prep", prep), ("clf", lgb.LGBMClassifier(
    n_estimators=300, learning_rate=0.03, num_leaves=15, min_child_samples=50, verbose=-1,
    random_state=0))]).fit(X_train, y_train)
for name, m in [("RandomForest (l\u00e0 s\u00e2u, overfit)", rf), ("LightGBM", gbm)]:
    print(f"{name:30s} AUC train={roc_auc_score(y_train, m.predict_proba(X_train)[: , 1]):.3f} "
          f"test={roc_auc_score(y_test, m.predict_proba(X_test)[: , 1]):.3f}")

```

10.2. Feature importance: MDI và permutation

[Docs] scikit-learn, *Permutation feature importance và Relation to impurity-based importance in trees*:

- **MDI** (mean decrease in impurity, `feature_importances_`) được tính trên **dữ liệu train**, nên có thể gán importance cao cho feature **không dự đoán được trên dữ liệu mới** khi mô hình overfit. MDI cũng **thiên vị mạnh** feature có **nhiều giá trị** (biến liên tục) so với biến nhị phân hay phân loại ít mức.
- **Permutation importance** đo mức **giảm điểm** khi xáo trộn một cột, tính được trên **dữ liệu held-out** và với **bất kỳ metric** nào. Lưu ý của tài liệu: feature ít quan trọng với một mô hình *tốt* có thể rất quan trọng với một mô hình *tốt*. Vì vậy hãy đánh giá mô hình trước, rồi mới tính importance.

$$i_j = s - \frac{1}{K} \sum_{k=1}^K s_{k,j}$$

```

mdi = pd.Series(rf["clf"].feature_importances_, index=rf["prep"].get_feature_names_out())
# rf đã tự song song khi predict (n_jobs=-1) -> permutation_importance chạy tuần tự để tránh song song l\u00f4ng nhau
perm_train = permutation_importance(rf, X_train, y_train, scoring="average_precision", n_repeats=5, random_state=0)
perm_test = permutation_importance(rf, X_test, y_test, scoring="average_precision", n_repeats=5, random_state=0)
cmp_imp = pd.DataFrame({"MDI (train)": mdi,
                        "perm (train)": pd.Series(perm_train.importances_mean, index=features),
                        "perm (test)": pd.Series(perm_test.importances_mean, index=features),
                        "perm_test_std": pd.Series(perm_test.importances_std, index=features)})
print(cmp_imp.sort_values("perm (test)", ascending=False).round(4))

```

Đọc kết quả: MDI xếp `random_num` và `data_usage_gb` (liên tục, không có tác động thật) **cao hơn** `contract` (3 mức, tác động mạnh). Permutation importance trên **test** đưa các biến ngẫu nhiên về ≈ 0 . Khoảng cách giữa permutation trên train và trên test cho thấy mức độ overfit theo từng feature.

Feature tương quan làm importance bị "chia nhỏ"

[Docs] *Misleading values on strongly correlated features*: khi xáo trộn một feature, mô hình vẫn lấy được thông tin qua feature tương quan với nó. Kết quả là **cả hai** đều có importance thấp, dù thông tin chung rất quan trọng. Tài liệu đề xuất **phân cụm các feature tương quan và giữ một đại diện** mỗi cụm (Chương 3.7).

```

Xc_train = X_train.assign(tenure_copy=X_train["tenure_months"] + rng.normal(0, 0.5, len(X_train)))
Xc_test = X_test.assign(tenure_copy=X_test["tenure_months"] + rng.normal(0, 0.5, len(X_test)))
prep_c = ColumnTransformer([("num", SimpleImputer(strategy="median"), num + ["tenure_copy"]),
                             ("cat", make_pipeline(SimpleImputer(strategy="constant", fill_value="missing"),
                                                       OrdinalEncoder(handle_unknown="use_encoded_value",
                                                                       unknown_value=-1)), cat)])

```

```
gbm_c = make_pipeline(prepare_c, lgb.LGBMClassifier(n_estimators=300, learning_rate=0.03, num_leaves=15,
                                                    min_child_samples=50, verbose=-1, random_state=0)).fit(Xc_train,
y_train)
pi_c = permutation_importance(gbm_c, Xc_test, y_test, scoring="average_precision", n_repeats=5, random_state=0)
pi_o = permutation_importance(gbm, X_test, y_test, scoring="average_precision", n_repeats=5, random_state=0)
print("tenure (không có bản sao):", round(pd.Series(pi_o.importances_mean, index=features)["tenure_months"], 4))
print("tenure + bản sao      :", pd.Series(pi_c.importances_mean, index=list(Xc_test.columns))
      [{"tenure_months", "tenure_copy"}].round(4).to_dict())
```

10.3. Partial Dependence (PDP), ICE và hiệu ứng tương tác

[Docs] *Partial Dependence and Individual Conditional Expectation plots:*

$$PD_S(X_S) = \mathbb{E}_{X_C} [f(X_S, X_C)] \approx \frac{1}{n} \sum_{i=1}^n f(X_S, x_C^{(i)})$$

- **PDP** là hiệu ứng *trung bình* của feature lên dự đoán. **ICE** vẽ một đường cho *mỗi* mẫu. ICE tỏ ra nhiều nghĩa là có tương tác hoặc dị biệt.
- `centered=True` đưa mọi đường về cùng điểm xuất phát, giúp so sánh độ dốc.
- **Giả định độc lập:** PDP thay giá trị x_S cho mọi mẫu, kể cả khi tổ hợp đó phi thực tế (tenure = 1 nhưng total_charges rất lớn). Khi feature tương quan mạnh, dùng **ALE** (Accumulated Local Effects; Apley & Zhu, 2020).
- Mặc định với cây/GBM của scikit-learn, `method="recursion"` rất nhanh nhưng chỉ cho `kind="average"`. ICE yêu cầu `method="brute"`.

```
from sklearn.inspection import PartialDependenceDisplay

# scikit-learn 1.9 từ chối PDP trên cột kiểu số nguyên (lưới giá trị bị làm tròn ngầm) -> ép sang float
int_cols = X_test.select_dtypes("integer").columns
sample = X_test.sample(1500, random_state=0).astype({c: "float64" for c in int_cols})
fig, axes = plt.subplots(1, 3, figsize=(16, 4.5))
PartialDependenceDisplay.from_estimator(
    gbm, sample, features=["support_calls", "tenure_months", "monthly_charges"], kind="both",
    subsample=150, centered=True, response_method="predict_proba", random_state=0, ax=axes,
    ice_lines_kw={"alpha": 0.15}, pd_line_kw={"color": "crimson", "lw": 2.5})
fig.suptitle("PDP (đỏ) + ICE (xám): support_calls tăng rủi ro, tenure giảm rủi ro")
plt.tight_layout()

fig, ax = plt.subplots(figsize=(6, 5))
PartialDependenceDisplay.from_estimator(gbm, sample, features=[("tenure_months", "support_calls")],
                                       kind="average", ax=ax)
ax.set_title("PDP 2 chiều: tương tác tenure x support_calls")
```

10.4. SHAP: Shapley values cho từng dự đoán

Lý thuyết

Lý thuyết trò chơi hợp tác (Shapley, 1953) phân chia "phần thưởng" (dự đoán) cho các "người chơi" (feature) một cách **duy nhất** thỏa bốn tiên đề: *efficiency, symmetry, dummy, additivity*:

$$\phi_j = \sum_{S \subseteq F \setminus \{j\}} \frac{|S|! (|F| - |S| - 1)!}{|F|!} [v(S \cup \{j\}) - v(S)]$$

SHAP (Lundberg & Lee, 2017) diễn giải mỗi dự đoán theo dạng **cộng tính**: $f(x) = \phi_0 + \sum_j \phi_j$, với ϕ_0 là giá trị kỳ vọng (base value).

Explainer	Mô hình	Ghi chú
TreeExplainer	Cây, RF, XGBoost, LightGBM, CatBoost	Chính xác, nhanh (đa thức). <code>feature_perturbation="tree_path_dependent"</code> (không cần background) hoặc <code>"interventional"</code> (cần background, đúng nghĩa "can thiệp" hơn)

Explainer	Mô hình	Ghi chú
LinearExplainer	Tuyến tính	Có xét tương quan feature
KernelExplainer	Bất kỳ	Model-agnostic nhưng chậm
Permutation / Exact	Bất kỳ	shap.Explainer tự chọn
Deep/GradientExplainer	Neural network	

Thang đo: với LightGBM nhị phân, SHAP mặc định nằm trên **thang log-odds** (raw margin). Cộng base value với tổng SHAP ra đúng logit của dự đoán.

```
import shap

X_test_t = gbm["prep"].transform(X_test)
explainer = shap.TreeExplainer(gbm["clf"])
sv = explainer(X_test_t.iloc[:2000])
values = sv.values[:, 1] if sv.values.ndim == 3 else sv.values # một số phiên bản trả 2 lớp
base = np.ravel(sv.base_values)[0] if np.ndim(sv.base_values) else sv.base_values

# Kiểm tra tính cộng (efficiency): base + Σφ = logit(dự đoán)
raw_margin = gbm["clf"].predict_proba(X_test_t.iloc[:2000], raw_score=True)
print("Sai số cộng tính lớn nhất:", float(np.abs(base + values.sum(axis=1) - raw_margin).max()))

global_shap = pd.Series(np.abs(values).mean(axis=0), index=X_test_t.columns).sort_values(ascending=False)
print("mean |SHAP| (log-odds):")
print(global_shap.round(4))
```

Đối chiếu ground truth (Chương 0.8): contract, support_calls, tenure_months có |SHAP| lớn nhất. region, data_usage_gb, random_num, random_cat gần 0. SHAP tìm đúng các yếu tố thật.

Biểu đồ SHAP chuẩn

```
plt.figure()
shap.plots.beeswarm(shap.Explanation(values, base_values=np.full(len(values), base),
data=X_test_t.iloc[:2000].values,
feature_names=list(X_test_t.columns)), max_display=10, show=False)
plt.title("Beeswarm: độ lớn + chiều tác động")
plt.tight_layout()

plt.figure()
shap.plots.scatter(shap.Explanation(values[:, list(X_test_t.columns).index("support_calls")],
data=X_test_t["support_calls"].iloc[:2000].values,
feature_names="support_calls"), show=False)
plt.title("Dependence: SHAP theo support_calls")
```

Reason codes cho từng khách hàng

```
READABLE = {"support_calls": "Gọi tổng đài hỗ trợ nhiều lần", "tenure_months": "Thời gian gắn bó",
"contract": "Loại hợp đồng", "monthly_charges": "Mức cước tháng", "payment_method": "Phương thức thanh toán",
"age": "Độ tuổi", "data_usage_gb": "Dung lượng sử dụng", "region": "Khu vực"}

def reason_codes(row_values: np.ndarray, row_data: pd.Series, k: int = 3) -> list[str]:
    contrib = pd.Series(row_values, index=row_data.index)
    top = contrib[contrib > 0].nlargest(k)
    return [f"{READABLE.get(f, f)} (giá trị={row_data[f]:.0f}, +{v:.2f} log-odds)" for f, v in top.items()]

proba = gbm["clf"].predict_proba(X_test_t.iloc[:2000])[:, 1]
i = int(np.argmax(proba))
print(f"Khách có rủi ro cao nhất: P(churn)={proba[i]:.2f}")
for r in reason_codes(values[i], X_test_t.iloc[i]):
    print(" -", r)
```

[Kinh nghiệm] Khi đưa reason codes ra cho người dùng cuối:

- Chỉ hiển thị **feature hành động được** hoặc dễ hiểu. Ẩn các feature kỹ thuật (cờ missing, ID mã hóa).
- Dùng ngôn ngữ nghiệp vụ, không dùng "log-odds".
- Kiểm tra **độ ổn định**: hai khách gần giống nhau phải có lý do gần giống nhau.

10.5. Mô hình tự giải thích được (glass-box) và mô hình thay thế

Hệ số Logistic chuẩn hóa: odds ratio trên 1 độ lệch chuẩn

```
from sklearn.linear_model import LogisticRegression

from churn.features.build import RAW_FEATURES, build_preprocessor

logit = make_pipeline(build_preprocessor(scale=True), LogisticRegression(max_iter=3000)).fit(
    train_df[RAW_FEATURES], y_train)
coef = pd.Series(logit[-1].coef_[0], index=logit[0].get_feature_names_out())
print(pd.DataFrame({"coef_per_sd": coef, "odds_ratio_per_sd": np.exp(coef)})
      .reindex(coef.abs().sort_values(ascending=False).index).head(8).round(3))
# Lưu ý: one-hot đầy đủ (không drop) + L2 -> hệ số các mức của cùng một biến chỉ có nghĩa TƯƠNG ĐỐI với nhau
# (two-year so với month-to-month), không diễn giải từng hệ số riêng lẻ.
```

Global surrogate: cây nông bắt chước mô hình hộp đen

```
from sklearn.metrics import r2_score
from sklearn.tree import DecisionTreeRegressor, export_text

surrogate = DecisionTreeRegressor(max_depth=3, min_samples_leaf=200, random_state=0)
target_logit = gbml["clf"].predict_proba(X_test_t, raw_score=True)
surrogate.fit(X_test_t, target_logit)
print(f"Độ trung thực (R² surrogate vs mô hình) = {r2_score(target_logit, surrogate.predict(X_test_t)):.3f}")
print(export_text(surrogate, feature_names=list(X_test_t.columns), decimals=1))
```

Surrogate chỉ đáng tin khi **độ trung thực (fidelity)** cao. Nó giải thích *mô hình*, không giải thích *dữ liệu*.

Explainable Boosting Machine (EBM, InterpretML của Microsoft) là GAM được huấn luyện bằng boosting, có thêm một số tương tác cặp. Độ chính xác thường gần GBM và **mỗi feature có đồ thị hiệu ứng riêng**. Đây là lựa chọn tốt khi cần cả hiệu năng lẫn minh bạch (ngân hàng, y tế).

```
from interpret.glassbox import ExplainableBoostingClassifier

ebm = ExplainableBoostingClassifier(interactions=10, random_state=0).fit(X_train, y_train)
ebm.explain_global().visualize() # đồ thị hiệu ứng từng feature
```

10.6. Fairness: công bằng giữa các nhóm

Tiêu chí	Định nghĩa	Ý nghĩa	fairlearn
Demographic parity	$P(\hat{Y} = 1 \mid A = a)$ bằng nhau	Tỷ lệ được chọn như nhau	demographic_parity_difference
Equal opportunity	TPR bằng nhau	Người "xứng đáng" có cơ hội như nhau	true_positive_rate theo nhóm
Equalized odds	TPR và FPR bằng nhau		equalized_odds_difference
Predictive parity	Precision bằng nhau	Dự đoán positive đáng tin như nhau	MetricFrame + precision
Calibration theo nhóm	Xác suất calibrate trong từng nhóm		Reliability diagram theo nhóm

Định lý bất khả thi (Kleinberg, Mullainathan & Raghavan, 2016; Chouldechova, 2017): khi tỷ lệ cơ sở (base rate) khác nhau giữa các nhóm, **không thể** đồng thời thỏa calibration theo nhóm và equalized odds, trừ trường hợp tầm thường. Chọn tiêu chí là **quyết định nghiệp vụ/đạo đức/pháp lý**, cần stakeholder tham gia.

```
from fairlearn.metrics import (MetricFrame, count, demographic_parity_difference, equalized_odds_difference,
                             false_positive_rate, selection_rate, true_positive_rate)
from sklearn.metrics import precision_score

threshold = 0.25
p_test = gbm.predict_proba(X_test)[: , 1]
pred = (p_test >= threshold).astype(int)

def age_bands(ages: pd.Series) -> pd.Series:
    """Nhóm tuổi; tuổi thiếu -> 'unknown'. Lưu ý pandas 3: .astype(str) GIỮ NaN là NaN (không thành 'nan'),
    nên phải gán nhãn tường minh, nếu không nhóm thiếu tuổi bị loại âm thầm khỏi phân tích fairness."""
    bands = pd.cut(ages, bins=[0, 30, 45, 60, 120], labels=["<30", "30-45", "45-60", "60+"])
    return bands.cat.add_categories("unknown").fillna("unknown").astype(str)

age_group = age_bands(X_test["age"])

mf = MetricFrame(metrics={"n": count, "base_rate": lambda y, p: np.mean(y), "selection_rate": selection_rate,
                        "TPR": true_positive_rate, "FPR": false_positive_rate,
                        "precision": lambda y, p: precision_score(y, p, zero_division=0)},
                y_true=y_test, y_pred=pred, sensitive_features=age_group)
print(mf.by_group.round(3))
print("Demographic parity difference:", round(demographic_parity_difference(y_test, pred,
sensitive_features=age_group), 3))
print("Equalized odds difference      :", round(equalized_odds_difference(y_test, pred, sensitive_features=age_group),
3))
```

Giảm thiểu bias

Giai đoạn	Kỹ thuật	Công cụ
Pre-processing	Cân bằng lại dữ liệu, loại biến proxy (mã vùng chỉ tiết có thể là proxy cho dân tộc/thu nhập)	fairlearn.preprocessing.CorrelationRemover
In-processing	Huấn luyện có ràng buộc fairness	fairlearn.reductions.ExponentiatedGradient
Post-processing	Ngưỡng khác nhau theo nhóm	fairlearn.postprocessing.ThresholdOptimizer

```
from fairlearn.postprocessing import ThresholdOptimizer
from sklearn.frozen import FrozenEstimator

# fairlearn không nhận NaN trong X. Điền NaN bằng đúng giá trị mà SimpleImputer của pipeline đã học
# (median cho số, "missing" cho phân loại) -> dự đoán của mô hình KHÔNG đổi.
fill = {**dict(zip(num, gbm["prep"].named_transformers_["num"].statistics_)), **{c: "missing" for c in cat}}

def no_nan(frame: pd.DataFrame) -> pd.DataFrame:
    return frame[features].fillna(fill)

assert np.allclose(gbm.predict_proba(no_nan(valid_df))[:, 1], gbm.predict_proba(valid_df[features])[:, 1])

# Fit bộ hậu xử lý trên VALID (độc lập với dữ liệu train mô hình), đánh giá trên TEST
X_fit, y_fit = no_nan(valid_df), valid_df["churn"]
postproc = ThresholdOptimizer(estimator=FrozenEstimator(gbm), constraints="true_positive_rate_parity",
                             objective="balanced_accuracy_score", predict_method="predict_proba", prefit=True)
postproc.fit(X_fit, y_fit, sensitive_features=age_bands(valid_df["age"])) # tuổi gốc (có thể thiếu)

age_te = age_bands(test_df["age"])
before = (gbm.predict_proba(test_df[features])[:, 1] >= threshold).astype(int)
after = postproc.predict(no_nan(test_df), sensitive_features=age_te, random_state=0)
```

```
for name, pr in [("trước", before), ("sau ThresholdOptimizer", after)]:
    tpr = MetricFrame(metrics=true_positive_rate, y_true=test_df["churn"], y_pred=pr, sensitive_features=age_te)
    print(f"TPR theo nhóm {name:22s}: {tpr.by_group.round(3).to_dict()} | chênh lệch={tpr.difference():.3f}")
```

[Kinh nghiệm] Dùng ngưỡng khác nhau theo nhóm (post-processing) yêu cầu **dùng thuộc tính nhạy cảm lúc dự đoán**. Điều này có thể bị luật cấm trong một số lĩnh vực (tín dụng, tuyển dụng). Hãy tham vấn pháp chế trước khi áp dụng.

10.7. Model Card: tài liệu hóa mô hình

Theo Mitchell et al. (2019), *Model Cards for Model Reporting*:

```
# Model Card: churn-classifier v3
## Chi tiết mô hình
- Loại: LightGBM (300 cây), pipeline tiền xử lý `churn.features.build`; MLflow model_id m-xxxx
- Chủ sở hữu: Data Science Team (ds-team@company.vn); ngày: 2026-10
## Mục đích sử dụng
- Dùng cho: xếp hạng khách có nguy cơ rời bỏ trong 30 ngày để CSKH liên hệ.
- KHÔNG dùng cho: từ chối dịch vụ, định giá cá nhân hóa, quyết định tín dụng.
## Dữ liệu
- Huấn luyện: khách đăng ký 2022-01 → 2023-09; đánh giá out-of-time 2023-10 → 2024-06.
- Nhận: hủy dịch vụ trong 30 ngày sau ngày chấm điểm (gap 7 ngày).
## Hiệu năng (test out-of-time, 95% CI bootstrap)
- PR-AUC 0.39 [0.36–0.42]; ROC-AUC 0.75 [0.73–0.77]; ngưỡng 0.33 (tối đa lợi nhuận kỳ vọng).
## Phân tích theo nhóm & fairness
- Chênh lệch TPR giữa các nhóm tuổi ≤ 0.05 sau hậu xử lý; kém chính xác hơn với khách < 3 tháng.
## Giải thích
- Yêu tố chính: loại hợp đồng, số cuộc gọi hỗ trợ, thời gian gắn bó (mean |SHAP|).
## Hạn chế & rủi ro
- Chưa kiểm định cho khách doanh nghiệp; nhạy với thay đổi chính sách giá (theo dõi drift).
## Giám sát
- PSI feature/score hằng tuần; retrain khi PR-AUC giảm > 10% hoặc PSI > 0.25.
```

Tài liệu bổ sung cho dữ liệu: **Datasheets for Datasets** (Gebru et al., 2021).

Checklist Chương 10

- ☐ Đánh giá mô hình trước, giải thích sau. Importance tính trên **held-out** (permutation/SHAP), không chỉ MDI.
- ☐ Kiểm tra phương pháp bằng biến ngẫu nhiên; xử lý feature tương quan (gom cụm) trước khi diễn giải.
- ☐ Hướng và hình dạng tác động (PDP/ICE/SHAP) hợp lý về nghiệp vụ; cân nhắc monotonic constraints.
- ☐ Reason codes dùng ngôn ngữ nghiệp vụ, chỉ gồm feature hiểu được, có kiểm tra độ ổn định.
- ☐ Đo fairness theo các nhóm liên quan; chọn tiêu chí cùng stakeholder; tham vấn pháp chế về giảm thiểu.
- ☐ Có Model Card ghi rõ mục đích, phạm vi, hiệu năng theo nhóm, hạn chế, chủ sở hữu.

Tài liệu tham khảo Chương 10

- scikit-learn User Guide: *Inspection (Partial Dependence and ICE plots; Permutation feature importance: Outline, Relation to impurity-based importance, Misleading values on strongly correlated features)*.
- scikit-learn Examples: *Permutation Importance vs Random Forest Feature Importance (MDI); Permutation Importance with Multicollinear or Correlated Features; Common pitfalls in the interpretation of coefficients of linear models*.
- SHAP documentation: *An introduction to explainable AI with Shapley values*; API `TreeExplainer`. shap.readthedocs.io
- Lundberg, S. & Lee, S.-I. (2017). *A Unified Approach to Interpreting Model Predictions*. NeurIPS.
- Lundberg, S. et al. (2020). *From local explanations to global understanding with explainable AI for trees*. Nature MI 2.
- Molnar, C. (2022). *Interpretable Machine Learning*, 2nd ed. christophm.github.io/interpretable-ml-book

- Apley, D. & Zhu, J. (2020). *Visualizing the effects of predictor variables in black box supervised learning models (ALE)*. JRSS-B 82(4).
- Breiman, L. (2001). *Random Forests*. Machine Learning 45. · Strobl, C. et al. (2007). *Bias in random forest variable importance measures*. BMC Bioinformatics 8.
- fairlearn User Guide: *Assessment* (MetricFrame), *Mitigation* (ThresholdOptimizer, ExponentiatedGradient). fairlearn.org
- Kleinberg, J., Mullainathan, S. & Raghavan, M. (2016). *Inherent Trade-Offs in the Fair Determination of Risk Scores*. · Chouldechova, A. (2017). *Fair prediction with disparate impact*. Big Data 5(2).
- Mitchell, M. et al. (2019). *Model Cards for Model Reporting*. FAT*. · Gebru, T. et al. (2021). *Datasheets for Datasets*. CACM.
- Nori, H. et al. (2019). *InterpretML: A Unified Framework for Machine Learning Interpretability*.

CHƯƠNG 11. TRỰC QUAN HÓA DỮ LIỆU & KỂ CHUYỆN BẰNG DỮ LIỆU

Mục tiêu chương: chọn đúng loại biểu đồ cho đúng câu hỏi, dựng biểu đồ **chính xác về nhận thức**, nhất quán về phong cách, tái lập được bằng code, và trình bày kết quả để **dẫn tới quyết định**.

11.1. Nền tảng nhận thức thị giác

Thứ bậc độ chính xác khi giải mã (Cleveland & McGill, 1984), từ chính xác nhất đến kém nhất:

1. Vị trí trên cùng một trục (dot plot, scatter, bar)
2. Vị trí trên các trục không thẳng hàng (small multiples)
3. Độ dài (bar)
4. Góc, độ dốc
5. Diện tích (bubble, treemap)
6. Thể tích, độ cong
7. Độ đậm nhạt, độ bão hòa màu

Hệ quả: pie chart (góc/diện tích) kém hơn bar chart (độ dài/vị trí) khi so sánh giá trị. Heatmap (màu) phù hợp cho *mẫu hình*, không phù hợp để đọc giá trị chính xác.

Nguyên tắc của Tufte (*The Visual Display of Quantitative Information*, 1983):

- **Data-ink ratio:** tối đa hóa phần mực dùng để thể hiện dữ liệu; bỏ khung, lưới đậm, hiệu ứng 3D, bóng đổ.
- **Không bóp méo:** "lie factor" = độ lớn hiệu ứng trên hình / độ lớn trong dữ liệu ≈ 1 . Biểu đồ cột **phải bắt đầu từ 0**.
- **Small multiples:** nhiều biểu đồ nhỏ cùng thang đo thường tốt hơn một biểu đồ chồng chất.

Grammar of Graphics (Wilkinson, 2005): một biểu đồ = **dữ liệu** + **ánh xạ thẩm mỹ** (x, y, màu, kích thước) + **hình học** (điểm, đường, cột) + **thống kê** (bin, smooth) + **hệ tọa độ** + **facet**. Đây là nền tảng của ggplot2, seaborn.objects, plotnine, Altair/Vega-Lite.

11.2. Chọn biểu đồ theo câu hỏi

Câu hỏi	Biểu đồ phù hợp	Tránh
Phân phối một biến số	Histogram (bin FD), KDE, ECDF , box/violin	Pie
So sánh phân phối giữa nhóm	Box/violin cạnh nhau, ECDF chồng, ridgeline	Nhiều histogram chồng đặc
So sánh giá trị giữa nhóm	Bar ngang (nhãn dài), dot plot , có khoảng tin cậy	3D bar, pie > 4 phần
Thành phần (tỷ lệ)	Stacked bar 100%, waffle, treemap	Pie nhiều lát
Xu hướng theo thời gian	Line (+ dải CI), area	Bar cho chuỗi dài
Quan hệ 2 biến số	Scatter (+ hexbin/2D KDE khi nhiều điểm), đường hồi quy/LOWESS	Nối điểm không có thứ tự
Nhiều biến	Heatmap tương quan, pairplot, parallel coordinates	
Địa lý	Choropleth (chuẩn hóa theo dân số!), bubble map	Choropleth số tuyệt đối
Đánh giá mô hình	ROC, PR, calibration, confusion matrix, lift/gain, residual	
Bất định	Error bar, dải CI, fan chart, gradient interval	Chỉ vẽ ước lượng điểm

11.3. Kiến trúc matplotlib và phong cách thống nhất

[Docs] matplotlib phân biệt hai giao diện. **Giao diện hướng đối tượng (OO)** (`fig`, `ax = plt.subplots()`), gọi phương thức trên `ax`) được **khuyến nghị** cho code tái sử dụng và biểu đồ phức tạp. **Giao diện pyplot** (state-based, `plt.plot`) chỉ phù hợp khi vẽ nhanh tương tác. Cấu trúc đối tượng: **Figure** → **Axes** (một hệ trục) → **Axis**, **Artist** (mọi thứ được vẽ).

Package tham chiếu có `churn/viz/style.py` để mọi biểu đồ trong dự án dùng chung phong cách:

```
PALETTE = {"primary": "#1f5fa8", "accent": "#d1495b", "neutral": "#9aa5b1", "good": "#2a9d8f"}

def set_style() -> None:
    sns.set_theme(style="whitegrid", context="notebook", palette="colorblind")
    mpl.rcParams.update({
        "figure.figsize": (10, 5.5), "figure.dpi": 110, "savefig.dpi": 200, "savefig.bbox": "tight",
        "axes.spines.top": False, "axes.spines.right": False, "axes.titleweight": "bold",
        "axes.titlelocation": "left", "font.family": "DejaVu Sans",      # font hỗ trợ tiếng Việt
    })

def save(fig, name, folder="reports/figures"):      # PNG cho slide + SVG cho in ấn
    for ext in ("png", "svg"):
        fig.savefig(Path(folder) / f"{name}.{ext}")
```

Thiết lập

```
import matplotlib as mpl
import matplotlib.pyplot as plt
import numpy as np
import pandas as pd
import seaborn as sns

from churn.data.synthetic import make_churn_data
from churn.features.clean import clean_churn
from churn.viz.style import PALETTE, save, set_style

set_style()
df = clean_churn(make_churn_data(n=20_000, seed=42))
df["churn_label"] = df["churn"].map({0: "Ở lại", 1: "Rời bỏ"})
```

11.4. Màu sắc có chủ đích

Loại bảng màu	Dùng cho	Ví dụ (matplotlib/seaborn)
Qualitative	Nhóm không có thứ tự	<code>colorblind</code> , <code>tab10</code> , <code>Set2</code>
Sequential	Giá trị từ thấp đến cao	<code>viridis</code> , <code>cividis</code> , <code>Blues</code>
Diverging	Có điểm giữa ý nghĩa (0, trung bình)	<code>RdBu_r</code> , <code>coolwarm</code> (đặt <code>center=0</code>)

Nguyên tắc:

- Dùng **màu xám cho ngữ cảnh** và **một màu nhấn** cho điều cần chú ý.
- Bảng màu **perceptually uniform** (`viridis`, `cividis`) giữ thứ tự khi in đen trắng và với người mù màu (khoảng 8% nam giới).
- Tránh `jet` / `rainbow`: tạo ranh giới giả và không đơn điệu về độ sáng.
- Không mã hóa cùng một biến bằng hai màu khác nhau ở hai biểu đồ cạnh nhau.

```
fig, ax = plt.subplots(figsize=(9, 4.5))
rates = df.groupby("region")["churn"].mean().sort_values()
colors = [PALETTE["accent"] if r == rates.idxmax() else PALETTE["neutral"] for r in rates.index]
ax.barh(rates.index, rates.values, color=colors)
```

```
ax.axvline(df["churn"].mean(), color="k", lw=1, ls="--")
ax.text(df["churn"].mean(), len(rates) - 0.5, "trung bình", va="top", fontsize=9)
ax.xaxis.set_major_formatter(mpl.ticker.PercentFormatter(1.0))
ax.set(title=f"Vùng {rates.idxmax()} cao nhất, nhưng mọi vùng nằm trong dao động ngẫu nhiên",
       xlabel="Tỷ lệ churn", ylabel="")
ax.tick_params(axis="y", labels=7)
save(fig, "churn_by_region")
```

Tiêu đề ở đây nói đúng **sự thật thống kê**: region không có tác động thật (Chương 0.8). Biểu đồ đặt màu nhấn vào vùng cao nhất **rất dễ gây hiểu lầm** nếu không kèm khoảng tin cậy. Xem cách sửa ở mục 11.5.

11.5. Biểu đồ EDA chuẩn mực

So sánh nhóm phải kèm độ bất định

```
from statsmodels.stats.proportion import proportion_confint

g = df.groupby("region")["churn"].agg(["sum", "count"])
lo, hi = proportion_confint(g["sum"], g["count"], method="wilson")
g = g.assign(rate=g["sum"] / g["count"], lo=lo, hi=hi).sort_values("rate")
fig, ax = plt.subplots(figsize=(9, 5))
ax.errorbar(g["rate"], range(len(g)), xerr=[g["rate"] - g["lo"], g["hi"] - g["rate"]], fmt="o",
           color=PALETTE["primary"], ecolor=PALETTE["neutral"], capsize=2, ms=4)
ax.axvline(df["churn"].mean(), color=PALETTE["accent"], ls="--", lw=1)
ax.set_yticks(range(len(g)), g.index, fontsize=7)
ax.xaxis.set_major_formatter(mpl.ticker.PercentFormatter(1.0))
ax.set(title="Khoảng tin cậy 95% của các vùng chồng lên nhau: không có vùng nào khác biệt thật",
       xlabel="Tỷ lệ churn (Wilson 95% CI)")
```

Phân phối theo nhóm: histogram mật độ, ECDF, violin

```
fig, axes = plt.subplots(1, 3, figsize=(17, 4.8))
sns.histplot(data=df, x="monthly_charges", hue="churn_label", stat="density", common_norm=False,
             element="step", bins="fd", ax=axes[0])
axes[0].set_xlim(0, 200)
axes[0].set(title="Khách rời bỏ có cước cao hơn", xlabel="Cước tháng (nghìn VND)")
sns.ecdfplot(data=df, x="tenure_months", hue="churn_label", ax=axes[1])
axes[1].axhline(0.5, color="grey", lw=0.8, ls=":")
axes[1].set(title="ECDF: median tenure của nhóm rời bỏ thấp hơn rõ", xlabel="Tenure (tháng)")
sns.violinplot(data=df, x="contract", y="support_calls", hue="churn_label", split=True, inner="quart",
               order=["month-to-month", "one-year", "two-year"], ax=axes[2])
axes[2].set(title="Số cuộc gọi hỗ trợ theo hợp đồng", xlabel="")
plt.tight_layout()
save(fig, "eda_distributions")
```

[Docs] seaborn phân biệt hàm **axes-level** (histplot, scatterplot ... vẽ lên một ax) và **figure-level** (displot, relplot, catplot ... tự tạo figure, hỗ trợ facet qua col / row). Dùng figure-level cho **small multiples**:

```
grid = sns.displot(data=df, x="tenure_months", hue="churn_label", col="contract", kind="ecdf", height=3.6,
                  aspect=1.1, col_order=["month-to-month", "one-year", "two-year"])
grid.set_titles("{col_name}")
grid.figure.suptitle("Small multiples: tenure theo từng loại hợp đồng", y=1.05)
```

Tỷ lệ theo biến liên tục (binned), có CI

```
fig, ax = plt.subplots(figsize=(8, 4.5))
for contract, sub in df.groupby("contract"):
    b = sub.assign(bin=pd.qcut(sub["tenure_months"], 8, duplicates="drop")).groupby("bin", observed=True)["churn"]
    stats_ = b.agg(["sum", "count"])
    lo, hi = proportion_confint(stats_["sum"], stats_["count"], method="wilson")
    mid = [iv.mid for iv in stats_.index]
    ax.plot(mid, stats_["sum"] / stats_["count"], marker="o", label=contract)
```

```
ax.fill_between(mid, lo, hi, alpha=0.15)
ax.yaxis.set_major_formatter(mpl.ticker.PercentFormatter(1.0))
ax.set(title="Churn giảm theo tenure ở mọi loại hợp đồng", xlabel="Tenure (tháng)", ylabel="Tỷ lệ churn")
ax.legend(title="Hợp đồng", frameon=False)
```

Quan hệ nhiều biến

```
num = ["age", "tenure_months", "monthly_charges", "total_charges", "support_calls", "data_usage_gb"]
corr = df[num].corr(method="spearman")
fig, ax = plt.subplots(figsize=(7, 6))
sns.heatmap(corr, mask=np.triu(np.ones_like(corr, dtype=bool)), annot=True, fmt=".2f", cmap="RdBu_r",
            center=0, vmin=-1, vmax=1, square=True, linewidths=0.5, cbar_kws={"shrink": 0.8}, ax=ax)
ax.set_title("Tương quan Spearman (tam giác dưới)")

sample = df.sample(1500, random_state=0)
pg = sns.pairplot(sample, vars=["tenure_months", "monthly_charges", "support_calls"], hue="churn_label",
                 corner=True, plot_kws={"s": 8, "alpha": 0.4}, diag_kind="kde")
pg.figure.suptitle("Pairplot (mẫu 1 500)", y=1.02)
```

Chuỗi thời gian có chú thích

```
monthly = df.set_index("signup_date").resample("MS")["churn"].agg(["mean", "count"])
fig, ax = plt.subplots(figsize=(11, 4))
ax.plot(monthly.index, monthly["mean"], color=PALETTE["neutral"], marker="o", ms=3)
roll = monthly["mean"].rolling(3, center=True).mean()
ax.plot(monthly.index, roll, color=PALETTE["primary"], lw=2.5, label="Trung bình trượt 3 tháng")
peak = monthly["mean"].idxmax()
ax.annotate(f"Đỉnh {monthly['mean'].max():.1%}", xy=(peak, monthly["mean"].max()), xytext=(15, 10),
           textcoords="offset points", arrowprops={"arrowstyle": "->", "color": PALETTE["accent"]})
ax.yaxis.set_major_formatter(mpl.ticker.PercentFormatter(1.0))
ax.set(title="Tỷ lệ churn theo cohort tháng đăng ký", ylabel="Tỷ lệ churn")
ax.legend(frameon=False)
```

11.6. Dashboard đánh giá mô hình (một hình, sáu panel)

[Docs] scikit-learn cung cấp **Display API** thống nhất cho biểu đồ đánh giá: `RocCurveDisplay`, `PrecisionRecallDisplay`, `DetCurveDisplay`, `ConfusionMatrixDisplay`, `CalibrationDisplay`, `PredictionErrorDisplay`, `LearningCurveDisplay`, `ValidationCurveDisplay`, `PartialDependenceDisplay`. Mỗi lớp có `from_estimator(...)` (tự tính dự đoán) và `from_predictions(...)` (dùng dự đoán có sẵn, nên dùng khi dự đoán tốn kém).

```
import lightgbm as lgb
from sklearn.calibration import CalibrationDisplay
from sklearn.metrics import ConfusionMatrixDisplay, PrecisionRecallDisplay, RocCurveDisplay
from sklearn.pipeline import make_pipeline

from churn.data.split import time_split
from churn.features.build import RAW_FEATURES, build_preprocessor

tr, va, te = time_split(df, "signup_date", "2023-10-01", "2024-01-01")
model = make_pipeline(build_preprocessor(scale=False), lgb.LGBMClassifier(
    n_estimators=300, learning_rate=0.03, num_leaves=15, min_child_samples=50, verbose=-1, random_state=0)
).fit(tr[RAW_FEATURES], tr["churn"])
y_true, proba = te["churn"].to_numpy(), model.predict_proba(te[RAW_FEATURES])[:, 1]

def model_dashboard(y_true: np.ndarray, proba: np.ndarray, threshold: float, title: str) -> plt.Figure:
    fig, axes = plt.subplots(2, 3, figsize=(18, 10))
    fig.suptitle(title, fontsize=16, fontweight="bold")
    RocCurveDisplay.from_predictions(y_true, proba, ax=axes[0, 0], plot_chance_level=True)
    axes[0, 0].set_title("ROC")
    PrecisionRecallDisplay.from_predictions(y_true, proba, ax=axes[0, 1], plot_chance_level=True)
    axes[0, 1].set_title("Precision-Recall")
    ConfusionMatrixDisplay.from_predictions(y_true, (proba >= threshold).astype(int), display_labels=["Ở lại", "Rời bỏ"],
    cmap="Blues", colorbar=False, ax=axes[0, 2])
```

```

axes[0, 2].set_title(f"Confusion matrix @ {threshold:.2f}")
for label, name in [(0, "Ở lại"), (1, "Rời bỏ")]:
    axes[1, 0].hist(proba[y_true == label], bins=40, density=True, alpha=0.55, label=name)
axes[1, 0].axvline(threshold, color="k", ls="--")
axes[1, 0].set(title="Phân phối điểm theo lớp", xlabel="P(churn)")
axes[1, 0].legend(frameon=False)
CalibrationDisplay.from_predictions(y_true, proba, n_bins=10, strategy="quantile", ax=axes[1, 1])
axes[1, 1].set_title("Calibration (reliability)")
order = np.argsort(-proba)
gain = np.cumsum(y_true[order]) / y_true.sum()
frac = np.arange(1, len(y_true) + 1) / len(y_true)
axes[1, 2].plot(frac, gain, label="Mô hình")
axes[1, 2].plot([0, 1], [0, 1], "k--", label="Ngẫu nhiên")
axes[1, 2].set(title="Cumulative gain", xlabel="% khách được liên hệ", ylabel="% churn bắt được")
axes[1, 2].legend(frameon=False)
fig.tight_layout()
return fig

```

```

fig = model_dashboard(y_true, proba, threshold=0.33, title="Churn model – test out-of-time (2024-H1)")
save(fig, "model_dashboard")

```

Biểu đồ phần dư cho hồi quy

```

from sklearn.ensemble import HistGradientBoostingRegressor
from sklearn.metrics import PredictionErrorDisplay

reg = HistGradientBoostingRegressor(random_state=0).fit(tr[["tenure_months", "monthly_charges"]],
tr["total_charges"])
pred = reg.predict(tr[["tenure_months", "monthly_charges"]])
fig, axes = plt.subplots(1, 2, figsize=(12, 4.8))
PredictionErrorDisplay.from_predictions(tr["total_charges"], pred, kind="actual_vs_predicted", subsample=1000,
ax=axes[0], random_state=0)
PredictionErrorDisplay.from_predictions(tr["total_charges"], pred, kind="residual_vs_predicted", subsample=1000,
ax=axes[1], random_state=0)
axes[0].set_title("Thực tế vs dự đoán")
axes[1].set_title("Phần dư vs dự đoán (hình phễu = phương sai không đều)")
plt.tight_layout()

```

11.7. Biểu đồ tương tác với Plotly

```

import plotly.express as px

s = df.sample(3000, random_state=0)
fig_px = px.scatter(s, x="tenure_months", y="monthly_charges", color="churn_label", opacity=0.6,
                    hover_data=["customer_id", "contract", "support_calls"], template="simple_white",
                    color_discrete_map={"Ở lại": PALETTE["neutral"], "Rời bỏ": PALETTE["accent"]},
                    title="Tenure vs cước tháng (di chuột để xem chi tiết)")
fig_px.update_yaxes(range=[0, 200])
fig_px.write_html("reports/figures/scatter_interactive.html", include_plotlyjs="cdn")

seg = (df.groupby(["contract", "payment_method"], observed=True)
        .agg(n=("churn", "size"), churn_rate=("churn", "mean")).reset_index())
fig_sb = px.sunburst(seg, path=["contract", "payment_method"], values="n", color="churn_rate",
                    color_continuous_scale="RdYlGn_r", title="Phân khúc: kích thước = số khách, màu = churn")
print(len(fig_sb.data[0]["ids"]), "nút trong sunburst")

```

11.8. Dashboard ứng dụng với Streamlit

[Docs] Streamlit chạy lại toàn bộ script mỗi khi người dùng tương tác. Vì vậy phải **cache** đúng cách:

`st.cache_resource` cho đối tượng dùng chung không nên copy (mô hình, kết nối DB); `st.cache_data` cho dữ liệu trả về có thể serialize (DataFrame), có `ttl`.

```
# app/dashboard.py - chạy: streamlit run app/dashboard.py
import joblib
import pandas as pd
import plotly.express as px
import streamlit as st

st.set_page_config(page_title="Churn Monitor", layout="wide")

@st.cache_resource
def load_model():
    return joblib.load("models/model.joblib")

@st.cache_data(ttl=3600)
def load_scores() -> pd.DataFrame:
    return pd.read_parquet("data/processed/scoring_latest.parquet")

model, data = load_model(), load_scores()
data["score"] = model.predict_proba(data)[:, 1]

st.title("Customer Churn Monitor")
threshold = st.sidebar.slider("Ngưỡng rủi ro", 0.05, 0.95, 0.33, 0.01)
contracts = st.sidebar.multiselect("Loại hợp đồng", sorted(data["contract"].dropna().unique()))
view = data[data["contract"].isin(contracts)] if contracts else data

c1, c2, c3 = st.columns(3)
c1.metric("Số khách", f"{len(view):,}")
c2.metric("Nguy cơ cao", f"{(view['score'] >= threshold).sum():,}")
c3.metric("Điểm trung bình", f"{view['score'].mean():.1%}")
st.plotly_chart(px.histogram(view, x="score", nbins=50), use_container_width=True)
st.dataframe(view.nlargest(100, "score")[["customer_id", "contract", "tenure_months", "score"]])
st.download_button("Tải danh sách gọi điện (CSV)",
                    view[view["score"] >= threshold].to_csv(index=False).encode("utf-8"), "call_list.csv")
```

Công cụ BI cho người dùng nghiệp vụ: **Power BI, Tableau, Looker, Apache Superset, Metabase**. Data Scientist nên cung cấp **bảng điểm đã tính sẵn** (batch scoring, Chương 12) cho các công cụ này, thay vì để BI gọi mô hình.

11.9. Kể chuyện bằng dữ liệu (Data Storytelling)

[Sách] Cole Nussbaumer Knaflic, *Storytelling with Data* (2015), sáu bài học:

1. **Hiểu bối cảnh:** ai là khán giả, họ cần quyết định gì, bạn muốn họ làm gì.
2. **Chọn hình ảnh phù hợp** (mục 11.2).
3. **Loại bỏ rối mắt (clutter).**
4. **Hướng sự chú ý:** màu nhấn, kích thước, vị trí.
5. **Tư duy như nhà thiết kế:** căn lề, khoảng trắng, phân cấp chữ.
6. **Kể một câu chuyện:** mở đầu (bối cảnh), cao trào (insight), kết thúc (hành động).

Cấu trúc bài trình bày cho lãnh đạo (một trang / năm slide)

Phần	Nội dung	Ví dụ
1. Vấn đề	Bối cảnh + con số tiền	"Mỗi tháng mất 2.1% thuê bao $\approx$ 12 tỷ doanh thu/năm"
2. Phát hiện	3 insight, mỗi insight 1 biểu đồ có tiêu đề là kết luận	"Khách hợp đồng tháng rồi bỏ gấp 3 lần"
3. Giải pháp	Mô hình làm gì, so với cách hiện tại	"Gọi top 10% bắt được 30% khách sắp rời bỏ, gấp 3 lần quy tắc hiện tại"
4. Đề xuất	Ai làm gì, khi nào, đo thế nào	"A/B test 4 tuần, 2 x 5 500 khách, đo tỷ lệ giữ chân 60 ngày"

Phần	Nội dung	Ví dụ
5. Rủi ro & bước tiếp	Hạn chế, kế hoạch giám sát	"Kém chính xác với khách < 3 tháng; giám sát PSI hàng tuần"

Tiêu đề biểu đồ là câu kết luận ("Khách hợp đồng tháng rời bỏ gấp 3 lần"), không phải mô tả ("Churn theo hợp đồng").

Checklist Chương 11

- ☐ Loại biểu đồ phù hợp câu hỏi và thứ bậc nhận thức; không dùng pie nhiều lát, 3D, trục cột không bắt đầu từ 0.
- ☐ So sánh nhóm luôn kèm độ bất định (CI); không đặt màu nhấn vào khác biệt do ngẫu nhiên.
- ☐ Bảng màu đúng loại (qualitative/sequential/diverging), thân thiện người mù màu; font hỗ trợ tiếng Việt.
- ☐ Dùng giao diện OO của matplotlib; phong cách chung qua một module; biểu đồ lưu tự động (PNG + SVG).
- ☐ Dashboard đánh giá mô hình chuẩn (ROC, PR, CM, phân phối điểm, calibration, gain) bằng Display API.
- ☐ Tiêu đề là kết luận; có đơn vị, nguồn, giai đoạn, cỡ mẫu.
- ☐ Bài trình bày đi từ vấn đề → insight → giải pháp → hành động → rủi ro.

Tài liệu tham khảo Chương 11

- matplotlib documentation: *Quick start guide* (Figure/Axes anatomy, OO vs pyplot), *Choosing Colormaps*. matplotlib.org
- seaborn documentation: *Overview of seaborn plotting functions* (axes-level vs figure-level), *Visualizing distributions*, *Choosing color palettes*, *The seaborn.objects interface*. seaborn.pydata.org
- scikit-learn User Guide: *Visualizations* (Display objects). · Plotly Python documentation. · Streamlit documentation: *Caching*.
- Cleveland, W. S. & McGill, R. (1984). *Graphical Perception*. JASA 79(387).
- Tufte, E. R. (2001). *The Visual Display of Quantitative Information*, 2nd ed. Graphics Press.
- Wilkinson, L. (2005). *The Grammar of Graphics*, 2nd ed. Springer.
- Wilke, C. O. (2019). *Fundamentals of Data Visualization*. O'Reilly (clauswilke.com/dataviz).
- Knaflitz, C. N. (2015). *Storytelling with Data*. Wiley.
- Harvard CS109: *Visualization lectures*.

CHƯƠNG 12. MLOps — ĐƯA MÔ HÌNH VÀO PRODUCTION VÀ GIỮ NÓ ĐÚNG

"Only a small fraction of real-world ML systems is composed of the ML code." — Sculley et al., Hidden Technical Debt in Machine Learning Systems (NeurIPS 2015).

Mục tiêu chương: biến mô hình trong notebook thành **hệ thống** tái lập được, kiểm thử được, triển khai an toàn, được giám sát và tự cải thiện. Toàn bộ ví dụ dùng package tham chiếu `code/src/churn`, đã được chạy kiểm chứng.

12.1. Vì sao MLOps: nợ kỹ thuật đặc thù của ML

Sculley et al. (2015) chỉ ra các dạng nợ kỹ thuật **chỉ có ở hệ thống ML**:

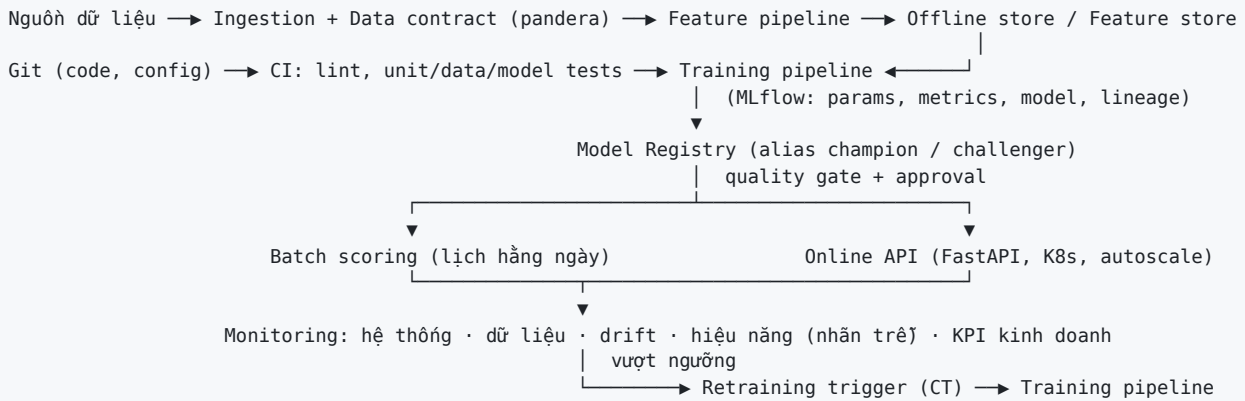
Dạng nợ	Mô tả	Biện pháp
CACE (<i>Changing Anything Changes Everything</i>)	Feature/tham số đan xen, sửa một chỗ ảnh hưởng mọi chỗ	Kiểm thử hành vi, so sánh champion/challenger
Hidden feedback loops	Dự đoán ảnh hưởng dữ liệu tương lai: khách được gọi thì không churn, nên nhãn bị lệch	Nhóm đối chứng (holdout) cố định
Undeclared consumers	Hệ thống khác âm thầm dùng output mô hình	Access control, hợp đồng API, versioning
Data dependency debt	Phụ thuộc dữ liệu không ổn định hoặc không cần thiết	Data contract, loại feature ít giá trị
Glue code / pipeline jungles	Code dán nối, pipeline chấp vá	Package hóa, pipeline khai báo (DAG)
Configuration debt	Cấu hình rải rác, không review	Config có kiểu, trong git, có test
Training-serving skew	Feature tính khác nhau giữa huấn luyện và phục vụ	Dùng chung code/pipeline, feature store

Mức độ trưởng thành MLOps (Google Cloud Architecture Center)

Mức	Đặc điểm	Dấu hiệu
Level 0: Thủ công	Notebook → file model → bàn giao cho kỹ sư; retrain hiếm	Không tái lập; không giám sát; "mô hình chạy được trên máy tôi"
Level 1: Tự động hóa pipeline ML	Pipeline huấn luyện tự động, continuous training (CT) theo lịch/trigger; data & model validation; feature store; metadata	Mô hình mới tự được huấn luyện và kiểm định
Level 2: CI/CD cho pipeline	Thay đổi code pipeline được test, build, deploy tự động	Thử nghiệm nhanh ý tưởng mới một cách an toàn

Mục tiêu thực tế cho phần lớn đội: **Level 1 vững chắc**.

12.2. Kiến trúc tham chiếu



Thiết lập

```

import json
import os
import subprocess
import sys
import time
from pathlib import Path

import numpy as np
import pandas as pd

os.environ["MLFLOW_TRACKING_URI"] = "sqlite:///mlflow.db"
import churn # noqa: E402

CODE_DIR = Path(churn.__file__).resolve().parents[2] # docs/data-science-handbook/code
print("Package churn:", churn.__version__, "| code dir:", CODE_DIR.name)

```

12.3. Kiểm thử hệ thống ML

ML Test Score (Breck et al., 2017, Google)

Rubric gồm **28 kiểm thử** chia 4 nhóm. Điểm của hệ thống bằng điểm **thấp nhất** trong 4 nhóm, vì hệ thống chỉ mạnh bằng mắt xích yếu nhất. Mỗi kiểm thử được 0.5 điểm nếu làm thủ công và 1 điểm nếu tự động.

Nhóm	Ví dụ kiểm thử (trích)
Features & Data	Kỳ vọng phân phối feature được ghi thành schema; mọi feature đều có ích (không thừa); chi phí của feature được đo; tuân thủ yêu cầu meta-level (PII); code tạo feature có unit test
Model Development	Mọi thay đổi mô hình được review và lưu trong VCS; metric offline tương quan với metric online; mọi siêu tham số đã được tune; độ "cũ" của mô hình được biết; mô hình được so với baseline đơn giản ; chất lượng được kiểm tra theo phân khúc
ML Infrastructure	Huấn luyện tái lập được; mô hình được unit test; pipeline được integration test end-to-end; chất lượng được kiểm định trước khi serve ; mô hình có thể debug từng ví dụ; canary trước khi phục vụ toàn bộ; rollback được
Monitoring	Phát hiện thay đổi dependency; dữ liệu đầu vào ổn định; training-serving skew ; mô hình không quá cũ; không có NaN/Inf trong output; không suy giảm hiệu năng tính toán; không suy giảm chất lượng dự đoán

Các tầng kiểm thử trong package tham chiếu

Tầng	File	Kiểm tra gì
Unit	tests/unit/test_transformers.py	Winsorizer học đúng ngưỡng, giữ NaN, qua check_transformer_general của sklearn; làm sạch chuỗi; dedup
Data	tests/data/test_contract.py , test_split.py	Contract bắt đúng lỗi; split theo thời gian không chồng lấn; phát hiện leakage
Model: hành vi	tests/model/test_behavior.py	Invariance (đổi ID không đổi dự đoán), directional (nhiều khiếu nại thì rủi ro tăng), robustness (giá trị lạ/thiếu)
Model: chất lượng	tests/model/test_quality_gate.py	AUC, PR-AUC vượt ngưỡng tối thiểu
API	tests/api/test_api.py	Schema, mã lỗi 422, /ready

```
# tests/model/test_behavior.py (trích) – kiểm thử hành vi theo Checklist (Ribeiro et al., 2020)
def test_invariance_to_customer_id(score):
    assert score(customer_id="C9999999") == pytest.approx(score())

def test_directional_support_calls(score):
    assert score(support_calls=8) > score(support_calls=0)

def test_robust_to_missing_and_unseen(score):
    assert 0.0 <= score(age=None, payment_method="crypto", region="r99") <= 1.0
```

Chạy toàn bộ test của package (CI làm đúng lệnh này):

```
result = subprocess.run([sys.executable, "-m", "pytest", "-q", "-p", "no:cacheprovider"], cwd=CODE_DIR,
                        capture_output=True, text=True, env={**os.environ, "MLFLOW_DISABLE_AGENT_HINT": "1"})
print(result.stdout.strip().splitlines()[-1])
assert result.returncode == 0
```

12.4. Experiment tracking & Model Registry (MLflow 3)

[Docs] Các khái niệm của MLflow 3:

- **Experiment / Run:** một lần huấn luyện, gồm params, metrics, tags, artifacts.
- **LoggedModel:** mô hình là thực thể hạng nhất, có `model_id`, được liên kết với run, metrics và dataset. Nạp bằng `models:/<model_id>`.
- **Registered Model + Version:** phiên bản được quản trị trong **Model Registry**.
- **Alias** (từ MLflow 2.9, thay cho `stages` Staging/Production đã deprecate): nhãn di động như `champion`, `challenger`. Code serving nạp `models:/churn-classifier@champion` mà **không cần biết số version**.
- **Backend:** file store `./mlruns` ở chế độ bảo trì; dùng database (SQLite cho local, PostgreSQL/MySQL cho production) cùng artifact store (S3/GCS/Azure Blob).

```
# Tracking server production: metadata trong PostgreSQL, artifact trong S3
mlflow server --backend-store-uri postgresql://mlflow:***@db:5432/mlflow \
              --artifacts-destination s3://ml-artifacts/mlflow --host 0.0.0.0 --port 5000
```

Quy trình champion/challenger chạy thật

```
import mlflow
from mlflow import MlflowClient

from churn.models.registry import promote_if_better
from churn.models.train import main as train_main
```

```

Path("configs").mkdir(exist_ok=True)
base_cfg = """
target: churn
random_state: 42
split: {date_col: signup_date, train_end: "2023-10-01", valid_end: "2024-01-01", gap_days: 0}
model_params: {n_estimators: %d, learning_rate: 0.03, num_leaves: %d, min_child_samples: 50,
               subsample: 0.8, subsample_freq: 1, colsample_bytree: 0.8, reg_lambda: 1.0}
"""
client = MlflowClient()
MODEL = "churn-classifier"
history = []
for version_cfg in [(60, 3), (400, 15), (400, 63)]: # 3 lần huấn luyện với cấu hình khác nhau
    Path("configs/train.yaml").write_text(base_cfg % version_cfg)
    metrics = train_main("configs/train.yaml", out_dir=".")
    run = mlflow.search_runs(experiment_names=["churn-prediction"], order_by=["start_time DESC"],
                           max_results=1).iloc[0]
    model_id = mlflow.search_logged_models(experiment_ids=[run["experiment_id"]], order_by=[
        {"field_name": "creation_timestamp", "ascending": False}], max_results=1, output_format="list")[0].model_id
    mv = mlflow.register_model(f"models://{model_id}", MODEL)
    promoted = promote_if_better(MODEL, mv.version, metric="valid_pr_auc", min_gain=0.005, client=client)
    history.append({"version": mv.version, "cfg(n_estimators,num_leaves)": version_cfg,
                  "valid_pr_auc": round(metrics["valid_pr_auc"], 4), "promoted": promoted})
print(pd.DataFrame(history).to_string(index=False))

champion = client.get_model_version_by_alias(MODEL, "champion")
print("Champion hiện tại: version", champion.version)
served = mlflow.sklearn.load_model(f"models://{MODEL}@{champion}") # serving chỉ cần alias
print(type(served).__name__, "đã nạp từ registry")

```

`promote_if_better` (trong `churn/models/registry.py`) chỉ gán alias `champion` khi phiên bản mới **vượt champion hiện tại ít nhất `min_gain`**. Nếu không, nó gán `challenger` để tiếp tục chạy shadow/A/B. Trong tổ chức lớn, bước `promote` cần thêm **phê duyệt thủ công** và ghi lại lý do (audit trail).

12.5. Serving: batch, online, streaming

Kiểu	Độ trễ	Khi nào	Công nghệ
Batch	Giờ/ngày	Danh sách gọi điện hàng ngày, báo cáo, BI	Airflow/Prefect + pandas/Spark, ghi ra bảng
Online (request/response)	ms	Chấm điểm khi khách thao tác, chống gian lận	FastAPI, BentoML, KServe, Triton, Seldon
Streaming	giây	Sự kiện liên tục	Kafka + Flink/Spark Structured Streaming
Edge / on-device	ms, offline	Mobile, IoT	ONNX Runtime, TFLite, Core ML

[Kinh nghiệm] Nếu nghiệp vụ chấp nhận được độ trễ, **ưu tiên batch**. Batch đơn giản hơn nhiều: không cần SLA độ trễ, dễ giám sát, dễ rollback, và có thể dùng mô hình nặng.

Batch scoring

```

import joblib

from churn.data.synthetic import make_churn_data
from churn.data.validate import validate
from churn.features.build import RAW_FEATURES
from churn.features.clean import clean_churn

def batch_score(raw: pd.DataFrame, model, threshold: float, model_ref: str) -> pd.DataFrame:
    """Làm sạch → kiểm tra contract → chấm điểm → output có metadata để audit và giám sát."""
    clean = clean_churn(raw)
    validate(clean.assign(churn=clean.get("churn", 0))) # contract chạy cả lúc chấm điểm
    out = clean[["customer_id"]].copy()

```

```

out["score"] = model.predict_proba(clean[RAW_FEATURES])[:, 1]
assert np.isfinite(out["score"]).all(), "Output có NaN/Inf" # ML Test Score: monitoring
out["is_high_risk"] = out["score"] >= threshold
out["score_date"] = pd.Timestamp.today().normalize()
out["model_ref"] = model_ref
return out

scores = batch_score(make_churn_data(n=5_000, seed=7), served, threshold=0.33,
                      model_ref=f"{MODEL}@champion(v{champion.version})")
scores.to_parquet("scores_latest.parquet", index=False)
print(scores.head(3))
print("Tỷ lệ khách nguy cơ cao:", round(scores["is_high_risk"].mean(), 3))

```

Online serving với FastAPI

Mã nguồn: `code/src/churn/serving/api.py`. Các điểm production:

- **Nạp mô hình một lần** trong `lifespan` (không nạp mỗi request).
- **Validate input** bằng **pydantic v2**: kiểu, miền giá trị, `Literal` cho phân loại. Input sai trả về **422** tự động.
- `/health (liveness)` và `/ready (readiness)` riêng biệt cho Kubernetes.
- Endpoint `def` (không phải `async def`) cho suy luận CPU-bound. **[Docs]** FastAPI chạy hàm `def` trong `threadpool`, nhờ đó không chặn event loop.
- Không trả stack trace cho client; log có `request_id`; không log PII.
- Cấu hình (đường dẫn mô hình, ngưỡng, version) đọc từ **biến môi trường**.

```

import joblib
from fastapi.testclient import TestClient

joblib.dump(served, "model.joblib")
os.environ.update({"MODEL_PATH": "model.joblib", "MODEL_VERSION": f"v{champion.version}"})
importlib

import churn.serving.api as api

importlib.reload(api) # đọc lại biến môi trường

payload = [{"customer_id": "C0000001", "signup_date": "2024-03-01T00:00:00", "age": 29,
            "tenure_months": 2, "monthly_charges": 95.0, "total_charges": 190.0, "contract": "month-to-month",
            "payment_method": "cash", "region": "r03", "support_calls": 5, "data_usage_gb": 4.2}]
with TestClient(api.app) as client_api: # chạy lifespan: nạp mô hình
    print(client_api.get("/ready").json())
    r = client_api.post("/predict", json=payload)
    print(r.status_code, r.json()[0])
    bad = client_api.post("/predict", json=[**payload[0], "contract": "lifetime"])
    print("Input sai ->", bad.status_code, bad.json()["detail"][0]["msg"])
    t0 = time.perf_counter()
    for _ in range(50):
        client_api.post("/predict", json=payload * 20)
    print(f"Độ trễ trung bình (batch 20 khách, in-process): {(time.perf_counter() - t0) / 50 * 1000:.1f} ms")

```

12.6. Đóng gói: Docker và Kubernetes

Dockerfile (trong package tham chiếu)

```

# syntax=docker/dockerfile:1
FROM python:3.12-slim AS builder
WORKDIR /build
COPY pyproject.toml ./
COPY src/ src/
RUN pip install --no-cache-dir --prefix=/install .

FROM python:3.12-slim AS runtime
RUN apt-get update && apt-get install -y --no-install-recommends libgomp1 \
    && rm -rf /var/lib/apt/lists/* \

```

```

&& useradd --create-home --uid 10001 app          # không chạy bằng root
COPY --from=builder /install /usr/local
WORKDIR /app
COPY models/model.joblib models/model.joblib
ENV MODEL_PATH=/app/models/model.joblib PYTHONUNBUFFERED=1
USER app
EXPOSE 8000
HEALTHCHECK --interval=30s --timeout=3s --retries=3 \
  CMD python -c "import urllib.request; urllib.request.urlopen('http://localhost:8000/health')"
CMD ["uvicorn", "churn.serving.api:app", "--host", "0.0.0.0", "--port", "8000", "--workers", "2"]

```

Thực hành tốt	Lý do
Multi-stage build, image <code>slim</code>	Image nhỏ, ít bề mặt tấn công
Ghim phiên bản base image và lock dependency	Tái lập; tránh thay đổi âm thầm
User không phải root (<code>USER app</code>)	Bảo mật
<code>libgomp1</code>	OpenMP runtime cho LightGBM/XGBoost
Không "nướng" secret vào image	Secret qua env/secret manager lúc chạy
Cẩn nhắc không nướng model vào image	Tách vòng đời model và code: nạp từ registry khi khởi động
Quét lỗ hổng (<code>trivy</code> , <code>pip-audit</code>), SBOM	Chuỗi cung ứng phần mềm

Kubernetes: Deployment, probes, resources, autoscaling

```

apiVersion: apps/v1
kind: Deployment
metadata: {name: churn-api, labels: {app: churn-api}}
spec:
  replicas: 3
  selector: {matchLabels: {app: churn-api}}
  strategy: {type: RollingUpdate, rollingUpdate: {maxUnavailable: 0, maxSurge: 1}}
  template:
    metadata: {labels: {app: churn-api}}
    spec:
      containers:
        - name: api
          image: ghcr.io/org/churn-api:1f2e3d4          # tag bất biến (git SHA), không dùng :latest
          ports: [{containerPort: 8000}]
          env:
            - {name: MODEL_VERSION, value: "v12"}
            - {name: DECISION_THRESHOLD, value: "0.33"}
            - {name: OMP_NUM_THREADS, value: "1"}      # khớp limits.cpu (xem Chương 8.11)
          resources:
            requests: {cpu: "500m", memory: "512Mi"}
            limits: {cpu: "1", memory: "1Gi"}
          readinessProbe: {httpGet: {path: /ready, port: 8000}, initialDelaySeconds: 5, periodSeconds: 10}
          livenessProbe: {httpGet: {path: /health, port: 8000}, initialDelaySeconds: 15, periodSeconds: 20}
      ---
apiVersion: autoscaling/v2
kind: HorizontalPodAutoscaler
metadata: {name: churn-api}
spec:
  scaleTargetRef: {apiVersion: apps/v1, kind: Deployment, name: churn-api}
  minReplicas: 3
  maxReplicas: 20
  metrics: [{type: Resource, resource: {name: cpu, target: {type: Utilization, averageUtilization: 65}}}]

```

Bài học từ Chương 8.11: trong container, số luồng OpenMP mặc định bằng số vCPU **của node**, không phải `limits.cpu` của pod. Không đặt `OMP_NUM_THREADS` sẽ gây oversubscription và độ trễ tăng hàng chục lần.

12.7. CI/CD/CT với GitHub Actions

```

name: churn-ci
on:
  pull_request:
  push: {branches: [main]}
jobs:
  test:
    # CI: mọi PR
    runs-on: ubuntu-latest
    defaults: {run: {working-directory: docs/data-science-handbook/code}}
    steps:
      - uses: actions/checkout@v4
      - uses: actions/setup-python@v5
        with: {python-version: "3.12", cache: pip}
      - run: pip install -e ".[dev]"
      - run: ruff check src tests
      - run: pytest -q
        # unit + data + behaviour + API
  train-and-gate:
    # CT: huấn luyện + quality gate
    needs: test
    if: github.ref == 'refs/heads/main'
    runs-on: ubuntu-latest
    env:
      MLFLOW_TRACKING_URI: ${ secrets.MLFLOW_TRACKING_URI }
    steps:
      - uses: actions/checkout@v4
      - uses: actions/setup-python@v5
        with: {python-version: "3.12", cache: pip}
      - run: pip install -e "docs/data-science-handbook/code[dev]"
      - run: python -m churn.models.train --config docs/data-science-handbook/code/configs/train.yaml
      - run: python -m churn.models.promote --model churn-classifier --metric valid_pr_auc --min-gain 0.005
  build-and-deploy:
    # CD: chỉ khi có champion mới
    needs: train-and-gate
    runs-on: ubuntu-latest
    permissions: {contents: read, packages: write, id-token: write}
    steps:
      - uses: actions/checkout@v4
      - uses: docker/build-push-action@v6
        with: {context: docs/data-science-handbook/code, push: true,
          tags: "ghcr.io/${ github.repository }/churn-api:${ github.sha }}"
      - run: kubectl set image deployment/churn-api api=ghcr.io/${ github.repository }/churn-api:${ github.sha }

```

`churn.models.promote` là CLI bọc quanh `promote_if_better`. Package tham chiếu cung cấp hàm, còn CLI là phần bài tập mở rộng.

12.8. Chiến lược triển khai an toàn

Chiến lược	Cách làm	Ưu	Nhược
Shadow	Mô hình mới nhận traffic thật, kết quả không được dùng; so với champion	Không rủi ro cho người dùng	Không đo được tác động kinh doanh; tốn tài nguyên
Canary	1% → 5% → 25% → 100% traffic, dừng nếu metric xấu	Giới hạn thiệt hại	Cần định tuyến và giám sát tốt
Blue/Green	Hai môi trường song song, chuyển traffic tức thì	Rollback tức thì	Gấp đôi tài nguyên
A/B test	Ngẫu nhiên hóa người dùng, đo OEC (Chương 4.6)	Đo được tác động nhân quả	Cần thời gian và cỡ mẫu
Interleaving	Trộn kết quả hai mô hình trong cùng một danh sách	Nhạy hơn A/B cho ranking	Chỉ cho xếp hạng
Multi-armed bandit	Tự dồn traffic cho mô hình tốt hơn	Ít chi phí cơ hội	Khó phân tích thống kê

```
def shadow_compare(champion, challenger, X: pd.DataFrame, threshold: float) -> dict:
    """Shadow mode: champion phục vụ, challenger chỉ được log; so sánh mức độ bất đồng."""
    p_c, p_n = champion.predict_proba(X)[:, 1], challenger.predict_proba(X)[:, 1]
    flip = (p_c >= threshold) != (p_n >= threshold)
    return {"rank_corr": float(pd.Series(p_c).corr(pd.Series(p_n), method="spearman")),
            "decision_flip_rate": float(flip.mean()), "mean_score_shift": float(p_n.mean() - p_c.mean())}

challenger_v = client.get_model_version(MODEL, history[-1]["version"])
challenger = mlflow.sklearn.load_model(f"models:{MODEL}/{challenger_v.version}")
print(shadow_compare(served, challenger, clean_churn(make_churn_data(n=3_000, seed=11))(RAW_FEATURES), 0.33))
```

Luôn có kế hoạch rollback (đổi alias `champion` về version cũ) và **fallback** (quy tắc baseline) khi mô hình lỗi hoặc timeout.

12.9. Giám sát (Monitoring)

Các dạng suy giảm

Dạng	Định nghĩa	Ví dụ	Phát hiện
Data drift (covariate shift)	$P(X)$ thay đổi	Chiến dịch thu hút khách trẻ hơn	PSI, KS, Wasserstein, Jensen-Shannon
Prior / label shift	$P(Y)$ thay đổi	Đối thủ giảm giá, churn tăng	Tỷ lệ dự đoán dương; tỷ lệ nhãn khi về
Concept drift	$P(Y X)$ thay đổi	Chính sách mới đổi quan hệ giá-churn	Metric hiệu năng khi có nhãn
Data quality	Pipeline lỗi	Cột null toàn bộ; đổi đơn vị	Contract, null rate, volume
Training-serving skew	Feature tính khác nhau	Logic SQL ở serving khác pandas ở training	So phân phối feature log ở serving với training

Population Stability Index (PSI) và các khoảng cách phân phối

$$PSI = \sum_{i=1}^B (A_i - E_i) \ln \frac{A_i}{E_i}$$

PSI	Diễn giải (quy ước ngành tín dụng)
< 0.1	Ổn định
0.1 - 0.25	Thay đổi vừa: theo dõi
> 0.25	Thay đổi lớn: điều tra, cân nhắc retrain

```
from churn.monitoring.drift import drift_report, psi

reference = clean_churn(make_churn_data(n=10_000, seed=1))
current = clean_churn(make_churn_data(n=5_000, seed=2))
# Mô phỏng sự cố: (1) chiến dịch thu hút khách trẻ; (2) nguồn dữ liệu đổi đơn vị cước; (3) kênh thanh toán mới
current["age"] = (current["age"] - 8).clip(lower=18)
current.loc[current.sample(frac=0.3, random_state=0).index, "monthly_charges"] *= 1000
current.loc[current.sample(frac=0.2, random_state=1).index, "payment_method"] = "qr-pay"

report = drift_report(reference, current,
                     num_cols=["age", "tenure_months", "monthly_charges", "support_calls", "data_usage_gb"],
                     cat_cols=["contract", "payment_method", "region"])
print(report.round(3).to_string(index=False))

score_psi = psi(served.predict_proba(reference[RAW_FEATURES])[:, 1], served.predict_proba(current[RAW_FEATURES])[:, 1])
print(f"PSI của điểm dự đoán = {score_psi:.3f}")
```

Đọc kết quả, một bài học quan trọng. Input trôi rất mạnh: `monthly_charges` bị nhân 1 000 ở 30% bản ghi, xuất hiện kênh thanh toán mới, khách trẻ hơn. Vậy mà **PSI của điểm dự đoán chỉ ≈ 0.02** . Lý do là Winsorizer trong pipeline cắt cước về phân vị 99% của train, còn mức `qr-pay` lạ được xử lý như "infrequent/unknown". Mô hình **bền vững**, nhưng chính sự bền vững đó **che giấu sự cố dữ liệu** (sai đơn vị ở nguồn billing). Vì vậy phải giám sát **cả input lẫn output**: chỉ nhìn phân phối điểm sẽ bỏ sót sự cố này.

Với dữ liệu lớn, KS test gần như luôn cho $p \approx 0$ kể cả khi khác biệt không đáng kể. Hãy dựa vào **độ lớn khác biệt** (PSI, KS statistic, Wasserstein chuẩn hóa) và **ngưỡng nghiệp vụ**, không dựa vào p-value.

Evidently: báo cáo drift tự động

[Docs] Evidently ≥ 0.7 dùng API mới: `Report([...presets]).run(current_data, reference_data)` trả về **Snapshot**, có thể `save_html`, `dict()` / `json()`, hoặc đẩy lên Evidently UI/Cloud. Các preset chính: `DataDriftPreset`, `DataSummaryPreset`, `ClassificationPreset`, `RegressionPreset`. Mặc định Evidently tự chọn phép đo theo kiểu cột và cỡ mẫu: K-S/ χ^2 cho mẫu nhỏ, **Wasserstein (chuẩn hóa)** / **Jensen-Shannon** cho mẫu lớn.

```
from evidently import Report
from evidently.preset import DataDriftPreset

cols = ["age", "tenure_months", "monthly_charges", "support_calls", "contract", "payment_method"]
snapshot = Report([DataDriftPreset()]).run(current_data=current[cols], reference_data=reference[cols])
snapshot.save_html("drift_report.html")
drift_metrics = [m for m in snapshot.dict()["metrics"] if m["metric_name"].startswith("ValueDrift")]
for m in drift_metrics:
    print(f"{m['metric_name'][:75]:75s} value={float(m['value']):.3f}")
```

Hiệu năng khi nhãn đến trễ

Nhãn churn chỉ có sau 30–37 ngày. Trong khoảng thời gian đó:

1. Giám sát **proxy**: drift input, drift điểm, tỷ lệ dự đoán dương, calibration theo cohort cũ đã có nhãn.
2. **Ước lượng hiệu năng không cần nhãn**: CBPE (*Confidence-Based Performance Estimation*, thư viện NannyML) dùng xác suất **đã calibrate** để ước lượng confusion matrix kỳ vọng. Phương pháp này chỉ đúng khi không có concept drift.
3. Khi nhãn về: tính metric thật **theo cohort chấm điểm**, so với lúc kiểm định.

```
def expected_confusion(proba: np.ndarray, threshold: float) -> dict:
    """Ước lượng TP/FP/FN/TN kỳ vọng từ xác suất đã calibrate (ý tưởng của CBPE)."""
    act = proba >= threshold
    return {"E[TP]": float(proba[act].sum()), "E[FP]": float((1 - proba[act]).sum()),
            "E[FN]": float(proba[~act].sum()), "E[TN]": float((1 - proba[~act]).sum())}

p_cur = served.predict_proba(clean_churn(make_churn_data(n=5_000, seed=21))[RAW_FEATURES][:, 1])
ec = expected_confusion(p_cur, 0.33)
print({k: round(v) for k, v in ec.items()},
      "| precision ước lượng =", round(ec["E[TP]"] / (ec["E[TP]"] + ec["E[FP]"]), 3))
```

Bảng giám sát đề xuất

Nhóm	Metric	Ngưỡng cảnh báo ví dụ	Tần suất
Hệ thống	p95/p99 latency, error rate, throughput, CPU/RAM	p95 > 200 ms; lỗi > 1%	Real-time (Prometheus/Grafana)
Dữ liệu	Null rate, schema, volume, category mới, freshness	Null tăng > 5 điểm %; volume $\pm 30\%$	Mỗi batch
Drift input	PSI/Wasserstein các feature quan trọng	PSI > 0.25	Hàng ngày/tuần

Nhóm	Metric	Ngưỡng cảnh báo ví dụ	Tần suất
Drift output	PSI điểm, tỷ lệ dự đoán dương	PSI > 0.1	Hàng ngày
Hiệu năng	PR-AUC, recall, calibration theo cohort khi có nhãn	Giảm > 10% so với kiểm định	Khi nhãn về
Kinh doanh	Tỷ lệ giữ chân nhóm được gọi so với holdout	Theo OEC	Hàng tháng

Mỗi cảnh báo phải có **người nhận (on-call)** và **runbook**: kiểm tra gì, ai quyết định rollback hay retrain.

12.10. Retraining và orchestration

Chiến lược	Mô tả	Phù hợp
Định kỳ	Hàng tuần/tháng	Đơn giản, dễ vận hành. Mặc định tốt
Theo trigger	Khi drift/hiệu năng vượt ngưỡng	Môi trường biến động
Online / incremental	Cập nhật liên tục	Gợi ý, quảng cáo, streaming

Retrain **không bao giờ deploy mù quáng**: validate dữ liệu → train → đánh giá out-of-time → **gate** so với champion → shadow/canary → promote (có phê duyệt).

```
# flows/retrain.py – Prefect (2.x/3.x): task có retry, flow có lịch
from prefect import flow, get_run_logger, task

@task(retries=3, retry_delay_seconds=60)
def ingest(snapshot: str) -> str:
    return f"data/raw/churn_{snapshot}.parquet"

@task
def validate_data(path: str) -> str:
    from churn.data.validate import validate
    validate(pd.read_parquet(path))
    return path

@task
def train(path: str) -> str:
    from churn.models.train import main
    main("configs/train.yaml")
    return latest_model_version()      # truy vấn registry

@task
def gate(version: str) -> bool:
    from churn.models.registry import promote_if_better
    return promote_if_better("churn-classifier", version)

@flow(name="churn-retrain")
def retrain_flow(snapshot: str = "latest"):
    promoted = gate(train(validate_data(ingest(snapshot))))
    get_run_logger().info("promoted=%s", promoted)

if __name__ == "__main__":
    retrain_flow.serve(name="monthly", cron="0 2 1 * *")
```

```
# dags/churn_retrain.py – Airflow (TaskFlow API)
import pendulum
from airflow.decorators import dag, task
```

```
@dag(schedule="0 2 1 * *", start_date=pendulum.datetime(2026, 1, 1, tz="Asia/Ho_Chi_Minh"),
      catchup=False, default_args={"retries": 2}, tags=["ml", "churn"])
def churn_retrain():
    @task
    def ingest() -> str: ...

    @task
    def validate(path: str) -> str: ...

    @task
    def train(path: str) -> str: ...

    @task
    def gate(version: str) -> None: ...

    gate(train(validate(ingest())))

churn_retrain()
```

12.11. Feature store: chống training-serving skew

[Docs] Feast tách **offline store** (dữ liệu lịch sử dùng để tạo tập huấn luyện bằng **point-in-time join**) khỏi **online store** (giá trị mới nhất, độ trễ thấp cho serving). Hai store dùng **cùng một định nghĩa feature**.

```
# feature_repo/features.py - Feast
from datetime import timedelta

from feast import Entity, FeatureView, Field, FileSource
from feast.types import Float32, Int64

customer = Entity(name="customer", join_keys=["customer_id"])
usage_source = FileSource(path="data/processed/usage_features.parquet", timestamp_field="event_timestamp")
usage_fv = FeatureView(
    name="customer_usage", entities=[customer], ttl=timedelta(days=90),
    schema=[Field(name="support_calls_90d", dtype=Int64), Field(name="avg_bill_6m", dtype=Float32)],
    source=usage_source,
)
# Huấn luyện: store.get_historical_features(entity_df=labels_with_snapshot_ts, features=[...]) -> point-in-time
# Serving : store.get_online_features(features=[...], entity_rows=[{"customer_id": "C0000001"}])
```

12.12. Tối ưu suy luận

Kỹ thuật	Lợi ích	Công cụ
Batch inference	Vector hóa, ít overhead	Gom request (micro-batching)
Chuyển định dạng	Nhanh 2-10 lần, không phụ thuộc Python	ONNX (skl2onnx , onnxmltools), Treelite (cây)
Lượng tử hóa / distillation	Mô hình nhỏ hơn	ONNX Runtime quantization; train mô hình nhỏ bắt chước mô hình lớn
Cache	Bỏ qua tính lại	Redis theo khóa feature hash
Tính trước	Độ trễ ≈ 0	Batch scoring + tra cứu

12.13. Bảo mật, quyền riêng tư, quản trị và chi phí

- **Secret:** biến môi trường hoặc secret manager (Vault, AWS Secrets Manager, GCP Secret Manager). **Không bao giờ** commit vào git.

- **Dữ liệu cá nhân:** giả danh/ẩn danh trước khi vào pipeline (Chương 2.12); phân quyền theo vai trò; tuân thủ **ND 13/2023/ND-CP**.
- **Lineage:** model version ← run ← data version (DVC hash) ← git commit ← config. MLflow lưu tag `git_commit`, `data_path`.
- **Audit log:** mọi dự đoán quan trọng gồm input hash, output, model version, thời gian.
- **Chuỗi cung ứng:** lock file; `pip-audit`; quét image (`trivy`); chỉ nạp model pickle/cloudpickle từ registry đáng tin. MLflow 3 mặc định dùng **skops** cho scikit-learn chính vì lý do này.
- **Quản trị rủi ro AI:** phân loại rủi ro (EU AI Act: unacceptable / high / limited / minimal), model card, đánh giá fairness định kỳ.
- **Chi phí:** theo dõi chi phí huấn luyện/serving trên mỗi dự đoán; autoscale về mức tối thiểu ngoài giờ.

Checklist Chương 12

- ☐ Mọi thí nghiệm được log: params, metrics, artifact, data version, git commit.
- ☐ Registry dùng alias champion/challenger; promote qua quality gate tự động và có phê duyệt.
- ☐ Có unit, data, behaviour, quality-gate và API tests trong CI; tự chấm theo ML Test Score.
- ☐ API validate input (pydantic), có `/health` + `/ready`, log có request_id, không lộ stack trace/PII.
- ☐ Docker image non-root, tag bất biến; Kubernetes có probes, resources, `OMP_NUM_THREADS` khớp `limits.cpu`.
- ☐ Triển khai qua shadow/canary; rollback = đổi alias; có fallback.
- ☐ Giám sát hệ thống, dữ liệu, drift, hiệu năng (cả khi nhàn rỗi), KPI kinh doanh; mỗi cảnh báo có runbook.
- ☐ Retraining có lịch/trigger và gate; feature dùng chung định nghĩa giữa training và serving.

Tài liệu tham khảo Chương 12

- Sculley, D. et al. (2015). *Hidden Technical Debt in Machine Learning Systems*. NeurIPS.
- Breck, E. et al. (2017). *The ML Test Score: A Rubric for ML Production Readiness and Technical Debt Reduction*. IEEE Big Data.
- Google Cloud Architecture Center. *MLOps: Continuous delivery and automation pipelines in machine learning*.
- MLflow 3 documentation: *Tracking, LoggedModel, Model Registry (aliases), Self-hosting / backend stores*. mlflow.org/docs
- FastAPI documentation: *Concurrency and async/await, Lifespan Events, Testing, Deployment (Docker)*. fastapi.tiangolo.com
- Docker documentation: *Building best practices*. · Kubernetes documentation: *Configure Liveness, Readiness and Startup Probes; Horizontal Pod Autoscaling*.
- Evidently documentation (≥ 0.7): *Reports, Presets, Data drift methods*. docs.evidentlyai.com · NannyML documentation: *CBPE*.
- Feast documentation: *Concepts (offline/online store, point-in-time joins)*. docs.feast.dev
- Prefect / Apache Airflow documentation (TaskFlow API).
- Ribeiro, M. T. et al. (2020). *Beyond Accuracy: Behavioral Testing of NLP Models with CheckList*. ACL.
- Huyen, C. (2022). *Designing Machine Learning Systems*, ch. 7–9. O'Reilly. · Mohandas, G. *Made With ML* (Testing, Serving, CI/CD, Monitoring). · DeepLearning.AI *MLOps Specialization* (Deployment & Monitoring). · Full Stack Deep Learning.

CHƯƠNG 13. CHECKLIST PRODUCTION, ANTI-PATTERNS & QUY TRÌNH REVIEW

Mục tiêu chương: cô đọng toàn bộ handbook thành các công cụ dùng hằng ngày: checklist go-live, danh mục anti-pattern (kèm chương tham chiếu), bộ câu hỏi review mô hình, và mẫu postmortem khi sự cố xảy ra.

13.1. Checklist trước khi Go-Live

A. Bài toán & dữ liệu (Chương 1-3)

- ☐ Mục tiêu kinh doanh, hành động sau dự đoán và người chịu trách nhiệm hành động được stakeholder ký duyệt.
- ☐ Đã cân nhắc giải pháp không dùng ML; có baseline heuristic để so sánh.
- ☐ Nhãn có cửa sổ quan sát, gap, label window; loại thực thể không còn hợp lệ tại thời điểm dự đoán.
- ☐ Mọi feature đảm bảo **point-in-time**; không dùng bảng bị ghi đè để tính feature quá khứ.
- ☐ Data contract tự động tại ingestion và trước khi chấm điểm; có kiểm tra volume/freshness.
- ☐ Dữ liệu thô bất biến, có phiên bản (DVC/snapshot); PII được giả danh/ẩn danh theo ND 13/2023/ND-CP.

B. Mô hình (Chương 4-10)

- ☐ Validation mô phỏng production (out-of-time / group); có kiểm tra leakage tự động.
- ☐ Mọi bước stateful nằm trong Pipeline; target encoding có cross-fitting; resampling nằm trong `imblearn.pipeline`.
- ☐ Vượt baseline **có ý nghĩa thống kê** (CI của chênh lệch không chứa 0); báo cáo test out-of-time có CI.
- ☐ Ngưỡng được chọn theo chi phí/năng lực vận hành trên validation/OOF, đóng gói bằng `FixedThresholdClassifier`.
- ☐ Xác suất đã kiểm tra calibration (calibrate lại nếu dùng trực tiếp).
- ☐ Slice analysis và fairness theo nhóm đã được review; có Model Card.
- ☐ Giải thích mô hình hợp lý về nghiệp vụ (permutation/SHAP trên held-out); behaviour tests đạt.

C. Kỹ thuật (Chương 0, 8, 12)

- ☐ Huấn luyện là **một lệnh** tái lập được từ (git commit, data version, config, seed); log vào MLflow.
- ☐ Unit, data, behaviour, quality-gate, API tests chạy trong CI; tự chấm **ML Test Score**.
- ☐ API: validate input, xử lý giá trị thiếu/lạ, `/health + /ready`, timeout, log có request_id, không lộ PII.
- ☐ Load test đạt SLA (p95/p99) với cấu hình CPU thật; số luồng khớp CPU được cấp.
- ☐ Image non-root, tag bất biến, phụ thuộc đã quét lỗ hổng; secret không nằm trong code/image.
- ☐ Dùng API hiện hành của thư viện (không còn cảnh báo deprecation).

D. Vận hành (Chương 12)

- ☐ Dashboard giám sát: hệ thống, dữ liệu, drift input/output, hiệu năng theo cohort, KPI kinh doanh.
- ☐ Mỗi cảnh báo có người nhận (on-call) và runbook.
- ☐ Rollback (đổi alias) và fallback (baseline) đã được **diễn tập**.
- ☐ Retraining có lịch/trigger và gate; triển khai qua shadow/canary; có nhóm holdout chống feedback loop.
- ☐ Kế hoạch đo tác động thật (A/B test với OEC và guardrail metrics).

13.2. Tự chấm ML Test Score

```
import pandas as pd

ml_test_score = {
    "Features & Data": {"schema kỳ vọng feature": 1, "mọi feature có ích": 0.5, "chi phí feature được đo": 0,
                        "tuân thủ PII": 1, "code feature có unit test": 1, "pipeline dữ liệu có test": 1,
                        "kiểm tra quyền riêng tư": 0.5},
    "Model Development": {"review & VCS mô hình": 1, "offline ~ online metric": 0, "siêu tham số đã tune": 1,
                           "độ cũ mô hình được biết": 0.5, "so với baseline đơn giản": 1,
                           "chất lượng theo phân khúc": 1, "kiểm tra fairness/inclusion": 0.5},
    "ML Infrastructure": {"huấn luyện tái lập": 1, "mô hình có unit test": 1, "integration test pipeline": 0.5,
                          "kiểm định trước khi serve": 1, "debug từng ví dụ": 0.5, "canary": 0, "rollback": 0.5},
    "Monitoring": {"thay đổi dependency": 0.5, "dữ liệu đầu vào ổn định": 1, "training-serving skew": 0.5,
                    "mô hình không quá cũ": 0.5, "không NaN/Inf output": 1, "hiệu năng tính toán": 1,
                    "chất lượng dự đoán": 0.5},
}

scores = pd.Series({section: sum(items.values()) for section, items in ml_test_score.items()})
print(scores)
print("ML Test Score (min của 4 nhóm) =", scores.min())
# Breck et al. (2017): 0 = nghiên cứu; (0,1] = sơ khai; (1,2] = cơ bản; (2,3] = khá;
# (3,5] = đủ cho hệ thống quan trọng; > 5 = xuất sắc
```

13.3. Danh mục anti-pattern

#	Anti-pattern	Hậu quả	Cách đúng	Chương
1	Fit scaler/imputer/encoder/feature selection trên toàn bộ dữ liệu	Metric ảo	Pipeline + CV	5, 7
2	Random split cho dữ liệu có thời gian	Hiệu năng production thấp hơn nhiều	Out-of-time, TimeSeriesSplit	7
3	SMOTE/oversample trước khi chia	Metric ảo nghiêm trọng	imblearn.pipeline	5
4	Target encoding bằng <code>fit(X).transform(X)</code>	Mô hình tin vào biến nhiều	TargetEncoder.fit_transform	5
5	Accuracy làm metric chính khi mất cân bằng	Mô hình "luôn đoán 0" trông tốt	PR-AUC, recall, MCC, lợi nhuận	9
6	Ngưỡng mặc định 0.5	Sai lệch chi phí	Threshold tuning theo chi phí	1, 9
7	Tune trên tập test	Ước lượng lạc quan	Validation / nested CV	7
8	Feature sinh ra sau sự kiện target	Leakage, mô hình vô dụng khi deploy	Kiểm tra thời điểm sinh feature	3, 7
9	Cùng thực thể ở train và test	Mô hình "nhớ" thực thể	Group split	7
10	Feature "số ngày kể từ một ngày cố định"	Giá trị lệch/âm trên dữ liệu tương lai	Đo tới thời điểm chấm điểm của từng dòng	5
11	Báo cáo một con số không có CI	Quyết định dựa trên nhiều	Bootstrap CI, kiểm định ghép cặp	4, 9
12	So sánh mô hình bằng t-test thường trên các fold	p-value quá nhỏ	Corrected resampled t-test	9
13	Tin MDI feature importance	Kết luận sai, thiên vị biến liên tục	Permutation/SHAP trên held-out	10
14	Diễn giải SHAP/hệ số như nhân quả	Hành động kinh doanh sai	A/B test, suy luận nhân quả	4, 10

#	Anti-pattern	Hậu quả	Cách đúng	Chương
15	Dùng xác suất sau resampling/class weight như xác suất thật	Lợi nhuận kỳ vọng sai	Calibration	5, 9
16	Biến đổi log target rồi báo cáo tổng/mean	Ước lượng thấp có hệ thống	Smearing; loss Poisson/Gamma	6
17	Notebook là "production code"	Không tái lập, không test	Package + test + CI	0, 12
18	Viết lại logic feature khi serving	Training-serving skew	Dùng chung pipeline; feature store	12
19	Song song lồng nhau ($n_{\text{jobs}} = -1 \times$ mô hình đa luồng) trong container	Chậm hàng chục, hàng trăm lần	Song song một cấp; <code>OMP_NUM_THREADS = CPU được cấp</code>	8, 12
20	Deploy xong là xong	Mô hình suy giảm âm thầm	Monitoring + retrain có gate	12
21	Retrain rồi tự động deploy không gate	Mô hình tệ lên production	Quality gate + shadow/canary	12
22	Hard-code secret, đường dẫn, tham số	Rò rỉ bảo mật, khó thay đổi	Env/secret manager, config YAML	0, 12
23	Nạp pickle từ nguồn không tin cậy	Thực thi mã độc	skops / registry kiểm soát	12
24	Bỏ qua cảnh báo deprecation	Pipeline vỡ khi nâng cấp thư viện	Lock phiên bản; chạy test với cảnh báo bật	0
25	Tối ưu metric offline, quên KPI online	Mô hình "tốt" nhưng không tạo giá trị	Hệ thống metric nhất quán; A/B test	1, 4

13.4. Bộ câu hỏi review mô hình (dành cho reviewer)

Bài toán & dữ liệu

1. Ai dùng dự đoán, dùng lúc nào, hành động gì? Nếu không có mô hình thì hiện tại làm thế nào?
2. Nhân được định nghĩa thế nào? Có gap không? Thực thể nào bị loại?
3. Feature quan trọng nhất có sẵn **tại thời điểm dự đoán** không? Nếu bỏ nó, hiệu năng giảm bao nhiêu?

Validation & metric

4. Cách chia dữ liệu có mô phỏng production không? Adversarial AUC giữa train và test là bao nhiêu?
5. Metric chính có gắn với chi phí kinh doanh không? Ngưỡng được chọn trên dữ liệu nào?
6. Cải thiện so với baseline có CI không? Có ý nghĩa kinh doanh không?

Rủi ro

7. Mô hình hoạt động thế nào với khách mới, giá trị thiếu, category chưa thấy, nguồn dữ liệu chậm 3 ngày?
8. Phân khúc nào mô hình kém nhất? Có nhóm nào bị đối xử bất lợi không?
9. Xác suất có được dùng trực tiếp không? Nếu có, đã calibrate chưa?

Vận hành

10. Ai nhận cảnh báo khi mô hình suy giảm, và họ làm gì?
11. Rollback trong 5 phút bằng cách nào? Fallback là gì?
12. Khi nào retrain, ai phê duyệt promote?

13.5. Mẫu postmortem (blameless) cho sự cố mô hình

```
# Postmortem: [Tên sự cố] – [Ngày]
## Tóm tắt
Một đoạn: chuyện gì xảy ra, ảnh hưởng (khách hàng, doanh thu), thời gian phát hiện và khắc phục.
## Dòng thời gian (giờ Việt Nam)
- 08:00 deploy model v13 (canary 5%)
- 09:30 cảnh báo PSI điểm = 0.41 trên dashboard
- 09:45 on-call xác nhận; 09:50 rollback alias champion -> v12
## Nguyên nhân gốc (5 Whys)
- Vì sao điểm trôi? -> monthly_charges tăng 1000 lần.
- Vì sao? -> nguồn billing đổi đơn vị VNĐ -> đồng mà không thông báo.
- Vì sao không bị chặn? -> data contract chỉ kiểm tra kiểu, không kiểm tra miền giá trị.
## Điều làm tốt / chưa tốt
## Hành động khắc phục (có chủ sở hữu, hạn chót)
- [ ] Thêm kiểm tra miền giá trị và phân phối vào contract (DS, 15/10)
- [ ] Thỏa thuận data contract với team billing (DE, 30/10)
- [ ] Thêm test tích hợp cho thay đổi đơn vị (ML Eng, 20/10)
```

13.6. Lộ trình áp dụng handbook trong một dự án 12 tuần

Tuần	Hoạt động	Sản phẩm	Chương
1-2	Framing, metric, design doc; khảo sát nguồn dữ liệu	ML Design Doc ký duyệt	1
2-3	Thu thập, data contract, versioning	Dataset v1 + contract	2
3-4	EDA, phân tích thống kê	Báo cáo EDA + quyết định tiền xử lý	3, 4
4-6	Pipeline tiền xử lý, baseline, mô hình ứng viên	Leaderboard với CI	5-8
6-7	Tuning, threshold, calibration, XAI, fairness	Model Card v1	8-10
7-8	Package hóa, test, CI, registry	Lệnh train tái lập + CI xanh	12
8-9	Serving (batch/API), Docker, giám sát	Môi trường staging	12
9-12	Shadow → A/B test → rollout; báo cáo kết quả	Quyết định go/no-go dựa trên OEC	4, 11, 12

Tài liệu tham khảo Chương 13

- Breck, E. et al. (2017). *The ML Test Score: A Rubric for ML Production Readiness and Technical Debt Reduction*. IEEE Big Data.
- Sculley, D. et al. (2015). *Hidden Technical Debt in Machine Learning Systems*. NeurIPS.
- Zinkevich, M. *Rules of Machine Learning*. Google for Developers.
- Google SRE Book: *Postmortem Culture: Learning from Failure*. sre.google
- Kapoor, S. & Narayanan, A. (2023). *Leakage and the reproducibility crisis in ML-based science*. Patterns.
- scikit-learn User Guide: *Common pitfalls and recommended practices*. · imbalanced-learn: *Common pitfalls*.

PHỤ LỤC

Phụ lục A. Cheatsheet thư viện theo giai đoạn

Giai đoạn	Thư viện	Dùng để	Tài liệu chính thức
Thao tác dữ liệu	pandas (3.x), polars, duckdb, pyarrow	DataFrame, SQL trên file, Arrow/Parquet	pandas.pydata.org · docs.pola.rs · duckdb.org
Dữ liệu lớn	pyspark, dask, ray	Xử lý phân tán	spark.apache.org · docs.dask.org
Thu thập	SQLAlchemy 2.x, httpx, tenacity, beautifulsoup4, playwright, scrapy, confluent-kafka	DB, API, scraping, streaming	docs.sqlalchemy.org · python-httpx.org
Chất lượng dữ liệu	pandera, great_expectations, soda-core	Data contract	pandera.readthedocs.io
Versioning	dvc, Delta Lake, lakeFS	Phiên bản dữ liệu	dvc.org/doc
EDA	ydata-profiling, sweetviz, missingno	Báo cáo tự động	
Thống kê	scipy.stats, statsmodels, pingouin	Kiểm định, hồi quy, GLM, survival, chuỗi thời gian	docs.scipy.org · statsmodels.org
Thực nghiệm & nhân quả	statsmodels.stats.power, DoWhy, EconML, CausalML	Power, A/B, uplift	
Tiền xử lý	scikit-learn, category_encoders, feature-engine, imbalanced-learn, optbinning	Pipeline, encoding, resampling	scikit-learn.org · imbalanced-learn.org
Văn bản tiếng Việt	underthesea, pyvi, transformers (PhoBERT), sentence-transformers	Tách từ, embedding	
Mô hình cổ điển	scikit-learn	Linear, cây, SVM, KNN, clustering	
Gradient boosting	lightgbm, xgboost, catboost	Mô hình bảng mạnh nhất	lightgbm.readthedocs.io · xgboost.readthedocs.io · catboost.ai
Deep learning	pytorch, lightning, transformers	NN, NLP, CV	pytorch.org
Chuỗi thời gian	statsforecast, sktime, darts	Dự báo	
Survival	lifelines, scikit-survival, statsmodels.duration	Thời gian đến sự kiện	
Tuning	optuna, ray[tune]	Tối ưu siêu tham số	optuna.readthedocs.io
Giải thích	shap, scikit-learn.inspection, interpret (EBM), alibi, dice-ml	XAI	shap.readthedocs.io
Fairness	fairlearn, aif360	Đo và giảm bias	fairlearn.org
Conformal	mapie	Khoảng dự đoán có bảo đảm	
Trực quan hóa	matplotlib, seaborn, plotly, altair	Biểu đồ	matplotlib.org · seaborn.pydata.org
Dashboard	streamlit, dash, gradio	Ứng dụng dữ liệu	docs.streamlit.io
Tracking/Registry	mlflow (3.x), wandb	Thí nghiệm, mô hình	mlflow.org/docs
Serving	fastapi, bentoml, kserve, onnxruntime	API, tối ưu suy luận	fastapi.tiangolo.com
Orchestration	airflow, prefect, dagster	Pipeline	

Giai đoạn	Thư viện	Dùng để	Tài liệu chính thức
Feature store	feast	Offline/online feature	docs.feast.dev
Monitoring	evidently (0.7+), nannyml, prometheus, grafana	Drift, hiệu năng	docs.evidentlyai.com
Chất lượng code	ruff, mypy, pytest, pre-commit, threadpoolctl	Lint, type, test, kiểm soát luồng	

Phụ lục B. Thay đổi API cần biết (đã kiểm chứng khi viết handbook)

Thư viện	Cũ (đã deprecate / đổi hành vi)	Mới
scikit-learn 1.8+	<code>LogisticRegression(penalty="l1")</code>	<code>LogisticRegression(l1_ratio=1, solver="liblinear" "saga"); C=np.inf</code> cho không penalty
scikit-learn 1.9	<code>TargetEncoder(shuffle=True, random_state=0)</code>	<code>TargetEncoder(cv=StratifiedKFold(5, shuffle=True, random_state=0))</code>
scikit-learn 1.6+	<code>CalibratedClassifierCV(model, cv="prefit")</code>	<code>CalibratedClassifierCV(FrozenEstimator(model))</code>
scikit-learn 1.4+	<code>mean_squared_error(..., squared=False)</code>	<code>root_mean_squared_error(...)</code>
scikit-learn 1.2+	<code>OneHotEncoder(sparse=False)</code>	<code>OneHotEncoder(sparse_output=False)</code>
scikit-learn 1.9	(mới)	<code>metric_at_thresholds</code> , <code>confusion_matrix_at_thresholds</code>
pandas 3.0	Chained assignment <code>df["a"][m] = v</code>	<code>df.loc[m, "a"] = v</code> (Copy-on-Write luôn bật)
pandas 3.0	Chuỗi kiểu <code>object</code>	Kiểu <code>str</code> mặc định; <code>datetime</code> không còn mặc định <code>ns</code>
pandas 3.x → 4.0	<code>df.sum(1)</code> (positional)	<code>df.sum(axis=1)</code>
MLflow 3	<code>log_model(model, artifact_path="model")</code>	<code>log_model(sk_model=model, name="model"); LoggedModel models: /<model_id></code>
MLflow 3	Tracking <code>./mlruns</code> (file store)	<code>sqlite:///mlflow.db</code> / PostgreSQL
MLflow 3	sklearn serialize mặc định pickle	<code>skops</code> (an toàn) ngoài Databricks; custom class cần <code>skops_trusted_types</code> hoặc <code>cloudpickle</code> + <code>code_paths</code>
MLflow 2.9+	Stages Staging / Production	Aliases (<code>@champion</code>)
SciPy 1.17	<code>stats.anderson(x)</code>	<code>stats.anderson(x, method="interpolate")</code> (Có <code>pvalue</code>)
SciPy 1.15+	<code>random_state=</code> trong <code>bootstrap/permutation_test</code>	<code>rng=</code>
statsmodels 0.15 → 0.16	<code>adfuller(x)[1]</code> , <code>kpss(x)[1]</code> (tuple)	<code>adfuller(x, result_object=True).pvalue</code>
imbalanced-learn 0.14	Pipeline lồng trong <code>imblearn.pipeline.Pipeline</code>	Trải phẳng các bước
Evidently 0.7	<code>from evidently.report import Report; report.run(reference_data=..., current_data=...)</code>	<code>from evidently import Report; Report([...]).run(current_data, reference_data)</code> trả về Snapshot
XGBoost 2+	<code>early_stopping_rounds</code> trong <code>fit</code>	Trong constructor

Phụ lục C. Bảng chọn metric nhanh

Bài toán?	
Phân loại	
Cần xác suất đúng?	Log loss, Brier (+ reliability diagram, ECE)
Positive hiếm (<10%)?	PR-AUC, Recall@Precision _x , F-β, MCC
Hành động trên top-K?	Precision@K, Lift@K, cumulative gain
Có ma trận chi phí?	Lợi nhuận kỳ vọng (+ threshold tuning)
Cân bằng, chi phí lỗi như nhau?	Accuracy, F1, ROC-AUC
Đa lớp	Macro-F1 (lớp hiếm quan trọng) / log loss
Hồi quy (theo đại lượng cần dự đoán)	
Mean	RMSE / R ² ; Poisson/Gamma/Tweedie deviance cho dữ liệu đếm/dương
Median	MAE
Quantile / khoảng	Pinball loss + coverage
Sai số tương đối	WAPE, sMAPE (MAPE chỉ khi y xa 0); MASE cho chuỗi thời gian
Xếp hạng / truy hồi	NDCG@K, MAP@K, MRR, Recall@K, F2
Phân cụm	Silhouette, Davies–Bouldin (+ ARI/NMI nếu có nhãn) + đánh giá nghiệp vụ

Phụ lục D. Bảng chọn thuật toán

Tiêu chí	Logistic/Linear	Random Forest	GBM (LGBM/XGB/Cat/HGB)	Neural Net	KNN	SVM
Hiệu năng dữ liệu bằng	Trung bình-Khá (tốt nếu quan hệ gần tuyến tính)	Khá	Cao nhất (thường)	Khá-Cao (dữ liệu lớn)	Thấp-TB	Khá
Cần scale	Có	Không	Không	Có	Có	Có
Xử lý NaN gốc	Không	Có (sklearn ≥ 1.4)	Có	Không	Không	Không
Biến phân loại gốc	Không	Không	Có (LGBM, Cat, HGB, XGB)	Embedding	Không	Không
Xác suất calibrate sẵn	Tốt (khi đúng đặc tả)	Kém (chữ S)	Khá	Khá-kém	Kém	Không có (cần Platt)
Giải thích	Rất tốt	TB	TB (SHAP)	Kém	TB	Kém
Ràng buộc đơn điệu	Dấu hệ số	Không	Có	Hạn chế	Không	Không
Tốc độ train	Rất nhanh	Nhanh	Nhanh	Chậm	Không cần	Chậm khi n lớn
Ngoại suy	Có	Không	Không	Có	Không	Hạn chế
Dữ liệu rất nhỏ	Tốt	Tốt	Khá	Kém (trừ TabPFN)	Tốt	Tốt

Phụ lục E. Thuật ngữ Anh - Việt

English	Tiếng Việt
Exploratory / Confirmatory Data Analysis	Phân tích khám phá / khẳng định
Point-in-time correctness	Đúng thời điểm (không nhìn trộm tương lai)
Data leakage / Target leakage	Rò rỉ dữ liệu / rò rỉ nhãn
Imputation / Missing indicator	Điền giá trị thiếu / cờ thiếu
Cross-fitting	Khớp chéo (mã hóa mỗi fold bằng thống kê các fold khác)
Stratified / Group / Out-of-time split	Chia phân tầng / theo nhóm / theo thời gian

English	Tiếng Việt
Nested cross-validation	Kiểm định chéo lồng nhau
Early stopping	Dừng sớm
Strictly proper scoring rule	Quy tắc chấm điểm thích hợp chặt
Calibration / Reliability diagram	Hiệu chuẩn xác suất / biểu đồ độ tin cậy
Decision threshold	Ngưỡng quyết định
Confusion matrix	Ma trận nhầm lẫn
Precision / Recall / Specificity	Độ chính xác dự đoán dương / độ bao phủ (độ nhạy) / độ đặc hiệu
Permutation importance	Độ quan trọng hoán vị
Partial dependence / ICE	Phụ thuộc riêng phần / kỳ vọng có điều kiện từng mẫu
Concept drift / Covariate shift	Trôi khái niệm / dịch chuyển phân phối đầu vào
Training-serving skew	Lệch giữa huấn luyện và phục vụ
Model registry / Alias	Kho quản lý mô hình / bí danh phiên bản
Champion / Challenger	Mô hình đương nhiệm / mô hình thách đấu
Shadow / Canary deployment	Triển khai bóng / triển khai thăm dò
Oversubscription	Quá tải luồng (nhiều luồng hơn số lỗi thật)
Overall Evaluation Criterion (OEC)	Tiêu chí đánh giá tổng thể của thí nghiệm
Sample Ratio Mismatch (SRM)	Lệch tỷ lệ phân bố mẫu

Phụ lục F. Bản đồ khóa học chuyên sâu theo chương

Chương	Khóa học / tài liệu chuyên sâu nên học kèm
0-1	Google <i>ML Problem Framing & Rules of ML</i> ; DeepLearning.AI <i>MLOps Specialization</i> (C1: Scoping); FSDL <i>Lecture: ML Projects</i> ; Chip Huyen <i>CS329S</i>
2	Harvard <i>CS109</i> (Data collection & scraping); Kaggle Learn <i>Pandas</i> ; DeepLearning.AI <i>MLOps</i> (C2: Data); pandas User Guide
3-4	Harvard <i>CS109</i> ; Kohavi <i>Trustworthy Online Controlled Experiments</i> ; Hernán & Robins <i>Causal Inference: What If</i> ; <i>Forecasting: Principles and Practice</i>
5-6	scikit-learn User Guide & Examples; Kaggle Learn <i>Feature Engineering, Intermediate ML</i> ; Kuhn & Johnson <i>FES</i>
7	scikit-learn <i>Cross-validation</i> ; Kaggle Learn <i>Data Leakage</i> ; Andrew Ng <i>Machine Learning Yearning</i>
8	Stanford <i>CS229</i> ; DeepLearning.AI <i>Machine Learning Specialization</i> ; fast.ai <i>Practical Deep Learning</i> ; tài liệu LightGBM/XGBoost/CatBoost/Optuna
9-10	scikit-learn <i>Model evaluation, Calibration, Inspection</i> ; Molnar <i>Interpretable ML</i> ; fairlearn User Guide
11	Wilke <i>Fundamentals of Data Visualization</i> ; Knaflitz <i>Storytelling with Data</i> ; Harvard <i>CS109</i> (Visualization)
12-13	<i>Made With ML</i> (MLOps); DeepLearning.AI <i>MLOps</i> (C3-C4: Deployment, Monitoring); FSDL; Google <i>MLOps levels</i> ; MLflow/FastAPI/Evidently docs

Phụ lục G. Cách dùng package tham chiếu và kiểm chứng handbook

```
cd docs/data-science-handbook/code
pip install -e ".[dev,extra]"
pytest -q                                # 18 test: unit, data, behaviour, quality gate, API
python -m churn.models.train --config configs/train.yaml # huấn luyện + log MLflow (sqlite:///mlflow.db)
make serve                               # FastAPI :8000
cd ..
```

```
python build/run_snippets.py
./build/build.sh
```

```
# chạy MỌI khối code của các chương
# ghép Markdown + xuất PDF
```

Phụ lục H. Tài liệu tham khảo tổng hợp

Tài liệu chính thức

- scikit-learn User Guide (1.9): scikit-learn.org/stable/user_guide.html · Common pitfalls · Examples gallery.
- pandas User Guide (3.0): pandas.pydata.org/docs/user_guide · SciPy reference: docs.scipy.org · statsmodels: statsmodels.org.
- LightGBM: lightgbm.readthedocs.io (Parameters Tuning) · XGBoost: xgboost.readthedocs.io (Notes on Parameter Tuning) · CatBoost: catboost.ai/docs.
- Optuna: optuna.readthedocs.io · MLflow 3: mlflow.org/docs · imbalanced-learn: imbalanced-learn.org · SHAP: shap.readthedocs.io · fairlearn: fairlearn.org · Evidently: docs.evidentlyai.com · pandera: pandera.readthedocs.io · FastAPI: fastapi.tiangolo.com · DVC: dvc.org/doc · Feast: docs.feast.dev.

Sách

- Géron, A. (2022). *Hands-On Machine Learning with Scikit-Learn, Keras & TensorFlow*, 3rd ed. O'Reilly.
- Hastie, T., Tibshirani, R. & Friedman, J. (2009). *The Elements of Statistical Learning*, 2nd ed. Springer.
- James, G. et al. (2023). *An Introduction to Statistical Learning with Applications in Python*. Springer.
- Kuhn, M. & Johnson, K. (2019). *Feature Engineering and Selection*. CRC Press.
- Huyen, C. (2022). *Designing Machine Learning Systems*. O'Reilly.
- Kohavi, R., Tang, D. & Xu, Y. (2020). *Trustworthy Online Controlled Experiments*. Cambridge.
- Molnar, C. (2022). *Interpretable Machine Learning*, 2nd ed.
- Provost, F. & Fawcett, T. (2013). *Data Science for Business*. O'Reilly.
- Kleppmann, M. (2017). *Designing Data-Intensive Applications*. O'Reilly.
- Wilke, C. O. (2019). *Fundamentals of Data Visualization*. O'Reilly.

Bài báo nền tảng

- Sculley et al. (2015) *Hidden Technical Debt in ML Systems* · Breck et al. (2017) *The ML Test Score* · Kapoor & Narayanan (2023) *Leakage and the reproducibility crisis*.
- Friedman (2001) *Gradient Boosting Machine* · Chen & Guestrin (2016) *XGBoost* · Ke et al. (2017) *LightGBM* · Prokhorenkova et al. (2018) *CatBoost* · Akiba et al. (2019) *Optuna*.
- Gneiting & Raftery (2007) *Strictly Proper Scoring Rules* · Niculescu-Mizil & Caruana (2005) *Predicting Good Probabilities* · Elkan (2001) *Cost-Sensitive Learning*.
- Lundberg & Lee (2017) *SHAP* · Mitchell et al. (2019) *Model Cards* · Grinsztajn et al. (2022) *Tree-based models vs deep learning on tabular data*.

Hết handbook. Phiên bản 2.0, tháng 10/2026. Mọi khối code được kiểm chứng tự động bằng `build/run_snippets.py`. Góp ý qua Pull Request.